

The DevOps Engineer's Lab Notebook

A guided-practice companion to the
DevOps Engineer — 14-Day Speedrun Roadmap

From “I can write a Dockerfile” to designing, automating, securing, observing and operating production cloud infrastructure — understanding *why* each layer exists, not just which command to run.

14

DAYS

42

GUIDED LABS

115+

HOURS DEPTH

1

CAPSTONE

This notebook belongs to: _____

Started on: _____ Target finish: _____



How to Use This Workbook

This is not an exam-prep book and not a second reference manual. It is a **laboratory manual** — a place to *do the engineering* beside the roadmap. The roadmap tells you *what* to learn; this workbook walks you through understanding *why it exists*, *how it works underneath*, and *what it costs you* — then makes you build, break and fix it.

The goal is not command recall. It is to leave each chapter reasoning like a senior engineer: able to justify a design decision, name its trade-offs, and say what you would do differently at ten times the scale.

The daily study loop

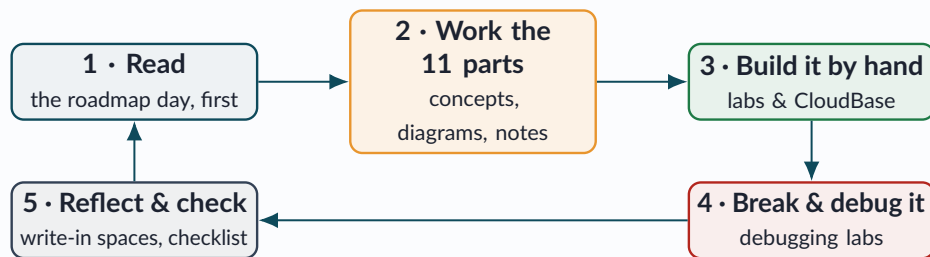


Fig 0.1 — Run this loop once per day. The learning is in steps 3 and 4, not step 1.

The eleven parts of every chapter

1	The Big Picture	What problem it solves, where it sits in a cloud architecture, how it links to earlier days.
2	Concept Map	One page holding the whole territory and its vocabulary.
3	Guided Learning	Each idea taught progressively: what / why it exists / how it works internally / when / when not / production example.
4	Visualizations	Redraw the architecture from memory to lock it in.
5	Guided Hands-On Lab	Build it step by step — the workbook guides the decisions, you write the code.
6	Thinking Exercises	Engineering reflections, not quizzes.
7	Debugging Practice	A real incident worked as Observe → Hypothesise → Investigate → Root cause → Fix → Prevent.
8	Mini-Labs	Three labs: Beginner, Intermediate, Production-style.
9	Engineering Notes	Senior judgment, trade-offs, cost and reliability thinking.
10	Build-Along Project	Extend CloudBase , one cloud platform built across all 14 days.
11	Chapter Summary	Takeaways, cheat sheet, CLI reference, official docs, production checklist.



Production tip

The write-in spaces and "redraw from memory" boxes are meant to be filled in *by hand*. Print the pages you are working, or annotate the PDF. Reproducing an architecture diagram yourself is what turns "I have seen that" into "I can whiteboard that in an interview."

Security warning • on copy-paste

This workbook deliberately does not hand you finished infrastructure. The labs guide you to build it. If you paste a Terraform module or a Kubernetes manifest you did not type and reason through, you will feel productive and learn nothing — and you will not be able to debug it at 3am. The struggle in Parts 5, 7 and 10 is the entire point.

Field Guide to the Engineering Notes

Coloured callouts flag different *kinds* of engineering thinking. Learn to read them at a glance.

Senior engineer insight

How an experienced engineer reasons about the decision — the judgment behind the code.

Cloud cost optimization

Where the cloud bill quietly comes from, and how to avoid it.

Production tip

A concrete practice that holds up under real production load.

Automation tip

Where to replace human effort with automation.

Infrastructure insight

Something the infrastructure is doing underneath the abstraction.

Operational excellence

Operational excellence: runbooks, on-call, and what "done" means.

Behind the scenes

What the tool or kernel is actually doing behind the command.

Disaster recovery tip

Disaster recovery: what happens when a whole component or region is gone.

Design trade-off

An honest design trade-off — what you gain and what you give up.

Performance note

A latency or throughput consideration worth measuring.

Scalability note

What changes when traffic, nodes or teams multiply by ten.

Interview insight

A question you will likely be asked, and how to answer it well.

Reliability principle

A principle that keeps the system up when something fails.

Common misconception

A belief many engineers hold that is subtly wrong.

Security warning

A security trap — read these twice.

And the working blocks you will act on:

🔧 Mini-Lab • Beginner / Intermediate / Production*graded practice*

Labs with a Goal, Business context, Architecture, Requirements, Constraints, Hints, Expected outcome, Production considerations and Common pitfalls.

🔍 Debugging Lab • A real production incident

Symptom and evidence — logs, `kubectl` output, a plan diff — then a guided Observe → Hypothesise → Investigate → Root cause → Fix → Prevent walkthrough.

🧠 Think it through

An engineering reflection with ruled space to write your answer by hand.

🔧 Build-Along • CloudBase

The one cloud platform you extend a little further every single day.

📅 The 14-Day Arc at a Glance

Each day earns its place and feeds the next. Read down the "CloudBase milestone" column to watch one platform grow from a bare Linux host into production infrastructure with GitOps delivery and real SLOs.

Day	Topic	CloudBase milestone you build
1	Linux & the Command Line	Base host: strict-mode bootstrap, service user, systemd unit.
2	Git & Version Control	Repo conventions: branch protection, CODEOWNERS, secret-blocking hook.
3	Networking for DevOps	Network design: /16 VPC, public/private subnets, bastion, least-privilege SGs.
4	Docker Deep Dive	Hardened multi-stage image, non-root, minimal build context.
5	Kubernetes Fundamentals	Deployment, Service, config/secrets, liveness & readiness probes.
6	Kubernetes in Production	Helm chart, Ingress with TLS, HPA, least-privilege RBAC.
7	Terraform (IaC)	The network in code, with remote state and locking.
8	Ansible (Config Mgmt)	Idempotent role replacing the bootstrap script; Vault secrets.
9	CI/CD Engineering	test → scan → build → push, SHA-tagged, promoted not rebuilt.
10	Cloud Infra on AWS	ALB, cluster, RDS in private subnets, least-privilege IAM.
11	Monitoring & Observability	Prometheus + Grafana, correlated logs, one real SLO and alert.
12	Security & Secrets	Secrets manager, image-scan gate, NetworkPolicies, RBAC audit.
13	GitOps & Deployment	Argo CD reconciliation, canary gated on SLO, rehearsed rollback.
14	Capstone	Full assembly, DR drill, runbook, architecture review.

🔑 Key idea

CloudBase is deliberately a *small* application carrying a *large* amount of operational weight — exactly the shape of a strong take-home assignment or a well-scoped first project at a new job. If you have worked Volume I of this series, the natural application to wrap in this infrastructure is the order API you built there. Depth over breadth, every day.



Contents

Day 01 – Linux & the Command Line That Actually Matters	6
Day 02 – Git & Version Control Mastery	21
Day 03 – Networking & Linux Systems for DevOps	42
Day 04 – Docker Deep Dive	63
Day 05 – Kubernetes Fundamentals	85
Day 06 – Kubernetes in Production	107
Day 07 – Infrastructure as Code with Terraform	132
Day 08 – Configuration Management with Ansible	154
Day 09 – CI/CD Pipeline Engineering	177
Day 10 – Cloud Infrastructure on AWS	199
Day 11 – Monitoring, Logging & Observability	223
Day 12 – Security & Secrets Management	246
Day 13 – GitOps & Deployment Strategies	267
Day 14 – Final Project – CloudBase	291
Appendix A – DevOps Interview Cheat Sheet	312
Appendix B – The CloudBase Build-Along Master Tracker	316
Appendix C – Command & Manifest Quick Reference	318
Appendix D – Glossary of Key Terms	320

DAY 01 • FOUNDATIONS

Linux & the Command Line That Actually Matters

The substrate every container, cluster and cloud VM is standing on

🕒 Estimated study time: 8–9 hours

🎯 By the end of this day, you will be able to...

- Navigate and reason about the Linux filesystem hierarchy like an operator, not a tourist.
- Read and set permissions and ownership correctly — including SUID and the sticky bit.
- Manage processes and signals, and say exactly what each signal does to a running program.
- Write and enable `systemd` units, and read their logs with `journalctl`.
- Write shell scripts that fail loudly instead of silently corrupting state.

1

The Big Picture

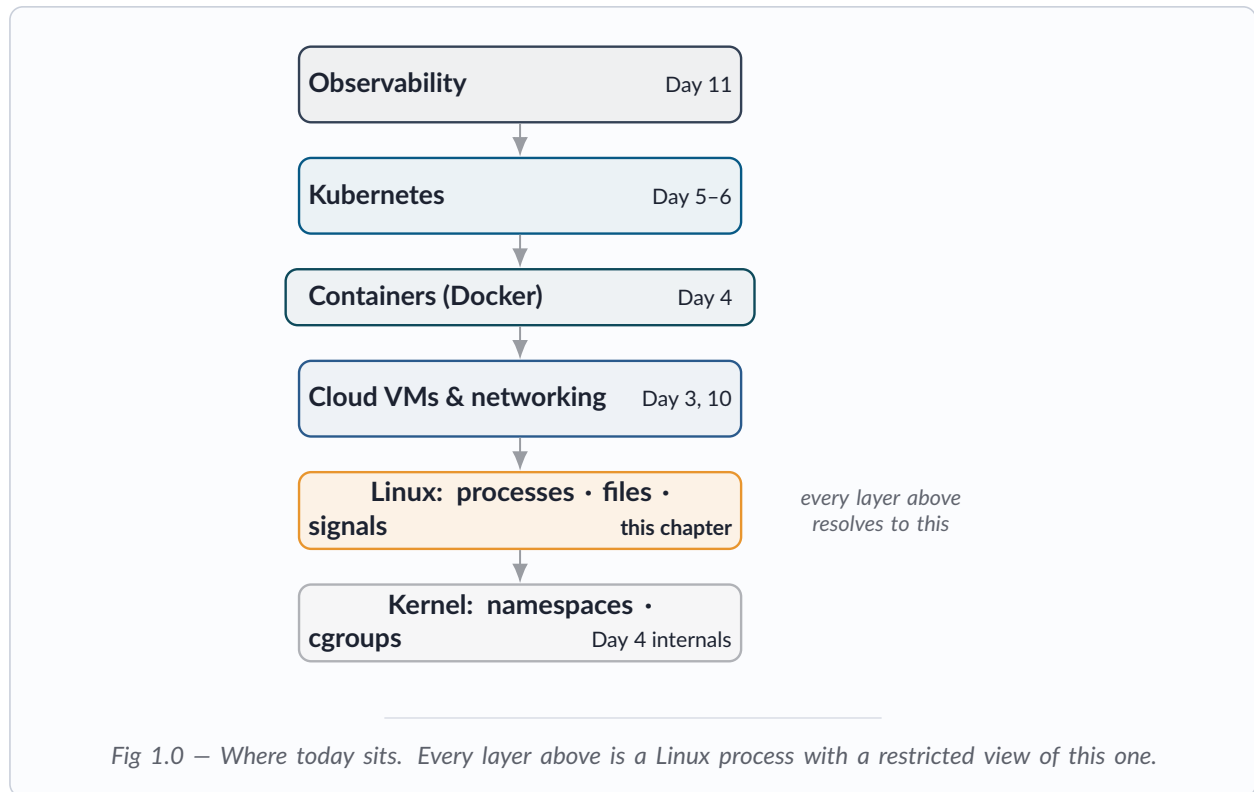
What problem this solves. Every abstraction you will build in the next thirteen days — a container, a pod, a Terraform-provisioned EC2 instance, a CI runner — is a Linux process with a Linux filesystem, Linux permissions and Linux signals underneath it. When a container exits with code 137, when a pod refuses to start because a mounted file is `root`-owned, when a deploy script silently deletes the wrong directory, the explanation is never in the tool's documentation. It is here.

Why DevOps engineers need it. Tools change every few years; the kernel interface does not. An engineer who understands processes, file descriptors and signals can debug Docker, Kubernetes, `systemd` and a CI runner with the same mental model. An engineer who only knows `kubectl` is stuck the moment the abstraction leaks — and abstractions always leak in production.

🔑 Key idea

Containers are not small virtual machines. They are ordinary Linux processes with restricted views of the filesystem, the network and the process table. Everything you learn today is what a container actually is — which is why Day 4 will feel like a re-reading of this chapter rather than a new topic.

Where it fits in a modern cloud architecture. Linux is the bottom layer of the entire stack you are about to build. Read the diagram upward: nothing above works if the layer below is misunderstood.



Where companies use it in production. Netflix runs its streaming control plane on tens of thousands of Linux instances and built its own base AMI tooling so that every machine boots from one hardened, reproducible image. Cloudflare's edge is Linux boxes running tuned kernel networking. Google's Borg – the ancestor of Kubernetes – is a scheduler placing Linux processes into cgroups. In all three the operational skill is identical: read the process table, read the logs, understand what the kernel is enforcing.

Common misconceptions. That "Linux knowledge" means memorising commands (it means understanding the process and file model); that `root` is the answer to a permissions problem (it is usually the question); that a container isolates a process completely (it isolates a view, and by default shares the kernel).

2

Concept Map

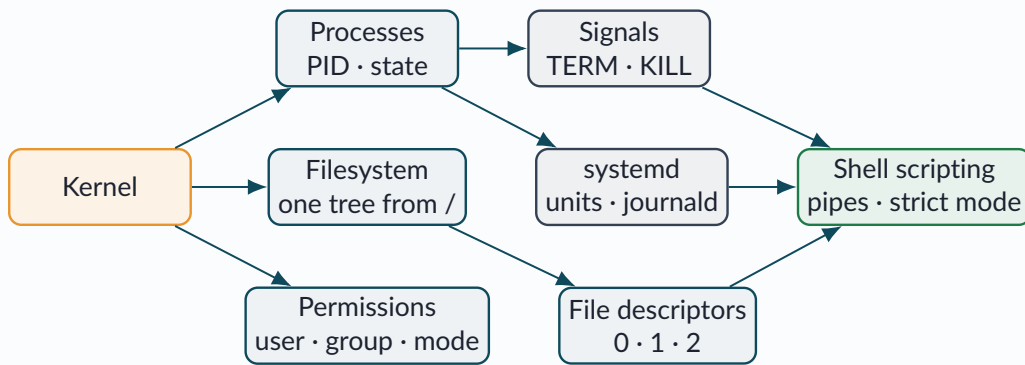


Fig 1.1 — The Day 1 territory: the kernel exposes resources as files, processes act on them, systemd supervises, and the shell composes it all.

Terminology to anchor: **everything is a file** → **file descriptor** → **process** → **signal** → **init (systemd)** → **unit** → **journal** → **pipeline**.

3 Guided Learning

3.1 The Filesystem Hierarchy & Why Everything Is a File

What is it? Linux organises the entire system as a single tree rooted at `/`. There is no `c:` drive; a second disk is *mounted* into the tree at some path.

Why does it exist? A single namespace means one set of tools works everywhere. Before this convention, every resource type needed its own API. By modelling devices, sockets and pipes as files, the kernel gives you *one* interface — `open`, `read`, `write`, `close` — for everything.

How does it work internally? Directories map names to inodes; inodes hold metadata and block pointers. Some filesystems are not on disk at all: `/proc` is generated by the kernel on read. `/proc/cpuinfo` does not exist until you read it — the kernel synthesises the bytes at that moment. This is why `/proc` is the canonical place to inspect live kernel state.

Path	What lives there, and why you will care
<code>/etc</code>	System-wide configuration, plain text. Config drift lives here (Day 8 fixes it with Ansible).
<code>/var</code>	Data that changes at runtime — logs, caches, spools. This is what fills up disks.
<code>/home</code>	User data.
<code>/usr</code>	Installed software and its resources.
<code>/opt</code>	Third-party, self-contained packages.
<code>/tmp</code>	Wiped on reboot. Never durable storage.
<code>/proc</code>	Not a real filesystem — a live window into kernel state; every PID is a directory.

👁 Behind the scenes

When you run `ps aux`, nothing scans memory. `ps` walks `/proc`, reading one directory per PID and parsing text the kernel generated on demand. The process table is a filesystem. This is also how container tools inspect processes — and why a container with a restricted `/proc` mount cannot see the host's processes.

3.2 Permissions, Ownership & the Bits Engineers Get Wrong

What is it? Every file carries three permission triads — owner, group, other — each with read, write and execute (*rw*x). `chmod 750 deploy.sh` sets owner = *rw*x, group = *r*-*x*, other = none.

Why does it exist? Multi-user systems needed a cheap, kernel-enforced answer to "who may touch this?" that did not require consulting a central service on every syscall. Three triads and nine bits fit in the inode.

How does it work internally? On a *directory*, the bits mean something different from a file, and this is the single most common source of confusion: *r* lists names, *w* creates/deletes entries, and *x* means "may traverse into it". A directory with read but no execute lets you list filenames but not `cd` into it or `stat` anything inside.

Symbol	Meaning	Common use
-rw-r--r--	Owner read/write; group and other read-only	Config files, source
-rwxr-xr-x	Owner full; everyone may read/execute	Scripts, binaries
drwxr-x--	Directory; group may enter but not write	Shared team directories
chmod u+s	SUID — runs as the file's <i>owner</i> , not the invoker	passwd
chmod +t	Sticky bit — only the file's owner may delete it	/tmp

`umask` subtracts permission bits from every newly created file (the common default `022` strips write from group and other). `chown user:group file` changes ownership — only *root* can give a file away.

Security warning • why `chmod 777` is almost never the fix

`chmod 777` says "any user or process on this host may read, modify and execute this." It resolves the symptom because it removes all enforcement. The correct move is to read the actual bits, identify which *identity* needs access, and grant that. This exact reflex is what Day 12's least-privilege IAM work is about — the principle is identical, only the enforcement point moves to the cloud.

Production tip

Prefer `sudo <command>` over an interactive root shell. Every invocation is logged with the real username, and `/etc/sudoers` can scope it to specific commands. "Who ran this?" is answerable afterwards — which matters enormously during an incident review.

3.3 Processes, Signals & systemd

What is it? Every running program is a process with a PID, a parent PID and a state (running, sleeping, zombie, stopped). `kill` does not "kill" — it *sends a signal*, and the process decides how to react.

Why does it exist? Signals are the kernel's minimal asynchronous notification mechanism: a way to tell a process something happened without a shared channel. They predate sockets and pipes as an IPC method.

Signal	Number	Behaviour
SIGHUP	1	Traditionally "reload config" — many daemons trap it instead of dying.
SIGINT	2	What Ctrl+C sends. Catchable.
SIGTERM	15	Polite shutdown request; the process can clean up. The default for <code>kill</code>, and what Docker and Kubernetes send first.
SIGKILL	9	Immediate, uncatchable termination by the kernel. No cleanup, no flushed buffers.

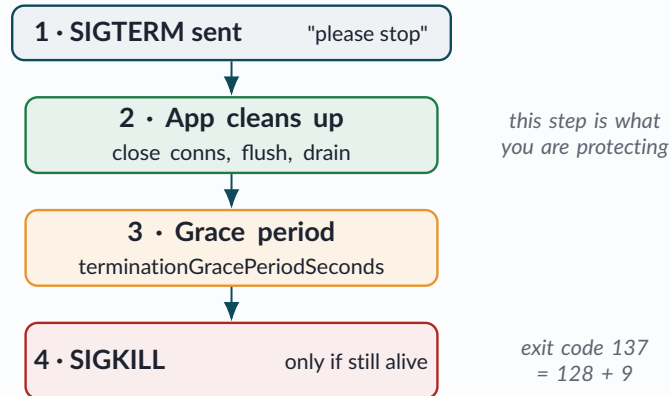


Fig 1.2 – The shutdown path Docker and Kubernetes both follow. The grace period is the whole game.

Reliability principle

A process killed with `-9` cannot close database connections, flush write buffers or remove lock files – this is precisely how you get corrupted files and orphaned locks. Always try SIGTERM and allow a grace period. When you see a container exit `137` in Day 4, you now know it means SIGKILL: the app ignored or outlasted its grace period.

How does it work internally? (systemd) `systemd` is PID 1: the first userspace process the kernel starts, and the ancestor of everything else. It reads declarative *unit* files, resolves a dependency graph between them, starts services in a valid order, restarts them on failure, and captures their stdout/stderr into the journal.

`/etc/systemd/system/cloudbase-api.service`

```
[Unit]
Description=CloudBase API
After=network.target

[Service]
ExecStart=/usr/bin/java -jar /opt/cloudbase/app.jar
Restart=on-failure
RestartSec=5
User=cloudbase
EnvironmentFile=/etc/cloudbase/env

[Install]
WantedBy=multi-user.target
```

managing the unit

```
sudo systemctl daemon-reload      # re-read unit files after ANY edit
sudo systemctl enable --now cloudbase-api
journalctl -u cloudbase-api -f     # follow live logs
systemctl status cloudbase-api     # state, PID, recent log lines
```

⚠ Common misconception

Editing a unit file does not change anything by itself. `systemd` caches the parsed unit graph in memory; without `systemctl daemon-reload` it keeps running the old definition. "I changed the config and nothing happened" is nearly always a missing `daemon-reload`.

When should I use it? Anything that must survive a reboot, restart on crash, start in a defined order, or have its logs collected — which is essentially every long-running service on a VM.

When should I not? Inside a container. A container should run one foreground process as PID 1 so the runtime's signals reach it directly. Running `systemd` inside a container re-introduces the supervision layer the orchestrator is already providing — that is Kubernetes' job from Day 5 onward.

3.4 Shell Scripting Engineers Actually Write

Why does it exist? Most production shell is *glue*: deploy steps, cron jobs, CI helpers, container entrypoints. It runs unattended, which means nobody is watching when it fails.

How does it work internally? Bash's defaults are optimised for interactive use, where a failed command is obvious because you see it. Unattended, those same defaults silently swallow the failures you most need to know about. Strict mode reverses them.

bootstrap.sh

```
#!/usr/bin/env bash
set -euo pipefail
# -e          exit immediately if any command fails
# -u          treat an unset variable as an error, not an empty string
# -o pipefail a failure anywhere in a pipe fails the whole pipe

ENVIRONMENT="${1:?Usage: bootstrap.sh <environment>}"
LOG_FILE="/var/log/bootstrap-${ENVIRONMENT}.log"

log() { echo "[$(date -Is)] $*" | tee -a "$LOG_FILE"; }

main() {
  log "Bootstrapping ${ENVIRONMENT}"
  systemctl restart cloudbase-api || { log "restart failed"; exit 1; }
}

main "$@"
```

👤 Security warning • the two failures strict mode prevents

Without `-u`, `rm -rf "$PREFIX/"` with `PREFIX` unset expands to `rm -rf /`. Without `-e`, `cd /some/typo-path && rm -rf ./*` carries on after the failed `cd` and deletes the *current* directory. Without `pipefail`, `grep pattern file | wc -l` reports success (the exit code of `wc`) even if `grep` crashed. These are not hypotheticals; they are recurring outage post-mortems.

3.5 Pipes, Redirection & Text Processing

How does it work internally? Every process starts with three open file descriptors: `stdin` (0), `stdout` (1) and `stderr` (2). `>` redirects `stdout`, `2>` redirects `stderr`, and `2>&1` points fd 2 at whatever fd 1 currently is. A pipe `|` is a kernel buffer wiring one process's fd 1 to the next process's fd 0 — the two run *concurrently*, not sequentially.

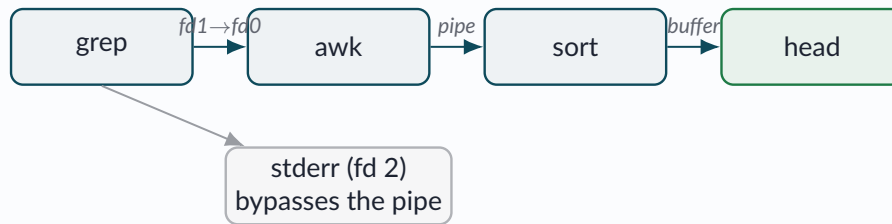


Fig 1.3 — A pipeline is concurrent processes joined by kernel buffers, not a sequence of steps.

log and disk triage pipelines

```
# Top 5 IPs causing 5xx responses in an access log
grep " 5[0-9][0-9] " access.log | awk '{print $1}' | sort | uniq -c | sort -rn | head -5

# What is filling the disk? Top 10 largest directories under /var
sudo du -h --max-depth=2 /var 2>/dev/null | sort -rh | head -10
```

💡 Senior engineer insight

Before reaching for a full observability stack, a senior engineer triages with a pipeline. Prometheus (Day 11) tells you *that* error rate rose; a five-command pipeline over the log tells you *which* endpoint and *which* client, in about twenty seconds, on the box. Both skills matter; the second one is the one juniors skip.

✅ Best practices — what experienced engineers do

- set `-euo pipefail` in every script that runs unattended.
- Use `systemd` units for anything that must survive a reboot — not a `nohup` & hack.
- Read the actual permission bits before running `chmod`; grant to an identity, not to everyone.
- Prefer `sudo <command>` over an interactive root shell — it keeps an audit trail.
- Quote every variable expansion ("`$VAR`"); unquoted expansion is how paths with spaces destroy things.

❌ Common mistakes beginners make

- Running `rm -rf` with an unquoted variable that can expand to empty.
- Sending `SIGKILL` by default instead of `SIGTERM` plus a grace period.
- Editing a unit file without `systemctl daemon-reload`, then wondering why nothing changed.
- Treating `/tmp` as durable storage — it is wiped on reboot.
- Reaching for `chmod 777` as a first response to any permission error.

4 Visualizations to Internalize

Redraw these from memory. Reproducing a diagram is what moves it from "I recognise that" to "I can explain that under pressure."

 Redraw: the SIGTERM → grace period → SIGKILL shutdown path, and where exit code 137 comes from

 Redraw: a four-stage pipeline showing fd 0, 1 and 2, and where stderr goes

5

Guided Hands-On Lab — Bootstrap a Production-Shaped Host

Goal. Provision a fresh Linux box the way a real team would: a dedicated service user, correct ownership, a supervised service, and a script that fails loudly. This becomes CloudBase's base-image prep, referenced again on Day 8 (Ansible) and Day 10 (AWS).

1 Create a dedicated non-root service user

Create a system user (no login shell, no home directory) that the application will run as.

 **Why:** If the app is compromised, the blast radius is that user's permissions. Running as root means the blast radius is the whole host — the same reasoning drives non-root containers on Day 4.

2 Lay out the application directory with correct ownership

Create `/opt/cloudbase`, own it by the service user, and set directory mode `750`.

 **Why:** You are practising "grant to an identity", not "open to everyone". Notice `execute` on a directory means `traverse`.

3 Write the systemd unit

Declare `ExecStart`, `User`, `Restart=on-failure`, `RestartSec`, and an `EnvironmentFile`.

 **Why:** Declaring restart policy and identity in the unit means the host enforces them — you are not relying on a human remembering.

4 Enable, start, and read the journal

`daemon-reload, enable -now`, then follow with `journalctl -u ... -f`.

 *Why:* You must be able to answer "is it running and what did it say?" without guessing. This is the same question `kubectrl logs` answers later.

5 Prove the restart policy works

Kill the process with `SIGKILL` and watch `systemd` bring it back. Then `systemctl stop` it and observe that it stays down.

 *Why:* Distinguishing "crashed" from "intentionally stopped" is exactly what Kubernetes probes encode on Day 6.

6 Wrap it all in a strict-mode bootstrap script

Put every step above into `bootstrap.sh` with `set -euo pipefail` and an environment argument.

 *Why:* An idempotent, loud-failing script is the bridge to configuration management — Day 8 replaces it with Ansible, and you will appreciate why.

7 Break it on purpose

Remove `set -u`, unset a variable used in a path, and observe what the script does. Then restore it.

 *Why:* You will never forget strict mode after watching what its absence does once.

Production tip

Run the script twice. A bootstrap script that fails the second time (because the user already exists) is not production-ready — **idempotency** is the property that lets automation run safely on an unknown starting state. That single idea is the foundation of Terraform (Day 7) and Ansible (Day 8).

6 Thinking Exercises

Think it through

Kubernetes sends `SIGTERM`, waits, then sends `SIGKILL`. Why does it not simply send `SIGKILL` and start a fresh pod immediately — wouldn't that be faster and more predictable?

Think it through

`/proc` is generated on read rather than stored on disk. What would break, or become much harder, if the kernel instead wrote process state to a real file every time it changed?

 Think it through

Running as a non-root user is standard advice, yet plenty of production containers still run as root. What makes the correct choice inconvenient, and what does that tell you about how security defaults should be designed?

7 Debugging Practice

 Debugging Lab • The service that "restarted successfully" but is serving old config

Symptom. A colleague edited `cloudbase-api.service` to add an environment variable, ran `systemctl restart`, saw no error — but the app still behaves as before.

```
systemctl status cloudbase-api

* cloudbase-api.service - CloudBase API
   Loaded: loaded (/etc/systemd/system/cloudbase-api.service; enabled)
   Active: active (running) since Mon 2026-07-19 09:12:04 UTC; 2min ago
     Main PID: 2841 (java)
  Warning: The unit file, source configuration file or drop-ins of
           cloudbase-api.service changed on disk. Run 'systemctl daemon-reload'.
```

- ✓ **Observe.** The service is running and restarted cleanly — but there is a warning about the unit file changing on disk.
- ✓ **Hypothesise.** `restart` restarts the process using the *cached* unit definition; the edit was never parsed.
- ✓ **Investigate.** Compare the file on disk with what systemd loaded: `systemctl cat cloudbase-api` versus the actual file.
- ✓ **Root cause.** A missing `systemctl daemon-reload` — systemd's in-memory unit graph is stale.
- ✓ **Fix.** `sudo systemctl daemon-reload && sudo systemctl restart cloudbase-api`, then confirm the variable is present.
- ✓ **Prevent.** Put `daemon-reload` in the deploy script so a human can never skip it — and treat that warning line as an error in review.

 Debugging Lab • Disk full at 3am, but `du` says the disk is nearly empty

Symptom. Writes fail with "No space left on device". `df` shows 100% used; `du -sh /` totals far less than the disk size.

df -h / and lsof

```

Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p1  50G   50G    0 100% /

COMMAND  PID  USER  FD  TYPE  SIZE/OFF      NODE NAME
java     2841 cloud  3w   REG  31457280000 (deleted) /var/log/cloudbase/app.log

```

- ✓ **Observe.** `df` (kernel's view) and `du` (directory walk) disagree — a classic signature.
- ✓ **Hypothesise.** A file was deleted while a process still held it open, so its blocks are not freed.
- ✓ **Investigate.** `sudo lsof +L1` lists open files with zero links — here a 30 GB deleted log still held by the app.
- ✓ **Root cause.** Log rotation deleted the file instead of truncating it; the process kept writing to the open descriptor.
- ✓ **Fix.** Restart (or `HUP`) the process to release the descriptor; space returns immediately.
- ✓ **Prevent.** Configure `logrotate` with `copytruncate` or a proper post-rotate signal — and alert on disk usage before it reaches 100%, not at it.

8 Mini-Labs

Mini-Lab • Beginner

30 min

Goal. Build a signal-handling script and watch each signal's real behaviour.

Business context. Your team keeps losing in-flight work when deploys roll pods.

Architecture. One bash script, one terminal, `kill` from a second terminal.

Requirements. Trap `SIGTERM` and `SIGINT`, log which arrived, clean up a lock file, exit 0.

Constraints. Do not trap `SIGKILL` (prove to yourself that you cannot).

Hints. `trap 'handler' TERM INT`; use a sleep loop as the workload.

Expected outcome. `TERM` and `INT` run cleanup; `kill -9` leaves the lock file behind.

Production considerations. This is exactly what a graceful-shutdown hook does in a container.

Common pitfalls. Trapping inside a subshell so the handler never fires.

Mini-Lab • Intermediate

50 min

Goal. Diagnose a "disk full" incident you create yourself.

Business context. Logging filled a production volume last quarter; you are writing the runbook.

Architecture. One VM (or container), a writer process, a deleted-but-open file.

Requirements. Reproduce the `df/du` discrepancy, find the holder with `lsof`, reclaim the space.

Constraints. Do not reboot — diagnose it live, as you would in production.

Hints. Write with a long-running redirect, then `rm` the file while the writer runs.

Expected outcome. A written runbook: symptom, three commands, root cause, fix.

Production considerations. Add the alert threshold you would set, and why not 100%.

Common pitfalls. Concluding "du is wrong" instead of understanding open descriptors.

 Mini-Lab • Production-style

90 min

Goal. Write `bootstrap.sh` that provisions a host for CloudBase, safely and repeatably.

Business context. New hosts are provisioned by whoever is on call; results must not vary by person.

Architecture. Fresh VM → service user → app directory → systemd unit → enabled service.

Requirements. Strict mode, an environment argument, structured log output, correct ownership and modes, unit installed and enabled.

Constraints. Must be **idempotent** – running it three times leaves the same state and exits 0 every time.

Hints. Guard each step (`id -u user &>/dev/null || useradd ...`); prefer `install -d -o -m` over `mkdir` plus `chmod`.

Expected outcome. Three consecutive runs, all exit 0, host identical each time.

Production considerations. What would you do differently for 500 hosts? (That answer is Day 8.)

Common pitfalls. Hardcoding a path or environment; forgetting `daemon-reload`; assuming a clean starting state.

9 Engineering Notes

 Operational excellence

The operational question is never "does it run?" but "what happens when it stops?" A service without a restart policy, a log destination and a known shutdown path is not deployed – it is merely started. Every layer in this book re-asks these three questions.

 Infrastructure insight

Immutability starts here. The reason teams move from "SSH in and fix it" to rebuilt images (Day 4) and declared infrastructure (Day 7) is that hand-edits on a host are invisible, unversioned and unreproducible. Every manual `vim` on a production box is future config drift.

 Cloud cost optimization

Unrotated logs are a real cloud bill. Filled volumes trigger emergency disk expansion, and expanded EBS volumes are rarely shrunk afterwards – so a forgotten `logrotate` config quietly becomes permanent monthly spend. Cost discipline starts with mundane hygiene, not with a dashboard.

 Interview insight

Expect: "What is the difference between `SIGTERM` and `SIGKILL`, and why does it matter?" The answer that lands is not the numbers. It is: `SIGTERM` is catchable so the app can drain connections and flush state; `SIGKILL` is enforced by the kernel and gives it no chance. Then connect it: that is why orchestrators send `TERM` first and why exit code 137 means the grace period was exceeded.

10 Build-Along Project – CloudBase

Across fourteen days you build **CloudBase**: the full production infrastructure and operational tooling around a small containerised API. Today you prepare the ground it runs on.

🔧 Build-Along • Day 1 — the base host

- Create the CloudBase repository and commit a `scripts/` directory (Day 2 adds conventions and protection).
- Write `bootstrap.sh`: strict mode, environment argument, structured logging.
- Provision a dedicated non-root service user and `/opt/cloudbase` with correct ownership and modes.
- Install and enable a `systemd` unit for a placeholder service, with `Restart=on-failure`.
- Verify with `systemctl status` and `journalctl -u`, and prove the restart policy by killing the process.
- Document the host's assumptions in `docs/host-baseline.md` — OS version, user, paths, ports.

Definition of done: `bootstrap.sh production` runs cleanly three times in a row on a fresh VM, leaves an enabled service running, and every step is reproducible from the repository with no manual edits.

11 Chapter Summary

Key takeaways

- ✓ Everything is a file — one kernel interface for devices, sockets, pipes and process state.
- ✓ `/proc` is synthesised on read; it is the live window into the kernel, and how `ps` works.
- ✓ Permission bits mean something different on directories: execute means *traverse*.
- ✓ `kill` sends a signal. `SIGTERM` is catchable and lets the app clean up; `SIGKILL` is not.
- ✓ `systemd` declares identity, restart policy and logging — and caches units until `daemon-reload`.
- ✓ `set -euo pipefail` converts silent corruption into a loud, early failure.

Things to remember

- Exit code `137 = 128 + 9 = SIGKILL`. You will see it constantly from Day 4 onward.
- `df` and `du` disagreeing means a deleted file is still held open.
- Idempotency is the property that makes automation safe to re-run — the seed of Terraform and Ansible.

Common pitfalls (last look)

- `chmod 777` as a reflex • `SIGKILL` as a habit • missing `daemon-reload`
- Unquoted variable expansion • treating `/tmp` as durable

 One-Page Cheat Sheet

Filesystem — one tree from `/`; `/var` fills disks; `/proc` is live kernel state.

Permissions — owner/group/other `x` `rw`; on a directory `x` = traverse; SUID runs as owner; `++` = only owner deletes.

Signals — TERM (15) catchable, graceful → KILL (9) uncatchable. Exit 137 = SIGKILL.

systemd — unit declares ExecStart, User, Restart; always `daemon-reload` after an edit.

Journald — `journalctl -u <unit> -f` to follow; `-p err` to filter by priority.

Strict mode — `set -euo pipefail` — fail fast, no unset vars, pipes propagate failure.

Descriptors — 0 stdin, 1 stdout, 2 stderr; `2>&1` points stderr at stdout's current target.

CLI quick reference

Day 1 commands worth memorising

```
# processes & signals
ps aux --sort=-%mem | head           # top memory consumers
kill -15 <pid>                       # polite stop (default)
kill -9 <pid>                        # last resort, no cleanup

# systemd
systemctl daemon-reload              # after ANY unit file edit
systemctl enable --now <unit>        # start now + on boot
journalctl -u <unit> -f --since "10 min ago"

# permissions
stat -c '%A %U:%G %n' <file>         # exact mode and ownership
install -d -o app -g app -m 750 /opt/app

# disk triage
df -h                                # kernel's view (authoritative)
du -h --max-depth=2 /var | sort -rh | head
sudo lsof +L1                        # deleted-but-open files
```

 Interview questions to master

- › What is the difference between SIGTERM and SIGKILL, and why does it matter for graceful shutdown?
- › Walk me through what `chmod 640` actually sets.
- › What is `/proc`, and why is it not a real filesystem on disk?
- › How does `systemd` decide the order in which services start?
- › What does `pipefail` change about a shell pipeline's exit code?
- › Why prefer `sudo` over logging in as root?

Further reading & official docs

- Filesystem Hierarchy Standard 3.0 — the specification behind `/etc`, `/var`, `/usr`.
- `man 7 signal` — the authoritative signal list and default dispositions.
- `man 5 systemd.service` and `man 5 systemd.unit` — every directive, with defaults.
- `man 1 bash`, the "SHELL BUILTIN COMMANDS" section on `set`.

- Google SRE Book, "Monitoring Distributed Systems" — why you alert before saturation, not at it.

Production checklist

- ☐ Services run as a dedicated non-root user, never `root`.
- ☐ Every long-running service has a unit with an explicit restart policy.
- ☐ Logs go to the journal (or a shipper), never to an unrotated flat file.
- ☐ Every unattended script starts with `set -euo pipefail` and quotes its variables.
- ☐ Provisioning is a script in version control, not steps in someone's memory.
- ☐ Disk usage alerts fire well before 100%.

Key idea

What you will learn next (Day 2). You just wrote a script that provisions a host. Tomorrow you make that script — and everything after it — *versioned, reviewable and recoverable*: Git's real object model, branching strategies that match a team's release cadence, and what to actually do when a secret is committed and pushed.

DAY 02 • FOUNDATIONS

Git & Version Control Mastery

02

The object graph underneath every command — and how not to lose work

🕒 Estimated study time: 7–8 hours

🎯 By the end of this day, you will be able to...

- Explain Git's object model — blobs, trees, commits, refs — well enough to reason about any command you have never run.
- Choose a branching strategy that matches your team's release cadence, and say why the others would hurt.
- State precisely when `rebase` is safe, when it destroys other people's work, and how `-force-with-lease` differs from `-force`.
- Make good practice the default with hooks, CODEOWNERS and branch protection — and know which of those is real enforcement.
- Execute a leaked-secret response end to end: rotate first, purge history second, prevent third.

1 The Big Picture

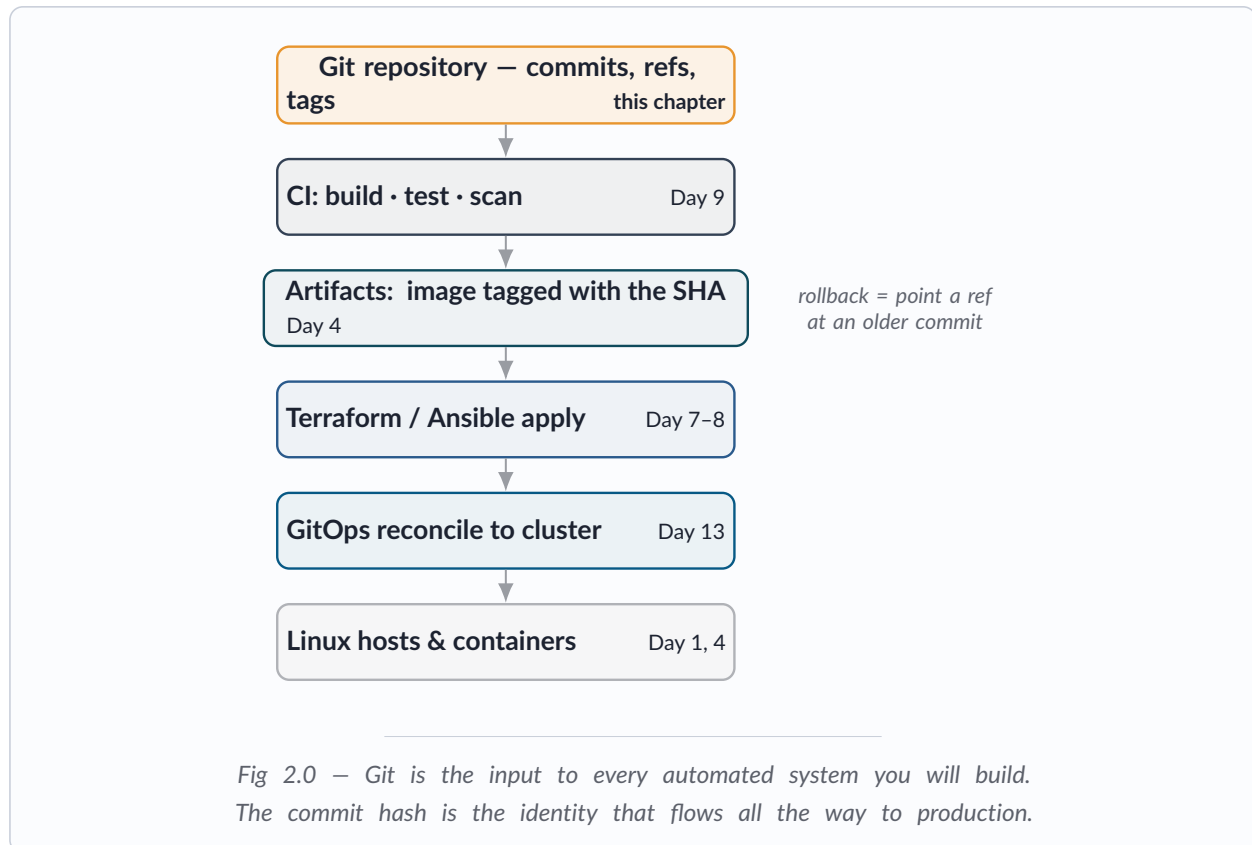
What problem this solves. On Day 1 you wrote `bootstrap.sh` and a systemd unit. Right now they exist as bytes on one host, in one person's home directory, with no answer to three questions every operations team must be able to answer instantly: *what exactly is running in production, who changed it and why, and how do we get back to the version that worked?* Version control is the mechanism that turns "the state of our infrastructure" from tribal memory into a queryable, auditable, reversible artifact.

Why DevOps engineers need it. Everything you build for the remaining twelve days is text in a repository. Terraform state comes from `.tf` files (Day 7). Ansible converges hosts from playbooks (Day 8). CI pipelines are YAML (Day 9). GitOps controllers reconcile a Kubernetes cluster against a Git commit (Day 13). In every one of those systems, **the commit hash is the deployment identity**. If you cannot reason about what a commit *is*, you cannot reason about what is deployed.

🔑 Key idea

Git is not a tool for storing files. It is a **content-addressable object database** with a small key-value store (refs) layered on top. Once you see that, every command stops being a spell to memorise and becomes an obvious manipulation of a graph: `commit` adds a node, `branch` adds a pointer, `merge` adds a node with two parents, `rebase` copies nodes and moves a pointer.

Where it fits in a modern cloud architecture. Git sits at the top of the delivery chain as the single source of truth. Every automated system downstream is, structurally, a function of a commit.



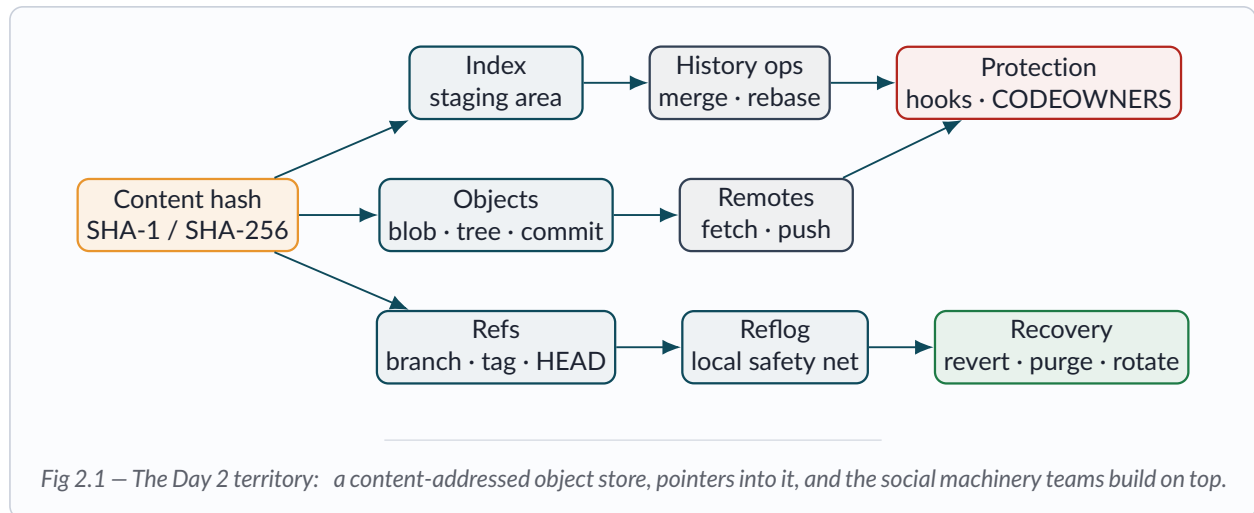
How it connects to previous chapters. Day 1 gave you a host you could rebuild by hand. Today makes that rebuild *reviewable and recoverable*: the same script, but with a history, an owner, a required review and a check that refuses to let a secret leave your laptop. From tomorrow onward, "it works on my machine" stops being a defence because the machine is described in the repository.

Where companies use it in production. Microsoft moved the Windows source tree — over 3.5 million files and more than 270 GB — into a single Git repository, which vanilla Git could not handle: a clone took over twelve hours and `git status` took roughly ten minutes. Their answer, GVFS (later generalised as Scalar and upstreamed into core Git), virtualises the working tree so files are fetched on first read; clone dropped to minutes and `status` to a few seconds. Google went the other way: unable to find any system that would scale to a billion-file repository, they built the proprietary **Piper** monorepo on Spanner across ten data centres, and practise trunk-based development with essentially no branches other than release branches. Two opposite engineering choices, one shared conclusion — the version-control model determines the shape of the whole delivery process.

Common misconceptions. That Git stores diffs (it stores whole snapshots, deduplicated by hash — diffs are computed on demand); that a branch is a copy of the code (it is a file containing one hash); that deleting a file removes it from history (the old object is still there, still reachable, still fetchable); and that a commit is "saved work" (until it is referenced by a ref *and* pushed, it is a garbage-collection candidate on one laptop).

2

Concept Map



Terminology to anchor: **content addressing** → **blob** → **tree** → **commit** → **ref** → **HEAD** → **index** → **merge base** → **reflog** → **branch protection**.

3 Guided Learning

3.1 The Object Model – Content, Addressed by Its Own Hash

What is it? Git stores four kinds of object in `.git/objects`, and every one of them is named by the cryptographic hash of its own contents:

Object	What it holds
blob	The raw bytes of one file version. No filename, no path, no history – just content.
tree	A directory listing: mode, type, hash and <i>name</i> for each entry. Trees reference blobs and other trees.
commit	One root tree hash, zero or more parent commit hashes, author, committer, timestamps, message.
tag	An annotated tag: a named, optionally signed pointer to another object.

Why does it exist? Earlier centralised systems – CVS, then Subversion – modelled history as a sequence of *changes* to files on a server. That made three operations expensive or impossible: branching copied the tree, history was only complete on the server, and verifying that what you received was what was sent required trusting the server. Linus Torvalds's design inverted it: name every piece of content by its hash, and integrity becomes structural. If a single byte of an old file changes, its blob hash changes, so every tree containing it changes, so every commit above it changes. **You cannot silently alter history in Git; you can only replace it with a visibly different history.**

How does it work internally? A commit is a small text object. Reading one is the fastest way to demolish the "commits are diffs" misconception:

```
git cat-file -p HEAD
```

```
tree 4d5e9f1a3c2b7e08f6a1d94c3b2e5f7a0c8d6b41
parent 9a3f2c81b04e7d6a5f39c2b81e0d47a6f3c95b2e
author  Amine <amine@cloudbase.dev> 1752921600 +0100
committer Amine <amine@cloudbase.dev> 1752921600 +0100

feat(bootstrap): fail fast on unset ENVIRONMENT
```

That is the entire commit. There is no diff in it. When you run `git show`, Git loads this commit's tree and its parent's tree, walks both, and computes the difference *at that moment*. Because identical content yields an identical hash, a file unchanged across a thousand commits is stored exactly once and referenced a thousand times — this is why a repository with deep history is far smaller than the naive sum of its versions.

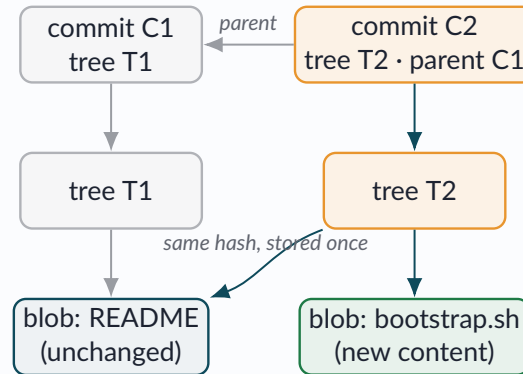


Fig 2.2 — Two commits, one changed file. The unchanged blob is shared; only the objects on the path to the change are new.

👁 Behind the scenes

`git commit` does not "save your files". It writes any new blobs, builds the tree objects that describe the staged directory structure, writes a commit object pointing at the root tree and the current `HEAD`, and finally rewrites one small file — the branch ref — to contain the new commit's hash. The last step is the only mutation; everything before it is append-only. That is why interrupted Git operations rarely corrupt anything.

When should I use it? This mental model pays off every time a command surprises you. "Where did my commit go?" becomes "which ref used to point at it?" — a question with an answer (`git reflog`), rather than a panic.

When should I *not* lean on it? Git is a terrible database for large binaries. Every version of a 200 MB video is a full blob that every clone must download forever. Use Git LFS or an artifact store; do not commit build outputs, and never commit `terraform.tfstate` (Day 7 explains what belongs in remote state instead).

⚠ Common misconception

"SHA-1 is broken, so Git is insecure." The SHAttered attack of February 2017 produced a real SHA-1 collision, and Git responded by shipping a hardened SHA-1 that detects collision attempts. The object format is nevertheless migrating: `extensions.objectFormat = sha256` exists today, and the Git project's stated plan is for SHA-256 to become the default in Git 3.0. For your purposes the practical consequence is small — but knowing *why* the transition is hard (every hash in every object of every clone would change) is a good demonstration of what content addressing costs you.

3.2 Refs, HEAD and the Index — the Three Trees

What is it? A **ref** is a file under `.git/refs` whose contents are one object hash. A branch ref for a SHA-1 repository is exactly 41 bytes: forty hexadecimal characters and a newline. `HEAD` is one indirection above that — normally the text `ref: refs/heads/main`, telling Git which branch you are "on".

Why does it exist? Because branching had to become free. In Subversion, creating a branch meant a server-side copy and switching meant a network round trip, so teams branched rarely and merged painfully. When a branch costs 41 bytes and a local write, developers branch per task — and the entire pull-request workflow the industry now runs on becomes possible.

How does it work internally? Three trees are in play at all times, and almost every confusing Git message is about a disagreement between them.

Tree	What it is	Moved by
Working directory	The actual files on disk you edit	Your editor
Index (staging)	A binary file listing paths, modes and blob hashes for the next commit	<code>git add</code> , <code>git restore -staged</code>
HEAD	The commit your branch currently points at	<code>git commit</code> , <code>git checkout</code> , <code>git reset</code>

`git status` is nothing more than two comparisons: HEAD versus index ("changes to be committed") and index versus working directory ("changes not staged"). `git reset` takes a target commit and a flag saying how far the reset propagates — `-soft` moves HEAD only, `-mixed` (the default) also rewrites the index, `-hard` additionally overwrites the working directory. That is the whole model, and it explains why only `-hard` can destroy uncommitted work.

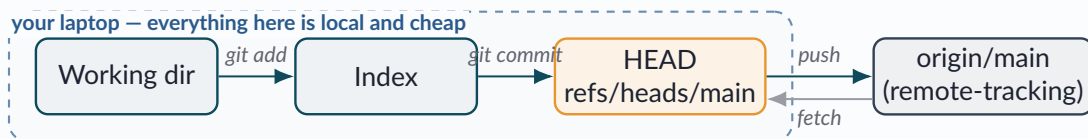


Fig 2.3 — Local repository versus remote. A push moves objects and then updates a ref; a fetch does the same in reverse, into a remote-tracking ref you do not edit.

💡 Senior engineer insight

`git pull` is `git fetch` followed by an integration step, and the second half is where teams hurt themselves. `fetch` is always safe: it only downloads objects and moves remote-tracking refs. Configure the integration deliberately — `pull.rebase=true` for a linear history, or `pull.ff=only` so that a pull which cannot fast-forward stops and makes you choose. The default silent merge commit is how repositories accumulate a hundred "Merge branch 'main' of github.com/..." commits that carry no information.

3.3 Branching Strategies — Match the Cadence, Not the Fashion

What is it? A branching strategy is a team agreement about which branches exist, how long they live, and what merging into each one means. It is a *social* protocol enforced by *technical* controls.

Why does it exist? Because integration cost grows superlinearly with divergence time. Two branches that diverged an hour ago almost never conflict; two that diverged three weeks ago conflict semantically even when Git reports no textual conflict — your colleague renamed the function you built on top of, and the merge succeeds into code that no longer compiles. Branching strategy is fundamentally a lever on how long divergence is allowed to last.

Strategy	Model	Fits a team that...
Trunk-based	Everyone commits small changes to <code>main</code> daily; unfinished work hides behind feature flags	Deploys many times a day and has fast, trustworthy CI
GitHub Flow	Short-lived branch → pull request → review → merge to <code>main</code> → deploy	Runs continuous deployment; the common default for SaaS
Git Flow	Long-lived <code>develop</code> , plus <code>release/*</code> and <code>hotfix/*</code> branches	Ships versioned artifacts on a schedule, supports old versions

Production example. Google's monorepo paper describes trunk-based development at extreme scale: tens of thousands of engineers, over a billion files, and effectively no branches except release branches — every change goes to trunk, guarded by mandatory review and a very large automated test system. That only works because the safety net is strong enough to make trunk always releasable. Adopt trunk-based development without that test coverage and you have not adopted a strategy; you have removed one.

Design trade-off

Git Flow gets mocked, but it exists for a real constraint: if you must patch version 2.3 for a customer who will not upgrade while 3.0 is in development, you need a long-lived branch, and no amount of feature-flagging removes that need. The mistake is not using Git Flow — it is using Git Flow when you deploy continuously from `main` and therefore maintain a `develop` branch that has no distinct meaning. Choose the strategy from your *release cadence*, not from a blog post.

Scalability note

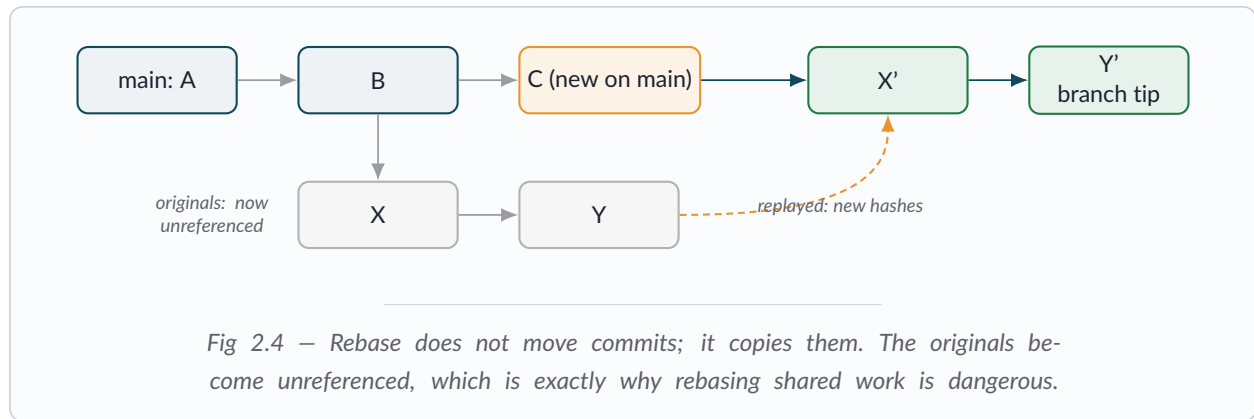
Strategy interacts with repository scale. Microsoft could not run trunk-based development on Windows until Git itself got faster — when `git status` takes ten minutes, developers batch work into long-lived branches out of self-defence. Their fix was engineering (GVFS, then Scalar: lazy file hydration, sparse checkout, commit-graph and multi-pack indexes now in core Git), not process. When a process convention keeps failing, check whether a tool is making the good path slow.

When should I not enforce a strategy hard? A two-person prototype does not need a release-branch policy; the ceremony will cost more than the conflicts it prevents. Introduce controls when the cost of a mistake exceeds the cost of the process — which for anything touching production is essentially immediately.

3.4 Merge vs Rebase — Same Destination, Different History

What is it? Both integrate one line of work into another. `git merge` finds the **merge base** (the most recent common ancestor of the two branches), performs a three-way merge, and records a *new commit with two parents*. Nothing existing is touched. `git rebase` instead takes each of your commits, computes its diff against its parent, and re-applies it onto a new base — producing *new commits with new hashes* and moving your branch pointer to the last of them. The old commits still exist, briefly, but nothing references them.

Why does it exist? Merge preserves the literal truth of what happened; rebase preserves the *readability* of what you meant. A history of forty commits including "wip", "fix typo" and "actually fix typo" is an honest record of your afternoon and a useless record for the engineer bisecting a production regression in eight months. Interactive rebase is how you turn the first into the second before anyone else sees it.



👤 Security warning • the one rule that prevents rebase disasters

Never rebase commits that have been pushed and pulled by someone else. Their clone still references the *old* commits; when you force-push the rewritten ones, their next pull produces a history with both copies, and the "fix" they reach for — merging — duplicates every commit. Rebase freely on your own unpublished branch; use `merge` or `revert` on anything shared. If you must rewrite a shared branch, that is an announced, coordinated operation, not a Tuesday afternoon.

cleaning up a feature branch before opening a PR

```
git fetch origin
git rebase -i origin/main    # squash wip commits, reword, reorder
git push --force-with-lease  # NOT --force
```

How does `-force-with-lease` work internally? A plain `-force` tells the server "set this ref to my hash, whatever it currently is". `-force-with-lease` sends an additional expectation: "set it to my hash *only if* it currently equals the value my remote-tracking ref recorded when I last fetched." The server performs a compare-and-swap and rejects the push if anyone else advanced the branch in the meantime. It is optimistic concurrency control on a ref — the same pattern as a conditional `PUT` with an ETag, or a DynamoDB conditional write.

⚠️ Common misconception

`-force-with-lease` is not magic protection. If your tooling (or an IDE, or a background `git fetch`) updates your remote-tracking ref without you looking at the new commits, the lease now matches and the force succeeds — overwriting work you never saw. The robust form pins the expected value explicitly: `git push -force-with-lease=refs/heads/main:<sha>`. Newer Git also offers `-force-if-includes`, which additionally requires that the commits you are about to overwrite are reachable from your local branch.

Production example. On GitHub, high-traffic repositories use a **merge queue** rather than letting each pull request merge directly. The queue groups pending pull requests with the latest base branch, runs the required checks against that combined candidate, and merges only if it passes. This exists to close a specific gap: two pull requests can each pass CI in isolation and still break `main` together — for example, one renames a function while the other adds a new call to it. Testing the branch, rather than the merge result, is a false sense of safety at high merge rates.

3.5 Making the Right Thing the Easy Thing — Hooks, CODEOWNERS, Protection

What is it? Three layers of control with very different strength. **Hooks** are executable scripts in `.git/hooks` that Git runs at defined points — `pre-commit` before a commit is created, `commit-msg` to

validate the message, `pre-push` before objects are sent. **CODEOWNERS** is a file mapping path patterns to teams, so the right reviewers are requested automatically. **Branch protection** is a server-side rule: this branch cannot be pushed to directly, requires N approving reviews, requires named status checks to pass, and cannot be force-pushed or deleted.

Why does it exist? Because process documents do not execute. "Everyone should run the linter" is a wish; a `pre-commit` hook that runs it is a mechanism. But note the asymmetry that matters most:

Control	Where it runs	Bypassable?
<code>pre-commit</code> hook	Your laptop, from a directory not under version control	Yes — <code>-no-verify</code>
CODEOWNERS	The forge, when a PR is opened	Advisory unless paired with protection
Required review	The forge, server-side	Only by an admin, and it is logged
Required status check	The forge, server-side	Only by an admin, and it is logged

💡 Senior engineer insight

Local hooks are a *fast feedback* mechanism, not a security control. They live in `.git/hooks`, which is not cloned, so a new teammate has none until something installs them; and any developer can skip them with `-no-verify`. Treat them as the ten-second version of a check whose authoritative copy runs in CI (Day 9). The correct architecture is: same check, two places — locally for speed, server-side for truth.

.github/hooks/pre-commit — block obvious secrets before they leave the laptop

```
#!/usr/bin/env bash
set -euo pipefail

# Scan only what is STAGED, not the working tree.
staged=$(git diff --cached --name-only --diff-filter=ACM)
[ -z "$staged" ] && exit 0

patterns='AKIA[0-9A-Z]{16}|BEGIN (RSA|EC|OPENSSH) PRIVATE KEY|xox[abprs]-'

if git diff --cached -U0 -- $staged | grep -nEq "$patterns"; then
  echo "BLOCKED: a credential-shaped string is staged." >&2
  echo "Remove it, use an env var or a secrets manager, then re-commit." >&2
  exit 1
fi
```

making the hook survive a fresh clone

```
# .git/hooks is not versioned, so version the hooks elsewhere
# and point Git at that directory (Git 2.9+):
git config core.hooksPath .githubhooks
chmod +x .githubhooks/*
# Put this line in bootstrap/setup so nobody has to remember it.
```

How does branch protection work internally? The forge intercepts the ref update. A push to a protected branch is a request to move `refs/heads/main`; the server evaluates its rules against that

update and rejects it with a message rather than writing the ref. Required status checks work the same way at merge time — the merge button is disabled until a check run with the configured name reports success for the head commit. This is why the check must be named exactly and must actually run on pull requests: a required check that never reports simply blocks the branch forever, which is the single most common configuration mistake.

Automation tip

Commit message conventions become useful the moment a machine reads them. Conventional Commits (`feat:`, `fix:`, `chore:`, `feat!:` for a breaking change) lets tooling derive the next semantic version and generate a changelog from history alone. Enforce it in `commit-msg` locally and as a required check on the pull-request title. The real payoff arrives on Day 9, when the pipeline tags an image from the commit rather than from a human typing a version number.

CODEOWNERS — last matching pattern wins

```
# Default owner for everything not matched below
*                               @cloudbase/platform

# Infrastructure needs a platform reviewer, always
/terraform/                    @cloudbase/platform @cloudbase/security
/.github/workflows/            @cloudbase/platform

# Anything security-relevant pulls in the security team
/scripts/bootstrap.sh          @cloudbase/platform @cloudbase/security
/SECURITY.md                   @cloudbase/security
```

Operational excellence

CODEOWNERS only creates real accountability when the protected branch also requires review *from code owners*. Without that box ticked, it merely suggests reviewers, and a pull request touching `/terraform/` can be approved by anyone. Audit this: pick a protected repository and check whether the owner requirement is actually enforced, or only implied.

3.6 When a Secret Is Committed — Rotate, Purge, Prevent (in That Order)

What is it? A credential reaches a commit — an AWS access key, a private key, a database URL with a password. This will happen to you or your team. What separates a senior response from a junior one is the ordering.

Why does the ordering matter? Because the moment a secret is pushed to a shared remote you must assume it is compromised. Public repositories are scraped by automated harvesters within seconds of a push. Purging history is *cleanup*; it does not un-disclose anything. If you spend forty minutes rewriting history before revoking the key, you have spent forty minutes leaving a live credential in the hands of whoever already copied it.

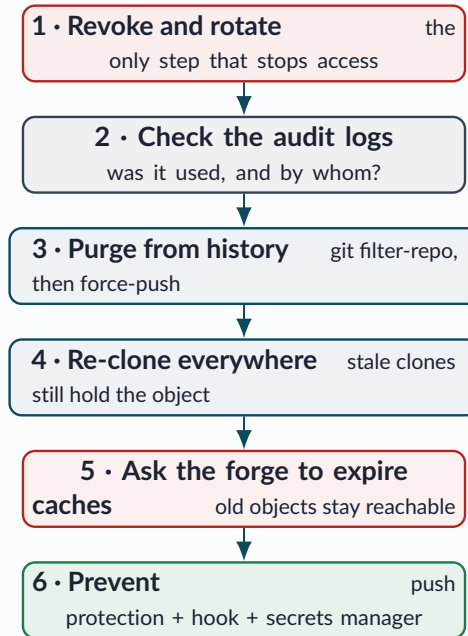


Fig 2.5 — Incident order for a leaked credential. Step 1 is the one that stops the bleeding; the rest is hygiene.

How does the purge work internally? You cannot delete a commit from the middle of history, because every later commit's hash is derived from it. What history rewriting actually does is *replay* the whole history, applying a filter to each commit, producing an entirely new chain of objects with new hashes. The tool the Git project recommends today is `git filter-repo`; `git filter-branch` is documented as slow and error-prone and should not be used. The BFG Repo-Cleaner is a faster alternative for the narrow case of deleting files or replacing strings.

purging a file from all history with `git filter-repo`

```
# Always work on a fresh mirror clone, never your working repo
git clone --mirror git@github.com:cloudbase/platform.git
cd platform.git

# Remove one path from every commit in every branch and tag
git filter-repo --invert-paths --path scripts/.env.production

# Rewritten history is a different history: this must be forced
git push --force --all
git push --force --tags
```

👤 Security warning • what the purge does not accomplish

The old objects remain reachable for a while even after a successful force-push: the forge keeps them behind pull-request refs and internal caches, teammates' clones still contain them, and any fork made before the rewrite retains the full original history — forks are outside your control entirely. This is precisely why revocation comes first. Ask your provider to expire cached views, and treat the secret as public regardless of how clean the history looks afterwards.

 Production tip

Server-side push protection is the control that would have prevented the incident. GitHub's secret scanning push protection inspects incoming pushes for known credential formats — provider partners register their token patterns — and blocks the push, telling the developer why. It can be bypassed with a stated reason, and each bypass raises an alert and is logged. That design is worth studying: an unbyypassable control gets disabled by a team under deadline pressure; a bypassable-but-loud control survives contact with reality and still produces the audit trail.

 Reliability principle

Prevention beats detection beats response, but you should still rehearse the response. Run the whole drill once on a throwaway repository — commit a fake key, push it, revoke it, purge it, re-clone — so that the first time you do it is not at 2am with an active credential in a public repository. A runbook nobody has executed is a hypothesis.

 Best practices — what experienced engineers do

- Write commit messages that explain *why*; the diff already says what.
- Keep feature branches short-lived — days, not weeks. Divergence is the cost driver.
- Use `-force-with-lease`, never bare `-force`, and never on a shared branch.
- Protect `main`: no direct pushes, required review, required status checks, no force-push.
- Run the same secret scan locally (fast) and in CI (authoritative).
- Revoke and rotate a leaked credential *before* touching history.

 Common mistakes beginners make

- Believing a later commit that deletes a file removes it from history.
- Rebasing a branch other people have already pulled, then force-pushing over their work.
- `git reset -hard` on a branch with uncommitted or unpushed work.
- Treating a local hook as a security control instead of a fast first check.
- Committing generated artifacts, large binaries or `terraform.tfstate`.
- Configuring a required status check whose name never reports, blocking every merge.

4 Visualizations to Internalize

Redraw these from memory. Reproducing a diagram is what moves it from "I recognise that" to "I can explain that under pressure."

 Redraw: `commit` → `tree` → `blob` for two commits where one file changed. Mark which objects are new and which are shared.

 Redraw: a rebase. Show the original commits, the copies, the moved branch pointer — and mark the moment where a teammate's clone diverges.

5

Guided Hands-On Lab — Repository Conventions for CloudBase

Goal. Take the loose `bootstrap.sh` from Day 1 and put it behind a real engineering process: conventions, ownership, automated checks and a protected branch. You are designing the guardrails, not typing commands — decide *what* each control must guarantee before you configure it.

1 Inspect the object graph you already have

Use `git cat-file -p HEAD` to print your latest commit, then follow the tree hash, then a blob hash. Do it by hand once.

 **Why:** Reading the raw objects converts the model from something you read about into something you have seen. Every later command becomes predictable.

2 Choose the branching strategy and write down why

Decide between trunk-based and GitHub Flow for CloudBase and record the decision in `docs/branching.md`, naming the release cadence that justifies it.

 **Why:** An undocumented convention is not a convention. Writing the justification forces you to notice if you are copying a strategy rather than choosing one.

3 Establish a commit-message convention

Adopt Conventional Commits and add a `commit-msg` hook that rejects a message not matching the

pattern.

 *Why: You are creating machine-readable history now so that Day 9's pipeline can derive versions and changelogs without a human deciding them.*

4 Write the pre-commit secret-blocking hook

Scan the *staged* diff for AWS access-key IDs, private-key headers and provider token prefixes. Exit non-zero with an actionable message.

 *Why: Scanning the staged diff (not the working tree) is the correct scope: it is exactly the content about to become an immutable object.*

5 Make the hooks survive a fresh clone

Move the hooks to a versioned `.githubhooks/` directory and set `core.hooksPath`. Add that line to the repository setup script.

 *Why: `.git/hooks` is never cloned. A control that only exists on the machine of whoever wrote it protects nobody.*

6 Add CODEOWNERS with meaningful boundaries

Map `/terraform/`, `/.github/workflows/` and any bootstrap or security path to the teams that must see changes there.

 *Why: Reviewer selection is a risk control. The people who understand the blast radius of a change should be the ones asked to approve it.*

7 Protect main

Require a pull request, at least one approving review, review from code owners, a named passing status check, and disallow force-push and deletion.

 *Why: This is the first control in the chapter that a determined developer cannot bypass silently. Everything else is speed; this is truth.*

8 Try to break each control, deliberately

Push directly to `main`. Commit with `-no-verify` and a fake AWS key. Open a pull request touching `/terraform/` without a code owner.

 *Why: You must know which of your controls are advisory and which are enforced. Discovering that during an incident is expensive; discovering it now is free.*

Production tip

Make the required status check a real job that runs on every pull request — even if today it only runs the secret scan and `shellcheck`. Protection rules that reference a check name which never reports leave the branch permanently unmergeable, and the usual "fix" is an admin turning protection off. Configure the check first, make it green, then require it.

6 Thinking Exercises

Think it through

Git recomputes diffs on demand from full snapshots rather than storing diffs. Storing diffs would obviously use less space for text. What does Git buy with that trade, and which operations

would become expensive in the diff-based design?

Think it through

A local `pre-commit` hook can be skipped with `-no-verify`, and push protection can be bypassed with a stated reason. Both are deliberately defeatable. Argue the case for and against making them impossible to bypass, and say what you would do for a repository holding production infrastructure.

Think it through

Google runs trunk-based development across a billion files with almost no branches; Microsoft had to rewrite parts of Git before Windows could work that way at all. What does that suggest about the relationship between tooling performance and team process — and where in your current workflow is a slow tool quietly shaping behaviour?

7

Debugging Practice

Debugging Lab • "My commits vanished after a rebase"

Symptom. An engineer ran `git rebase -i` on a feature branch, hit a conflict, panicked, ran `git rebase -abort` and then `git reset --hard origin/feature`. Three commits of work are gone from `git log`.

git reflog

```
a91c4de HEAD@{0}: reset: moving to origin/feature
7f2b8c1 HEAD@{1}: rebase (abort): returning to refs/heads/feature
3e5d902 HEAD@{2}: rebase (start): checkout origin/main
3e5d902 HEAD@{3}: commit: feat(hooks): block AWS key patterns
b04a7f6 HEAD@{4}: commit: feat(hooks): add commit-msg validation
c72e15a HEAD@{5}: commit: chore: add .github directory
```

- ✓ **Observe.** `git log` shows nothing, but the reflog still lists the three commits at `HEAD@{3}` and below.
- ✓ **Hypothesise.** The commit objects were never deleted — the branch ref was moved, so nothing points at them any more.
- ✓ **Investigate.** `git show 3e5d902` still prints the commit. The objects are intact; only reachability was lost. `git fsck --lost-found` would find them too.
- ✓ **Root cause.** `reset --hard` moved the ref; unreferenced commits become invisible to `log` but survive until garbage collection (default expiry is 30 days for unreachable objects, 90 for reflog entries).
- ✓ **Fix.** `git branch rescue 3e5d902`, verify the content, then reset the feature branch to it or cherry-pick what is needed.
- ✓ **Prevent.** Before any history-rewriting operation, tag the current tip: `git tag backup/pre-rebase`. It costs nothing and turns recovery from archaeology into a one-liner.

🔧 Debugging Lab • The secret was removed from the repo, but the key is still being used

Symptom. An AWS access key was committed. The developer deleted the file, committed the deletion, and reported it fixed. Two days later CloudTrail shows API calls from an unfamiliar IP using that key.

git log -all --oneline -- scripts/.env.production

```
d19f4a3 chore: remove accidentally committed env file
7c02b5e feat(bootstrap): add production environment file

# the blob is still fully reachable from the earlier commit:
$ git show 7c02b5e:scripts/.env.production | head -1
AWS_ACCESS_KEY_ID=AKIA.....
```

- ✓ **Observe.** Deleting a file adds a commit whose tree omits it; every earlier commit still references the blob, and `git show` retrieves it in one command.
- ✓ **Hypothesise.** The credential was never revoked, so removal from `HEAD` changed nothing about its validity — and it was public for two days regardless.
- ✓ **Investigate.** Deactivate the key in IAM, then read CloudTrail for every call made with that access-key ID: which principal, which region, which resources.
- ✓ **Root cause.** A secret in source control plus a response that treated a disclosure incident as a file-management problem.
- ✓ **Fix.** Revoke and rotate first. Then purge with `git filter-repo` on a mirror clone, force-push, have everyone re-clone, and ask the forge to expire cached views of the old objects.

- ✓ **Prevent.** Enable server-side push protection, ship the `pre-commit` scan via `core.hooksPath`, move real values into a secrets manager, and commit only a `.env.example` with placeholder values.

8 Mini-Labs

🔧 Mini-Lab • Beginner

35 min

Goal. Read Git's object database by hand and prove that commits are snapshots.

Business context. You are about to be responsible for a repository whose history is an audit record.

Architecture. One local repository, four commits, one file edited twice.

Requirements. Use `cat-file -p` to walk commit → tree → blob for two commits; record which blob hashes are identical across them.

Constraints. No GUI, no `git log -p` — follow the hashes manually.

Hints. `git rev-parse HEAD` for the hash; `git cat-file -t <sha>` tells you the object type.

Expected outcome. A written explanation of exactly which objects a commit creates and which it reuses.

Production considerations. Explain from this why committing a 200 MB binary is permanent.

Common pitfalls. Assuming the tree hash changes when only the message changes (it does not — but the commit hash does).

🔧 Mini-Lab • Intermediate

55 min

Goal. Cause a rebase disaster in a controlled environment, then recover from it.

Business context. You want to be the person who fixes this calmly, not the person who caused it.

Architecture. One bare repo as "origin", two clones acting as two developers.

Requirements. Both clones push to a shared branch; clone A rebases and force-pushes; observe what clone B sees on the next pull, then recover B's lost commit via the reflog.

Constraints. Recover without asking A for anything — work only from B's local objects.

Hints. Compare a plain `-force` with `-force-with-lease` after B has pushed; note which is rejected.

Expected outcome. A short runbook: symptom, three commands, recovery, and the rule that prevents it.

Production considerations. Which branch protection setting would have made this impossible?

Common pitfalls. Fixing the divergence with a merge, which duplicates every rewritten commit.

🔧 Mini-Lab • Production-style

95 min

Goal. Build the full guardrail set for CloudBase and prove each control by attacking it.

Business context. The repository will soon hold Terraform that can create and destroy real cloud resources.

Architecture. Repository → versioned `.githubhooks/` → CODEOWNERS → protected `main` → required check.

Requirements. Conventional-commit validation, staged-diff secret scan, CODEOWNERS with at least three meaningful path rules, protection requiring one review plus code-owner review

plus one named check.

Constraints. A fresh clone must acquire every local control by running one setup command — no manual instructions.

Hints. `core.hooksPath` for distribution; make the required check green *before* you require it.

Expected outcome. A table of every control, where it runs, and the exact command that defeats it (or "cannot be defeated silently").

Production considerations. Which of these should be organisation-wide policy rather than per-repository settings?

Common pitfalls. Requiring a status check that never reports; assuming CODEOWNERS is enforced without the matching protection option.

9

Engineering Notes

 Infrastructure insight

Git's design choice — distributed, content-addressed, append-only — is why GitOps works at all. A Kubernetes controller can reconcile a cluster against a commit hash precisely because that hash is a cryptographic statement about exact content, verifiable without trusting the transport. Day 13's GitOps model is this chapter's object model applied to cluster state.

 Disaster recovery tip

Every developer's clone is a full backup of the history — but not of the metadata your team actually depends on: pull requests, reviews, issues, CI configuration, protection rules and secrets all live in the forge, not in the repository. Ask what your recovery plan is if the forge account is lost. Mirroring the repository is easy; reconstructing four years of review context is not.

 Cloud cost optimization

Repository hygiene is a real cost line. Committed build artifacts and large binaries are downloaded by every clone, every CI job and every container build, forever — history cannot be slimmed without a coordinated rewrite. On a pipeline running hundreds of jobs a day, a 500 MB history is measurable egress and measurable runner minutes. Enforce a size limit in CI before the repository grows, not after.

 Interview insight

Expect: "A *secret* was committed three commits ago and already pushed — what is your process?" The answer that fails starts with `filter-repo`. The answer that lands starts with: assume it is compromised, revoke and rotate immediately, then check the audit logs for use, then purge history, then re-clone everywhere, then close the gap with push protection and a secrets manager. Ordering demonstrates that you understand it is a disclosure incident, not a Git problem.

 Behind the scenes

`git bisect` is the object model paying dividends. Because commits form a directed acyclic graph with full snapshots, Git can binary-search the history for the commit that introduced a regression, checking out arbitrary points in constant time. With a scripted test, `git bisect run ./test.sh` finds a bad commit among a thousand in about ten steps — which is why atomic, self-contained commits are an operational asset, not an aesthetic preference.

10 Build-Along Project — CloudBase

Day 1 left you with a base Linux host: a strict-mode `bootstrap.sh`, a dedicated service user, and a `systemd` unit that restarts on failure. All of it exists on one machine and in one person's head. Today you make everything that follows *versioned, reviewable and recoverable* — so that from Day 3 onward, every artifact you add inherits these guarantees for free.

↗ Build-Along • Day 02 — repository conventions and guardrails

- Move Day 1's `scripts/bootstrap.sh`, the `systemd` unit and `docs/host-baseline.md` into a clean CloudBase repository with a `README` stating what the project is and how to set it up.
- Record the branching decision in `docs/branching.md`: strategy chosen, release cadence it serves, maximum branch lifetime the team accepts.
- Adopt Conventional Commits and enforce it with a versioned `commit-msg` hook.
- Write `.githubhooks/pre-commit` that scans the *staged* diff for AWS access-key IDs, private-key headers and provider token prefixes, and blocks the commit with an actionable message.
- Distribute the hooks with `core.hooksPath` wired into a `make setup` (or `scripts/setup.sh`) so a fresh clone is protected in one command.
- Add `CODEOWNERS` routing infrastructure, workflow and security-relevant paths to the teams that must review them.
- Protect `main`: no direct pushes, one approving review, code-owner review required, one named required status check, force-push and deletion disabled.
- Add `.gitignore` and a `.env.example` with placeholders only; enable server-side push protection if your forge offers it.
- Rehearse the incident: commit a fake key to a throwaway branch, then run the full drill — rotate (simulated), purge with `git filter-repo` on a mirror clone, re-clone — and write it up as `docs/runbooks/leaked-secret.md`.

Definition of done: a fresh clone of CloudBase becomes fully protected by running one setup command; a direct push to `main` is rejected by the server; a commit containing `AKIA`-shaped text is blocked locally; a pull request touching `/terraform/` automatically requests the platform and security owners; and `docs/runbooks/leaked-secret.md` describes a drill you have actually performed, rotation first.

11 Chapter Summary

Key takeaways

- ✓ Git is a content-addressed object store: blobs, trees, commits and tags, each named by the hash of its own content.
- ✓ A commit is a full snapshot plus parent pointers — diffs are computed on demand, never stored.
- ✓ A branch is a 41-byte file holding one hash. That is why branching is instant and why rebasing rewrites rather than moves.

- ✓ `merge` adds a commit and touches nothing; `rebase` copies commits with new hashes and abandons the originals.
- ✓ `-force-with-lease` is compare-and-swap on a ref; bare `-force` is an unconditional overwrite.
- ✓ Local hooks are speed; server-side branch protection is enforcement. Ship both.
- ✓ A leaked secret is a disclosure incident: rotate first, purge second, prevent third.

Things to remember

- `git reflog` is the local safety net — almost nothing is truly lost for weeks.
- Deleting a file in a new commit leaves the blob fully reachable from every earlier commit.
- Forks made before a history rewrite keep the original objects, permanently, outside your control.
- A required status check whose name never reports blocks every merge forever.

Common pitfalls (last look)

- Bare `-force` on a shared branch • rebasing published commits • purging before rotating
- Hooks that a fresh clone never gets • committing state files and build artifacts

📋 One-Page Cheat Sheet

Objects — blob = file bytes; tree = directory listing; commit = tree + parents + metadata; all named by content hash.

Refs — a branch is a file containing one hash (41 bytes for SHA-1); `HEAD` points at the current branch.

Three trees — working directory → index → `HEAD`; `reset` propagates as far as `-soft`, `-mixed` or `-hard` says.

Merge vs rebase — merge = new commit, two parents, safe on shared branches; rebase = new hashes, local-only work.

Force safety — `-force-with-lease` is a conditional update; add `-force-if-includes` for stronger guarantees.

Undo — `revert` adds a commit and is always safe when shared; `reset -hard` rewrites and is local-only.

Secret leak — rotate → audit logs → `git filter-repo` on a mirror → re-clone → push protection.

CLI quick reference

Day 2 commands worth memorising

```
# read the object graph directly
git rev-parse HEAD           # hash of the current commit
git cat-file -p <sha>        # print any object: commit, tree, blob
git cat-file -t <sha>        # what type is this object?

# history that is actually readable
git log --oneline --graph --decorate --all
git log -S 'AKIA' --all      # which commit introduced this string?

# integrate work
git fetch origin             # always safe: no working tree changes
git rebase -i origin/main    # clean up YOUR unpublished commits
```

```

git push --force-with-lease           # conditional overwrite, never bare --force

# undo, safely
git revert <sha>                     # new commit undoing it: safe when shared
git reset --soft HEAD~1              # uncommit, keep index and files
git reflog                           # every position HEAD has held; your net

# guardrails
git config core.hooksPath .git/hooks # versioned, cloneable hooks
git commit --no-verify               # proves local hooks are NOT enforcement

```

Interview questions to master

- › What actually happens under the hood when you create a branch?
- › Merge vs rebase — when is each the right call, and what exactly does rebase rewrite?
- › Why is `--force-with-lease` safer than `--force`, and when does it still fail to protect you?
- › A secret was committed three commits ago and already pushed — walk me through your process.
- › What is the difference between `git revert` and `git reset`?
- › How would you enforce that nobody merges to `main` without a passing test suite?
- › Why does Git store snapshots rather than diffs, and what does that buy you?

Further reading & official docs

- *Pro Git* (Chacon & Straub), chapter 10 "Git Internals" — the authoritative walkthrough of objects, refs and packfiles. Free at git-scm.com/book.
- Git documentation: [hash-function-transition](#) — the official SHA-1 to SHA-256 migration design, and why it is hard.
- `man git-filter-repo` and the GitHub Docs page "Removing sensitive data from a repository" — the recommended purge procedure, which begins with revoking the credential.
- GitHub Docs: "About protected branches", "Available rules for rulesets" and "Managing a merge queue" — what each server-side control actually enforces.
- GitHub Docs: "About push protection" — how server-side secret scanning blocks a push, and how bypasses are logged.
- Potvin & Levenberg, "Why Google Stores Billions of Lines of Code in a Single Repository", *Communications of the ACM*, 2016 — trunk-based development at extreme scale.
- The GitHub Blog, "The Story of Scalar", and the Azure DevOps blog posts on GVFS — how Microsoft made Git viable for the Windows repository.

Production checklist

- ☐ `main` rejects direct pushes and force-pushes, and cannot be deleted.
- ☐ Every merge to `main` requires an approving review and a named passing status check.
- ☐ `CODEOWNERS` exists *and* code-owner review is a required protection option.
- ☐ Secret scanning runs locally (fast) and server-side (authoritative); pushes with credentials are blocked.

- ☐ Hooks are versioned and installed by the setup script — not left in `.git/hooks`.
- ☐ A leaked-secret runbook exists, starts with rotation, and has been rehearsed at least once.
- ☐ No state files, build artifacts or large binaries are tracked; a size check runs in CI.
- ☐ Commit messages follow a machine-readable convention so tooling can derive versions.

Key idea

What you will learn next (Day 3). Your code and your host baseline are now versioned, reviewed and recoverable. Next you take that repository off your laptop: cloud fundamentals — regions and availability zones, the shared responsibility model, VPCs and subnets, and the identity model that decides which of your automated systems is allowed to touch which resource. The guardrails you built today are the pattern; from tomorrow the enforcement point moves into the cloud provider.

DAY 03 • NETWORKING

Networking & Linux Systems for DevOps 03

TCP/IP, DNS, CIDR, firewalls and TLS — the layer every outage eventually reaches

 Estimated study time: 8 hours By the end of this day, you will be able to...

- Reason about TCP/IP, ports and the three-way handshake well enough to debug a connectivity incident from first principles.
- Read and *design* CIDR blocks fluently, and size a real VPC that will not need re-architecting in a year.
- Trace the DNS resolution chain end to end, and treat TTL as a decision you make days before a migration.
- State the practical difference between stateful security groups and stateless network ACLs — and write correct rules for both.
- Configure SSH key-based authentication with a bastion and [ProxyJump](#), and say precisely what TLS does and does not guarantee.

1 The Big Picture

What problem this solves. Two computers that have never met must exchange bytes reliably, across equipment owned by strangers, without a central coordinator, and must survive links going down mid-conversation. Networking is the stack of agreements that makes this work. For you it solves something narrower and more urgent: when a deploy succeeds but the service is unreachable, the network is where the answer is — and "the network" is not a black box. It is four or five mechanisms you can enumerate, each of which fails in a recognisable way.

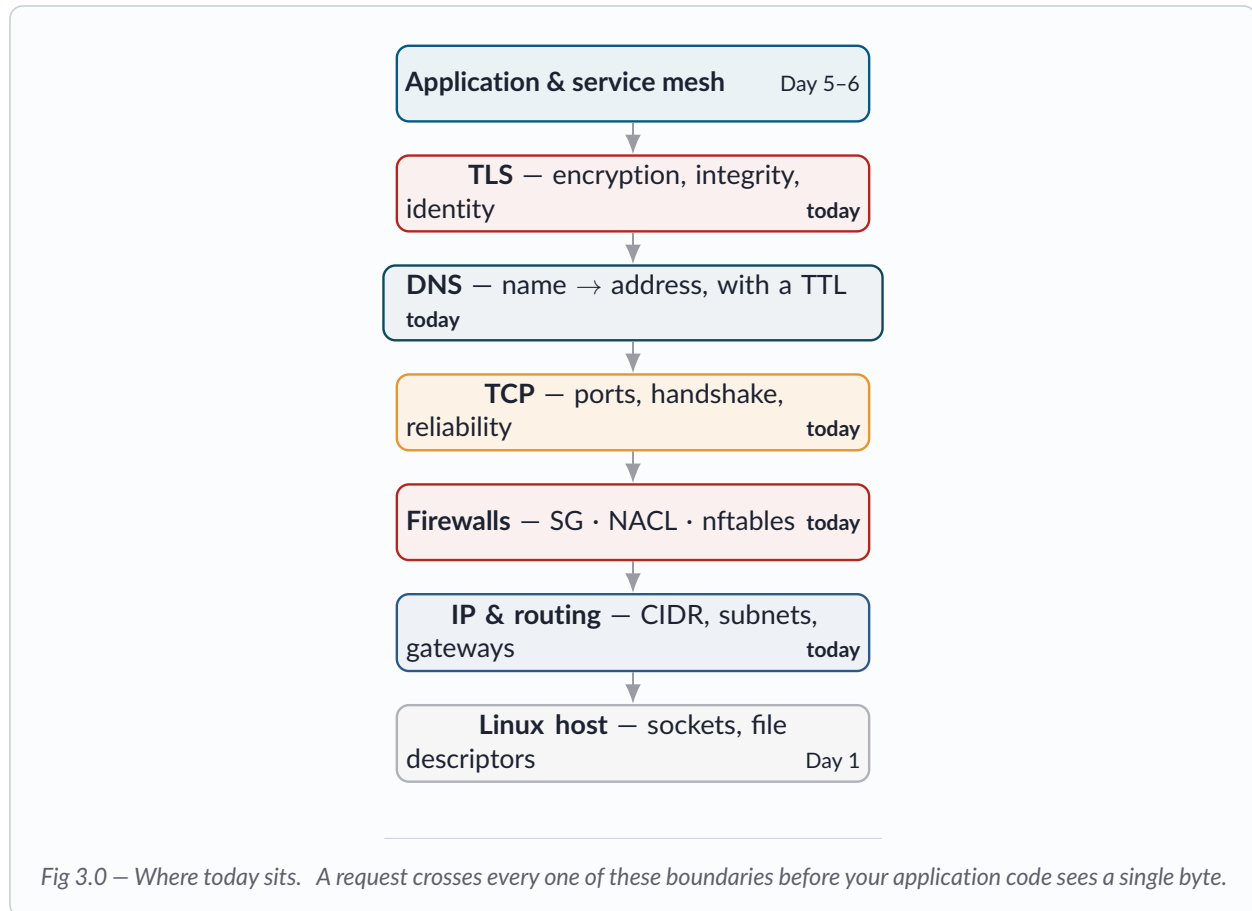
Why DevOps engineers need it. Almost every production incident you will ever debug is, at some layer, a networking incident. A pod cannot reach a database (security group, or CoreDNS). A certificate expired (TLS and automation). A cutover took six hours instead of six minutes (TTL). A "random" 5xx spike traces to one availability zone (routing, or a NAT gateway). Tools change; the packet does not. Day 1 gave you the host. Today gives you what happens the moment a process on that host opens a socket.

 Key idea

Networking is the only Day in this book where you design something before you can run it. Every other topic you can try, break and retry. A VPC CIDR block is chosen once and lives for years — AWS will not let you shrink it, and overlapping ranges block the VPC peering and Transit Gateway connections you will want later. That is why today's build-along is done on paper, and why Day 7 (Terraform) and Day 10 (AWS) only *transcribe* what you decide now.

Where it fits in a modern cloud architecture. Read the diagram from the bottom. Every layer above

depends on the address and the route below it resolving correctly.



How it connects to previous chapters. Day 1 taught you that a process holds file descriptors; a socket is just another one, and `ss -tlnp` is the network's `ps aux`. Day 2 gave you a repository and review discipline — today's network design is a document that belongs *in* that repository, reviewed like code, because it is about to become Terraform on Day 7.

Where companies use it in production. Slack rebuilt its AWS network in a project called Whitecastle: two `/16` CIDR ranges per region (over 130,000 addresses), all inside the `10.0.0.0/8` summary route so that nothing overlaps, with a Transit Gateway in each region and inter-region peering between them; every VPC route table points `10.0.0.0/8` at its local Transit Gateway. Their subnets come in two flavours — public ones carrying a `0.0.0.0/0` route to an internet gateway, and private ones with no such route at all; private workloads that need the internet egress through a Squid proxy stack in the public subnets, which Slack built instead of NAT instances specifically to satisfy internal logging and compliance requirements. That is the same public/private split you are about to design, at a scale where getting the CIDR plan wrong costs years.

Common misconceptions. That a successful `telnet host 443` means the service works (it means a socket accepted a connection). That security groups behave like `iptables` (they do not — they are stateful and have no deny rules). That lowering a DNS TTL takes effect immediately (it takes effect one old-TTL later). That TLS proves the server is trustworthy (it proves you reached the name on the certificate — nothing about what that server does with your data).

2

Concept Map

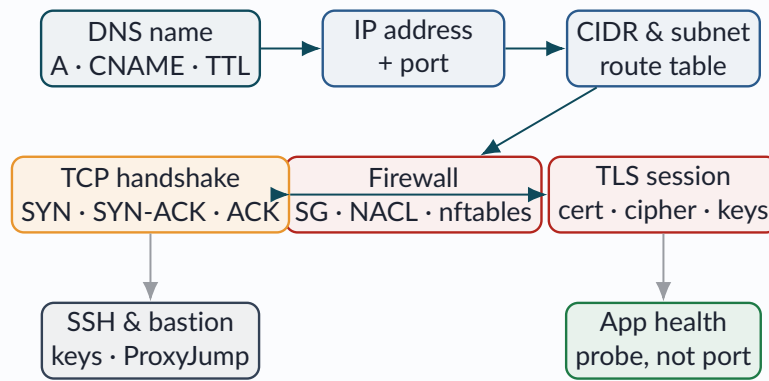


Fig 3.1 — The Day 3 territory. A name becomes an address, an address becomes a route, a route crosses filters, and a socket becomes an encrypted session.

Terminology to anchor: **packet** → **IP address** → **port** → **socket** → **three-way handshake** → **CIDR block** → **route table** → **security group** → **resolver** → **TTL** → **certificate chain**.

3

Guided Learning

3.1 TCP/IP in the Amount of Depth You Will Actually Use

What is it? IP gets a packet to the right *host*. TCP gets a byte stream to the right *process* on that host, in order, without gaps, retransmitting what the network dropped. The port number is the process address: `10.20.34.7:8080` identifies not a machine but a listening socket on a machine.

Why does it exist? Before TCP, an application that wanted reliable delivery had to implement sequencing, acknowledgement, retransmission and flow control itself — badly, and differently every time. TCP factors that work out of every application and into the kernel, once. UDP still exists precisely because that reliability costs latency, and some workloads (DNS queries, video frames, QUIC's own reliability layer) would rather handle loss themselves.

How does it work internally? A connection is established with a three-way exchange. The client sends `SYN` carrying an initial sequence number; the server replies `SYN-ACK` with its own sequence number and an acknowledgement of the client's; the client sends `ACK`. Only now does the kernel move the connection from the `SYN queue` to the `accept queue`, and only when the application calls `accept()` does it leave that queue.

That last sentence is the whole lesson of today. **The handshake is completed by the kernel, not by your application.** A process that is deadlocked, out of database connections, or stuck in a garbage-collection pause still has a kernel that answers `SYN` with `SYN-ACK`. Your connection succeeds. Your request then hangs.

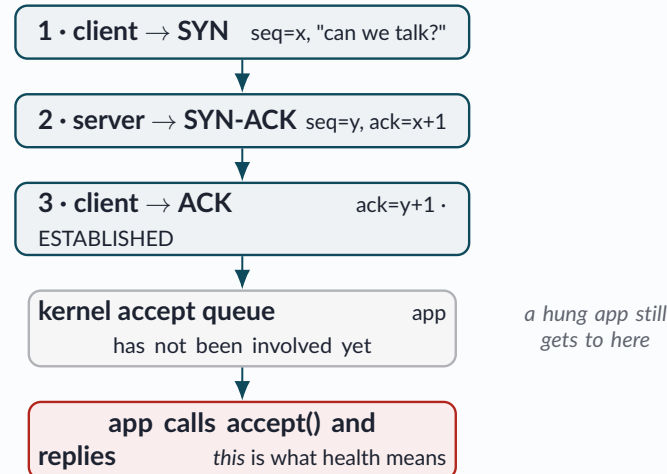


Fig 3.2 – The three-way handshake, and the exact point where "connected" stops meaning "working".

⚠ Common misconception

`nc -zv db.internal 5432` returning "succeeded" proves three things: the route exists, the firewall allows it, and something is bound to that port. It proves nothing about whether the database can answer a query. This is exactly why Kubernetes distinguishes a TCP socket probe from an HTTP probe on Day 6 – and why a `/health` endpoint that returns `200` without touching its dependencies is only marginally better than the port check.

When should I use it? TCP for anything where losing a byte is unacceptable and a few milliseconds of setup are not: HTTP, SSH, database protocols, replication.

When should I not? When head-of-line blocking hurts more than loss does. DNS uses UDP for a single-packet query because retrying is cheaper than a handshake. HTTP/3 runs over QUIC on UDP for the same reason: one lost packet in TCP stalls every stream sharing the connection.

👁 Behind the scenes

`netstat` and `ss` do not query the network. They read `/proc/net/tcp` – the kernel's socket table, rendered as text on demand, exactly like the `/proc` you met on Day 1. When you see a pile of sockets in `TIME_WAIT`, that is the kernel deliberately holding the four-tuple for twice the maximum segment lifetime so a late-arriving packet from the closed connection cannot be mistaken for data on a new one. It is correctness, not a leak.

socket-level triage on a Linux host

```

ss -tlnp                                # what is listening, and which PID owns it
ss -tn state established '( dport = :5432 )' # live conns to Postgres
nc -zv db.internal 5432                  # can I complete a handshake? (NOT health)
curl -sS -o /dev/null -w '%{http_code} %{time_total}\n' https://api.internal/health
ip route get 10.20.34.7                  # which interface and gateway would be used
traceroute -T -p 443 api.internal        # TCP traceroute; ICMP is often blocked
  
```

3.2 Subnetting & CIDR – Reading and Designing the Notation

What is it? `10.20.32.0/20` means: the first 20 bits are the fixed network prefix; the remaining 12 bits identify hosts inside it. That is $2^{12} = 4096$ addresses, spanning `10.20.32.0` to `10.20.47.255`. The slash

number is a count of fixed bits — nothing more.

Why does it exist? The original internet allocated addresses in classes: a Class A was a fixed /8 (16.7 million addresses), a Class B a /16, a Class C a /24. An organisation needing 300 addresses had to take a Class B and waste 65,000, or take two Class Cs and route them separately. Classless Inter-Domain Routing (RFC 4632, 1993) replaced the classes with an arbitrary prefix length, which both stopped the waste and let providers *aggregate* many customer prefixes into one routing-table entry — the reason the global routing table is merely large rather than impossible.

How does it work internally? A router does a longest-prefix match. It masks the destination address against every route's prefix and picks the most specific route that matches. This is why 0.0.0.0/0 works as a default route (it matches everything, with zero specificity, so it loses to any other match), and why a /32 host route always wins. In AWS the same rule governs your VPC route tables: the implicit local route for the VPC CIDR is more specific than 0.0.0.0/0, so intra-VPC traffic never leaves via the gateway.

CIDR	Addresses	Usable (AWS)	Typical use
/32	1	—	A single host route or a one-address SG rule
/28	16	11	Smallest subnet AWS allows; a NAT or endpoint subnet
/24	256	251	A classic application subnet
/20	4,096	4,091	A roomy per-AZ tier subnet
/16	65,536	65,531	An entire VPC (AWS maximum for one block)

Infrastructure insight

The "usable" column is smaller in AWS than the textbook figure because AWS reserves **five** addresses in every subnet, not two: the network address, the VPC router, the DNS resolver, one address reserved for future use, and the broadcast address. In a /28 that is five of sixteen — nearly a third gone. Beginners size a /28 for a fourteen-instance service and are surprised when the eleventh launch fails.

Designing rather than reading. A usable VPC plan answers four questions before it assigns a single address. **One:** which private range, and does it collide with anything you might ever peer with (your office, a partner VPC, your Kubernetes pod CIDR)? **Two:** how many availability zones — and reserve for one more than you use today. **Three:** how many *tiers* per AZ (public edge, private application, private data)? **Four:** how much space stays unallocated for growth. Here is the CloudBase plan you will build on Day 7.

Block	CIDR	Addresses	Purpose
VPC	10.20.0.0/16	65,536	The whole CloudBase network
public-a	10.20.0.0/20	4,096	ALB, NAT gateway, bastion (AZ a)
public-b	10.20.16.0/20	4,096	ALB, NAT gateway (AZ b)
app-a	10.20.32.0/20	4,096	Application instances / nodes (AZ a)
app-b	10.20.48.0/20	4,096	Application instances / nodes (AZ b)
data-a	10.20.64.0/20	4,096	RDS, cache (AZ a)
data-b	10.20.80.0/20	4,096	RDS, cache (AZ b)
unallocated	10.20.96.0/19 10.20.128.0/17	40,960	A third AZ and future tiers

Why these boundaries. Each /20 starts on a multiple of 16 in the third octet, so the maths is readable at a glance: public tiers occupy 0–31, app 32–63, data 64–95. Anyone can see from an address which tier it is in — which matters enormously at 3am. More than half the VPC is deliberately unspent, because AWS permits adding a secondary CIDR to a VPC but never shrinking or renumbering the

primary one, and because your future Kubernetes cluster (Day 5) will want addresses per *pod*, not per node.

Design trade-off

Big subnets waste nothing — private address space is free — but they encourage flat, permissive networks where a single security-group mistake exposes everything. Small subnets force explicit boundaries but run out. The resolution most teams land on: generous subnets, strict security groups. Space is cheap; blast radius is not.

3.3 DNS — The Lookup Chain, and TTL as a Planning Decision

What is it? A globally distributed, hierarchical, cached key-value store mapping names to records. A gives an IPv4 address, AAAA an IPv6 one, CNAME aliases one name to another, MX routes mail, TXT carries arbitrary strings (domain verification, SPF), NS delegates a zone to nameservers.

Why does it exist? In the early ARPANET every host downloaded a single file, `HOSTS.TXT`, maintained by one organisation and distributed by FTP. It did not scale: one authority, one bottleneck, no delegation, and updates measured in days. DNS (RFCs 1034 and 1035, 1987) replaced it with *delegated* authority — the root delegates `.com` to registry operators, who delegate `example.com` to whoever runs it — and *caching*, so that the vast majority of lookups never leave your ISP.

How does it work internally? Your machine asks a **recursive resolver** (your ISP's, or `1.1.1.1`, or `169.254.169.253` inside an AWS VPC). If the answer is not cached, the resolver walks the hierarchy: it asks a root server, which does not know the answer but returns the nameservers for `.com`; it asks those, which return the nameservers for `example.com`; it asks those, which are *authoritative* and return the record. Every answer carries a TTL, and the resolver caches it for that many seconds.

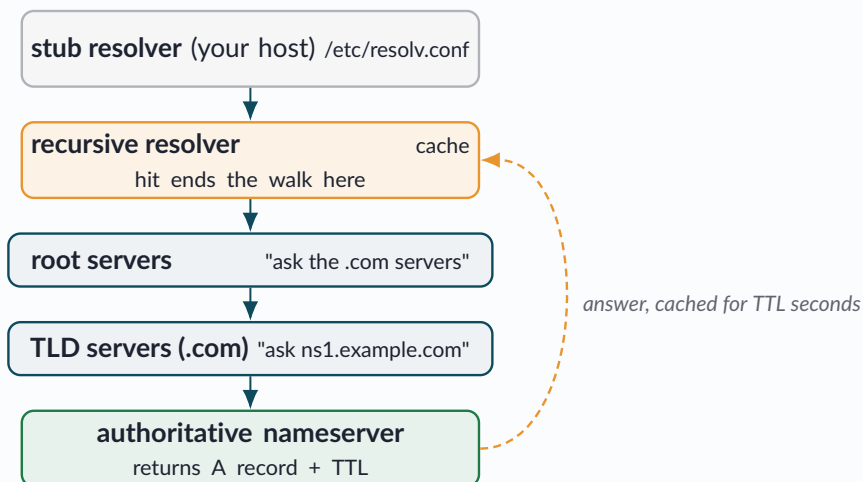


Fig 3.3 — The resolution chain. Each step returns a delegation, not an answer, until the last one.

tracing a resolution yourself

```

dig +trace api.cloudbase.example      # walk root -> TLD -> authoritative
dig +noall +answer api.cloudbase.example # just the record and its TTL
dig @1.1.1.1 api.cloudbase.example    # bypass your local resolver's cache
dig +nssearch cloudbase.example       # who is authoritative for the zone
dig -x 10.20.34.7                     # reverse lookup (PTR)
resolvectl status                     # what systemd-resolved is actually using

```

Production example. Cloudflare's public resolver `1.1.1.1` is announced by anycast from hundreds of cities: the same address is advertised from every point of presence, and BGP routes each client to the topologically nearest one. Cloudflare runs it on the same network that already hosts authoritative records for millions of domains, so a large fraction of recursive walks terminate locally instead of crossing the internet. It also applies aggressive negative caching and query-name minimisation, and offers DNS over TLS and DNS over HTTPS so the query itself is not readable in transit. The architectural lesson: DNS latency is dominated by *distance and cache hit rate*, not by protocol overhead.

Security warning • why TTL is an incident-prevention decision, not an afterthought

If a record's TTL is 86400, resolvers all over the internet may hold the old answer for up to 24 more hours. Lowering the TTL five minutes before a cutover changes nothing — caches already populated do not re-check early. The correct sequence is: lower the TTL to 60 several days ahead, wait for the *old* TTL to fully expire everywhere, then cut over, verify, and raise the TTL again once traffic has settled. A low TTL is not free: it multiplies query volume against your authoritative servers, so you lower it deliberately and put it back.

Common misconception

A CNAME cannot coexist with other records at the same name, which is why you cannot put a CNAME at a zone apex (`example.com` must carry SOA and NS records). Providers solve this with non-standard record types — Route 53's ALIAS, others' ANAME — that resolve server-side and return an A record. If you have ever wondered why the apex is configured differently from `www`, that is why.

3.4 Firewalls, Security Groups & Network ACLs

What is it? Three different filters, applied at three different places, that beginners routinely conflate. `nftables` (which replaced `iptables` as the Linux kernel's packet-filter framework) filters on the host itself. An AWS **security group** filters at the elastic network interface of an individual instance. A **network ACL** filters at the subnet boundary.

Why does it exist? Filtering at the host alone means every host must be configured correctly, forever, by whoever last touched it — Day 1's config-drift problem, now with a security blast radius. Cloud security groups move the control plane off the host: the rule is declared once, applied by the hypervisor's network stack, and cannot be disabled by someone with root on the instance. That is the real argument for them, and it is a good one.

How does it work internally — the difference that matters. A security group is **stateful**: it tracks connections. If you allow inbound `tcp/443`, the response packets are automatically permitted outbound, because the group remembers the flow. A network ACL is **stateless**: it evaluates each packet independently, so allowing inbound `443` without also allowing outbound to the client's *ephemeral* port range simply drops the reply. On Linux, `nftables` can do either, and in practice you write a stateful ruleset using connection tracking (`ct state established,related accept`) — exactly what a security group does for you.

Property	Security group	Network ACL
Stateful?	Yes — return traffic is automatic	No — both directions explicit
Applies to	An instance / ENI	An entire subnet
Rule types	Allow only	Allow <i>and</i> deny
Evaluation	All rules evaluated; any match allows	Ordered by number; first match wins
Can reference	Other security groups	CIDR blocks only
Typical use	Day-to-day least privilege	Coarse subnet-wide guardrails

💡 Senior engineer insight

The most powerful security-group feature is not the CIDR field — it is that a rule can reference *another security group* as its source. `sg-data` allows `tcp/5432` from `sg-app`, not from `10.20.32.0/20`. Now autoscaling can replace every application instance, addresses can change hourly, and the rule stays correct. You have written an identity-based rule instead of a location-based one, which is the same conceptual move as Day 12's IAM roles versus long-lived keys.

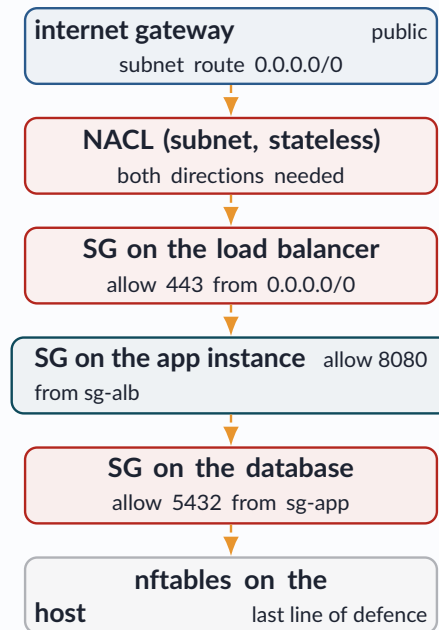


Fig 3.4 — Every filter a packet crosses on its way from the internet to the database. Each one can silently drop it.

a minimal deny-by-default nftables ruleset

```
#!/usr/bin/env nft -f
flush ruleset
table inet filter {
  chain input {
    type filter hook input priority 0; policy drop;
    ct state established,related accept # stateful: replies to our traffic
    ct state invalid drop
    iif lo accept # loopback is always fine
    ip protocol icmp accept # keep ping and PMTU working
    tcp dport 22 ip saddr 10.20.0.0/20 accept # SSH only from the bastion subnet
    tcp dport 8080 ip saddr 10.20.0.0/16 accept # app port only from inside the VPC
  }
  chain forward { type filter hook forward priority 0; policy drop; }
  chain output { type filter hook output priority 0; policy accept; }
}
```

When should I *not* use host firewalls? When the cloud control plane already expresses the rule. Maintaining a hand-written `nftables` file on every instance *in addition* to security groups gives you two sources of truth that will disagree, and the one that drops your traffic will be the one nobody remembers. Use host firewalls for defence in depth on things the cloud cannot see (containers sharing a host, on-premises machines), and let the security group be the primary control.

Cloud cost optimization

The permissive-rule reflex has a bill attached. A security group open to `0.0.0.0/0` on a database port invites internet-wide scanners, and every one of those probes crossing a NAT gateway is billed per gigabyte processed. Worse, the private-subnet-plus-NAT-gateway pattern charges both an hourly rate per gateway per AZ and a data-processing rate — which is why Slack's proxy stack and AWS's own VPC endpoints exist: keep S3 and internal traffic off the NAT path entirely.

3.5 SSH, Key-Based Authentication & the Bastion Pattern

What is it? SSH gives you an authenticated, encrypted channel to a remote shell. Key authentication is asymmetric: you hold a private key; the server holds only the matching public key in `~/.ssh/authorized_keys`.

Why does it exist? Its predecessors — `telnet`, `rlogin`, `rsh`, `ftp` — sent passwords in cleartext across shared networks. Anyone on the path read them. SSH replaced the entire family in the late 1990s. Key auth then removed the password itself, because a password is a shared secret the server must store and a human must remember, and humans reuse them.

How does it work internally? After the transport layer negotiates session keys (an ephemeral Diffie-Hellman exchange, so recording the traffic today and stealing the host key tomorrow does not decrypt it — forward secrecy), the client offers a public key. The server checks `authorized_keys`, and if the key is listed, sends a challenge; the client signs it with the private key; the server verifies the signature. **The private key never crosses the wire, and the server never learns it.** That is why a compromised server cannot replay your credential against another server — the crucial property a password does not have.

`/.ssh/config` — jump through a bastion transparently

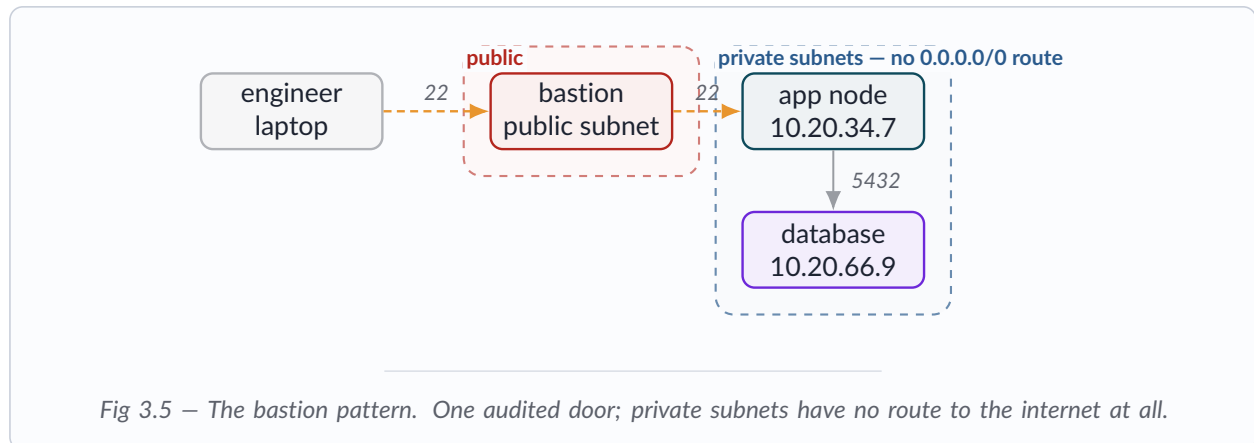
```
Host bastion
  HostName bastion.cloudbase.example
  User ops
  IdentityFile ~/.ssh/cloudbase_ed25519
  IdentitiesOnly yes

Host app-prod-*
  User deploy
  IdentityFile ~/.ssh/cloudbase_ed25519
  IdentitiesOnly yes
  ProxyJump bastion          # hop via the bastion, no agent forwarding

Host app-prod-1
  HostName 10.20.34.7

# ssh app-prod-1    <- one command; the hop is invisible
```

Why ProxyJump and not agent forwarding. `ProxyJump` (`-J`) opens a TCP tunnel through the bastion and completes the SSH handshake *end to end with the target*. The bastion sees encrypted bytes and never handles your key. The older `ssh -A` agent forwarding, by contrast, exposes a socket on the bastion that anyone with root there can use to sign challenges as you. If you take one operational habit from this section: stop using `-A`.



Production example. Netflix gives every engineer SSH access to production instances — and solved the credential problem by removing long-lived keys entirely. Their open-source tool BLESS (Bastion's Lambda Ephemeral SSH Service) runs an SSH certificate authority as an AWS Lambda function, with the CA private key protected by KMS. An engineer authenticates through SSO, and the Lambda issues a *short-lived SSH certificate* rather than installing a key on any host. Because it depends only on AWS services, the CA can be brought up without circular dependencies on Netflix's own infrastructure — a genuine bootstrapping concern for any auth system. Netflix pairs this with automated scanning of SSH usage and alerting on anomalies. The pattern to internalise: *credentials that expire in minutes turn key management from a revocation problem into a non-problem*, and `authorized_keys` files scattered across a fleet are exactly the thing you are trying to eliminate.

Security warning • hardening the door

Disable password authentication (`PasswordAuthentication no`) and root login (`PermitRootLogin no`) in `sshd_config`, and prefer `ed25519` keys over RSA. Note the ordering trap: verify that key auth works in a *second, still-open* session before you disable passwords, or you will lock yourself out of a host whose only access path you just removed. Moving SSH to a non-standard port is not security — it reduces log noise from scanners and nothing more.

3.6 TLS — What It Actually Guarantees

What is it? A protocol that wraps a TCP connection in three guarantees: **confidentiality** (an observer sees ciphertext), **integrity** (tampering is detected, because every record is authenticated), and **authentication** (the server proves it controls the name on its certificate, via a signature chaining to a CA your system already trusts).

Why does it exist? HTTP, SMTP and everything else were designed for a network of mutually trusting research institutions. Once the path between client and server ran through arbitrary ISPs, coffee-shop access points and border routers, "nobody on the path is hostile" stopped being a defensible assumption. SSL, and then TLS, retrofitted the missing properties without changing the applications above them.

How does it work internally? In TLS 1.3 the handshake costs one round trip. The client sends `ClientHello` with its supported cipher suites *and* a Diffie-Hellman key share, guessing the group the server will pick. The server replies `ServerHello` with its own key share — at which point both sides can derive the shared secret — then sends its certificate chain and a `CertificateVerify` signature over the transcript, all already encrypted. The client validates the chain to a trusted root, checks the name and the validity dates, and both sides send `Finished`. TLS 1.3 (RFC 8446) also removed the legacy weak options that made earlier versions negotiable into insecurity: no RSA key transport, no

static DH, no renegotiation, no compression.

TLS guarantees	TLS does <i>not</i> guarantee
The path cannot read the bytes	That the operator is honest — a phishing site can hold a perfectly valid certificate
Tampering is detected	That the data is safe once it lands — encryption at rest is Day 12's problem
You reached the name on the certificate	That the software behind the name is patched, or that it is who you meant to visit

What changed recently. Certificate lifetimes are collapsing. Let's Encrypt made 90-day, automatically renewed certificates normal; in January 2025 it announced six-day certificates and certificates for IP addresses, and in January 2026 both became generally available — the short-lived profile is valid for 160 hours, just over six days, and IP certificates are required to use it because addresses change hands more readily than domain names. The engineering consequence is unambiguous: **manual renewal is no longer a viable operating model.** If your renewal is not automated and monitored, a certificate whose lifetime is measured in days will expire during a holiday. Automate issuance with ACME, alert on days-to-expiry well ahead of time, and treat an expiring certificate as a failing pipeline, not a calendar reminder.

♥ Reliability principle

Certificate expiry is one of the most predictable outages in the industry — and one of the most common, because the failure is silent until the exact second it is total. Monitor the expiry date as a metric with an alert threshold, not as a ticket. On Day 11 you will scrape exactly this as a Prometheus gauge; the alert should fire with enough runway for a human to act during working hours.

inspecting what a server actually presents

```
openssl s_client -connect api.cloudbase.example:443 -servername api.cloudbase.example </dev/null
2>/dev/null | openssl x509 -noout -subject -issuer -dates
curl -vI https://api.cloudbase.example 2>&1 | grep -Ei 'subject|issuer|expire|TLS'
openssl s_client -connect api.cloudbase.example:443 -tls1_2 </dev/null # is old TLS still
allowed?
```

✓ Best practices — what experienced engineers do

- Design the CIDR plan before provisioning anything, and leave over half the VPC unallocated.
- Reference security groups from other security groups instead of hardcoding CIDR blocks.
- Default every security group to deny-all, then open the single port the single caller needs.
- Lower DNS TTLs days before a planned migration, and raise them again after it settles.
- Disable SSH password auth and root login; use `ProxyJump`, never `ssh -A`.
- Automate certificate issuance and renewal, and alert on days-to-expiry as a metric.

✗ Common mistakes beginners make

- Opening `0.0.0.0/0` "to make it work," then never closing it.
- Assuming a successful TCP connect means the application behind it is healthy.
- Writing NACL rules that allow inbound but forget the outbound ephemeral port range.
- Sizing subnets from today's instance count with no headroom for a third AZ or per-pod addressing.
- Leaving the TTL high right up until a cutover, then wondering why traffic did not move.
- Treating "it has HTTPS" as evidence that a service is trustworthy.

4 Visualizations to Internalize

Redraw these from memory. If you can draw the CloudBase network with correct CIDR arithmetic on a whiteboard, you can defend it in a design review — which is the actual test.

 Redraw: the CloudBase VPC — one /16, three tiers, two AZs. Write each subnet's CIDR and its first and last address.

 Redraw: the DNS resolution chain, and mark every point where an answer can be served from cache instead.

5 Guided Hands-On Lab — Design the CloudBase Network on Paper

Goal. Produce a network design document good enough that Day 7 can transcribe it into Terraform without a single new decision. Nothing is provisioned today; nothing costs money today. Every choice you make here is one you would otherwise make badly under deadline pressure in three weeks.

1 Choose the private range and prove it does not collide

Pick `10.20.0.0/16` from RFC 1918 space. Write down every other network CloudBase might ever need to reach: the office VPN, a partner VPC, the Kubernetes pod and service CIDRs from Day 5. Confirm no overlap.

 **Why:** Overlapping CIDRs make VPC peering and Transit Gateway attachments impossible — the routing table cannot decide which side of the peering owns an address. This is the mistake that forces a full re-numbering later, and it is entirely avoidable with ten minutes of arithmetic now.

2 Decide the AZ count and reserve for one more

Design for two availability zones and leave enough address space for a third.

 **Why:** Two AZs is the minimum for surviving a zone failure. A third is what you add when a regional service requires

it or a quorum-based datastore needs an odd number — and adding it should be an address allocation, not a redesign.

3 Carve the tiers

Split the `/16` into six `/20` subnets: public, app and data, in each of two AZs. Write out the first and last address of each by hand, without a calculator.

 **Why:** Doing the bit arithmetic once, manually, is what converts CIDR from notation you look up into notation you read. In a review you will be asked "does that overlap?" and be expected to answer in seconds.

4 Write the route tables

Public subnets get `0.0.0.0/0` to an internet gateway. Private subnets get `0.0.0.0/0` to a NAT gateway — one per AZ. Data subnets may get no default route at all.

 **Why:** A single NAT gateway shared across AZs is both a single point of failure and a cross-AZ data-transfer charge on every packet. One per AZ costs more per hour and less per outage. Data subnets with no egress route cannot exfiltrate to the internet even if compromised.

5 Design the security groups as a chain, not a list

Four groups: `sg-alb` (443 from the internet), `sg-app` (8080 from `sg-alb`), `sg-data` (5432 from `sg-app`), `sg-bastion` (22 from your corporate range only). Each references the previous group, never a CIDR.

 **Why:** This makes the security groups a readable dependency graph of who may talk to whom. Anyone can audit it in thirty seconds, and it stays correct through autoscaling because it names identities rather than addresses.

6 Place the bastion and define the SSH policy

Bastion in `public-a`. No other instance accepts port 22 from anywhere except `sg-bastion`. Write the `~/.ssh/config` with ProxyJump.

 **Why:** One audited door means one place to collect session logs, one place to rotate access, and one place an attacker must get through. Notice you have made access an identity question again — the same principle as the security group chain and Day 12's IAM.

7 Plan the DNS records and their TTLs

Decide the names: `api.cloudbase.example` (the ALB), `db.internal` (private zone). Write the record types and, for each, the steady-state TTL and the pre-migration TTL.

 **Why:** Writing the migration TTL now, while calm, is the only time you will get it right. During an incident you cannot retroactively shorten a cached answer.

8 Write down the failure modes

For each design choice, note what breaks if it is wrong: overlapping CIDR, subnet too small, NACL missing the ephemeral range, TTL too high, certificate not automated.

 **Why:** A design document that lists its own failure modes is a runbook in advance. This section is what your future on-call self will actually read.

Production tip

Commit the design as `docs/network-design.md` in the CloudBase repository with the CIDR table, an ASCII diagram and the security-group chain, and open it as a pull request. Day 2 gave you review discipline for exactly this: a network design reviewed by a second engineer catches the overlapping-CIDR mistake in five minutes, where the alternative is catching it in five months.

6

Thinking Exercises

 Think it through

Security groups have no deny rules — only allows, all evaluated together. NACLs have both, evaluated in order. Why would AWS deliberately remove the ability to express "deny" from the control most engineers use every day? What class of mistake does that design prevent, and what does it make harder?

 Think it through

DNS caching is what makes the system scale, and it is also what makes migrations slow and stale answers possible. Suppose you could redesign DNS with instant global invalidation. What would that cost — in infrastructure, in failure modes, and in who would have to be trusted?

 Think it through

Certificate lifetimes have gone from years, to 90 days, to six. Each reduction makes automation mandatory rather than optional. Is forcing automation by shortening a deadline a legitimate engineering strategy, or is it externalising cost onto operators? Where else in this book does the same pattern appear?

7

Debugging Practice

 Debugging Lab • The cutover that only half moved

Symptom. You migrated `api.cloudbase.example` to a new load balancer at 09:00 and updated the A record. Two hours later the new target is serving traffic, but roughly 30% of requests are still hitting the old one, and the old one is due to be terminated at noon.

dig from two vantage points

```
$ dig +noall +answer api.cloudbase.example
api.cloudbase.example. 43012 IN A 203.0.113.10      # OLD address, 11h left

$ dig @1.1.1.1 +noall +answer api.cloudbase.example
api.cloudbase.example. 58 IN A 198.51.100.24      # NEW address

$ dig +noall +answer +ttlunits api.cloudbase.example @ns1.cloudbase.example
api.cloudbase.example. 1m IN A 198.51.100.24      # authoritative: new, TTL 60
```

- ✓ **Observe.** The authoritative server returns the new address with TTL 60. One resolver already has the new answer; another is holding the old one with eleven hours remaining on a 12-hour TTL.
- ✓ **Hypothesise.** The TTL was lowered to 60 at the same time as the record change, so resolvers that had already cached the answer under the *old* 43200-second TTL never re-queried.
- ✓ **Investigate.** Query several public resolvers directly and compare remaining TTLs; check the zone's change history for when the TTL was actually reduced.
- ✓ **Root cause.** TTL reduction and record change were done in the same operation. Lowering a TTL only affects answers cached *after* the change.
- ✓ **Fix.** Do not terminate the old target. Keep both endpoints serving until the longest old TTL has fully expired — here, until roughly 21:00 — then decommission.
- ✓ **Prevent.** Make "lower TTL to 60" a step executed at least one full old-TTL period before any cutover, tracked as its own change ticket. Add a pre-cutover check that queries several public resolvers and confirms the observed TTL is already low.

🔧 Debugging Lab • The subnet where connections hang instead of failing

Symptom. After a security review added a network ACL to the app subnet, the application can no longer reach an external payment API. Connections do not get refused — they hang for 30 seconds and time out. The security group is unchanged and permits all outbound.

from an app instance

```
$ curl -v https://payments.example.com
* Trying 198.51.100.77:443...
* connect to 198.51.100.77 port 443 failed: Connection timed out

$ aws ec2 describe-network-acls --network-acl-id acl-0ab1 # abridged
Inbound    100    allow  tcp  443    0.0.0.0/0
Inbound    *      deny   all
Outbound   100    allow  tcp  443    0.0.0.0/0
Outbound   *      deny   all
```

- ✓ **Observe.** Timeout, not "connection refused". A refusal means a RST came back; a timeout means the packet vanished — the signature of a silent drop by a filter.
- ✓ **Hypothesise.** The NACL is stateless. Outbound 443 is allowed, so the *SYN* leaves; the *SYN-ACK* returns to the client's *ephemeral* source port, which the inbound rules do not permit, so the reply is dropped.
- ✓ **Investigate.** Enable VPC flow logs and look for *REJECT* entries on the return path

with a high destination port. Confirm the instance's local ephemeral range with `sysctl net.ipv4.ip_local_port_range` (commonly 32768–60999).

- ✓ **Root cause.** An inbound NACL rule allowing the ephemeral port range for return traffic is missing. The security group masked the concept because it is stateful and needed no such rule.
- ✓ **Fix.** Add an inbound allow for `tcp/1024–65535` from the required destinations, and an outbound allow for the same range so inbound connections' replies also work.
- ✓ **Prevent.** Treat NACLs as coarse subnet guardrails only, express least privilege in security groups, and add a review checklist item: "for every NACL rule, name the return-path rule that pairs with it."

8 Mini-Labs

Mini-Lab • Beginner

40 min

Goal. Trace a full DNS resolution from the root to the authoritative answer and explain every hop.

Business context. Your team is about to migrate a domain and nobody can say where the caching actually happens.

Architecture. One Linux host, `dig`, and a domain you do not control.

Requirements. Run `dig +trace`, annotate each delegation, then re-query and show the TTL counting down in a resolver cache.

Constraints. Explain the difference between the answer from your stub resolver and the answer from the authoritative server.

Hints. `dig +trace` bypasses your recursive resolver; `dig @1.1.1.1` does not bypass *its* cache.

Expected outcome. A one-page annotated trace naming root, TLD and authoritative servers, and the TTL on each record.

Production considerations. Note where a private hosted zone (Day 10) would short-circuit this chain entirely.

Common pitfalls. Confusing the TTL as configured at the authority with the TTL remaining in a cache.

Mini-Lab • Intermediate

60 min

Goal. Write and test a deny-by-default `nftables` ruleset allowing only SSH and HTTPS.

Business context. A host outside the cloud (a build runner) needs the same posture as your VPC instances.

Architecture. One VM you can reach out-of-band, plus a second host to test from.

Requirements. Policy `drop` on input, connection-tracking `accept` for established traffic, SSH restricted by source CIDR, HTTPS open, everything else dropped.

Constraints. You must not lock yourself out — keep a second session open and use a timed rollback.

Hints. `ct state established,related accept` must come first; test with `nc -zv` from the other host for both an allowed and a blocked port.

Expected outcome. A committed `nft` file, plus evidence that a blocked port times out while an

allowed one connects.

Production considerations. Compare, line by line, with the equivalent security group. Which rules disappear, and why?

Common pitfalls. Forgetting the loopback rule; dropping ICMP entirely and breaking path-MTU discovery.

Mini-Lab • Production-style

90 min

Goal. Produce the CloudBase network design document that Day 7 will implement verbatim.

Business context. A design review with a second engineer is scheduled; overlapping CIDRs must be caught now, not after provisioning.

Architecture. One $/16$ VPC, three tiers, two AZs, one bastion, four security groups, per-AZ NAT.

Requirements. A CIDR table with first and last address per subnet, route tables per subnet, the security-group chain, the SSH policy, the DNS records with steady-state and migration TTLs.

Constraints. Over half the VPC must remain unallocated; no security group may reference a CIDR except the bastion's corporate source and the ALB's $0.0.0.0/0$ on 443.

Hints. Do the bit arithmetic by hand first, then verify with `ipcalc`. Draw the diagram before writing the table.

Expected outcome. `docs/network-design.md` merged via pull request, with a failure-mode list.

Production considerations. Where would VPC endpoints remove NAT gateway cost? What changes if a third AZ is added?

Common pitfalls. Allocating the whole $/16$ immediately; sizing subnets from today's instance count; forgetting AWS reserves five addresses per subnet.

9 Engineering Notes

Infrastructure insight

Network topology is the one part of your infrastructure that is genuinely hard to change after the fact. Compute is disposable — terminate an instance and launch a new one. Address space is not: it is baked into route tables, peering connections, firewall rules, allow-lists held by third parties and, eventually, into someone's spreadsheet. Spend disproportionate care here relative to how little of the fourteen days it occupies.

Reliability principle

Design the network so that losing an availability zone is a capacity event, not an outage. That means per-AZ NAT gateways, subnets in at least two zones for every tier, and no single-AZ dependency hidden inside a "temporary" resource. The test is simple and worth writing down: if AZ *a* vanished right now, name every component that would stop working. If you cannot answer in under a minute, the design is not understood well enough.

Security warning • the two rules that cause most cloud breaches

An inbound $0.0.0.0/0$ on a management or database port, and a long-lived credential with more permission than its task needs. Both are shortcuts taken to unblock something at 6pm on a Friday, and both survive because nothing subsequently fails. Automated detection is the only reliable control —

a scheduled check that flags any security group open to the internet on a port other than 80 or 443, reported as a failing build rather than an email.

Cloud cost optimization

NAT gateways are one of the most common surprises on an AWS bill: charged per hour *per gateway* and per gigabyte processed. A private subnet whose instances pull container images from a public registry on every deploy sends all of that through NAT. VPC endpoints for S3 and ECR remove that path entirely, and gateway endpoints for S3 carry no hourly charge. Design the egress path deliberately, not as whatever happens by default.

Interview insight

Expect: "Walk me through what happens between typing a URL and the page loading." The answer that lands is a chain with no gaps: browser cache, then OS stub resolver, then recursive resolver walking root → TLD → authoritative, TTL governing all of it; then a TCP handshake to port 443; then a TLS 1.3 handshake with certificate validation; then the HTTP request, which crosses a load balancer, a security group, and finally the application. Naming the layer where each thing can fail is what separates a senior answer from a memorised one.

10 Build-Along Project — CloudBase

Day 1 gave CloudBase a bootstrapped host with a service user, a systemd unit and a strict-mode provisioning script. Day 2 gave it a repository with branch protection, commit conventions and review. Today CloudBase gets the thing that is hardest to change later and cheapest to get right now: its network, designed completely, on paper, before a single dollar of cloud spend. Day 7 turns this document into Terraform; Day 10 applies it to AWS.

Build-Along • Day 03 — the network design

- Create `docs/network-design.md` in the CloudBase repository (the one Day 2 set up) and open it as a pull request — this design gets reviewed like code.
- Allocate the VPC `10.20.0.0/16` and record why: an RFC 1918 range that collides with nothing CloudBase may peer with.
- Carve six `/20` subnets across two AZs — `public-a/b`, `app-a/b`, `data-a/b` — and tabulate the first and last address of each, computed by hand.
- Leave `10.20.96.0/19` and `10.20.128.0/17` explicitly unallocated, labelled "third AZ and future tiers".
- Write route tables: `public` → internet gateway; `app` → per-AZ NAT gateway; `data` → no default route.
- Define the security-group chain: `sg-alb` 443 from the internet; `sg-app` 8080 from `sg-alb`; `sg-data` 5432 from `sg-app`; `sg-bastion` 22 from the corporate range only. No CIDR sources anywhere else.
- Place the bastion in `public-a`, and commit the `ssh/config` fragment using `ProxyJump` with agent forwarding explicitly disabled.
- Record the DNS plan: `api.cloudbase.example` with a steady-state TTL of 300 and a

documented pre-migration TTL of 60, plus the private zone for `db.internal`.

- Add a "failure modes" section naming what breaks for each wrong choice, and a "cost notes" section identifying where VPC endpoints would remove NAT charges.

Definition of done: `docs/network-design.md` is merged after review; every subnet CIDR is stated with its first and last address and none overlap; the security-group chain contains no CIDR source except the ALB's `0.0.0.0/0` on 443 and the bastion's corporate range; over half the VPC is documented as unallocated; and a second engineer can read the document and provision the network without asking a single question.

11 Chapter Summary

Key takeaways

- ✓ The three-way handshake is completed by the kernel; a successful connect proves reachability, never application health.
- ✓ CIDR is a count of fixed prefix bits, and routers choose by longest-prefix match — which is why `0.0.0.0/0` loses to everything.
- ✓ AWS reserves five addresses in every subnet, and a VPC CIDR can be extended but never shrunk or renumbered.
- ✓ DNS answers are cached for their TTL; lowering the TTL only affects lookups made *after* the change.
- ✓ Security groups are stateful and allow-only; NACLs are stateless and ordered, and need explicit return-path rules.
- ✓ SSH key auth never sends the private key; `ProxyJump` keeps the bastion out of the authentication path entirely.
- ✓ TLS guarantees confidentiality, integrity and name authentication — not that the operator deserves your data.

Things to remember

- Timeout means a silent drop by a filter; "connection refused" means a RST came back and something answered.
- A security group referencing another security group survives autoscaling; a hardcoded CIDR does not.
- Certificate lifetimes are shrinking — Let's Encrypt's short-lived profile is 160 hours — so renewal must be automated and monitored.
- Design for two availability zones and leave address space for a third.

Common pitfalls (last look)

- `0.0.0.0/0` left open • TTL lowered too late • NACL return path forgotten
- Subnets sized from today's count • `ssh -A` habit • manual certificate renewal

 One-Page Cheat Sheet

TCP handshake — SYN → SYN-ACK → ACK, completed by the kernel. Connect ≠ healthy.

CIDR — /n = n fixed bits; /24=256 addrs (251 usable in AWS), /20=4096, /16=65536.

Routing — Longest-prefix match wins; 0.0.0.0/0 is the default route of last resort.

DNS — stub → recursive → root → TLD → authoritative; TTL governs every cache on the path.

SG vs NACL — SG: stateful, per-ENI, allow-only. NACL: stateless, per-subnet, ordered, allow+deny.

SSH — public key in `authorized_keys`; private key never transmitted; use `ProxyJump`, never `-A`.

TLS 1.3 — one round trip, forward secrecy, encrypted certificate; proves the name, nothing more.

CLI quick reference

Day 3 commands worth memorising

```
# sockets & reachability
ss -tlnp                                # listening sockets and owning PIDs
nc -zv host 5432                         # handshake test (not a health check)
ip route get 10.20.34.7                  # which route would this address take
traceroute -T -p 443 host                # TCP traceroute; ICMP is often blocked

# DNS
dig +trace name                          # walk root -> TLD -> authoritative
dig +noall +answer name                  # the record and its remaining TTL
dig @1.1.1.1 name                        # ask a specific resolver

# firewalling
nft list ruleset                         # current nftables rules
nft -f /etc/nftables.conf                # load a ruleset atomically

# ssh
ssh -J bastion app-prod-1                # one-off ProxyJump
ssh-keygen -t ed25519 -C "ops@cloudbase"

# tls
openssl s_client -connect host:443 -servername host </dev/null | openssl x509 -noout -dates
```

 Interview questions to master

- › Walk me through what happens between typing a URL and the page loading.
- › What is the difference between a security group and a network ACL?
- › Why can a TCP handshake succeed while the application is still broken?
- › How would you safely cut a domain over to a new server with minimal downtime?
- › Explain /24 versus /16 in terms of usable hosts — and how many AWS actually gives you.
- › Why disable SSH password authentication in production, and why prefer `ProxyJump` over agent forwarding?
- › What does TLS guarantee, and what does it not?

Further reading & official docs

- RFC 9293 — *Transmission Control Protocol*, August 2022. The consolidated TCP specification; it

obsoletes RFC 793 and folds in decades of piecemeal updates.

- RFC 4632 — *Classless Inter-Domain Routing (CIDR)*, and RFC 1918 — private address allocation.
- RFC 1034 and RFC 1035 — the DNS concepts and specification; RFC 2308 for negative caching and TTL semantics.
- RFC 8446 — *TLS 1.3*. Read the handshake section; it explains why the round-trip count dropped.
- AWS VPC User Guide — "Subnets for your VPC" and "Subnet CIDR blocks", at docs.aws.amazon.com/vpc/latest/userguide/, for the five reserved addresses and the /16-/28 size limits.
- Cloudflare's introduction to the 1.1.1.1 resolver — anycast, negative caching and DNS over TLS/HTTPS, at blog.cloudflare.com/dns-resolver-1-1-1-1/.
- Slack Engineering, "Building the Next Evolution of Cloud Networks at Slack" and its retrospective — the Whitecastle VPC and Transit Gateway design.
- Let's Encrypt, "Announcing Six Day and IP Address Certificate Options in 2025" and the January 2026 general-availability post; plus Netflix's BLESS, an SSH certificate authority on AWS Lambda that issues short-lived certificates instead of distributing keys.

Production checklist

- ☐ The VPC CIDR is documented, non-overlapping with every network you may ever peer with, and over half unallocated.
- ☐ Every tier has a subnet in at least two availability zones, with a per-AZ NAT gateway.
- ☐ No security group allows 0.0.0.0/0 on any port except 80 and 443 on the load balancer.
- ☐ Security groups reference other security groups, not CIDR blocks, wherever an identity exists.
- ☐ SSH password authentication and root login are disabled; access is through one audited bastion.
- ☐ DNS records have a documented steady-state TTL and a rehearsed pre-migration TTL procedure.
- ☐ Certificate issuance and renewal are automated, and days-to-expiry is monitored with an alert.
- ☐ A written failure-mode list exists for the network design, and someone other than the author has reviewed it.

Key idea

What you will learn next (Day 4). You now have a host and a network design. Tomorrow you package the thing that runs on them: containers — what an image layer actually is, how namespaces and cgroups make a process believe it owns the machine, why a container's networking is a Linux bridge and a virtual interface pair, and why the CIDR arithmetic you just did reappears the moment a container needs an address of its own.

DAY 04 • CONTAINERS

Docker Deep Dive

04

Images, layers, namespaces and cgroups — what a container actually is

🕒 Estimated study time: 8 hours

🎯 By the end of this day, you will be able to...

- Explain what a container is at kernel level — namespaces, cgroups and a root filesystem — without saying "lightweight VM".
- Reason about image layers, content addressing and copy-on-write well enough to make a slow build fast.
- Write a production-grade multi-stage Dockerfile: pinned base, non-root `USER`, build stage discarded.
- Choose deliberately between a named volume, a bind mount and `tmpfs`, and defend the choice.
- Diagnose container lifecycle failures — exit 137, OOMKill, PID 1 signal handling, a ballooning build context.

1 The Big Picture

What problem this solves. "It works on my machine" is not a joke about developers; it is a statement about *dependency resolution happening at the wrong time*. A traditional deploy resolves dependencies on the target host, at deploy time, against whatever library versions that host happens to carry. Every host drifts differently, so every host fails differently. A container image moves that resolution to *build time* and freezes the result: the application, its runtime, its shared libraries and its filesystem layout are sealed into one artifact that is byte-for-byte identical everywhere it runs. What ships is not "the code" — it is the whole userspace.

Why DevOps engineers need it. Every layer you build from here on consumes container images. Kubernetes (Day 5–6) schedules them. CI (Day 9) builds and pushes them. Image scanning (Day 12) is a supply-chain control on them. If you treat the Dockerfile as a magic incantation copied from a blog post, everything above inherits that fog: builds that take eleven minutes because the cache is busted on every commit, images that ship a full compiler toolchain into production, and containers that die at exit 137 for reasons nobody on the team can articulate.

🔑 Key idea

A container is not a small virtual machine. It is one ordinary Linux process — scheduled by the same kernel scheduler as everything else on the host — given a restricted *view* of the system (namespaces) and a restricted *budget* of resources (cgroups), with its root filesystem assembled from stacked read-only image layers. Every "container" concept in this chapter resolves to one of those three mechanisms. There is no hypervisor, no guest kernel, no emulation.

Where it fits in a modern cloud architecture. The image is the unit of exchange between every tool in the rest of this book. Read the diagram downward: each layer hands the next one an image reference and nothing else.

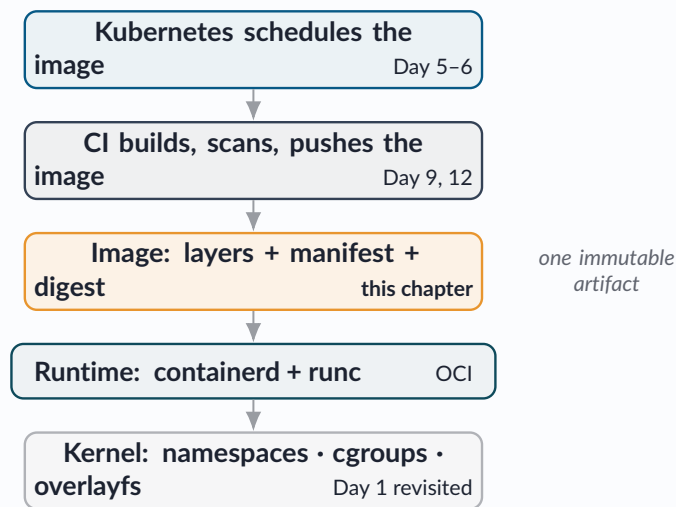


Fig 4.0 — The image is the contract. Everything above the container layer consumes an immutable digest; everything below is the Linux you learned on Day 1.

How it connects to previous chapters. Day 1 taught processes, signals, permissions and `/proc`. Today you discover that was not preparation for containers — it *was* containers, viewed from underneath. Exit code 137 is still `128 + 9`. A non-root `USER` in a Dockerfile is the same reasoning as the dedicated service user in `bootstrap.sh`. Day 2's Git discipline is what makes an image reproducible from a commit, and Day 3's networking is what the container's bridge, veth pair and NAT rules are made of.

Where companies use it in production. Netflix built **Titus**, its own container management platform, before standardising on Kubernetes-era tooling: a leader-elected *Titus Master* scheduler placing work onto a large pool of EC2 *Titus Agents*, each agent launching containers with Docker and wiring them into AWS VPC networking, IAM roles and security groups so that a container — not the VM — is the identity boundary. Uber, which builds on the order of a hundred thousand container images per week, wrote **Makisu** specifically because the standard builder needed access to `/proc` to launch nested containers, which their unprivileged build environment would not allow; their distributed layer cache cut build time by up to ninety percent on some repositories. Both stories are the same lesson: at scale, the image *build* becomes the bottleneck long before the container runtime does.

Common misconceptions. That containers "virtualise the OS" (they share one kernel — a container cannot run a different kernel version than its host); that images are tarballs of a running machine (they are ordered, content-addressed layer sets plus a JSON manifest); that `docker run` "boots" something (it forks a process); and that a container is a security boundary equivalent to a VM (by default it is not — Day 12 covers what actually hardens it).

2

Concept Map

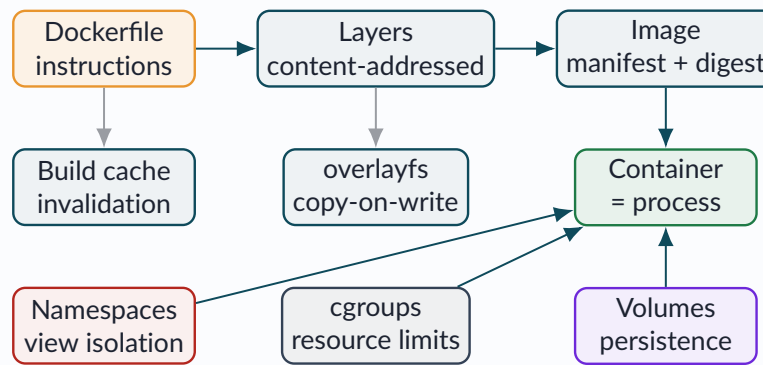


Fig 4.1 — The Day 4 territory. The Dockerfile produces layers; the kernel produces isolation; the two meet in a running container.

Terminology to anchor: **Dockerfile** → **layer** → **digest** → **image manifest** → **registry** → **copy-on-write** → **namespace** → **cgroup** → **volume**.

3 Guided Learning

3.1 What a Container Actually Is

What is it? A container is a normal Linux process, started by a normal `clone()` syscall, that the kernel has been told to lie to. It sees a filesystem that is not the host's, a process table that starts at PID 1 and contains only its own children, and network interfaces that belong only to it. Nothing about the process itself is special — `ps aux` on the host shows it sitting alongside `sshd` and `cron`.

Why does it exist? Engineers wanted isolation without paying for a second kernel. The lineage is long: `chroot` (1979) changed a process's apparent root directory but isolated nothing else; FreeBSD Jails (2000) and Solaris Zones (2004) added process and network isolation; Google contributed *process containers* — later renamed **cgroups** — to Linux in 2007 to bound CPU and memory; the `namespaces` family landed incrementally through the 2000s and 2010s. LXC glued these together around 2008. All of that existed before Docker. What Docker added in 2013 was not isolation — it was a *portable, layered, content-addressed image format* and a one-command developer workflow. That is why the image, not the runtime, is the durable contribution.

? Why does this exist?

Before containers, the reproducibility answer was a full virtual machine per service: a whole guest kernel, a whole init system, gigabytes of disk and tens of seconds of boot, all to isolate one Java process. The alternative was shared hosts with dependency conflicts — two applications needing incompatible OpenSSL versions on one box. Containers exist to give VM-grade dependency isolation at process-grade cost.

How does it work internally? Three kernel mechanisms, applied to one process.

Namespace	Isolates	What you observe
mnt	Mount points	The container's / is the image, not the host's root.
pid	Process IDs	The entrypoint is PID 1; it cannot see or signal host processes.
net	Interfaces, routes, iptables	Its own <code>eth0</code> , its own port space, its own routing table.
uts	Hostname, domain	<code>hostname</code> returns the container ID, not the host's name.
ipc	Shared memory, semaphores	No accidental SysV IPC collisions with the host.
user	UID/GID mapping	Root inside can map to an unprivileged UID outside.
cgroup	cgroup root view	The container cannot see the wider cgroup hierarchy.

Namespaces answer "what can this process see?" **cgroups** answer the orthogonal question "how much can this process consume?" — CPU shares and quota, memory ceiling, block-I/O weight, PID count. On modern distributions cgroup v2 is the default: a single unified hierarchy where each container gets a directory with files such as `memory.max` and `cpu.max`. Writing a byte count into `memory.max` is literally all that `docker run -m 512m` does.

The third mechanism is the root filesystem: `pivot_root` (a safer descendant of `chroot`) swaps the process's / for the assembled image. Then the runtime drops Linux capabilities, applies a seccomp filter restricting which syscalls are allowed, and `execs` your binary.

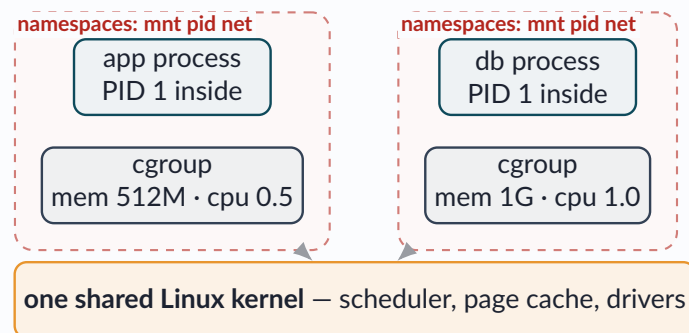


Fig 4.2 — One host kernel, two containers. The dashed boundaries are namespaces (view) and the meters are cgroups (budget). Nothing here is virtualised.

👁 Behind the scenes

Run `ps aux` on the host while a container is running and you will find its process listed with an ordinary host PID. Then look at `/proc/<pid>/ns/` — a directory of symbolic links, one per namespace, each pointing at an inode identifier. Two processes sharing a namespace share that inode. That is the entire implementation of "being in the same container", and it is why Kubernetes sidecars — separate containers in one pod — work: they are simply given the same net and ipc namespace inodes.

⚠ Common misconception

"Containers are more secure than processes." They are the same processes, with the kernel attack surface fully shared. A kernel privilege-escalation bug is exploitable from inside a container. Namespaces raise the cost of escape; they do not create a hypervisor boundary. When you genuinely need that boundary — untrusted multi-tenant code — the answer is a sandboxed runtime such as gVisor or Firecracker microVMs, not more Docker flags.

3.2 Images Are Layers; Containers Are a Writable Layer on Top

What is it? An image is an ordered stack of read-only **layers** plus a JSON manifest describing them. Each layer is a tar archive of filesystem changes — files added, modified or deleted relative to the layer beneath. Each is identified by the SHA-256 digest of its content, which makes layers **content-addressed**: identical content is the same layer everywhere in the world.

Why does it exist? Naive packaging ships the whole filesystem on every release. If your base OS is 120 MB and your application jar is 40 MB, a one-line code change should not cost a 160 MB transfer. Layers make the delta explicit: the base layer is pulled once per host and shared by every image built on it, and the registry, the host and the CI cache all deduplicate by digest. Twenty microservices on the same base image consume that base's disk space exactly once.

How does it work internally? The default storage driver, `overlay2`, is a union filesystem. It stacks the image layers as `lowerdir` (read-only), adds one empty `upperdir` (read-write, created per container), and presents the union as `merged`, which becomes the container's `/`. Reads fall through the stack top-down until a file is found. On the *first* write to an existing file, `overlayfs` performs a **copy-up**: it copies the whole file from the lower layer into the upper layer and modifies the copy. Deletion does not delete anything — it writes a *whiteout* entry in the upper layer that masks the lower file.

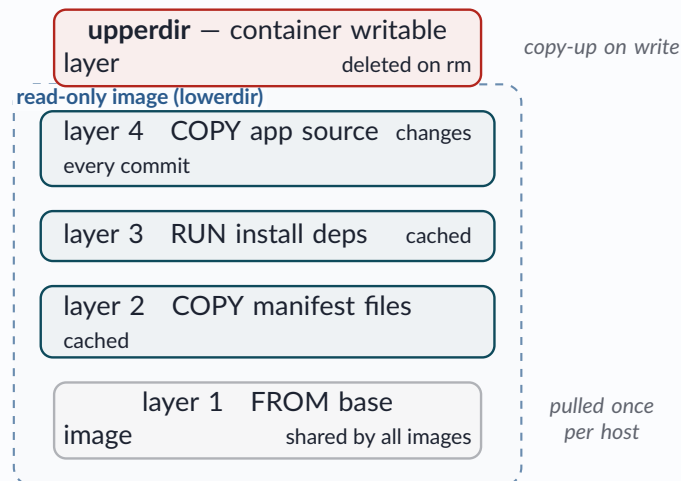


Fig 4.3 — Copy-on-write. The image layers are shared and never mutated; only the thin upper layer belongs to this container, and it is destroyed with it.

This is why containers start in milliseconds: nothing is copied at start, only an empty directory is created and a mount is assembled. It is also why data written inside a container vanishes when the container is removed — the upper layer goes with it. And because `overlayfs` supports page-cache sharing, one hundred containers reading the same library file from the same lower layer share one page-cache entry, not one hundred.

The build cache. During a build, each instruction produces a layer and a cache key. For `RUN`, the key is derived from the command string itself; for `COPY` and `ADD`, it is derived from the *checksum of the copied files*. On a rebuild, the builder walks instructions from the top, reusing cached layers while keys match. The moment one key differs, that layer and *every layer after it* is rebuilt — a cache miss is irreversible for the rest of the file.

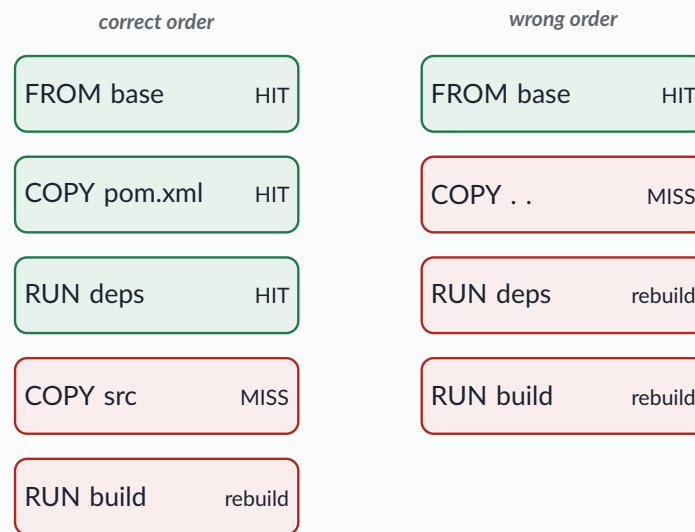


Fig 4.4 — Cache invalidation cascades downward. Copying source before installing dependencies means every commit reinstalls the world.

⚡ Performance note

The single highest-leverage change most teams can make to their CI time is reordering four lines of a Dockerfile. Copy only the dependency manifest (`pom.xml`, `package-lock.json`, `requirements.txt`, `go.sum`), install dependencies, *then* copy the source. Dependencies change weekly; source changes hourly. Uber built an entire distributed layer cache around this principle and reported up to ninety percent build-time reductions on some repositories — but the ordering rule is free and gets you most of the way.

⚠ Common misconception

"RUN `rm -rf /var/cache` at the end of my Dockerfile shrinks the image." It does not. Deleting a file in a later layer writes a whiteout; the bytes still sit in the earlier layer and still travel over the network. To actually remove them, the deletion must happen inside the *same* RUN instruction that created them — or, far better, the artifact must never enter the final image at all, which is what multi-stage builds are for.

3.3 Multi-Stage Builds

What is it? A single Dockerfile containing several `FROM` instructions. Each `FROM` starts a new stage with its own filesystem; a later stage can reach back with `COPY --from=<stage>` to take specific paths. Only the last stage becomes the published image.

Why does it exist? Building software needs a compiler, a package manager, dev dependencies, headers and test fixtures. Running it needs a runtime and one artifact. Before multi-stage builds, teams either shipped the entire toolchain to production — a 900 MB image to run a 40 MB jar — or maintained a fragile "builder container plus outer script plus second Dockerfile" pipeline that only worked on the CI machine. Multi-stage collapses that into one reviewable file.

Dockerfile — CloudBase API, production-grade multi-stage

```
# syntax=docker/dockerfile:1
```

```
# ----- stage 1: build (never shipped) -----
FROM eclipse-temurin:21-jdk-alpine AS build
WORKDIR /src

# dependency manifest first -> this layer is cached across code commits
COPY pom.xml ./
COPY .mvn/ .mvn/
COPY mvnw ./
RUN ./mvnw -B dependency:go-offline

# source last -> only this layer and below rebuild on a commit
COPY src/ src/
RUN ./mvnw -B -DskipTests package && \
    mv target/cloudbase-*.jar /src/app.jar

# ----- stage 2: runtime (this is what ships) -----
FROM eclipse-temurin:21-jre-alpine AS runtime

# non-root identity, created explicitly with a fixed UID
RUN addgroup -g 10001 -S cloudbase && \
    adduser -u 10001 -S cloudbase -G cloudbase

WORKDIR /app
COPY --from=build --chown=cloudbase:cloudbase /src/app.jar ./app.jar

USER 10001:10001
EXPOSE 8080
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75"

HEALTHCHECK --interval=30s --timeout=3s --start-period=40s --retries=3 \
    CMD wget -qO- http://127.0.0.1:8080/actuator/health || exit 1

ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/app.jar"]
```

How does it work internally? BuildKit — the default builder since Docker 23 — does not execute the Dockerfile top to bottom. It parses the whole file into a dependency **DAG**, prunes stages the final target does not need, and executes independent stages *in parallel*. If a stage's output is not reachable from the target, it is never built at all. This is why adding a `test` stage to a Dockerfile costs nothing when you build `-target runtime`.

Two details in the file above earn their place. `-chown` on the `COPY` sets ownership as the layer is written; doing it afterwards with `RUN chown` would copy-up every file into a second layer and double the image's size contribution. And `USER 10001:10001` uses the numeric UID rather than the name, because Kubernetes' `runAsNonRoot` check (Day 6) can only verify a numeric UID — a named user makes the admission check fail.

Design trade-off

Smaller is not automatically better. `alpine` uses `musl libc`, not `glibc`; some JVM workloads, native extensions and DNS resolution edge cases behave differently under `musl`. `distroless` images are smaller and have almost no CVE surface, but they contain no shell — so `docker exec -it ... sh` to debug a production incident is impossible, and your `HEALTHCHECK` cannot shell out. The senior move is to pick per-workload: `distroless` for stable services with good remote observability, a slim `glibc` base for anything you still need to poke at by hand.

Security warning • build secrets never belong in a layer

`ARG NPM_TOKEN` followed by `RUN npm install` bakes the token into the layer's metadata and history — `docker history` will show it, and anyone who pulls the image can read it. Deleting it in a later layer does nothing (whiteouts, again). Use BuildKit secret mounts — `RUN --mount=type=secret,id=npmtoken ...` — which expose the value on a tmpfs for the duration of that one instruction and leave nothing in the layer.

3.4 Container Networking

What is it? Each container gets its own network namespace: its own interfaces, routing table, iptables rules and port space. Docker offers several ways to populate that namespace.

Mode	Behaviour	When to use
bridge (default)	Private virtual switch on the host; a veth pair per container; outbound traffic is NATed	Local dev, Compose, single host
host	No net namespace at all — the container uses the host's stack directly	Latency-critical or high packet-rate agents; rare
none	Only a loopback interface	Batch jobs that must not touch the network
overlay	VXLAN tunnel spanning hosts	Swarm / multi-host, superseded by CNI in K8s

How does it work internally? On bridge mode the runtime creates a **veth pair** — a virtual cable with two ends. One end is moved into the container's namespace and renamed `eth0`; the other stays on the host and is enslaved to the `docker0` bridge, a software switch. Outbound packets are source-NATed by a MASQUERADE rule so they leave with the host's address. A published port (`-p 8080:8080`) installs a DNAT rule that rewrites the destination of arriving packets to the container's private address. There is no proxy in the data path — it is kernel packet rewriting, the same mechanism as any Linux router.

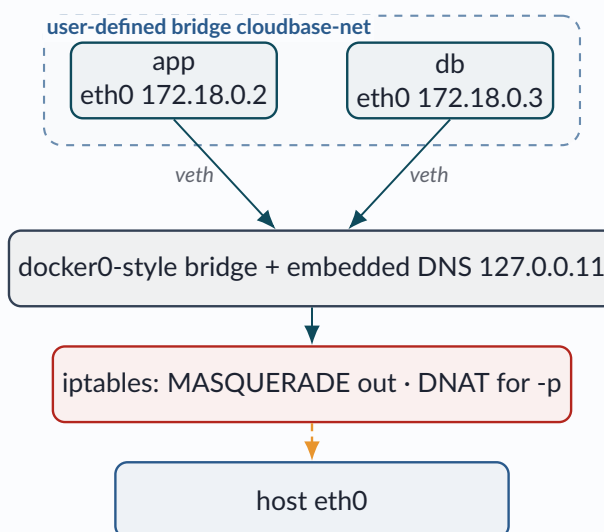


Fig 4.5 — Bridge networking. Each container owns one end of a veth pair; the host end plugs into a software switch, and iptables does the address translation.

Service discovery. On a *user-defined* bridge — not the legacy default bridge — Docker runs an embedded DNS resolver at `127.0.0.11` inside each container's namespace. A lookup of `db` returns the current IP of the container named `db`, refreshed as containers restart and change addresses. This is why your application connects to `db:5432`, never to `localhost:5432` and never to a hardcoded IP: `localhost` inside a container is that container's own loopback, so `localhost:5432` is a promise that nothing is listening. Kubernetes generalises exactly this idea into Services and cluster DNS on Day 5.

Scalability note

Port publishing is a single-host concept. It does not compose: two replicas of the same container cannot both publish host port 8080. The moment you need more than one replica you need a scheduler that assigns addresses and a load balancer that fans out to them — which is precisely the gap Kubernetes fills. If you find yourself scripting port arithmetic in Compose, you have outgrown Compose.

3.5 Volumes, Bind Mounts & Persistence

What is it? A container's writable layer is ephemeral by design. To keep data you mount storage into the container that is not part of the image. Three kinds exist, and choosing among them is a real engineering decision.

Kind	What it is	Correct use
Named volume	A directory managed by the engine, outside any container	Databases, queues, any state
Bind mount	A specific host path mapped into the container	Local dev live-reload; host sockets
tmpfs mount	RAM-backed, never touches disk	Secrets, scratch files

Why the distinction matters. A named volume is addressed by *name*; the engine decides where it lives and manages its lifecycle, so the container is portable to any host. A bind mount is addressed by *host path*, which hardcodes an assumption about the machine: the directory must exist, with the right ownership, on every host that ever runs this container. It also bypasses the image entirely — a bind mount over `/app` means the code you tested in the image is not the code that runs. That is a feature during development and a production incident waiting to happen.

How does it work internally? Both are ordinary Linux bind mounts performed inside the container's mount namespace before the entrypoint runs. Reads and writes go straight to the host filesystem at native speed — they do *not* pass through overlaysfs, which is a second reason databases belong on volumes: the copy-up penalty on first write to a large data file is avoided entirely.

compose.yaml — CloudBase local stack

```
services:
  app:
    build:
      context: .
      target: runtime
    image: ghcr.io/aymix/cloudbase-api:dev
    ports: ["8080:8080"]
    environment:
      DB_HOST: db
      DB_PASSWORD_FILE: /run/secrets/db_password
    depends_on:
      db:
        condition: service_healthy
```

```

restart: unless-stopped
stop_grace_period: 30s
deploy:
  resources:
    limits: { memory: 512M, cpus: "0.50" }

db:
  image: postgres:16.4-alpine
  environment:
    POSTGRES_USER: cloudbase
    POSTGRES_PASSWORD: ${DB_PASSWORD:?set DB_PASSWORD in .env}
  volumes:
    - pgdata:/var/lib/postgresql/data # named volume: survives down/up
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U cloudbase"]
    interval: 5s
    retries: 10

volumes:
  pgdata:

```

🚨 Disaster recovery tip

A named volume is durability, not a backup. `docker compose down -v` destroys it instantly and irreversibly, and a volume lives on exactly one host — lose the host, lose the data. Treat volumes as the write path and a scheduled `pg_dump` into object storage as the recovery path. "Where is the data?" and "how do I get it back?" are two different questions, and only the second one matters at 3am.

💡 Senior engineer insight

Notice `depends_on` with `condition: service_healthy` rather than a bare `depends_on: [db]`. The bare form only waits for the container to be *started*, not for Postgres to accept connections — so the app races the database on every cold start and the team learns to "just run it twice". Encoding readiness as a healthcheck is the same idea as a Kubernetes readiness probe, and building the habit here means Day 6 is a rename rather than a new concept.

3.6 Lifecycle, Signals & Why Containers Exit 137

What is it? A container lives exactly as long as its PID 1 process. When that process exits, the container exits, and its exit code becomes the container's exit code. There is no init system keeping it alive unless you put one there.

How does it work internally? `docker stop` sends SIGTERM to PID 1, waits (ten seconds by default, or `stop_grace_period`), and then sends SIGKILL. Because $128 + 9 = 137$, that path produces exit code 137 — and so does the kernel's OOM killer, for the same reason. Distinguishing them is a daily skill: `docker inspect` reports `OOMKilled: true` for the memory case; the shutdown-timeout case leaves that false.

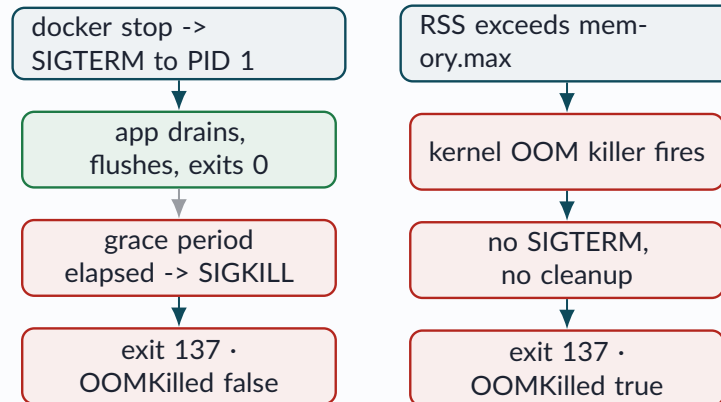


Fig 4.6 – Two roads to exit 137. The exit code is identical; the root cause, and therefore the fix, is not.

The PID 1 problem. If your entrypoint is written in shell form (`ENTRYPOINT java -jar app.jar`), Docker runs it as `/bin/sh -c "java -jar app.jar"`. The shell becomes PID 1, and most shells do *not* forward signals to their children. SIGTERM arrives, the shell ignores it, the JVM never hears it, the grace period elapses, SIGKILL – exit 137 on every single deploy, with connections cut mid-request. The fixes are exec form (`ENTRYPOINT ["java","-jar","app.jar"]`) or, if you genuinely need shell expansion as in the Dockerfile above, an explicit `exec` so the shell replaces itself with the JVM. Additionally, PID 1 has no default signal handlers and does not reap orphaned children, so a container that spawns subprocesses should run a tiny init (`docker run -init`, or `tini`) to avoid accumulating zombies.

♥ Reliability principle

The grace period is a contract between the orchestrator and your application, and both sides must honour it. The platform promises to send SIGTERM first and wait; the application promises to stop accepting new work, finish in-flight requests, flush buffers and exit. If either side defaults, you get truncated writes and 502s during every routine deploy. Day 1 taught you this with `systemd`; Day 6 will enforce it with `terminationGracePeriodSeconds`. It is one idea in three costumes.

📦 Cloud cost optimization

Memory limits are a bill, not just a safety net. Set them too low and you pay in OOMKill restarts and incident time; set them too high – or omit them – and the scheduler reserves capacity nobody uses, so you rent nodes to hold air. Measure actual working-set size under load, set the limit with real headroom above the p99, and revisit it. On Kubernetes this becomes the request/limit distinction on Day 6, where the same numbers directly determine how many nodes you pay for.

✓ Best practices – what experienced engineers do

- Order instructions least-changing first: base, then dependency manifest, then dependency install, then source.
- Multi-stage always – no compiler, package manager or test fixture in a runtime image.
- Pin base images to a specific tag, and to a digest for anything security-sensitive. Never `latest`.
- Run as an explicit numeric non-root UID, and `-chown` during `COPY` rather than after it.
- Commit a `.dockerignore` on day one; it shrinks context, speeds builds and keeps `.git` and secrets out.
- Named volumes for state, bind mounts for development only, `tmpfs` for anything secret.

- Exec-form `ENTRYPOINT` (or an explicit `exec`) so PID 1 receives `SIGTERM`.

✖ Common mistakes beginners make

- `COPY . .` before installing dependencies — every commit reinstalls the world.
- Running as root "for now" and never revisiting it.
- Bind-mounting a host directory as a production database's data directory.
- No `.dockerignore`, so a multi-gigabyte context (including `.git` and `node_modules`) is uploaded on every build.
- Passing secrets via `ARG` or `ENV` and assuming a later `rm` removes them.
- Treating exit 137 as "Docker being flaky" instead of reading `OOMKilled`.

4 Visualizations to Internalize

Redraw these from memory. If you can draw the copy-on-write stack and the two paths to exit 137 on a whiteboard, you can debug almost any container problem you will meet this year.

 Redraw: the overlayfs layer stack — `lowerdir`, `upperdir`, merged — and mark where a copy-up happens and what a whiteout does

 Redraw: the two roads to exit 137 (stop timeout vs OOM kill), and name the one field that tells them apart

5 Guided Hands-On Lab — Containerize CloudBase Properly

Goal. Turn the CloudBase application into an image a Kubernetes cluster would be happy to run: small, pinned, non-root, cache-friendly and correctly signalled. You are practising *decisions*, not typing — at each step, predict the outcome before you measure it.

1 Build a deliberately naive single-stage image and record its size

One `FROM`, `COPY . .`, install everything, run it as root. Note the size and the build time.

Why: You need a baseline you personally produced. "Multi-stage is smaller" is a claim; a number you measured is knowledge, and it is the number you will quote when a colleague asks why the extra stage is worth it.

2 Measure the build context before writing a `.dockerignore`

Watch the "Sending build context to Docker daemon" figure, then add `.dockerignore` excluding `.git`, build output, local env files and dependency directories. Rebuild.

Why: The context is tarred and uploaded to the daemon before the first instruction runs. A repository with deep Git history routinely ships hundreds of megabytes that no layer will ever use — and `COPY . .` would silently bake credentials from a stray `.env` into the image.

3 Split into build and runtime stages

Compile in a JDK stage; copy only the artifact into a JRE stage. Compare against your baseline.

Why: You are separating the two different dependency sets a program has — what it needs to be made, and what it needs to run. Everything in the first set is attack surface and CVE noise if it ships.

4 Deliberately break the cache, then fix it

Put `COPY . .` above the dependency install. Change one character in a source file, rebuild, time it. Move it back below, repeat.

Why: Feeling the difference between a nine-second and a four-minute rebuild is what makes the ordering rule stick. Notice also that the cache miss cascades — every instruction below the miss reruns, regardless of whether its own inputs changed.

5 Add a non-root user and prove enforcement

Create a group and user with fixed UID 10001, `-chown` the artifact, set `USER 10001:10001`. Then `docker exec` in and try to write to `/etc`.

Why: You want to see the permission denial with your own eyes. This is Day 1's ownership model, unchanged — the container did not invent a new one, it just gave you a clean place to apply it.

6 Wire up Compose with a named volume and a healthcheck gate

App plus Postgres on a user-defined network, `pgdata` named volume, `depends_on` gated on `service_healthy`.

Why: Two lessons at once: the app resolves `db` through Docker's embedded DNS rather than an IP, and startup ordering is expressed as a readiness condition rather than a hopeful sleep.

7 Prove persistence, then prove its limit

Insert a row. `docker compose down` then `up` — the row survives. Now `down -v` and repeat.

Why: Knowing precisely which command destroys data is not trivia; it is the difference between a routine restart and an outage. Write the distinction into your runbook while it is fresh.

8 Reproduce both exit codes 137

Run with `-m 64m` and allocate past it; inspect `OOMKilled`. Then use a shell-form entrypoint that ignores `SIGTERM` and `docker stop` it; inspect again.

 **Why:** Two identical exit codes, two completely different fixes — raise the limit versus fix PID 1. Being able to tell them apart in thirty seconds is a senior-engineer signal.

Production tip

Add `docker history <image>` to your routine. It shows every layer, its size and the instruction that created it, so a 400 MB layer with a suspicious command is immediately visible — as is any secret you accidentally passed as a build argument. Reviewing image history should be as normal as reading a diff, and on Day 9 you will make it a CI step alongside vulnerability scanning.

6 Thinking Exercises

Think it through

A container shares the host's kernel, yet the whole point is isolation. What classes of failure does that sharing make possible that a virtual machine would contain — and given that, why did the industry still choose containers as the default unit of deployment?

Think it through

Image layers are content-addressed, so identical content is stored once regardless of who built it. What does that property make possible for a registry, a CI cache and a supply-chain audit — and what new problem does it create when a base image you depend on is found to contain a vulnerability?

Think it through

Bind mounts are convenient in development precisely because they let you bypass the image. Every convenience that bypasses the artifact under test weakens the guarantee the artifact was supposed to give. Where else in a delivery pipeline does that same trade-off appear, and how would you keep the convenience without losing the guarantee?

7

Debugging Practice

 Debugging Lab • Every deploy takes 11 minutes and the container dies at 137 on shutdown

Symptom. CI builds are slow even when only one source file changed, and rolling restarts drop in-flight requests. The container always exits 137 after roughly ten seconds.

docker build output and docker inspect excerpt

```
Sending build context to Docker daemon 1.42GB
Step 3/9 : COPY . .
--> 8a2f1c0d9e11
Step 4/9 : RUN ./mvnw -B dependency:go-offline
--> Running in 4d1c2f7a8b30 # NOT cached - reruns every build

$ docker inspect --format '{{.State.ExitCode}} {{.State.OOMKilled}}' api
137 false
$ docker inspect --format '{{json .Config.Entrypoint}}' api
["/bin/sh","-c","java -jar /app/app.jar"]
```

- ✓ **Observe.** Two independent defects: a 1.42 GB context with an uncached dependency step, and a 137 exit with `OOMKilled` false.
- ✓ **Hypothesise.** `COPY . .` sits above the dependency install, so any source change invalidates it. Separately, PID 1 is `/bin/sh`, which does not forward `SIGTERM`.
- ✓ **Investigate.** Confirm no `.dockerignore` exists and that `.git` dominates the context. Confirm the shell-form entrypoint, and time a `docker stop` — it takes exactly the grace period.
- ✓ **Root cause.** Cache-hostile instruction ordering plus a missing ignore file; and a shell wrapper swallowing the shutdown signal.
- ✓ **Fix.** Copy the dependency manifest and install before copying source; add `.dockerignore`; switch to exec-form `ENTRYPOINT` or prefix the command with `exec`.
- ✓ **Prevent.** Assert build-context size and a maximum image size in CI, and add a shutdown test that stops the container and requires exit code 0 within the grace period.

 Debugging Lab • The database "loses all its data" every few days

Symptom. A staging Postgres periodically comes back empty. Nobody admits to deleting anything, and the Compose file "definitely has a volume".

docker inspect on the db container

```
"Mounts": [
  {
    "Type": "volume",
    "Name": "d41d8cd98f00b204e9800998ecf8427e",
    "Source": "/var/lib/docker/volumes/d41d8cd.../_data",
    "Destination": "/var/lib/postgresql/data",
    "Driver": "local"
  }
]
# note: random hex name -> this is an ANONYMOUS volume, not the named one
```

- ✓ **Observe.** The mount exists and is a volume — but its name is a random hex string, not pgdata.
- ✓ **Hypothesise.** The Postgres image declares `VOLUME /var/lib/postgresql/data`, so when the Compose mapping does not match that exact path the engine creates a fresh anonymous volume per container.
- ✓ **Investigate.** Compare the destination in the Compose file with the image's declared `VOLUME`; list volumes and count how many anonymous ones have accumulated.
- ✓ **Root cause.** A path typo (or a `down -v` in a teardown script) means each recreate gets a brand-new empty volume; the old data is orphaned, not deleted, which is why disk usage kept climbing.
- ✓ **Fix.** Map the named volume to the exact declared path, migrate data from the most recent anonymous volume, then prune the orphans.
- ✓ **Prevent.** Ban `-v` from any shared teardown script, alert on anonymous-volume count, and back up with `pg_dump` to object storage — a volume is durability, not recovery.

8 Mini-Labs

Mini-Lab • Beginner

35 min

Goal. Prove that a container is just a process, with your own eyes.

Business context. Your team keeps calling containers "mini VMs" in design reviews and reasoning badly as a result.

Architecture. One Linux host, one long-running container, two terminals.

Requirements. Find the container's host PID; compare `/proc/<pid>/ns/` on the host against PID 1's; show the same process listed inside and outside with different PIDs.

Constraints. No orchestrator, no Compose — `docker run` and `ps` only.

Hints. `docker inspect` exposes the host PID; `readlink` the namespace links to see the inode numbers.

Expected outcome. A short write-up naming which namespaces differ from the host's and which are shared.

Production considerations. Explain to a colleague why a container cannot run a different kernel version than its host.

Common pitfalls. Concluding that identical inode numbers mean "no isolation" — shared

namespaces are a deliberate default for some modes.

Mini-Lab • Intermediate

60 min

Goal. Cut an image from hundreds of megabytes to tens, and measure every step.

Business context. Pull time dominates cold-start latency during an autoscaling event.

Architecture. One application, four builds: naive, plus-dockerignore, multi-stage, slim-base.

Requirements. Record image size, build-context size, cold build time and warm rebuild time for all four in one table.

Constraints. The application must still start and pass its healthcheck in every variant — shrinking is not a win if it breaks.

Hints. `docker history` attributes size to instructions; try both alpine and a slim glibc base and note any behavioural difference.

Expected outcome. A four-row table plus one paragraph on which change gave the best return per unit of effort.

Production considerations. Estimate the saving across a hundred nodes pulling this image during a scale-out.

Common pitfalls. Comparing warm builds against cold ones; forgetting that removing files in a later layer does not shrink the image.

Mini-Lab • Production-style

110 min

Goal. Ship the CloudBase image the way a platform team would accept it.

Business context. This exact image is what CI will build on Day 9 and what Kubernetes will schedule on Day 5–6; defects here become everyone's defects.

Architecture. Multi-stage Dockerfile → Compose stack (app plus Postgres on a user-defined network with a named volume) → a verification script.

Requirements. Pinned base tags, numeric non-root `USER`, `.dockerignore`, `HEALTHCHECK`, `exec-form` entrypoint, memory and CPU limits, healthcheck-gated `depends_on`.

Constraints. No secret may appear in `docker history`; the app must exit 0 within its grace period on `docker stop`.

Hints. Use a BuildKit secret mount if the build needs a private registry token; verify shutdown with `docker stop` plus an exit-code assertion in the script.

Expected outcome. `make verify` builds, starts, health-checks, writes data, restarts, confirms the data survived, stops cleanly and asserts exit 0.

Production considerations. Which of these checks belong in CI as hard gates, and which are advisory?

Common pitfalls. Named `USER` instead of numeric (breaks `runAsNonRoot` later); `latest` tags; secrets via `ARG`; a healthcheck with no `start-period` that fails during JVM warm-up.

9

Engineering Notes

Infrastructure insight

Docker is no longer the interesting noun. The industry standardised on the **OCI** image and runtime specifications, so what you build is an OCI image and what runs it in a cluster is typically containerd invoking runc — Docker Engine itself became one of several producers and consumers of the same

artifact. Kubernetes removed its Docker-specific shim in v1.24. The practical consequence: build with whatever tool you like (BuildKit, Buildah, Kaniko), because the output is a spec-defined artifact, not a vendor format.

Automation tip

Never let a human decide an image tag. Tag with the immutable Git SHA in CI and let a moving tag like `staging` be a pointer that is updated, never a build input. Deploying by digest (`image@sha256:...`) rather than by tag removes an entire class of "the same tag now means different bytes" incidents — and it is what makes a rollback a genuine return to previously-running code rather than a rebuild and a hope.

Operational excellence

Containers are cattle, but their logs are not. Write to stdout and stderr and let the platform collect them — writing to a file inside the container puts your incident evidence in the writable layer, which is destroyed exactly when the container crashes and you most need it. Day 11's logging pipeline assumes this discipline was established here.

Scalability note

At Netflix's Titus scale, the container platform's hardest problems were not isolation but placement and integration: a leader-elected scheduler assigning containers to a large fleet of EC2 agents, and each container receiving its own VPC network identity, IAM role and security group so that authorization applies to the workload rather than the machine. That reframing — identity belongs to the container, not the host — is exactly what Day 12 formalises with per-workload IAM.

Interview insight

Expect: "How is a container different from a virtual machine?" The weak answer is "it's lighter". The answer that lands: a VM virtualises hardware and runs its own kernel under a hypervisor; a container is a host process with namespaced views and cgroup-bounded resources, sharing the host kernel. Then draw the consequences without being asked — millisecond starts and no guest OS overhead, but a shared kernel attack surface and no ability to run a different kernel or OS family.

10 Build-Along Project — CloudBase

Days 1–3 gave CloudBase a hardened base host with a dedicated service user and a supervised `systemd` unit, a Git repository with branch protection and reviewable conventions, and a network design with routable and private ranges. Until today the application was a jar copied onto a host. From today it is an *artifact*: one immutable, content-addressed image that CI will build (Day 9), Kubernetes will schedule (Day 5–6) and a scanner will gate (Day 12). The `systemd` unit and its restart policy do not disappear — they are replaced by a scheduler that does the same job across many hosts, which is why understanding the unit first made this easy.

Build-Along • Day 04 — containerize the application

- Write `Dockerfile` as a two-stage build: a JDK build stage that compiles, and a pinned slim JRE runtime stage that receives only the artifact via `COPY --from=build`.
- Order instructions for cache reuse — base, dependency manifest, dependency resolution, then source. Prove it: change one source line and confirm the dependency layer is a cache

hit.

- Add `.dockerignore` covering `.git`, build output, local env files and IDE directories. Record the build-context size before and after in `docs/image-baseline.md`.
- Create the numeric non-root user (UID 10001) in the runtime stage, `-chown` the artifact during `COPY`, and set `USER 10001:10001`.
- Pin every base tag explicitly (never `latest`); note the resolved digest of each base image in `docs/image-baseline.md` so the build is auditable.
- Use `exec-form ENTRYPOINT` (or an explicit `exec`) so PID 1 is the JVM and receives `SIGTERM`; add a `HEALTHCHECK` with a `start-period` that tolerates JVM warm-up.
- Write `compose.yaml`: the app plus `postgres:16.4-alpine` on a user-defined network, the app reaching the database as `db` through embedded DNS, a `pgdata` named volume, `depends_on` gated on `service_healthy`, and explicit memory and CPU limits.
- Record the exit-code experiment in `docs/runbook-containers.md`: how to tell an `OOMKill` from a shutdown-timeout 137, and the fix for each.
- Extend `scripts/verify.sh` to build, start the stack, wait for health, write a row, recreate the app container, confirm the row survived, then `docker stop` and assert exit code 0.

Definition of done: `scripts/verify.sh` passes from a clean checkout on a host with nothing cached; the runtime image contains no compiler or package manager, runs as UID 10001, is built from pinned tags, and stops with exit code 0 inside its grace period; `docker history` reveals no secret; and data written to the named volume survives `docker compose down` followed by `up`.

11 Chapter Summary

Key takeaways

- ✓ A container is a Linux process with namespaces (restricted view) and cgroups (restricted budget), sharing the host kernel.
- ✓ An image is an ordered stack of content-addressed read-only layers plus a manifest; a container adds one thin writable layer.
- ✓ overlays copies a file up on first write and masks deletions with whiteouts — so removing files in a later layer never shrinks an image.
- ✓ A cache miss cascades: every instruction below it rebuilds. Order least-changing first.
- ✓ Multi-stage builds keep the toolchain out of production; only `COPY --from` results ship.
- ✓ On a user-defined bridge, embedded DNS resolves container names — connect to `db`, never `localhost`.
- ✓ Named volumes are portable and managed; bind mounts hardcode a host path and bypass the image.
- ✓ Exit 137 means `SIGKILL`. `OOMKilled` tells you whether the cause was memory or an ignored `SIGTERM`.

Things to remember

- Shell-form `ENTRYPOINT` makes `/bin/sh` PID 1, and it will not forward `SIGTERM`.
- Numeric `USER` (not a name) is what Kubernetes' `runAsNonRoot` check can verify.
- The build context is uploaded before the first instruction runs — `.dockerignore` is not optional.
- Build-time `ARG` values are visible in `docker history`; use BuildKit secret mounts instead.

Common pitfalls (last look)

- `COPY . .` too early
- running as root
- latest tags
- no `.dockerignore`
- Bind mount for production state
- secrets in `ARG`
- `down -v` in a shared script

📄 One-Page Cheat Sheet

Container — a host process + namespaces (view) + cgroups (budget) + an image rootfs. One shared kernel.

Layer — immutable, content-addressed tar of filesystem changes; deduplicated by digest everywhere.

Copy-on-write — lowerdir (image) + upperdir (container) = merged; copy-up on first write, whiteout on delete.

Build cache — `RUN` keyed on the command, `COPY` on file checksums; one miss invalidates everything below.

Multi-stage — several `FROM`s; only the last stage ships; BuildKit runs independent stages in parallel.

Networking — user-defined bridge + veth pair + iptables NAT; embedded DNS at 127.0.0.11 resolves service names.

Storage — named volume = portable managed state; bind mount = host path, dev only; tmpfs = RAM, secrets.

Exit 137 — 128 + `SIGKILL`. `OOMKilled:true` = memory limit; `false` = grace period expired.

CLI quick reference

Day 4 commands worth memorising

```
# build & inspect
docker build --target runtime -t cloudbase-api:$(git rev-parse --short HEAD) .
docker history cloudbase-api:dev           # size per layer + the instruction that made it
docker image inspect --format '{{.RootFS.Layers}}' cloudbase-api:dev

# what actually happened to a container
docker inspect --format '{{.State.ExitCode}} {{.State.OOMKilled}}' api
docker logs --since 10m --timestamps api
docker stats --no-stream                   # live cgroup usage per container

# it is just a process
docker inspect --format '{{.State.Pid}}' api      # host PID
sudo ls -l /proc/<hostpid>/ns/                  # one link per namespace

# networking & storage
docker network inspect cloudbase-net
docker exec api getent hosts db                # embedded DNS resolving a service name
docker volume ls && docker volume inspect pgdata
```

```
# lifecycle & cleanup
docker stop -t 30 api           # SIGTERM, wait 30s, then SIGKILL
docker system df               # where the disk went
docker image prune -a --filter "until=168h"
```

Interview questions to master

- › What is a container, precisely, at the kernel level — and how does it differ from a virtual machine?
- › Why do multi-stage builds reduce both final image size and attack surface?
- › What is the practical difference between a named volume and a bind mount?
- › How do containers on the same user-defined bridge network resolve each other?
- › Why does layer ordering in a Dockerfile matter for build speed, and what exactly invalidates a layer?
- › What happens to data written inside a container with no volume when it is removed?
- › A container exits with code 137. Walk me through your diagnosis.
- › Why does `RUN rm -rf` in a later layer fail to shrink an image?

Further reading & official docs

- Docker Docs — "Building best practices" (docs.docker.com/build/building/best-practices/): layer ordering, `.dockerignore`, multi-stage, digest pinning, non-root `USER`.
- Docker Docs — "Optimize cache usage in builds" (docs.docker.com/build/cache/optimize/): cache mounts, and why apt needs `sharing=locked`.
- Docker Docs — "Use the OverlayFS storage driver": `lowerdir/upperdir/merged`, `copy-up`, `whiteouts`, `page-cache` sharing.
- `man 7 namespaces` and `man 7 cgroups` — the authoritative kernel-side description of both mechanisms.
- The Linux kernel documentation for Control Group v2 — `memory.max`, `memory.high`, `cpu.max` and `memory.oom.group`.
- The OCI Image Specification and Runtime Specification (github.com/opencontainers) — what an image and a runtime actually are, independent of Docker.
- Netflix TechBlog — "Titus, the Netflix container management platform, is now open source": scheduler, agents, and AWS VPC/IAM integration per container.
- Uber Engineering — "Makisu" and "uBuild": why unprivileged image builds and a distributed layer cache matter at 100k builds per week.

Production checklist

- ☐ Every Dockerfile is multi-stage; no compiler or package manager in the runtime image.
- ☐ Every base image is pinned to an explicit tag, and to a digest where supply chain matters.
- ☐ Every image runs as an explicit numeric non-root UID.
- ☐ A `.dockerignore` exists and the build context is measured, not assumed.
- ☐ `ENTRYPOINT` is `exec` form (or `exec`-prefixed) so PID 1 receives `SIGTERM`.

- ☐ Memory and CPU limits are set from measured usage, with headroom above p99.
- ☐ All state is on named volumes, and every volume has a backup path that is not the volume.
- ☐ Logs go to stdout and stderr, never to a file in the writable layer.
- ☐ No secret is ever passed as `ARG` or baked into `ENV`.

Key idea

What you will learn next (Day 5). You now have one immutable artifact and a way to run it on one host. Tomorrow you hand that artifact to something that runs it on *many* hosts and keeps it running: Kubernetes' architecture — the API server as the single source of truth, etcd as its store, controllers reconciling desired state against observed state, and the scheduler placing Pods. Every container concept from today reappears there wearing a different name, which is exactly why today had to come first.

DAY 05 • ORCHESTRATION

Kubernetes Fundamentals

05

Control plane, scheduling, and the reconciliation loop that runs everything

🕒 Estimated study time: 9 hours

🎯 By the end of this day, you will be able to...

- Explain what each control-plane component does, and why the API server is the only writer to `etcd`.
- Describe the reconciliation loop precisely enough to predict what the cluster will do next.
- Write a Deployment, Service, ConfigMap and Secret that a reviewer would approve.
- Trace how a Service finds its Pods through labels, EndpointSlices and `kube-proxy` — not a fixed IP list.
- Separate liveness from readiness correctly, and explain why confusing them causes fleet-wide restart storms.

1 The Big Picture

What problem this solves. On Day 4 you produced a hardened container image. An image is inert. To run it in production you must answer questions a container runtime does not answer: which machine should this run on, how many copies, what happens when the machine dies at 3am, how does traffic find the copies when every restart gives them a new IP, how do I change the image version without dropping requests, and how does configuration reach the process without being baked into the image. Kubernetes exists to answer exactly that list, once, for every workload, in a way that survives the humans who wrote it leaving the company.

Why DevOps engineers need it. Before orchestrators, the answers were a pile of bespoke glue: a Jenkins job that SSHed into a named host, a load balancer config edited by hand, a spreadsheet mapping services to machines, a pager rotation whose job was to be the failover mechanism. That glue is unversioned, untested and known only to its author. Kubernetes replaces "a human executes a procedure" with "a controller enforces a declaration" — and once you understand that substitution, every object in the API stops being trivia and becomes an instance of the same idea.

🔑 Key idea

Kubernetes is not a deployment tool. It is a database of desired state (`etcd`, behind an API server) plus a set of programs that continuously work to make reality match that database. Everything else — Deployments, Services, probes, autoscalers, operators you have never heard of — is a controller playing that same game. Learn the game once; the objects become vocabulary.

Where it fits in a modern cloud architecture. Kubernetes sits directly above the container runtime you met yesterday and directly below the delivery pipeline you will wire up later. It consumes images and declarations; it produces running, addressable, self-healing workloads.

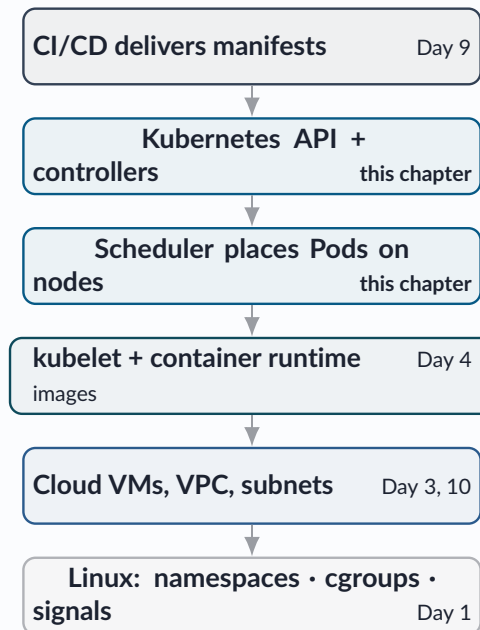


Fig 5.0 — Where today sits. Kubernetes turns a static image plus a declaration into a running, addressable, self-healing workload.

How it connects to previous chapters. Day 1 gave you processes, signals and the SIGTERM-then-SIGKILL shutdown path — Kubernetes uses precisely that path when it evicts a Pod. Day 3 taught you that a TCP handshake completing tells you a socket is listening and nothing more; today that lesson becomes the difference between a probe that protects users and a probe that lies. Day 4 gave you an immutable image with a non-root user and a small attack surface; today that image finally gets somewhere to run.

Where companies use it in production. Spotify ran a homegrown orchestrator called Helios on top of Docker for years, then migrated to Kubernetes from late 2018 onward — explicitly because a small internal team could not keep pace with the feature velocity of the wider community. Their published case study reports service provisioning dropping from roughly an hour to seconds or minutes, and CPU utilisation improving by a factor of two to three through bin-packing and multi-tenancy. That second number is the whole economic argument for orchestration: a scheduler packing many workloads onto shared nodes uses hardware far better than one-service-per-VM ever could. Airbnb runs a multi-cluster fleet and introduced a *cluster type* abstraction so that clusters of the same type share one configuration, letting heterogeneous workloads share nodes for better bin-packing while upgrades roll one cluster at a time.

Common misconceptions. That Kubernetes "runs your containers" (the `kubelet` does; Kubernetes decides *what should be running*); that `kubectl apply` deploys (it writes a desired state and returns — controllers do the work asynchronously); that a Secret is encrypted (by default it is base64-encoded and stored in plain text in `etcd`); that a Service is a load balancer process traffic flows through (it is usually a set of packet-rewriting rules on every node).

2

Concept Map

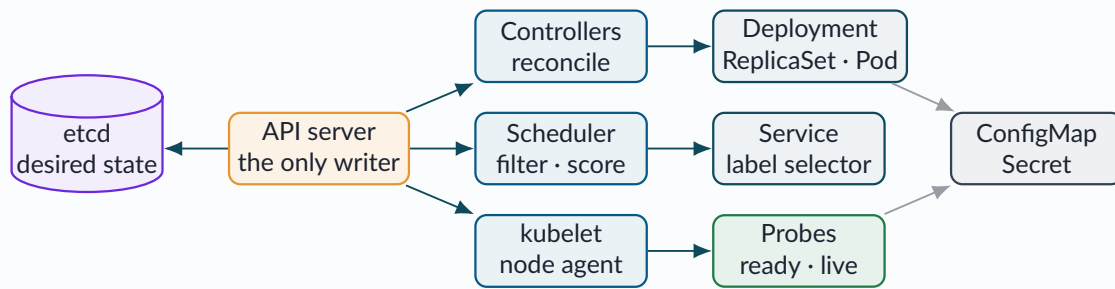


Fig 5.1 — The Day 5 territory. Everything on the right is a consequence of the loop in the middle.

Terminology to anchor: **desired state** → **API server** → **etcd** → **watch** → **controller** → **reconciliation** → **scheduler** → **kubelet** → **label selector** → **EndpointSlice** → **probe**.

3 Guided Learning

3.1 The Control Plane: Who Decides, Who Runs

What is it? A Kubernetes cluster splits into two halves. The **control plane** decides what should be running. The **worker nodes** run it. A node can host control-plane components too, but the responsibilities never blur: nothing on a worker node ever decides where a Pod belongs.

Why does it exist? Because the alternative — every machine deciding for itself — has no way to answer global questions. "Are three replicas running?" is unanswerable if there is no single place that knows what "three replicas" means. Centralising *desired state* while distributing *execution* is the classic split, and it is exactly what Google's Borg did before Kubernetes existed. Before orchestrators, teams centralised state in a wiki page and distributed execution to on-call engineers.

How does it work internally? Five components matter today.

Component	Responsibility, and the detail that matters
kube-apiserver	The only component that reads or writes <code>etcd</code> . Every <code>kubectl</code> command, every controller, every <code>kubelet</code> goes through it. It authenticates, authorises, runs admission control, validates and persists.
etcd	A distributed key-value store using the Raft consensus algorithm. It holds the whole cluster's declared state. Lose it without a backup and you have lost the cluster's intent, not just its uptime.
kube-scheduler	Watches for Pods with no node assigned; picks one. It does not start anything — it only writes a node name into the Pod object.
kube-controller-manager	A single binary hosting many independent control loops (Deployment, ReplicaSet, Node, Job, EndpointSlice, and more).
kubelet	The node agent. Watches the API server for Pods assigned to <i>its</i> node, tells the container runtime to start them, and reports status back.

👁 Behind the scenes

`kubectl apply -f deploy.yaml` does not deploy anything. It performs an HTTP request to the API server, which authenticates you, checks RBAC, passes the object through mutating then validating

admission plugins, validates it against the schema, and writes it to `etcd`. Then it returns `200`. At that instant nothing is running. Everything that follows — ReplicaSet created, Pods created, node chosen, container pulled and started — happens asynchronously in controllers reacting to that write. This is why `kubectl apply` succeeding tells you almost nothing about whether your app works.

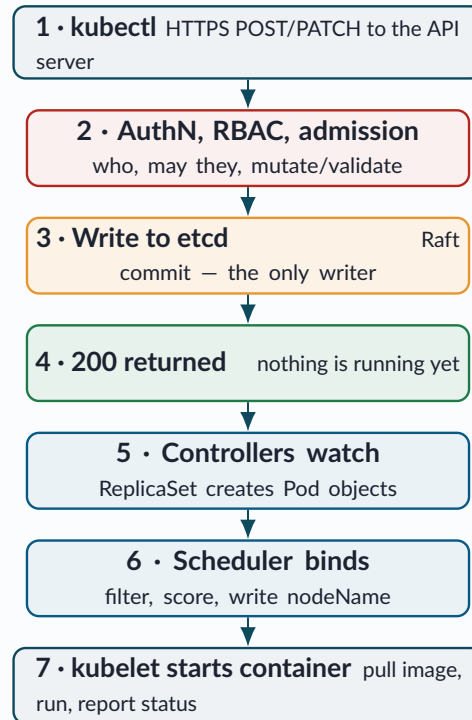


Fig 5.2 — The path of one `apply`. Note that the client is done at step 4; everything after is a reaction.

How the scheduler actually chooses. Scheduling is a two-step operation per Pod. **Filtering** eliminates every node on which the Pod *cannot* run: insufficient allocatable CPU or memory for the Pod's `requests`, a taint the Pod does not tolerate, a node selector or affinity rule that does not match, a required volume that cannot attach on that node. If filtering leaves an empty set, the Pod stays `Pending` and the event log says so. **Scoring** then ranks the survivors — spreading across nodes and zones, preferring nodes that already have the image, balancing resource usage — and the highest score wins, with ties broken at random. The result is written back as a binding: a node name on the Pod object.

⚠ Common misconception

The scheduler uses `requests`, not actual usage, when filtering. A Pod that requests 2 CPUs but uses 0.1 still occupies 2 CPUs of schedulable capacity. This is why a cluster can show 20% CPU utilisation on the dashboards and still refuse to schedule anything: you have not run out of CPU, you have run out of *promised* CPU. Over-requesting is the single most common cause of an expensive, idle, "full" cluster.

👤 Security warning • etcd is the crown jewels

Every Secret in the cluster lives in `etcd`, and by default the API server stores plain-text representations there with no at-rest encryption. Anyone with a copy of an `etcd` snapshot has every credential in the cluster. Production clusters enable an `EncryptionConfiguration` on the API server (`-encryption-`

provider-config) so that at minimum `secrets` are encrypted — ideally with a `kms` provider backed by a cloud KMS rather than a key sitting in a file next to the data it protects.

3.2 The Reconciliation Loop — The Idea Everything Else Is Built On

What is it? You never tell Kubernetes *how* to do something. You declare the end state you want, and a controller repeatedly compares that declaration with observed reality and takes one small action to narrow the gap. Then it does it again. Forever.

Why does it exist? Because imperative automation is not resilient to the world changing underneath it. A script that says "start three copies" is correct at the instant it runs and wrong forever after — when a node dies, nobody re-runs it. Declarative reconciliation is level-triggered rather than edge-triggered: it does not care *why* reality diverged, only that it did. A missed event, a network partition, a node reboot, a human running `docker kill` — all converge back to the same state, because the loop reads current reality every pass instead of trusting a memory of past events.

How does it work internally? Controllers do not poll the API server in a loop; that would not scale. They open a **watch** — a long-lived streaming connection — and receive add/update/delete events, which they use to maintain a local in-memory cache (the informer). Changed objects are pushed into a work queue, and worker goroutines dequeue an object key, read the *current* state of the world from the cache, compute the difference against desired state, and issue at most a small corrective write. Failures go back on the queue with exponential backoff. The result is idempotent by construction: running the same reconcile twice does nothing the second time.

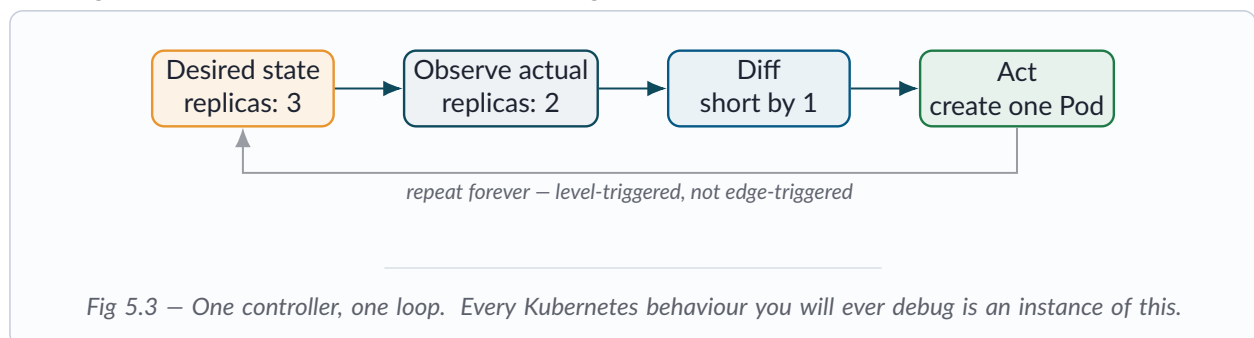


Fig 5.3 — One controller, one loop. Every Kubernetes behaviour you will ever debug is an instance of this.

When should I use it? Whenever you find yourself writing automation that says "when X happens, do Y". Rewrite it as "state Z should hold" and let a loop enforce it. This is the mental shift that carries into Terraform on Day 7 and Ansible on Day 8 — both are the same idea with different blast radii.

When should I not? One-shot, genuinely imperative operations: a database migration, a data backfill, a report. Kubernetes models these as Jobs precisely because they are *not* reconciled to a steady state — their desired state is "has completed once", not "is running".

💡 Senior engineer insight

When something in Kubernetes surprises you, ask three questions in order: what is the desired state (`kubectl get -o yaml`), what is the observed state (`kubectl describe`, `kubectl get events`), and which controller is responsible for closing that gap? Nine incidents in ten resolve at question two, because the controller has already written the reason into an Event and nobody read it. Juniors go straight to `logs`; there are no logs when a Pod never started.

3.3 Pods, ReplicaSets and Deployments

What is it? A **Pod** is the smallest schedulable unit: one or more containers that share a network

namespace (so they reach each other on `localhost`), share storage volumes, and are always placed on one node together. A **ReplicaSet** keeps N identical Pods alive. A **Deployment** owns ReplicaSets and orchestrates the transition from one to the next, which is what makes a rolling update possible.

Why does it exist? The three-layer split looks like bureaucracy until you need a rollback. Because each image version gets its own ReplicaSet, "roll back" is not a redeploy from source — it is scaling the previous ReplicaSet up and the current one down, using an image that already exists and Pods whose spec is already known good. Before this, rollback meant re-running the build pipeline on an old commit and hoping the result was byte-identical.

Object	What it is	Why it exists
Pod	One or more containers sharing network and storage, scheduled as a unit	The atom of scheduling; ephemeral by design
ReplicaSet	Keeps exactly N Pods matching a selector alive	Self-healing; you rarely write one by hand
Deployment	Owns ReplicaSets; performs rolling updates and rollbacks	The standard way to run a stateless app
Service	Stable virtual IP and DNS name in front of a label-selected set of Pods	Pod IPs change constantly; clients need one that does not
ConfigMap	Non-secret key/value config, injected as env vars or files	Decouples config from the image
Secret	Same shape, access-controlled separately, base64-encoded	Credentials, tokens, keys
Namespace	A scope for names, quotas and policy	Separating teams and environments on shared infrastructure

k8s/deployment.yaml — CloudBase API

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: cloudbase-api
  namespace: cloudbase
  labels: { app: cloudbase-api }
spec:
  replicas: 3
  revisionHistoryLimit: 5
  selector:
    matchLabels: { app: cloudbase-api }
  template:
    metadata:
      labels: { app: cloudbase-api }
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
      containers:
        - name: api
          image: ghcr.io/aymix/cloudbase-api:1.4.2
          ports:
            - name: http
              containerPort: 8080
          envFrom:

```

```

- configMapRef: { name: cloudbase-config }
- secretRef:    { name: cloudbase-db }
resources:
  requests: { cpu: "100m", memory: "256Mi" }
  limits:   {               memory: "512Mi" }

```

How does it work internally? (the rolling update) Editing the Pod template creates a new ReplicaSet with a new pod-template-hash label. The Deployment controller then walks both ReplicaSets toward their targets under two constraints: `maxUnavailable` (how far below `replicas` you may dip) and `maxSurge` (how far above you may go). Default is 25% each, so a 4-replica Deployment runs between 3 and 5 Pods during the transition. New Pods only count toward availability once they pass their *readiness* probe — which is what actually makes a rolling update safe, and what makes a missing readiness probe silently turn a rolling update into a brief outage.

Design trade-off

Pinning an image by mutable tag (`:latest`, `:main`) makes the manifest look stable while the running software changes underneath you — and worse, it breaks rollback, because the old ReplicaSet's tag now resolves to the new image too. Pinning by digest (`@sha256:...`) is exactly reproducible but makes every release a manifest change, which is only pleasant if a pipeline writes it for you. Production teams generally pin an immutable semantic tag in Git and let CI update it; the digest is recorded in the release notes, not typed by hand.

Common misconception

A bare Pod has no controller watching it. If the node it sits on is drained, cordoned or dies, that Pod is gone permanently — nothing recreates it. `kubectl run` exists for a one-off debugging shell and essentially nothing else. Similarly, editing a live Pod (`kubectl edit pod ...`) changes something the Deployment did not write, so the next reconcile or the next replacement erases it. Change the Deployment, always.

3.4 How a Service Actually Finds Its Pods

What is it? A Service is a stable virtual IP and a DNS name that fronts a dynamic set of Pods. Crucially, a Service does not know which Pods exist. It declares a **label selector**, and membership is computed continuously by a controller.

Why does it exist? Pod IPs are ephemeral — a rolling update replaces every IP in the set within a minute. Any client holding a Pod IP holds a value that will be wrong shortly. Before Kubernetes, teams solved this with a service-discovery sidecar plus a load balancer whose backend pool was edited by a deploy script; the failure mode was a load balancer still routing to terminated hosts.

How does it work internally? There is no proxy process in the request path for a normal `ClusterIP` Service. The chain is:

Step	What happens
Label match	The EndpointSlice controller watches Pods, selects those matching the Service's selector, and keeps only the ones that are <i>Ready</i> .
EndpointSlice	Those Pod IPs and ports are written into EndpointSlice objects (which replaced the older single Endpoints object so large Services do not ship one huge object on every change).
kube-proxy	On every node, <code>kube-proxy</code> watches EndpointSlices and programs kernel rules — iptables or IPVS — that DNAT the Service's virtual IP to one of the backend Pod IPs.
CoreDNS	Resolves <code>cloudbase-api.cloudbase.svc.cluster.local</code> to the Service's virtual IP, so clients use a name, never an IP.

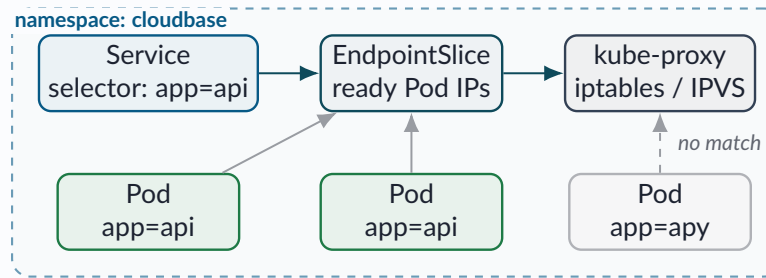


Fig 5.4 — A Service is an indirection through labels, not a list of IPs. Break the label match and the middle of this chain empties out.

Service type	What it gives you
ClusterIP	A virtual IP reachable only inside the cluster. The default, and correct for most internal services.
NodePort	Additionally opens a high port on every node. Mostly a building block, and a blunt one.
LoadBalancer	Asks the cloud provider for an external load balancer pointing at the NodePort. One cloud LB per Service — which is why Ingress exists (Day 6).
Headless (clusterIP: None)	No virtual IP; DNS returns the Pod IPs directly. For clients that must address individual replicas, e.g. databases.

k8s/service.yaml — stable front door for the Deployment

```

apiVersion: v1
kind: Service
metadata:
  name: cloudbase-api
  namespace: cloudbase
spec:
  type: ClusterIP
  selector:
    app: cloudbase-api    # must match the POD labels, not the Deployment's
  ports:
    - name: http
      port: 80             # the stable port clients call
      targetPort: http     # the named containerPort on the Pod
  
```

💡 Senior engineer insight

Point `targetPort` at a *named* port rather than a number. The Service then keeps working when the app moves from 8080 to 9090, because the name travels with the container spec. It also documents intent in a way a bare integer never does. This costs nothing and removes an entire category of "the Service is up but returns connection refused" incidents.

⚠️ Common misconception

The Service's selector matches *Pod* labels, not the Deployment's `metadata.labels`. These are different fields, and a manifest can easily have them disagree. When they do, `kubectl get svc` looks perfectly healthy — the Service exists, it has a ClusterIP — and every request times out, because the EndpointSlice is empty. `kubectl get endpointslices` is the two-second check that ends this argument.

3.5 ConfigMaps, Secrets and the Twelve-Factor Line

What is it? A ConfigMap holds non-sensitive key/value configuration; a Secret holds sensitive values. Both can be injected as environment variables or mounted as files.

Why does it exist? Because the same image must run in dev, staging and production. If configuration is baked into the image, "promote to production" means rebuild, which means the artifact you tested is not the artifact you shipped. Day 4's immutability promise only holds if configuration enters at runtime.

How does it work internally? The difference between the two injection styles matters more than most people realise. Environment variables are read once, at process start — the container's environment block is fixed by the kernel at `exec` time, so updating the ConfigMap changes nothing until the Pod is recreated. A *volume-mounted* ConfigMap or Secret, by contrast, is projected by the kubelet into a tmpfs directory and refreshed periodically (on the order of a minute, plus the kubelet's cache sync), so a running process that re-reads the file can pick up changes without a restart.

k8s/config.yaml — ConfigMap and Secret

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cloudbase-config
  namespace: cloudbase
data:
  APP_ENV: "production"
  LOG_LEVEL: "info"
  DB_HOST: "cloudbase-db.cloudbase.svc.cluster.local"
---
apiVersion: v1
kind: Secret
metadata:
  name: cloudbase-db
  namespace: cloudbase
type: Opaque
stringData:
  # plain text here; the API server base64-encodes it
  DB_USER: "cloudbase"
  DB_PASSWORD: "replaced-by-external-secrets-in-production"
```

Security warning • base64 is encoding, not encryption

`echo cGFzc3dvcmQ= | base64 -d` is not an attack; it is a decoding. A Secret is safer than a ConfigMap for three real reasons — it can be RBAC-scoped separately, it is kept out of most logging and `describe` output, and the API server can be configured to encrypt it at rest — but the value itself is not protected by base64 in any sense. Never commit a real Secret manifest to Git. Production teams inject values from an external store (AWS Secrets Manager, Vault) via the External Secrets Operator or the Secrets Store CSI driver, so the credential exists in the cluster only as a projected file with a short life.

Automation tip

Add a checksum of the ConfigMap to the Pod template's annotations in your templating layer. Changing config then changes the Pod template hash, which creates a new ReplicaSet, which performs a normal rolling restart. Without this, a config change to an env-var-consuming app is a no-op until somebody manually restarts the Deployment — and "I applied it and nothing happened" is one of the most common Kubernetes confusions.

3.6 Probes: Readiness Gates Traffic, Liveness Restarts

What is it? Three probe types, run by the `kubelet` against each container. **Readiness** answers "may this Pod receive traffic right now?" — failing it removes the Pod from EndpointSlices, and nothing is restarted. **Liveness** answers "is this container unrecoverably stuck?" — failing it kills and restarts the container. **Startup** answers "has it finished booting?" — while it is failing, the other two are suspended entirely.

Why does it exist? Recall Day 3: a completed TCP handshake proves a socket is bound and the kernel accepted a connection. It proves nothing about the application. A JVM that has bound port 8080 but is still warming caches, loading a schema and opening a connection pool will accept your TCP connection and then sit on the request for forty seconds. Without a readiness probe, Kubernetes would route production traffic there the moment the container started, and every rolling update would drop requests. Before probes, teams solved this with a sleep in the deploy script and a prayer.

How does it work internally? The `kubelet` — not the control plane — executes probes locally against the container. Each probe has `initialDelaySeconds`, `periodSeconds`, `timeoutSeconds`, `failureThreshold` and `successThreshold`. On readiness failure, the `kubelet` flips the Pod's `Ready` condition to false; the `EndpointSlice` controller notices and removes the Pod's IP; `kube-proxy` rewrites the node's rules. On liveness failure, the `kubelet` kills the container (SIGTERM then SIGKILL — exactly Day 1's path) and the restart policy brings it back, with the familiar exponential `CrashLoopBackOff` delay.

deployment.yaml — the probe block that belongs on every container

```
startupProbe:           # slow boot? this buys time safely
  httpGet: { path: /healthz, port: http }
  periodSeconds: 5
  failureThreshold: 30  # up to 150s to start, then give up
readinessProbe:         # gates traffic; checks dependencies
  httpGet: { path: /readyz, port: http }
  periodSeconds: 5
  timeoutSeconds: 2
  failureThreshold: 3
livenessProbe:          # restarts; checks ONLY the process
  httpGet: { path: /healthz, port: http }
  periodSeconds: 10
  timeoutSeconds: 2
  failureThreshold: 3
```

Question	Readiness (/readyz)	Liveness (/healthz)
On failure	Removed from the Service	Container restarted
Check the database?	Yes — if you cannot serve, say so	Never
Check the cache, queue, downstream API?	Yes, if requests genuinely need them	No
Should it ever be slow or heavy?	Keep it cheap; it runs constantly	Keep it trivial
What it really means	"send me work"	"I am not deadlocked"

♥ Reliability principle

The rule that prevents the worst Kubernetes outage there is: **a liveness probe must never check an external dependency**. Suppose your liveness probe queries the database. The database has a thirty-

second blip. Every replica's liveness probe fails simultaneously, so the kubelet restarts every container in the fleet at once. Restarting containers reconnect on cold pools, all at the same instant, and hammer the recovering database — which fails again. You have converted a brief dependency degradation into a self-sustaining, cluster-wide `CrashLoopBackOff` that outlives the original fault. The correct design puts the dependency check in *readiness*: traffic drains away, nothing restarts, and when the database returns, every Pod becomes Ready again within one probe period with no cold starts.

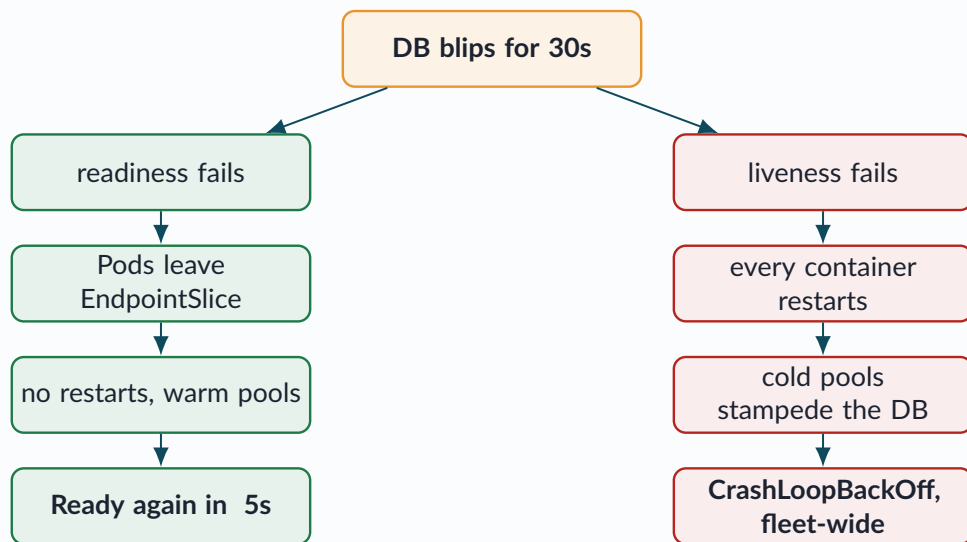


Fig 5.5 — Same failure, two probe designs. The left column is a recovery; the right is an outage you caused.

When should I not use a liveness probe? More often than you would guess. If your application has no plausible unrecoverable-but-still-running state — no deadlock-prone thread pool, no known hang — a liveness probe adds risk without adding recovery. A crashed process is already restarted by the restart policy; liveness only helps for the narrow case of "alive but permanently wedged". A conservative default is: always a readiness probe, a startup probe if boot is slow, and a liveness probe only where you can name the hang it catches.

⚡ Performance note

Probe timing is a budget you should compute, not guess. Time to restart on liveness failure is roughly `initialDelaySeconds` plus `failureThreshold × periodSeconds`, plus up to one `timeoutSeconds`. With the defaults above that is about 30 seconds — deliberately longer than a GC pause or a brief lock contention spike, and deliberately shorter than a human noticing. If your probe endpoint takes longer than `timeoutSeconds` under load, the probe fails under load, which is precisely when you least want a restart.

✓ Best practices — what experienced engineers do

- Deploy through a Deployment, never a bare Pod, outside a debugging session.
- `kubectl describe` before `kubectl logs` — scheduling and image failures appear in Events, and there are no logs for a container that never ran.
- Set resource `requests` on every container so the scheduler can do its job; set a memory `limit` to bound the blast radius.
- Readiness probes check dependencies; liveness probes check only the process.

- Keep config in ConfigMaps and Secrets, never baked into the image, and never a real credential in Git.
- Use namespaces to separate environments, and attach quotas so one team cannot consume the cluster.

✖ Common mistakes beginners make

- Putting a database check in the liveness probe – the fleet-wide restart storm above.
- Editing a running Pod instead of the Deployment; the change vanishes on the next replacement.
- Assuming a Service holds a fixed list of Pod IPs rather than a live label selector.
- Storing real credentials in a ConfigMap, or assuming base64 in a Secret is encryption.
- Omitting resource requests, so the scheduler packs blindly and one Pod starves its node.
- Believing `kubectl apply` succeeding means the application is running.

4 Visualizations to Internalize

Redraw these from memory. If you can draw the second one under pressure, you can debug most Kubernetes networking incidents without opening a browser.

 Redraw: the path of one `kubectl apply` – auth, admission, etcd, then which components react and in what order

 Redraw: Service → label selector → EndpointSlice → kube-proxy → Pod, and mark exactly where a readiness failure removes a Pod

5

Guided Hands-On Lab — Deploy CloudBase to a Cluster

Goal. Take yesterday's hardened CloudBase image and give it a real home: a namespace, a Deployment with requests and limits, a Service, externalised configuration, and probes that are correct rather than merely present. Use `kind` or `minikube` locally — the manifests are identical against EKS on Day 10.

1 Create a cluster and a namespace, and set your context

Stand up a local cluster, create the `cloudbase` namespace, and make it your default context namespace.

Why: Working in `default` is how manifests end up in the wrong place. Namespaces are also where quotas and NetworkPolicies attach later — getting one from the start costs nothing.

2 Write the Deployment with requests and limits from the start

Three replicas, the pinned image tag from Day 4, `runAsNonRoot`, a CPU request and a memory request plus a memory limit.

Why: Requests are what the scheduler filters on; without them it cannot reason about capacity and will happily overcommit a node into OOM. Set the memory limit but consider leaving CPU unlimited — a CPU limit throttles, a memory limit kills.

3 Apply it and watch the reconciliation happen

Apply, then immediately watch Pods appear. Note the Pod name structure: Deployment name, ReplicaSet hash, Pod suffix.

Why: That name is the ownership chain made visible. Reading it tells you which ReplicaSet a Pod belongs to, which is how you tell old from new mid-rollout.

4 Delete a Pod by hand and time the replacement

Delete one Pod and watch what happens within seconds.

Why: This is the reconciliation loop, observed. Nothing rescheduled a job or fired a webhook — the ReplicaSet controller simply noticed 2 does not equal 3.

5 Add the Service and prove the label indirection

Create the ClusterIP Service, confirm endpoints exist, then deliberately change the selector to a label no Pod carries and observe the endpoints empty out.

Why: You are building the reflex to check `endpointslices` when a Service "is up" but nothing responds. Break it on purpose now, so the muscle memory exists at 3am.

6 Externalise configuration into a ConfigMap and Secret

Move environment settings to a ConfigMap and credentials to a Secret, injected via `envFrom`.

Why: The same image must be promotable across environments unchanged. This is the boundary that makes the Day 4 immutability promise real rather than aspirational.

7 Change the ConfigMap and observe that nothing happens

Edit a value, apply, and confirm the running Pods still serve the old value. Then restart the Deployment and watch it take effect.

Why: Environment variables are fixed at process start. Discovering this deliberately, once, is far cheaper than discovering it during an incident where you are certain you already applied the fix.

8 Add readiness, liveness and startup probes — then break them

Add all three. Then temporarily point the liveness probe at an endpoint that depends on the database, stop the database, and watch the restart storm form.

 **Why:** Reading about restart storms convinces nobody. Watching every replica enter `CrashLoopBackOff` because of one probe line is the lesson that sticks for a career.

Production tip

Get into the habit of `kubectl rollout status deployment/cloudbase-api` instead of watching Pods. It exits non-zero if the rollout stalls or times out, which makes it the correct thing for a CI job to run — and it is a natural gate for Day 9's pipeline. A deploy step that cannot fail is not a deploy step.

6 Thinking Exercises

Think it through

The API server is the only component permitted to read or write `etcd`; everything else goes through its HTTP API even though the store is right there. What does this buy that direct `etcd` access from each controller would lose — and what does it cost?

Think it through

Kubernetes controllers are level-triggered: each pass reads the current state of the world rather than trusting a record of past events. Design the opposite — an edge-triggered orchestrator that acts only on events. Name three concrete situations in which it silently ends up in the wrong state.

Think it through

A readiness probe removes a Pod from the Service; a liveness probe restarts it. Now imagine Kubernetes had shipped with only one probe type that did both. Which real failures would that single probe handle acceptably, and which would it turn into an outage?

7

Debugging Practice

 Debugging Lab • Every replica in CrashLoopBackOff after a brief database hiccup

Symptom. A managed database failed over for about forty seconds. Twenty minutes later the database is healthy but all twelve API Pods are still restarting, and the error rate is 100%.

```
kubectl get pods -n cloudbase and kubectl describe pod
```

NAME	READY	STATUS	RESTARTS	AGE
cloudbase-api-7d9f4c8b6-2xk4m	0/1	CrashLoopBackOff	9 (24s ago)	3h
cloudbase-api-7d9f4c8b6-9jqvt	0/1	CrashLoopBackOff	9 (11s ago)	3h
cloudbase-api-7d9f4c8b6-hb2rn	0/1	Running	8 (41s ago)	3h

Events:

```
Warning Unhealthy 2m (x27 over 19m) kubelet Liveness probe failed:
HTTP probe failed with statuscode: 503
Normal Killing 2m (x9 over 18m) kubelet Container api failed
liveness probe, will be restarted
```

- ✓ **Observe.** The kill decision comes from the *liveness* probe returning 503, and it has repeated nine times per Pod. The database recovered long ago.
- ✓ **Hypothesise.** The liveness endpoint checks the database. The original blip failed it everywhere at once; each restart then starts with a cold connection pool, and the restarts themselves keep the dependency saturated.
- ✓ **Investigate.** Read what `/healthz` actually does in the application code, and compare it to `/readyz`. Check whether both point at the same handler — they very often do.
- ✓ **Root cause.** An external dependency check inside a liveness probe converts a transient dependency fault into a self-sustaining, fleet-wide restart loop.
- ✓ **Fix.** Point liveness at a trivial in-process handler that touches nothing external; move the database check to readiness. Raise `failureThreshold` so a single slow probe cannot kill a container.
- ✓ **Prevent.** Make it a review rule: liveness endpoints may not perform I/O to anything outside the process. Add a startup probe so slow boots never reach the liveness probe at all.

 Debugging Lab • The Service is healthy and every request times out

Symptom. `kubectl get svc` shows the Service with a ClusterIP. Pods are 1/1 Running. Requests from another Pod hang until they time out.

kubectl get endpointslices and the two label blocks

```

NAME                ADDRESSTYPE  PORTS  ENDPOINTS  AGE
cloudbase-api-x7f2q  IPv4         <unset> <unset>    14m

# service.yaml
spec:
  selector:
    app: cloudbase-api

# deployment.yaml (pod template)
template:
  metadata:
    labels:
      app.kubernetes.io/name: cloudbase-api

```

- ✓ **Observe.** The EndpointSlice exists but has no endpoints — the Service is fronting an empty set, so packets are DNAT'd nowhere.
- ✓ **Hypothesise.** Either no Pod carries the selected label, or the Pods carry it but are not Ready. Here the Pods report 1/1, so readiness is fine.
- ✓ **Investigate.** `kubectl get pods -l app=cloudbase-api` returns nothing, while `-l app.kubernetes.io/name=cloudbase-api` returns three. The label keys differ.
- ✓ **Root cause.** Someone standardised the Pod template on the recommended `app.kubernetes.io/*` label set without updating the Service's selector. The Service is selecting a label nothing has.
- ✓ **Fix.** Align the selector with the Pod template labels. Note that a Deployment's own `spec.selector` is immutable — if that is what drifted, you must recreate the Deployment.
- ✓ **Prevent.** Generate both from one source (Kustomize `commonLabels`, or a Helm helper), and add a CI check that every Service selector matches at least one Pod template in the same directory.

8 Mini-Labs

Mini-Lab • Beginner

40 min

Goal. Prove the reconciliation loop to yourself in four experiments.

Business context. You are explaining self-healing to a sceptical colleague who thinks it is a cron job.

Architecture. One local cluster, one 3-replica Deployment, no Service yet.

Requirements. Delete a Pod and time the replacement; scale to 5 and back to 2; delete the ReplicaSet and watch it return; `kubectl edit` a live Pod's image and watch the change be discarded.

Constraints. Use `kubectl get pods -w` throughout — you are observing, not fixing.

Hints. `kubectl get rs` shows old and new ReplicaSets and their desired/current counts.

Expected outcome. A written one-paragraph explanation of who acted, in what order, in each case.

Production considerations. Which of these four would page you, and which are routine?

Common pitfalls. Concluding the Pod "restarted" when it was in fact replaced by a new one with a new name and IP.

Mini-Lab • Intermediate

60 min

Goal. Build the probe set for a slow-starting application and measure each threshold.

Business context. A JVM service takes 45 seconds to warm up; deploys currently drop requests.

Architecture. One Deployment with an artificial startup delay, one Service, one client loop generating traffic.

Requirements. Add a startup probe that tolerates the boot, a readiness probe that gates traffic, and a liveness probe that touches nothing external. Measure requests lost during a rolling update, before and after.

Constraints. No `sleep` in the deploy process — the cluster must know when the app is ready.

Hints. Drive traffic with a tight `curl` loop and count non-200 responses; compare against a run with the readiness probe removed.

Expected outcome. Zero failed requests during a rolling update, with the numbers to prove it.

Production considerations. Compute the worst-case restart delay from your thresholds and state it explicitly.

Common pitfalls. Using the same handler for liveness and readiness; setting `initialDelaySeconds` so high that a genuinely wedged container survives for minutes.

Mini-Lab • Production-style

2 hours

Goal. Produce the complete, reviewable manifest set for CloudBase in one namespace.

Business context. Another engineer must be able to stand up the environment from your repository alone.

Architecture. Namespace → ConfigMap → Secret → Deployment (3 replicas, requests/limits, probes, non-root) → ClusterIP Service.

Requirements. Every container has requests, a memory limit, readiness and liveness probes and a named port; no credential is committed in plain text; a `README` documents the apply order and the verification commands.

Constraints. Must apply cleanly onto an empty cluster *and* onto a cluster where it already exists — idempotency, exactly as on Day 1.

Hints. Keep manifests in `k8s/` numbered by apply order; verify with `rollout status` plus an endpoint check, not by eye.

Expected outcome. `kubectl apply -f k8s/` twice in a row, both clean, with a healthy rollout and non-empty endpoints.

Production considerations. Note where the Secret's values will come from in a real environment, and who would be allowed to read them.

Common pitfalls. Selector and Pod labels drifting apart; mutable image tags; forgetting that the namespace must exist before anything in it.

9

Engineering Notes

 Scalability note

The economic case for Kubernetes is bin-packing, not convenience. One service per VM wastes the difference between peak provisioning and average usage on every machine, all the time. A scheduler packing many workloads with honest resource requests onto shared nodes reclaims most of that gap — Spotify's published case study reports a two-to-threefold CPU utilisation improvement from bin-packing and multi-tenancy. That improvement is entirely contingent on requests being accurate: garbage requests in, garbage packing out.

 Cloud cost optimization

The most expensive line in a Kubernetes manifest is a resource request nobody measured. Requests are what you pay for, because they are what the scheduler reserves; a service requesting 2 CPUs and using 0.15 wastes 92% of that reservation on every one of its replicas, forever, across every environment. Measure actual usage with the Vertical Pod Autoscaler in recommendation-only mode before committing numbers, and revisit them — request values set during a launch panic have a way of becoming permanent.

 Disaster recovery tip

Back up `etcd`, and rehearse the restore. A cluster whose `etcd` is gone has not merely lost uptime — it has lost every declaration of what should be running, every Secret, every RBAC binding. Managed control planes (EKS, GKE, AKS) handle this for you, which is a strong argument for using one. If you run your own, take periodic snapshots, store them off-cluster and encrypted, and prove at least once that you can rebuild from one. An untested backup is a belief, not a control.

 Operational excellence

Kubernetes Events are the cluster telling you exactly what it tried and why it failed — and they expire, typically after about an hour. If your only record of an incident is `kubectl describe` run after the fact, the evidence is already gone. Ship Events to your logging system on day one; on Day 11 you will wire the metrics half of this, but Events answer "why did nothing start?" in a way no metric does.

 Interview insight

Expect: "How does a Service find its Pods if Pod IPs change constantly?" The weak answer names the label selector and stops. The strong answer walks the chain — the selector matches Pod labels, the EndpointSlice controller keeps only *Ready* Pods in the slice, `kube-proxy` on each node programs iptables or IPVS rules from those slices, and CoreDNS resolves the Service name to the virtual IP — and then names the failure mode: a selector that matches nothing produces a perfectly healthy-looking Service with an empty EndpointSlice and total timeouts.

10 Build-Along Project — CloudBase

Day 1 gave CloudBase a hardened base host and a strict-mode bootstrap script. Day 2 put it under version control with reviewable history. Day 3 established how traffic actually reaches it across the network. Day 4 turned it into a small, non-root, pinned container image. Today that image stops being an artifact and becomes a running service that survives a node dying.

 Build-Along • Day 05 — CloudBase on Kubernetes

- Create a `k8s/` directory in the CloudBase repository, with manifests numbered by apply order so the sequence is documented by the filenames.
- Declare a `cloudbase` namespace and use it everywhere — nothing lands in `default`.
- Write the Deployment: 3 replicas, the Day 4 image pinned to an immutable tag, `runAsNonRoot`, CPU and memory `requests` and a memory `limit` on every container.
- Add a `ClusterIP` Service selecting the Pod labels by a named `targetPort`, and verify with `kubectl get endpointslices` that it is non-empty.
- Externalise all configuration into a ConfigMap and all credentials into a Secret, injected with `envFrom`; commit the ConfigMap, never the Secret's real values.
- Add a startup probe, a readiness probe that checks the database, and a liveness probe that checks *nothing outside the process* — then write one sentence in the README explaining that split to the next engineer.
- Prove self-healing: delete a Pod and confirm replacement; break the Service selector and diagnose it from `endpointslices` alone; restore it.
- Record the verification commands in `docs/k8s-runbook.md`: `rollout status`, `get endpointslices`, `describe pod`, `get events --sort-by`.

Definition of done: `kubectl apply -f k8s/` runs cleanly twice in a row, `kubectl rollout status deployment/cloudbase-api -n cloudbase` exits 0, the Service has three ready endpoints, deleting any Pod restores three within seconds, and stopping the database drains traffic through readiness *without a single container restart*.

11 Chapter Summary**Key takeaways**

- ✓ The control plane decides, the nodes run. The API server is the only writer to `etcd`.
- ✓ `kubectl apply` writes a declaration and returns; controllers do the work asynchronously.
- ✓ Reconciliation is level-triggered: observe reality, diff against desired, take one corrective action, repeat.
- ✓ The scheduler filters nodes on `requests` and constraints, scores the survivors, and writes a node name.
- ✓ A Service is a label selector resolved into EndpointSlices and programmed into kernel rules by `kube-proxy`.
- ✓ Readiness gates traffic; liveness restarts. Only readiness may check external dependencies.

Things to remember

- Environment variables from a ConfigMap are fixed at process start — changing the ConfigMap changes nothing until the Pod is replaced.
- Secrets are base64-encoded, not encrypted; at-rest encryption in `etcd` is opt-in on the API server.

- A Deployment's `spec.selector` is immutable — getting it wrong means recreating the object.
- Empty endpoints with a healthy Service is nearly always a label mismatch.

Common pitfalls (last look)

- Dependency check in liveness • editing a live Pod • no resource requests
- Mutable image tags • real credentials in a ConfigMap • trusting a successful `apply`

📋 One-Page Cheat Sheet

Control plane — API server (only etcd writer) • etcd (Raft, source of truth) • scheduler (filter, score) • controller-manager (loops).

Node — kubelet (starts containers, runs probes) • kube-proxy (programs Service rules) • container runtime.

Reconciliation — observe → diff → act → repeat; level-triggered, so it self-corrects after any missed event.

Deployment — owns ReplicaSets; rolling update bounded by `maxSurge` and `maxUnavailable`; rollback = scale the old ReplicaSet back up.

Service — selector → EndpointSlice (Ready Pods only) → kube-proxy rules → CoreDNS name.

Probes — startup suspends the others • readiness gates traffic • liveness restarts • never do external I/O in liveness.

Resources — `requests` = what the scheduler reserves and you pay for; memory `limit` = OOMKill boundary; CPU `limit` = throttling.

CLI quick reference

Day 5 commands worth memorising

```
# what is the cluster doing right now
kubectl get pods -n cloudbase -o wide           # includes node and Pod IP
kubectl get events -n cloudbase --sort-by=.lastTimestamp
kubectl describe pod <pod> -n cloudbase          # Events: why it will not start

# the declaration itself
kubectl apply -f k8s/                           # write desired state
kubectl get deploy cloudbase-api -o yaml         # what the API server stored
kubectl diff -f k8s/                             # what apply WOULD change

# rollouts
kubectl rollout status deployment/cloudbase-api -n cloudbase
kubectl rollout history deployment/cloudbase-api -n cloudbase
kubectl rollout undo deployment/cloudbase-api -n cloudbase
kubectl rollout restart deployment/cloudbase-api -n cloudbase

# service wiring -- the two-second timeout diagnosis
kubectl get endpointslices -n cloudbase
kubectl get pods -n cloudbase -l app=cloudbase-api

# inspection
kubectl logs -f <pod> -n cloudbase --previous    # logs from the crashed instance
kubectl exec -it <pod> -n cloudbase -- /bin/sh
kubectl top pods -n cloudbase                    # actual usage vs your requests
```


Interview questions to master

- › What is the difference between a Pod, a ReplicaSet and a Deployment, and why are there three?
- › How does a Service find the right Pods if Pod IPs change constantly?
- › What is `etcd`, and what exactly have you lost if you lose it?
- › Explain the reconciliation loop, and why level-triggered matters more than edge-triggered.
- › Why is a ConfigMap the wrong place for a database password, given that a Secret is only base64?
- › What does the scheduler actually use when it decides a node has room?
- › Why must a liveness probe never check the database?

Further reading & official docs

- *Kubernetes Components* — kubernetes.io/docs/concepts/overview/components/ — one page naming every control-plane and node component.
- *Kubernetes Scheduler* — kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/ — the filtering and scoring two-step, and scheduling profiles.
- *Liveness, Readiness and Startup Probes* — kubernetes.io/docs/concepts/workloads/pods/probes/ and the companion task page under [/docs/tasks/configure-pod-container/](https://kubernetes.io/docs/tasks/configure-pod-container/).
- *Encrypting Confidential Data at Rest* — kubernetes.io/docs/tasks/administer-cluster/encrypt-data/ — confirms the API server stores plain text in `etcd` by default.
- *Service and EndpointSlices* — both under kubernetes.io/docs/concepts/services-networking/.
- Spotify's Kubernetes case study — kubernetes.io/case-studies/spotify/ — the Helios migration, provisioning time and utilisation figures quoted in this chapter.
- "Dynamic Kubernetes Cluster Scaling at Airbnb" on the Airbnb Tech Blog — multi-cluster operation and the cluster-type abstraction.

Production checklist

- ☐ Every workload is a Deployment (or StatefulSet/Job), never a bare Pod.
- ☐ Every container has CPU and memory `requests` and a memory `limit`.
- ☐ Every container has a readiness probe; liveness only where a real hang exists, and it performs no external I/O.
- ☐ Images are pinned to immutable tags or digests, never `latest`.
- ☐ Configuration lives in ConfigMaps, credentials in Secrets sourced from an external store.
- ☐ At-rest encryption is enabled for Secrets on the API server, and `etcd` backups are tested.
- ☐ Namespaces separate environments, with resource quotas attached.
- ☐ Events are shipped off-cluster before they expire.

Key idea

What you will learn next (Day 6). CloudBase now runs and stays running — but it is only reachable from inside the cluster, it scales only when you type a number, and a bad rollout still reaches every user at once. Tomorrow: Ingress and TLS termination so one entry point serves many services, the Horizontal

Pod Autoscaler reacting to real metrics, persistent volumes for state that must outlive a Pod, and rollout strategies that let you fail a deploy in front of 1% of traffic instead of 100%.

DAY 06 • ORCHESTRATION

Kubernetes in Production

Helm, Ingress, TLS, RBAC, autoscaling and StatefulSets

06

 Estimated study time: 9 hours By the end of this day, you will be able to...

- Package a directory of raw manifests as a versioned, rollback-able Helm chart with per-environment values.
- Explain what an Ingress controller *is* — a reverse proxy that watches the API server and rewrites its own configuration — and route traffic through one.
- Terminate TLS at the edge of the cluster and automate certificate issuance and renewal with ACME.
- Derive the Horizontal Pod Autoscaler's replica count from its metric-ratio formula, and configure probes and resource requests that make that number meaningful.
- Apply least-privilege RBAC with per-application ServiceAccounts, and say precisely when a StatefulSet is required instead of a Deployment.

1 The Big Picture

What problem this solves. On Day 5 you got a workload *running* on Kubernetes: a Deployment, a Service, a ConfigMap, applied with `kubectl apply -f`. That is a demo. A production system additionally has to answer six questions that raw manifests do not: how do I ship the same application to three environments without maintaining three copies of the YAML? How does traffic from the public internet reach a Pod that has an ephemeral, cluster-internal IP? Who issues and renews the TLS certificate, at 3 am on a Sunday, without a human? How many replicas should be running *right now*? What is this application allowed to ask the API server for? And where does the data live when the Pod that owns it is deleted? Today answers all six.

Why DevOps engineers need it. The gap between "it runs on my cluster" and "it survives a Black Friday, an expired certificate and a compromised container" is exactly this chapter. Every item here is a control loop or a packaging boundary, and each one has a failure mode that beginners walk into: a chart whose values do not merge the way they expect, an Ingress annotation silently ignored because it targets a different controller, an HPA that never scales because no metrics API is installed, a ServiceAccount bound to `cluster-admin` "to make an error go away."

Where it fits in a modern cloud architecture. Day 6 is the layer between the cloud load balancer you will provision on Day 10 and the Pods you scheduled on Day 5. Read the diagram top-down as a request travelling inward.

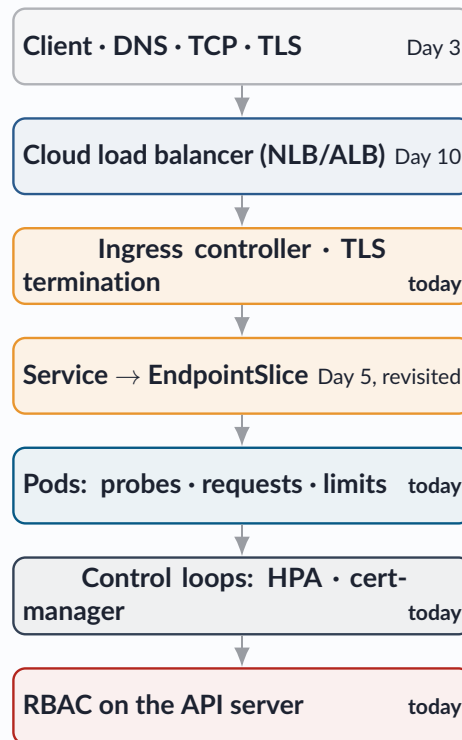


Fig 6.0 — The production request path. Everything between the cloud load balancer and the Pod is what you build today.

Key idea

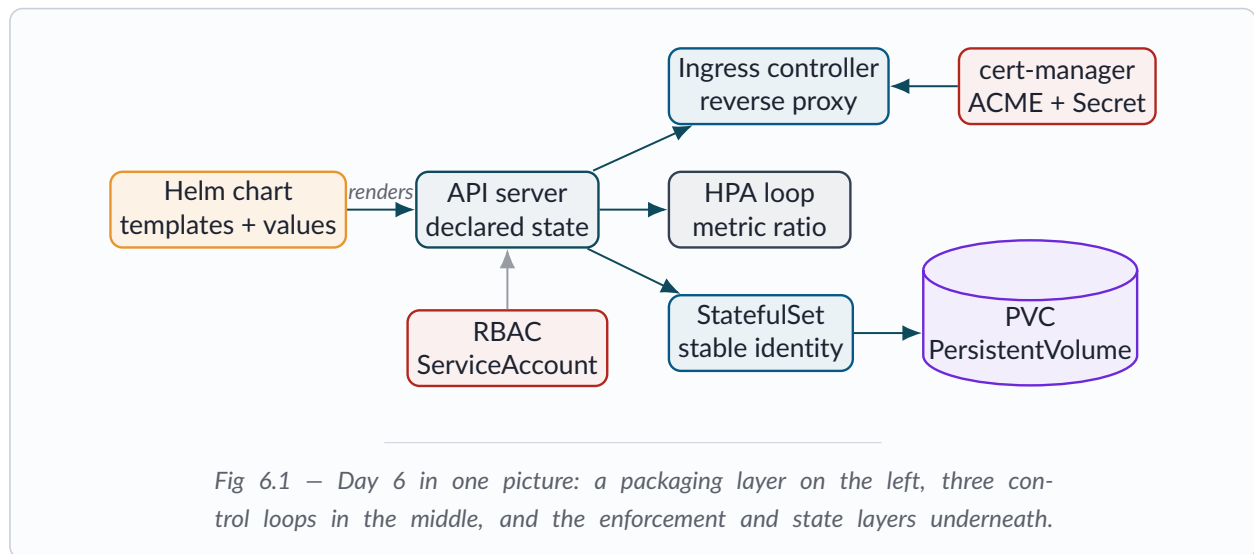
Kubernetes is not a deployment tool. It is a set of **control loops**: independent processes that continuously compare an observed state to a declared state and act to close the gap. The Ingress controller, cert-manager, the HPA and the StatefulSet controller in this chapter are four instances of exactly the same pattern. Once you see them that way, "how does X work?" always has the same shape of answer — what does it watch, what does it compute, and what does it write?

How it connects to previous chapters. Day 1's SIGTERM/grace-period path is what `terminationGracePeriodSeconds` governs, and exit code 137 is what an exceeded memory limit produces. Day 4's image layers and non-root user are what the Pod runs. Day 5 gave you Deployments, Services and the reconciliation model; today every new object is another controller reconciling against it. Day 7's Terraform will provision the cluster and the DNS records that point at your Ingress, and Day 12's least-privilege IAM work is the same principle as RBAC with the enforcement point moved from the cluster API to the cloud API.

Where companies use it in production. Shopify runs `ingress-nginx` such that, in their own description, nearly every request they serve passes through it at some point; they hit a real bottleneck where NGINX reloaded its configuration on every deploy, adding latency, and contributed a Lua-based dynamic endpoint-update path so that Pod churn updates an in-memory shared dictionary instead of forcing a reload. Zalando runs well over a hundred Kubernetes clusters on AWS and did not use the common controllers at all: they built `kube-ingress-aws-controller`, which watches Ingress objects and orchestrates real AWS load balancers, in front of Skipper, their own HTTP router; and `kube-metrics-adapter`, which feeds the HPA with request-rate, SQS-queue-depth and arbitrary Prometheus queries rather than CPU. Both architectures are worth understanding because they show the same interfaces — Ingress, the external metrics API — with completely different implementations behind them.

Common misconceptions. That Helm is a deployment tool (it is a template renderer plus a release ledger; the cluster still reconciles). That an `Ingress` object routes traffic (it does not; it is a piece of data that a controller reads). That the HPA reads Prometheus (it reads the metrics APIs, and something must implement them). That RBAC can deny something (it cannot — it is purely additive). That a `StatefulSet` gives you a highly available database (it gives you stable identity and storage; the replication is your database's problem).

2 Concept Map



Terminology to anchor: **chart** → **values** → **release revision** → **Ingress** → **IngressClass** → **TLS termination** → **readiness** → **request/limit** → **metric ratio** → **ServiceAccount** → **Role** → **volumeClaimTemplate**.

3 Guided Learning

3.1 Helm — Packaging, Values and the Release Ledger

What is it? A **chart** is a directory of Go-templated Kubernetes manifests plus a `values.yaml` of defaults and a `Chart.yaml` of metadata. Helm renders the templates against merged values, sends the resulting plain manifests to the API server, and records what it sent as a numbered **release revision**.

Why does it exist? A real service is a dozen YAML files, and between dev, staging and production the only differences are usually the image tag, the replica count, the resource sizes, the hostname and a handful of environment variables. Before Helm, teams solved this three ways, all bad: copy the directory per environment and let it drift; run `sed` over the YAML in CI and discover typos at apply time; or hand-roll a Python script that emits manifests, which nobody else on the team can read. Helm's contribution is not the templating — it is the **release**: a versioned, named, upgradeable and rollback-able unit, so that "what is deployed in production" is a question with a machine-readable answer.

How does it work internally? Rendering is entirely client-side. Helm walks `templates/`, executes each file as a Go template with a context containing `.Values` (defaults deep-merged with every `-f`

file and `-set` flag, later overriding earlier), `.Release` (name, namespace, revision, and whether this is an install or an upgrade), `.Chart` and `.Capabilities` (the cluster's Kubernetes version and available API groups). The output is ordinary YAML — there is nothing Helm-specific left in what the cluster receives, which is why `helm template` is such a useful debugging command.

The interesting part is state. Helm 3 removed the Tiller server component and stores each release revision as a Secret in the release's own namespace, named `sh.helm.release.v1.<release>.v<N>`. Its `release` key is a base64-encoded, gzip-compressed JSON blob containing the chart, the merged values, the rendered manifests and the release metadata. That is what makes `helm rollback 3` instantaneous: revision 3's rendered manifests are already stored, so a rollback is a re-apply, not a re-render from source you may no longer have.

On upgrade, Helm computes a **three-way strategic merge patch**: it compares the old rendered manifest, the object's *live* state in the cluster, and the new rendered manifest. This matters enormously in practice. If another controller (or a human) added a field to the live object, a two-way patch — what Helm 2 did — would silently wipe it. The three-way patch preserves changes that Helm did not make and did not intend to remove.

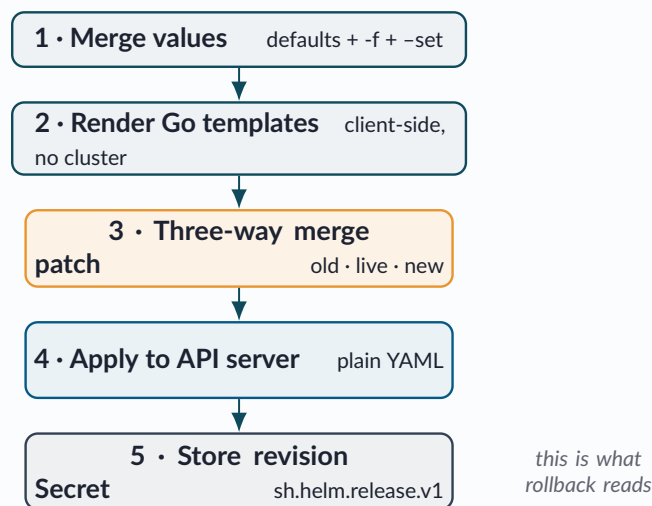


Fig 6.2 — What Helm actually does. Everything left of the API server is client-side text processing; the cluster only ever sees plain manifests.

chart/values.yaml – defaults, deliberately safe

```

replicaCount: 2
image:
  repository: ghcr.io/cloudbase/api
  tag: "" # empty -> template falls back to .Chart.AppVersion
  pullPolicy: IfNotPresent
resources:
  requests: { cpu: "250m", memory: "256Mi" }
  limits: { memory: "512Mi" }
ingress:
  enabled: false # off by default; each environment opts in
  host: ""
autoscaling:
  enabled: false
  minReplicas: 2
  maxReplicas: 10
  targetCPUUtilizationPercentage: 70

```

chart/values-prod.yaml - only the deltas

```

replicaCount: 5
image:
  tag: "1.4.2"
resources:
  requests: { cpu: "500m", memory: "512Mi" }
  limits:   { memory: "1Gi" }
ingress:
  enabled: true
  host: api.cloudbase.example
autoscaling:
  enabled: true
  minReplicas: 3
  maxReplicas: 20

```

chart/templates/deployment.yaml - the templated fragment

```

spec:
  # replicas is omitted entirely when the HPA owns it, so that a
  # helm upgrade cannot fight the autoscaler back down to the default
  {{- if not .Values.autoscaling.enabled }}
  replicas: {{ .Values.replicaCount }}
  {{- end }}
  template:
    spec:
      serviceAccountName: {{ include "cloudbase.serviceAccountName" . }}
      containers:
        - name: api
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag
            | default .Chart.AppVersion }}"
          resources: {{- toYaml .Values.resources | nindent 12 }}

```

⚠ Common misconception

"Helm deploys my application." Helm renders text and posts it to the API server; the Deployment controller, the ReplicaSet controller and the scheduler do the deploying. This distinction is not pedantry — it explains why `helm upgrade` returning success means "the API server accepted the manifests", not "the new Pods are healthy". You need `-wait` (and honestly, a real readiness gate in CI) before treating a Helm exit code as a deployment result.

💡 Senior engineer insight

The `replicas` template guard above is one of the most valuable eight lines in a production chart. If the chart hardcodes `replicas: 5` while an HPA has scaled the Deployment to 14, the next unrelated `helm upgrade` sets the field back to 5, the API server kills nine Pods, and the HPA spends the next few minutes scaling back up — during which you are under-provisioned. Omit fields another controller owns.

When should I use it? As soon as you have more than one environment, or more than one team consuming the same deployment shape, or a third-party component you would otherwise install by pasting a vendor's YAML.

When should I not? For a single application in a single environment, a `kustomize` overlay is simpler and involves no templating language — Kustomize patches structured YAML rather than generating text, so it cannot produce syntactically invalid output. And once a chart's templates are more `if` than manifest, the chart has become a program; that is the signal to split it, not to add another conditional.

Design trade-off

Helm templates *text*, not YAML. That is why indentation helpers like `nindent` exist and why a stray space produces a parse error hundreds of lines from its cause. Kustomize manipulates parsed YAML structures, so it is structurally safe but far less expressive — there are no loops and no conditionals. Teams commonly land on: Helm for anything published or shared across teams, Kustomize overlays for internal per-environment tweaks. Both are first-class; `kubectl` ships Kustomize built in.

3.2 Ingress, Ingress Controllers and TLS Termination

What is it? An `Ingress` is an API object describing HTTP routing rules: for host `api.example.com`, path `/orders`, send traffic to Service `order-api` on port 8080; and here is the Secret holding the TLS certificate for that host. An **Ingress controller** is a separate program running in the cluster that reads those objects and makes them true.

Why does it exist? On Day 5 you exposed a Service with `type: LoadBalancer`, and the cloud provider provisioned a load balancer. That is fine for one service. For forty services it means forty load balancers, forty public IPs, forty DNS records, forty certificates and forty monthly line items — and no shared place to put authentication, rate limiting or request logging. Before Ingress, teams solved this by running their own NGINX or HAProxy in the cluster and regenerating its config file from a script that polled the API server. Ingress standardised that pattern: the routing intent became an API object, and the config-regenerating proxy became a supported component.

How does it work internally? An Ingress controller is a reverse proxy plus a control loop, in the same process or Pod. The loop opens a *watch* on the API server for Ingress, Service, EndpointSlice and Secret objects. A watch is a long-lived streamed connection — the controller does not poll; the API server pushes ADDED/MODIFIED/DELETED events as they happen. On each event the controller recomputes its own desired configuration, and if it differs from what the proxy is currently running, it applies the difference and updates the Ingress object's `status.loadBalancer` field so `kubectl get ingress` shows an address.

The subtle part is *how* the difference is applied, and it is exactly where Shopify's engineering effort went. A naive controller writes a new `nginx.conf` and reloads. NGINX reloads gracefully — old workers drain, new workers start — but during a deploy, Pod churn produces a reload every few seconds, each one spawning worker processes and dropping keepalive connections; Shopify measured this as real added end-user latency. Their fix, contributed upstream, was to split configuration into two classes: *structural* changes (a new host, a new path, a changed annotation) still require a reload, while *endpoint* changes (a Pod came or went) are pushed over an internal HTTP endpoint into a Lua shared dictionary that the request path reads on every request. Pod churn therefore stopped causing reloads at all. That two-tier design — slow path for structure, fast path for membership — is the standard architecture for every serious data-plane component you will meet.

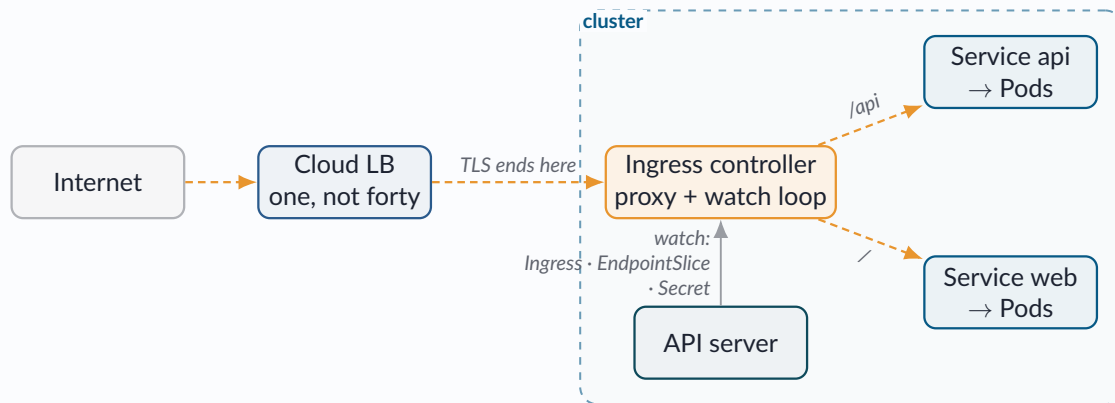


Fig 6.3 — The Ingress controller as a control loop: it watches the API server and rewrites its own data plane. One load balancer serves every host and path.

ingress.yaml – host and path routing with TLS

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: cloudbase
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  ingressClassName: nginx      # WHICH controller owns this object
  tls:
    - hosts: [api.cloudbase.example]
      secretName: cloudbase-tls # cert-manager creates and renews this
  rules:
    - host: api.cloudbase.example
      http:
        paths:
          - path: /orders
            pathType: Prefix
            backend:
              service:
                name: order-api
                port: { number: 8080 }

```

🔍 Behind the scenes

`ingressClassName` exists because a cluster can run several controllers at once — an internal one for private traffic and an external one for public. Each controller only acts on Ingress objects whose class matches its own. Omit the field on a cluster with no default IngressClass and *nothing happens*: the object is created, `kubectl get ingress` shows an empty ADDRESS column forever, and there is no error anywhere, because no controller ever considered the object its business. This is the single most common "my Ingress does not work" cause.

TLS termination. Terminating TLS means the controller completes the handshake, decrypts, and forwards plaintext HTTP to the Pod over the cluster network. The certificate and private key live in a `kubernetes.io/tls` Secret; the controller loads them and selects one per hostname using SNI, the TLS extension that carries the requested hostname in the ClientHello *before* the certificate is chosen. That is what allows one IP and one port to serve dozens of certificates.

Automating issuance is cert-manager's job, and it is another control loop. You declare an `Issuer` or `ClusterIssuer` pointing at an ACME directory (RFC 8555 — Let's Encrypt is one implementation). cert-manager watches Ingress objects for its annotation, creates a `Certificate`, then an `Order`, then a `Challenge`. For an HTTP-01 challenge it spins up a temporary solver Pod, a Service and a temporary Ingress so that the ACME server's request to `/.well-known/acme-challenge/<token>` on your public hostname is routed to the solver, which returns the expected key authorisation. On success cert-manager writes the certificate into the Secret, deletes the temporary objects, and schedules renewal well before expiry — typically at around two-thirds of the certificate's lifetime, so a failure has weeks of runway rather than hours.

Security warning • terminating TLS is not the same as having encryption

After termination, traffic from the controller to your Pods is plaintext on the cluster network. For most workloads that is an accepted trade-off, because the pod network is considered a trusted zone. It stops being acceptable the moment you handle regulated data, run untrusted tenants in the same cluster, or cross an availability zone on a shared network. The answers are re-encryption to the backend, or a service mesh doing mutual TLS between every Pod — and the honest cost is another control plane to operate. Decide deliberately; do not let "we have HTTPS" end the conversation at the load balancer.

Operational excellence • what changed recently, and why it matters to you

The Kubernetes project announced in November 2025 that `ingress-nginx` — for years the default choice, and by Datadog's measurement present in roughly half of cloud-native environments — would be retired in March 2026, with no further releases including security fixes. The stated migration path is the **Gateway API**, the successor to Ingress, which splits the single overloaded object into role-oriented resources: `GatewayClass` and `Gateway` for the platform team, `HTTPRoute` for the application team. Existing installs keep working; they simply stop receiving patches. Learn Ingress — it is what the overwhelming majority of running clusters use and what you will be handed — but write new platform work against Gateway API, and know that "which controller, and is it maintained?" is now a live production question rather than a detail.

3.3 Probes, Requests and Limits — The Contract With the Node

What is it? Probes are periodic checks the kubelet performs against a container. Requests and limits are the numbers the scheduler and the kernel use to decide where a Pod runs and how much it may consume.

Why does it exist? "The process is running" is a uselessly weak definition of healthy. A JVM that has deadlocked every request thread is running. A service whose connection pool is exhausted is running. Kubernetes needs two different and independent answers: *should I restart this?* and *should I send it traffic?* Before orchestrators, both questions were answered by an external monitoring system and a human paged at 3 am.

Probe	Question it answers	Action on failure
Liveness	Is this process wedged beyond recovery?	kubelet kills the container; restart policy applies
Readiness	Can it serve a request <i>right now</i> ?	Pod is removed from the Service's EndpointSlice — no restart
Startup	Has a slow boot finished?	Liveness and readiness are suppressed until it passes; failure kills the container

How does it work internally? The kubelet on the node — not the control plane — runs every probe. On failure it increments a counter and only acts after `failureThreshold` consecutive failures, which is

why a probe's *effective* timeout is `periodSeconds × failureThreshold`, not `timeoutSeconds`. A readiness failure causes the kubelet to update the Pod's status condition; the endpoints controller observes that and removes the Pod's address from the EndpointSlice; kube-proxy (or the Ingress controller reading EndpointSlices directly) stops sending it traffic. Notice this is several hops of eventual consistency — there is a real window, typically a second or two, during which a not-ready Pod still receives requests. That window is why a graceful shutdown that keeps serving during its termination grace period matters, and it is Day 1's SIGTERM discussion resurfacing.

deployment.yaml - probes and resources, deliberately separated

```
readinessProbe:
  httpGet: { path: /actuator/health/readiness, port: 8080 }
  periodSeconds: 5
  failureThreshold: 2      # out of rotation fast
livenessProbe:
  httpGet: { path: /actuator/health/liveness, port: 8080 }
  periodSeconds: 10
  failureThreshold: 6      # restart slow: 60s of sustained failure
startupProbe:
  httpGet: { path: /actuator/health/readiness, port: 8080 }
  periodSeconds: 5
  failureThreshold: 30     # allows a 150s cold start, once
resources:
  requests: { cpu: "250m", memory: "256Mi" }
  limits:   { memory: "512Mi" }
```

Reliability principle

Never let a downstream dependency influence the liveness probe. If the liveness endpoint checks the database and the database has a slow ten minutes, every replica fails liveness simultaneously, every replica is killed simultaneously, every replica cold-starts and reopens connections into an already-struggling database, and you have converted a degradation into an outage. Liveness answers "is *this* process wedged?" and nothing else. Readiness may check dependencies, because its consequence — being removed from rotation — is reversible and does not amplify load.

Requests and limits, honestly. A **request** is a scheduling reservation: the scheduler will only place the Pod on a node whose unreserved capacity covers it, and the sum of requests on a node can never exceed its allocatable capacity. A **limit** is a runtime ceiling enforced by kernel cgroups — exactly the cgroups from Day 4.

The two resources behave completely differently at the ceiling, and this is a favourite interview question with a mechanical answer. CPU is *compressible*: the kernel's CFS bandwidth controller gives the cgroup a quota of microseconds per 100 ms period, and when it is exhausted the tasks are simply descheduled until the next period. The process does not fail; it goes slow, and `container_cpu_cfs_throttled_seconds_total` goes up. Memory is *incompressible*: there is no way to give a process less memory than it has already allocated, so when the cgroup exceeds its limit the kernel OOM killer terminates a process in it. The container exits 137 — Day 1's 128 + 9 — and Kubernetes reports `OOMKilled`.

Performance note

CPU *limits* are the most contested setting in Kubernetes. Setting a low limit on a latency-sensitive service causes throttling in exactly the bursty moments that matter, and the resulting p99 latency spikes look like an application bug for weeks. Many mature teams set CPU requests carefully and omit CPU limits entirely, relying on requests for fair share under contention, while always setting memory requests

and limits at the same value for critical services — which additionally earns the Pod the Guaranteed QoS class, making it the last thing evicted when a node runs short. Whatever you choose, choose it from a load test, not from a template someone copied in 2021.

3.4 The Horizontal Pod Autoscaler Control Loop

What is it? A controller that periodically adjusts a Deployment's replica count so that an observed metric stays near a target you declare.

Why does it exist? Capacity planning by hand means provisioning for peak and paying for it at 4 am. Before autoscaling, teams either over-provisioned permanently or ran a cron job that scaled up at 08:00 and down at 20:00 — which handles the daily curve and nothing else, and fails precisely on the unusual day when it matters.

How does it work internally? This is the part to memorise, because it explains nearly every surprising HPA behaviour. Every 15 seconds by default, the controller reads the current metric for each Pod belonging to the target, computes the mean across ready Pods, and evaluates:

```
desiredReplicas = ceil[ currentReplicas × ( currentMetricValue / desiredMetricValue ) ]
```

Three consequences follow immediately. First, it is a *ratio*, so at 90% observed against a 70% target with 4 replicas you get $\text{ceil}(4 \times 90/70) = \text{ceil}(5.14) = 6$. Second, because of `ceil`, small clusters over-shoot: with 1 replica any excess at all produces at least 2. Third, the controller applies a **tolerance** — historically a fixed 10%, and configurable per-direction since Kubernetes v1.33 — and does nothing at all when the ratio is within it. That hysteresis is deliberate: without it the loop would oscillate around the target forever. Scale-down is further damped by a stabilisation window (five minutes by default) during which the controller uses the *highest* recommendation it has seen, so a brief dip does not tear down capacity you are about to need.

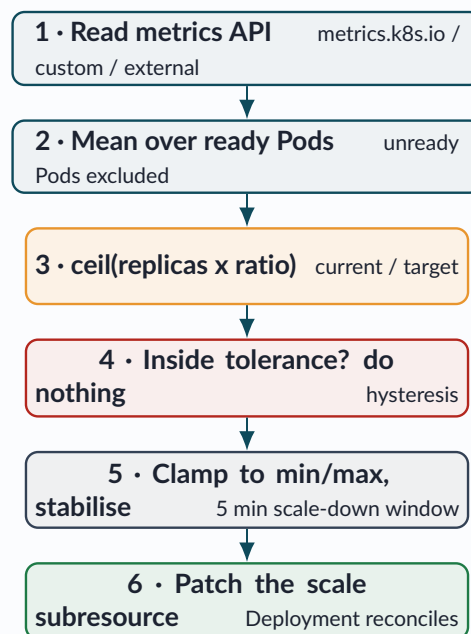


Fig 6.4 — The HPA loop. Note the two dampers: the tolerance band and the scale-down stabilisation window.

hpa.yaml – autoscaling/v2, with explicit behaviour

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: order-api }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: order-api }
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 }
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 0      # react immediately to load
      policies: [{ type: Percent, value: 100, periodSeconds: 30 }]
    scaleDown:
      stabilizationWindowSeconds: 600    # shed capacity slowly
      policies: [{ type: Percent, value: 25, periodSeconds: 60 }]

```

⚠ Common misconception

`averageUtilization: 70` does not mean "70% of a CPU core" or "70% of the node". It is 70% of the container's CPU *request*. A Pod requesting 250m and using 200m is at 80% utilisation and will trigger scale-up, even though it is using a fifth of a core on a machine that is 90% idle. Changing a request therefore silently changes every HPA target derived from it – which is why request tuning and autoscaler tuning have to happen in the same conversation.

🔍 Behind the scenes

The HPA does not read Prometheus, and it does not read the kubelet. It reads three aggregated APIs: `metrics.k8s.io` for CPU and memory (implemented by metrics-server, which is not installed by default in a vanilla cluster), `custom.metrics.k8s.io` for metrics attached to Kubernetes objects, and `external.metrics.k8s.io` for metrics with no Kubernetes object at all – an SQS queue depth, a Kafka consumer lag. Something must implement each. Zalando's `kube-metrics-adapter` implements the latter two, letting them autoscale on Skipper's request rate, SQS backlog or an arbitrary Prometheus query. "The HPA shows <unknown> for the metric" almost always means the API exists but nothing is serving it.

📈 Scalability note

CPU is a lagging indicator for request-driven services: by the time CPU is at 70%, latency has usually already degraded. Scaling on requests-per-second or on queue depth is leading, and it is why Zalando built the adapter rather than living with the default. Note also that the HPA only adds Pods – if the cluster has no room for them the new Pods sit `Pending` until a *cluster* autoscaler adds a node, which takes minutes. Pod autoscaling without node autoscaling has a hard ceiling you will find during an incident.

3.5 RBAC – Least Privilege on the API Server

What is it? Role-Based Access Control decides whether a given identity may perform a given verb on a given resource. Four object types: `Role` and `ClusterRole` carry the permissions; `RoleBinding` and `ClusterRoleBinding` attach them to subjects (users, groups, ServiceAccounts).

Why does it exist? Every Pod can reach the API server. Without authorisation, a compromised container – or a buggy one – could list every Secret in the cluster, including database passwords and

the TLS private keys from the previous section. RBAC is the enforcement point that makes container compromise a contained incident rather than a cluster compromise.

How does it work internally? A request arrives at the API server and passes through three phases: authentication (who are you?), authorisation (may you?), then admission controllers (should this specific object be allowed, and should it be mutated?). RBAC is an authoriser. It gathers every Role and ClusterRole bound to the subject and asks whether any rule matches the request's apiGroup, resource, verb and — for a namespaced request — namespace. **RBAC is purely additive: there are no deny rules.** If no rule matches, the request is denied by default; if any rule matches, it is allowed. You cannot "subtract" a permission by adding an object — you have to remove the binding that granted it. The scoping rules trip everyone up once, so learn them as a two-by-two: a Role is namespaced and grants inside its namespace; a ClusterRole is cluster-scoped; a RoleBinding grants only within its own namespace; a ClusterRoleBinding grants everywhere. The useful diagonal is a RoleBinding referencing a ClusterRole — that grants the ClusterRole's permissions but only inside the binding's namespace, which is how you define a reusable "developer" permission set once and bind it to twelve team namespaces.

rbac.yaml - the ServiceAccount CloudBase's API actually needs

```
apiVersion: v1
kind: ServiceAccount
metadata: { name: cloudbase-api, namespace: cloudbase }
automountServiceAccountToken: false # this app never calls the API
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: config-reader, namespace: cloudbase }
rules:
- apiGroups: [""]
  resources: ["configmaps"]
  resourceNames: ["cloudbase-feature-flags"] # ONE object, not all
  verbs: ["get", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: config-reader, namespace: cloudbase }
subjects:
- kind: ServiceAccount
  name: cloudbase-api
roleRef:
  kind: Role
  name: config-reader
  apiGroup: rbac.authorization.k8s.io
```

Security warning • the default ServiceAccount is a real attack path

Every namespace has a `default` ServiceAccount, and any Pod that does not name one gets it. By default its token is mounted into the container at `/var/run/secrets/kubernetes.io/serviceaccount/token`, so anyone with code execution in that container has a cluster credential in a known path. Two disciplines fix this: give every workload its own ServiceAccount with an explicitly scoped Role, and set `automountServiceAccountToken: false` on every Pod that never talks to the API server — which is most of them. Never bind `cluster-admin` to a workload identity to clear a `Forbidden` error; read the error, which names the exact apiGroup, resource and verb that was refused, and grant that.

👁 Behind the scenes

Since Kubernetes 1.24, ServiceAccounts no longer get a permanent Secret containing a never-expiring JWT. Pods receive a *projected* token: short-lived, automatically rotated by the kubelet, and bound both to an audience and to the Pod object itself — so it is invalidated when the Pod is deleted and is rejected by any service that checks the audience. If you find a long-lived ServiceAccount token in a CI system or a `.kube/config`, that is a legacy credential and it is a finding, not a configuration.

💡 Production tip

```
kubectl auth can-i -list -as=system:serviceaccount:cloudbase:cloudbase-api prints everything an identity may do, without needing that identity's credentials. Run it against your own workloads before an audit runs it against you, and put it in CI as an assertion: if the list grows, the pull request that grew it has to say why.
```

3.6 StatefulSets and PersistentVolumes

What is it? A StatefulSet manages Pods that need a *stable identity* and *stable storage*: ordinal names (`db-0`, `db-1`), a stable DNS name per Pod via a headless Service, and a PersistentVolumeClaim per Pod that follows that ordinal across rescheduling.

Why does it exist? A Deployment's Pods are deliberately interchangeable — random name suffixes, any Pod can replace any other, and a shared volume would be shared by all of them. That model is wrong for anything that keeps data or forms a cluster. A PostgreSQL replica must know it is replica 2 and must find the same 200 GB it had before the node was drained. A Kafka broker's identity determines which partitions it leads. Before StatefulSets, teams ran databases on pet VMs outside the cluster precisely because the orchestrator had no vocabulary for identity.

How does it work internally? Three mechanisms. First, ordinals: Pods are created `0`, `1`, `2`, ... in order, and by default the controller waits for each to become Ready before creating the next; deletion runs in reverse. Second, network identity: paired with a headless Service (`clusterIP: None`), each Pod gets a resolvable DNS record `db-0.db.cloudbase.svc.cluster.local` that survives rescheduling. Third, storage: `volumeClaimTemplates` makes the controller create one PVC *per ordinal* rather than one shared claim. That PVC binds to a PersistentVolume through a StorageClass, whose CSI driver provisions the real disk — an EBS volume, a Ceph RBD image. When `db-1` is rescheduled onto another node, the CSI driver detaches the volume from the old node and attaches it to the new one, and the same data comes back.

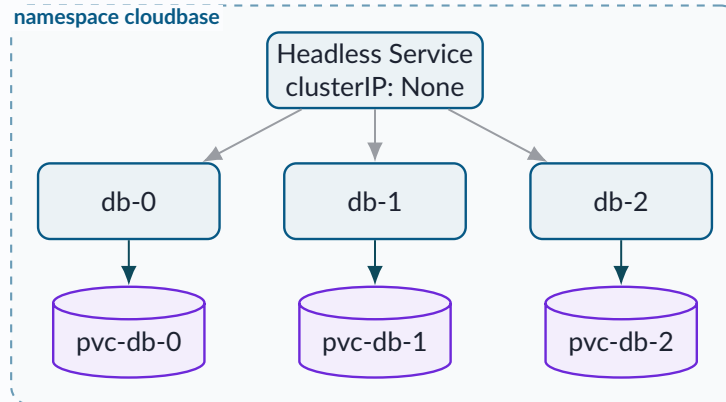


Fig 6.5 — StatefulSet identity: each ordinal owns a DNS name and its own volume, both of which survive rescheduling.

statefulset.yaml - identity and per-Pod storage

```

apiVersion: apps/v1
kind: StatefulSet
metadata: { name: db }
spec:
  serviceName: db          # MUST match the headless Service name
  replicas: 3
  podManagementPolicy: OrderedReady
  template:
    spec:
      terminationGracePeriodSeconds: 60 # let the DB checkpoint
      containers:
        - name: postgres
          volumeMounts:
            - { name: data, mountPath: /var/lib/postgresql/data }
      volumeClaimTemplates:
        - metadata: { name: data }
          spec:
            accessModes: ["ReadWriteOnce"]
            storageClassName: gp3
            resources: { requests: { storage: 200Gi } }

```

⚠ Common misconception

A StatefulSet does not replicate your data, elect a leader, or handle failover. It gives three Pods stable names and three separate disks; making `db-1` a replica of `db-0` is entirely your database's configuration and your responsibility. This is why production teams either run a purpose-built **operator** — a controller encoding an expert's runbook for backup, failover and version upgrade — or use a managed database and keep the cluster stateless. "We put Postgres in a StatefulSet" is the beginning of the work, not the end of it.

🚨 Disaster recovery tip

Deleting a StatefulSet does not delete its PVCs, and that default has saved a lot of data. It also means orphaned volumes accumulate and keep billing — a genuine and commonly overlooked cloud cost. More importantly: a replicated database is not a backup. Replication faithfully copies a `DROP TABLE` to every replica within milliseconds. You need snapshots with a tested restore path, and the restore has to be tested on a schedule, because an untested backup is a hypothesis.

When should I not use a StatefulSet? When a managed service will do. Running your own database on Kubernetes means owning backups, failover, version upgrades, storage expansion and the CSI driver's failure modes. Choose it when you have a genuine reason — cost at scale, a database with no managed equivalent, a strict data-residency constraint — and staff it accordingly. For most teams below a certain size, RDS plus a stateless cluster is the correct engineering decision, and saying so out loud is a senior move.

✓ Best practices — what experienced engineers do

- One chart, many values files. Environments differ by data, never by a forked copy of the templates.
- Omit `replicas` from the chart whenever an HPA owns the Deployment.
- Always set `ingressClassName`; never rely on a cluster default you did not configure.
- Separate liveness from readiness, and never put a dependency check in liveness.
- Set memory requests and limits on every container; derive them from a load test, not a template.
- Give every workload its own ServiceAccount, and disable token automount where it is not needed.
- Automate certificate renewal, then alert on days-to-expiry anyway — automation fails silently.

✗ Common mistakes beginners make

- Exposing every Service with `type: LoadBalancer` instead of one Ingress.
- Leaving resource limits unset, letting one runaway Pod starve every neighbour on its node.
- Wiring a database check into the liveness probe, producing cluster-wide restart storms.
- Binding the default ServiceAccount to `cluster-admin` to make a `Forbidden` error go away.
- Expecting an HPA to work without metrics-server installed, then debugging the HPA.
- Assuming a StatefulSet gives you replication, failover or backups. It gives you none of them.

4 Visualizations to Internalize

Redraw these from memory. If you can draw the loop, you can debug the loop.

✍ Redraw: a request from the internet to a Pod — LB, Ingress controller, TLS termination, Service, EndpointSlice — and mark where the controller's watch connection to the API server sits

 Write out the HPA formula, then work it: 6 replicas, target 70%, observed 40%. Does it scale down? Now observed 76% — does it scale up? Say why in one line each

5 Guided Hands-On Lab — Production-Shaping CloudBase

Goal. Take Day 5's raw manifests and turn them into something you would defend in a design review: a chart, an Ingress with real TLS, honest probes, a working HPA and a scoped ServiceAccount. Work in a local cluster (kind or minikube) with an ingress controller and metrics-server installed.

1 Convert the manifests into a chart, mechanically first

Create the chart skeleton, move Day 5's YAML into `templates/` unchanged, and confirm `helm template` reproduces byte-identical output before you parameterise anything.

 **Why:** Separating "restructure" from "parameterise" means that when the diff is wrong you know which of the two steps broke it. This is the same discipline as never mixing a refactor with a behaviour change in one commit.

2 Parameterise only what actually differs between environments

Extract image tag, replica count, resources, hostname and the autoscaling and ingress toggles. Leave everything else hardcoded.

 **Why:** Every value you expose is a value someone can set wrong. A chart with forty knobs has forty untested configurations; a chart with six has six.

3 Install with two values files and inspect the release ledger

Install to a `dev` namespace and a `staging` namespace from the same chart, then read the release Secret to see what Helm stored.

 **Why:** Seeing the gzipped manifests inside `sh.helm.release.v1` makes rollback stop being magic — you can point at where the previous revision physically lives.

4 Add the Ingress and prove the class matters

Enable the Ingress, then deliberately remove `ingressClassName` and observe that the object is created, no controller claims it, and `ADDRESS` stays empty with no error anywhere.

 **Why:** Creating this failure once, on purpose, immunises you against the most common Ingress support ticket in existence.

5 Terminate TLS and watch the ACME objects appear and vanish

Add a ClusterIssuer and the cert-manager annotation, then watch Certificate, Order and Challenge objects, plus the temporary solver Pod, while the certificate is issued.

 **Why:** Watching the temporary objects appear and be cleaned up teaches the challenge flow far better than reading it, and it is exactly what you will inspect when issuance is stuck.

6 Separate the probes and break each one independently

Give readiness a fast threshold and liveness a slow one. Make readiness fail (a toggle endpoint) and confirm the Pod leaves the EndpointSlice without restarting. Then make liveness fail and watch the restart count climb.

 **Why:** The distinction only becomes real once you have seen the two different consequences with your own `kubectl get pods -w`.

7 Load-test, then set requests and limits from the measurement

Generate load, record actual CPU and memory, and set requests near the observed steady state. Then set a deliberately low CPU limit and watch throttling metrics rise while the Pod stays alive.

 **Why:** This is the difference between throttled and OOMKilled, felt rather than read — and it is where your requests stop being a guess.

8 Enable the HPA and predict its arithmetic before it acts

With the HPA active, apply load and, before looking, compute the replica count the formula demands. Then remove the load and time how long scale-down takes.

 **Why:** Predicting the number turns the HPA from a black box into arithmetic. The scale-down delay you measure is the stabilisation window, and it will explain a future "why is it still at 12 replicas" question in seconds.

Production tip

Run `helm upgrade` while the HPA has scaled the Deployment above `replicaCount`. If the chart still hardcodes `replicas`, you will watch the API server delete Pods you need. Fix it with the template guard from Section 3.1 and run the same test again — this is a five-minute experiment that prevents a real, embarrassing incident.

6 Thinking Exercises

Think it through

The HPA rounds up with `ceil` and refuses to act inside a tolerance band. Both choices cost efficiency: you run more Pods than the ratio strictly demands, and you tolerate being off-target. What failure would each choice's absence produce, and what does that tell you about designing any feedback controller that acts on a noisy signal?

 Think it through

RBAC is purely additive and has no deny rules. Cloud IAM (Day 12) does have explicit denies. What class of mistake does each design make easy, and which would you rather debug at 2 am with an incident channel watching?

 Think it through

Shopify's fix was to split the Ingress controller's configuration into a slow path (reload for structural change) and a fast path (in-memory update for endpoint change). Name two other systems you have met that use the same split, and say what property makes a change eligible for the fast path.

7

Debugging Practice

 Debugging Lab • The HPA that will not scale during a traffic spike

Symptom. CPU on the order-api Pods is pinned, latency is climbing, and the HPA has been sitting at `minReplicas` for twenty minutes.

```
kubectl get hpa and describe
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
order-api	Deployment/order-api	<unknown>/70%	3	20	3

Conditions:

Type	Status	Reason	Message
AbleToScale	True	SucceededGetScale	the HPA controller was able to get
ScalingActive	False	FailedGetResourceMetric	unable to get metrics for resource cpu: no metrics returned from resource metrics API

- ✓ **Observe.** The target reads `<unknown>`, not a number. The HPA is not making a wrong decision — it has no input at all.
- ✓ **Hypothesise.** Either nothing is serving `metrics.k8s.io`, or it is serving but has no data for these Pods.
- ✓ **Investigate.** `kubectl get apiservices | grep metrics` to see whether the aggregated API is registered and Available; then `kubectl top pods`, which uses the same API and fails the

same way.

- ✓ **Root cause.** metrics-server is not installed — it is not part of a vanilla cluster. (The near neighbour: it is installed but its Pod is CrashLooping on kubelet TLS verification.)
- ✓ **Fix.** Install metrics-server; confirm `kubectl top pods` returns values; the HPA target populates within one loop period.
- ✓ **Prevent.** Treat metrics-server as a cluster prerequisite provisioned with the cluster, and alert on the HPA's `ScalingActive=False` condition — a silently inert autoscaler is worse than none, because you believe you are protected.

🔧 Debugging Lab • Certificate served is the wrong one, and browsers refuse the site

Symptom. After adding a new hostname, clients get `NET::ERR_CERT_COMMON_NAME_INVALID`. The controller is serving its own default self-signed certificate instead of the Let's Encrypt one.

`kubectl describe challenge and get secret`

```
Name:      cloudbase-tls-1-2937482
Status:    pending
Reason:    Waiting for HTTP-01 challenge propagation: failed to perform self check
            GET request: Get "http://api.cloudbase.example/.well-known/acme-
            challenge/6Xk...": dial tcp 203.0.113.9:80: connect: connection refused
```

NAME	TYPE	DATA	AGE
cloudbase-tls	kubernetes.io/tls	0	14m

- ✓ **Observe.** The Secret exists but holds zero keys — issuance never completed — and the Challenge is stuck on its own self-check over plain HTTP on port 80.
- ✓ **Hypothesise.** Either DNS does not point at this controller's load balancer, or port 80 is closed, or an HTTPS redirect is rewriting the challenge path before the solver sees it.
- ✓ **Investigate.** Resolve the hostname and compare with the Ingress `ADDRESS`; check the load balancer's port 80 listener and security group; `curl` the challenge path from outside the cluster and see what answers.
- ✓ **Root cause.** The load balancer was configured for 443 only, so the ACME server's HTTP-01 request could never arrive. Port 80 must stay reachable for the challenge even on an HTTPS-only site.
- ✓ **Fix.** Open port 80 to the controller and exempt `/.well-known/acme-challenge/` from the force-HTTPS redirect; the Challenge clears and the Secret fills within a minute.
- ✓ **Prevent.** Alert on `Certificate` objects whose `Ready` condition is `False` and on `days-to-expiry` below thirty — so a broken renewal surfaces weeks early rather than as an outage. Consider DNS-01 challenges instead, which need no inbound HTTP at all and are the only option for wildcard certificates.

8

Mini-Labs

 Mini-Lab • Beginner

45 min

Goal. Convert a two-file manifest set into a chart and perform a real rollback.

Business context. A bad image tag reached staging and the team took eleven minutes to revert by hand.

Architecture. One chart, one Deployment, one Service, two values files.

Requirements. Install, upgrade to a deliberately broken image tag, observe the failure, roll back to the previous revision.

Constraints. No `kubectl edit` — everything through Helm, so the ledger stays truthful.

Hints. `helm history` shows revisions; `helm get manifest <rel> -revision N` shows exactly what was applied.

Expected outcome. Rollback under thirty seconds, with the release history explaining what happened.

Production considerations. What makes a rollback *unsafe*? (Start with database migrations.)

Common pitfalls. Editing objects with `kubectl` between Helm operations, so the ledger and reality diverge.

 Mini-Lab • Intermediate

75 min

Goal. Route two services through one Ingress with TLS, and prove the load-balancer saving.

Business context. Finance flagged the cloud bill: fourteen load balancers for fourteen services.

Architecture. One Ingress controller, one Ingress object, two Services, one hostname, two paths.

Requirements. Path-based routing to both backends, TLS terminating at the controller, HTTP redirected to HTTPS.

Constraints. Exactly one `LoadBalancer` Service in the whole cluster — the controller's.

Hints. Use a local issuer or self-signed certificate if you have no public DNS; the flow is identical.

Expected outcome. Both paths reachable over HTTPS through a single address, with the certificate served by SNI.

Production considerations. Write down what breaks if the controller Pod is a single replica, and fix it.

Common pitfalls. A missing `ingressClassName`; a wrong `pathType`; the TLS Secret in the wrong namespace — it must be in the Ingress's namespace.

 Mini-Lab • Production-style

2.5 h

Goal. Take CloudBase from "running" to "defensible in a design review".

Business context. A traffic event is scheduled in two weeks; the service must scale and must not be a lateral-movement path.

Architecture. Chart with dev/staging/prod values → Ingress with TLS → Deployment with probes and measured resources → HPA 3–20 on CPU → scoped ServiceAccount.

Requirements. Load-test-derived requests and limits; separated probes; HPA with explicit scale-up and scale-down behaviour; a Role granting only what the app calls; token automount disabled if it calls nothing.

Constraints. No `cluster-admin` anywhere. `helm upgrade` must be safe while the HPA is active.

Hints. Record the load-test numbers in the repository next to the values file so the chosen figures are auditable.

Expected outcome. A documented scaling event: load applied, replica count over time, the formula's prediction beside the observed number.

Production considerations. Add a PodDisruptionBudget so a node drain cannot take you below safe capacity.

Common pitfalls. Setting a CPU limit equal to the request and then blaming the application for p99 latency spikes.

9 Engineering Notes

Cloud cost optimization

Ingress consolidation is one of the few Kubernetes changes with an immediately legible line-item saving: forty `LoadBalancer` Services become one, and forty hourly load-balancer charges plus their data-processing fees become one. The subtler savings are on the other side of the same chapter — honest requests stop you paying for reserved capacity nobody uses, and scale-down that actually happens stops you paying peak rates at 4 am. Orphaned PVCs from deleted StatefulSets are the classic silent cost; nothing reminds you they exist except the invoice.

Reliability principle

Add a PodDisruptionBudget to anything that matters. Requests, limits and an HPA govern steady state; a PDB governs *voluntary* disruption — a node drain during a cluster upgrade, a rebalance. Without one, a routine upgrade can evict every replica of a service at once, and it will do so at exactly the moment your attention is on the upgrade rather than on the application.

Senior engineer insight

Everything in this chapter is the same shape: watch a declared state, compute a desired state, write the difference, repeat. The Ingress controller does it to a proxy config, cert-manager to a Secret, the HPA to a replica count, the StatefulSet controller to Pods and PVCs. When you meet an unfamiliar Kubernetes component, three questions get you to competence fast: what does it watch, what does it compute, and what does it write? Then `kubectl describe` that object and read the events, which are the loop narrating itself.

Interview insight

Expect: "Why do CPU limits throttle but memory limits kill?" The answer that lands is mechanical, not definitional: CPU is compressible — the kernel's CFS bandwidth controller hands out a quota per period and simply deschedules the cgroup when it is exhausted, so the process gets slower. Memory is incompressible — there is no way to take back bytes already allocated, so the kernel's only lever is the OOM killer, and the container exits 137. Then connect it forward: that is why CPU limits show up as p99 latency mysteries and memory limits show up as restart loops.

Automation tip

Render and validate charts in CI before they reach a cluster: `helm lint`, then `helm template` piped into a policy checker, then a diff of the rendered output against the currently deployed release. Reviewing a diff of *rendered manifests* rather than of template source is the single highest-value habit in Helm-based delivery — template changes look small and produce large diffs surprisingly often.

10 Build-Along Project — CloudBase

Day 1 gave CloudBase a hardened host and a strict bootstrap script; Day 2 put it under version control with reviewable history; Day 3 gave it a network and DNS story; Day 4 turned it into a lean, non-root container image; Day 5 got that image running on Kubernetes as raw manifests — a Deployment, a Service and a ConfigMap applied by hand. Today those manifests stop being a demo.

🔧 Build-Along • Day 06 — package it, expose it, scale it, scope it

- Convert Day 5's manifests into `charts/cloudbase` with `Chart.yaml`, `values.yaml` and `templates/`. Verify `helm template` reproduces the Day 5 output before parameterising anything.
- Add `values-dev.yaml`, `values-staging.yaml` and `values-prod.yaml` carrying *only* the deltas: image tag, replicas, resources, hostname, and the ingress and autoscaling toggles.
- Guard `replicas` behind `if not .Values.autoscaling.enabled` so an upgrade can never fight the autoscaler.
- Add an Ingress with an explicit `ingressClassName`, host-and-path routing to the API, and a `tls` block naming `cloudbase-tls`.
- Install cert-manager, declare a ClusterIssuer against the ACME staging endpoint first, and let it issue and populate the Secret. Move to the production endpoint only once the staging flow is clean.
- Separate the probes: readiness on the dependency-aware endpoint with a fast threshold, liveness on a process-only endpoint with a slow one, plus a startup probe sized to a real cold start.
- Load-test the API and set requests and limits from what you measured. Commit the numbers alongside the values file.
- Add an HPA, 3–20 replicas at 70% CPU, with `behavior` set for fast scale-up and damped scale-down. Record predicted versus observed replica counts during a load run.
- Create a `cloudbase-api` ServiceAccount with a Role scoped to the exact resources the app reads, and set `automountServiceAccountToken: false` if it reads none.
- Add a StatefulSet plus headless Service and `volumeClaimTemplates` for CloudBase's datastore, and write `docs/stateful-decision.md` arguing for either this or a managed database — with the operational cost of each stated plainly.

Definition of done: `helm upgrade -install cloudbase ./charts/cloudbase -f values-prod.yaml` brings up the full stack from a clean namespace; the public hostname serves a valid, automatically issued certificate; a load test moves the replica count between 3 and 20 and back down on its own; `kubectl auth can-i -list` for the app's ServiceAccount shows only the permissions you deliberately granted; and a `helm rollback` to the previous revision restores the working state in under a minute.

11 Chapter Summary

Key takeaways

- ✓ A Helm chart is templates plus values; a release is a numbered revision stored as a Secret, which

is why rollback is instant.

- ✓ Helm upgrades use a three-way merge — old, live, new — so fields other controllers set are preserved.
- ✓ An Ingress is data. An Ingress controller is a reverse proxy watching the API server and rewriting its own config; `ingressClassName` decides which one owns the object.
- ✓ TLS termination ends encryption at the controller. cert-manager automates ACME issuance and renewal through Certificate, Order and Challenge objects.
- ✓ Liveness restarts, readiness de-routes. Never let a dependency failure reach liveness.
- ✓ CPU is compressible (throttle), memory is not (OOMKill, exit 137). Set both from measurement.
- ✓ $\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} \times \text{currentMetric} / \text{targetMetric})$, damped by a tolerance band and a scale-down stabilisation window.
- ✓ RBAC is additive with no deny rules; per-workload ServiceAccounts are the unit of least privilege.
- ✓ StatefulSets give identity and storage — never replication, failover or backups.

Things to remember

- `averageUtilization` is a percentage of the container's *request*, not of a core or a node.
- An HPA without metrics-server shows `<unknown>` and does nothing, silently.
- The TLS Secret must live in the same namespace as the Ingress that references it.
- `ingress-nginx` retires in March 2026 with no further security fixes; new platform work should target the Gateway API.

Common pitfalls (last look)

- Hardcoded `replicas` fighting an HPA
- missing `ingressClassName`
- DB check in liveness
- `cluster-admin` as a debugging tool
- limits copied from a blog
- port 80 closed, so ACME never completes

📄 One-Page Cheat Sheet

Chart — templates + values + `Chart.yaml`; rendered client-side into plain manifests.

Release — a numbered revision stored as `sh.helm.release.v1.<name>.v<N>`; the basis of rollback.

Ingress — routing data; a controller with a matching `ingressClassName` makes it real.

TLS termination — handshake ends at the controller; certificate chosen per hostname via SNI.

Probes — liveness restarts · readiness de-routes · startup suppresses both until boot completes.

Requests/limits — request = scheduling reservation; limit = cgroup ceiling. CPU throttles, memory OOM-kills.

HPA — $\text{ceil}(\text{replicas} \times \text{current}/\text{target})$, plus tolerance and a scale-down stabilisation window.

RBAC — Role/ClusterRole + binding; additive only, no deny; a RoleBinding to a ClusterRole scopes it to one namespace.

StatefulSet — ordinals + headless DNS + one PVC per Pod. Identity and storage only.

CLI quick reference

Day 6 commands worth memorising

```
# helm: render, inspect, ship, revert
helm template cloudbase ./charts/cloudbase -f values-prod.yaml # no cluster
helm upgrade --install cloudbase ./charts/cloudbase -f values-prod.yaml --wait
helm history cloudbase # every revision and its status
helm get manifest cloudbase --revision 3
helm rollback cloudbase 3

# ingress & tls
kubectl get ingress -A # empty ADDRESS = no controller claimed it
kubectl get ingressclass
kubectl describe certificate cloudbase-tls
kubectl get order,challenge -A # ACME flow in progress

# probes, resources, scaling
kubectl get pods -w # watch READY vs RESTARTS separately
kubectl top pods # uses the same API the HPA reads
kubectl describe hpa order-api # conditions explain every non-decision
kubectl get events --sort-by=.lastTimestamp | tail -20

# rbac
kubectl auth can-i --list --as=system:serviceaccount:cloudbase:cloudbase-api
kubectl get rolebinding,clusterrolebinding -A -o wide | grep cluster-admin

# stateful
kubectl get pvc -n cloudbase # orphaned claims still cost money
kubectl get statefulset db -o wide
```

👤 Interview questions to master

- › What is the difference between a liveness and a readiness probe, concretely — and what breaks if you conflate them?
- › Why do CPU limits throttle but memory limits kill?
- › When would you reach for a StatefulSet instead of a Deployment, and what does it *not* give you?
- › How does an Ingress Controller reduce the number of cloud load balancers you need, and what does it do when a Pod comes or goes?
- › What does the Horizontal Pod Autoscaler actually watch, and what formula does it apply?
- › Why is RBAC least-privilege important even inside a single cluster?
- › What does Helm store in the cluster, and how does that make `helm rollback` instant?
- › Walk me through what happens between `kubectl apply -f ingress.yaml` and a browser getting a valid certificate.

Further reading & official docs

- Kubernetes docs — *Horizontal Pod Autoscaling*: the exact algorithm, tolerance behaviour and the `behavior` field (kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/).
- Kubernetes docs — *Using RBAC Authorization*: Role vs ClusterRole, binding semantics, and the privilege-escalation prevention rules (kubernetes.io/docs/reference/access-authn-authz/rbac/).
- Kubernetes blog — *Ingress NGINX Retirement: What You Need to Know* (11 November 2025), and the Gateway API guides at gateway-api.sigs.k8s.io.
- Kubernetes blog — *Kubernetes v1.33: HorizontalPodAutoscaler Configurable Tolerance*, on per-

direction tolerance.

- Helm docs — *Changes Since Helm 2* (helm.sh/docs/v3/faq/changes_since_helm2/): the removal of Tiller, Secret-based release storage and the three-way merge patch.
- cert-manager docs — *HTTP01 challenge type* (cert-manager.io/docs/configuration/acme/http01/): the solver Pod, Service and temporary Ingress.
- RFC 8555 — *Automatic Certificate Management Environment (ACME)*: the protocol cert-manager and Let's Encrypt implement.
- Shopify Engineering — *Iterating Towards a More Scalable Ingress*: the NGINX reload bottleneck and the Lua dynamic-endpoint fix.
- Zalando — [kube-ingress-aws-controller](#) and [kube-metrics-adapter](#) on GitHub: autoscaling on request rate and queue depth in a 100+ cluster estate.

Production checklist

- ☐ Every environment ships from one chart; environments differ only by values files.
- ☐ `replicas` is omitted from any Deployment an HPA owns.
- ☐ Every Ingress sets `ingressClassName` explicitly.
- ☐ Certificate renewal is automated *and* monitored, with an alert on days-to-expiry.
- ☐ Liveness, readiness and startup probes check different things, deliberately.
- ☐ Every container has memory requests and limits derived from a load test.
- ☐ Every HPA has a metrics source that is verified working, and an alert on `ScalingActive=False`.
- ☐ Every workload has its own ServiceAccount; no workload is bound to `cluster-admin`.
- ☐ Token automount is disabled for Pods that never call the API server.
- ☐ Anything stateful has a tested restore, not just a replica.
- ☐ Critical services have a PodDisruptionBudget.

Key idea

What you will learn next (Day 07). You now have an application that packages, exposes, scales and defends itself — on a cluster that someone created by hand. Tomorrow you remove that last manual step: Terraform's declarative model, the state file and why it is the most dangerous artefact you own, the plan/apply cycle, modules, and how to describe an entire cluster, its network and its DNS as reviewable code.

DAY 07 • INFRASTRUCTURE AS CODE

Infrastructure as Code with Terraform 07

Providers, state, the plan DAG, modules and remote backends

🕒 Estimated study time: 9 hours

🎯 By the end of this day, you will be able to...

- Explain the declarative model: describe the end state, let the tool compute the steps.
- Describe what a provider actually is — a plugin process with a CRUD schema — and why versions get pinned.
- Reason about **state** as the map from configuration addresses to real-world resource IDs, and treat it as a secret.
- Read a plan as a walk over a dependency graph, and predict create versus update versus replace.
- Design modules with clean inputs and outputs, and detect, explain and correct drift.

1 The Big Picture

What problem this solves. On Day 3 you designed a network on paper: a VPC, public and private subnets across two availability zones, an internet gateway, a NAT gateway, route tables. On Days 5 and 6 you ran a cluster on top of it. Every one of those resources currently exists because a human clicked a console or typed a CLI command. Nobody can say with certainty what the security group rules are, what changed last Tuesday, or how to rebuild the whole thing in a second region by Friday. Infrastructure as Code replaces "what somebody did" with "what the repository says", and makes the gap between the two visible, reviewable and correctable.

Why DevOps engineers need it. Because the console does not scale past one person and one environment. Three environments times forty resources is a hundred and twenty hand-made objects that nobody remembers creating, and the first time production differs from staging in a way that matters, you will spend a day comparing dropdowns. IaC turns infrastructure into something you can diff, review in a pull request, revert with `git revert`, and reproduce from scratch. That is not a productivity nicety; it is how a disaster-recovery plan becomes credible.

What engineers did before it. First there was ClickOps — the console, plus a wiki page that went stale within a month. Then shell scripts wrapping the cloud CLI, which worked exactly once, because scripts are *imperative*: rerunning "create a VPC" on a second run either errors or creates a second VPC. Then vendor-native declarative tools (AWS CloudFormation in 2011), which fixed idempotency but locked you into one cloud and one very verbose template language. Terraform's contribution in 2014 was to put a plugin boundary between the engine and the cloud, so one language and one workflow could drive AWS, an on-prem vSphere, a DNS provider and a SaaS product in a single plan.

Key idea

Terraform is a *reconciliation engine*, exactly like the Kubernetes controllers you met on Day 5. You declare desired state; it observes actual state; it computes and executes the difference. The only structural distinction is when the loop runs. Kubernetes reconciles continuously, forever, on its own. Terraform reconciles once, when a human runs `apply`. Everything strange about Terraform — state files, drift, locking — follows from that one difference.

Where it fits in a modern cloud architecture. Terraform sits at the bottom of the delivery stack, below the cluster and beside the pipeline. It provisions the boxes and the wires; other tools configure and fill them.

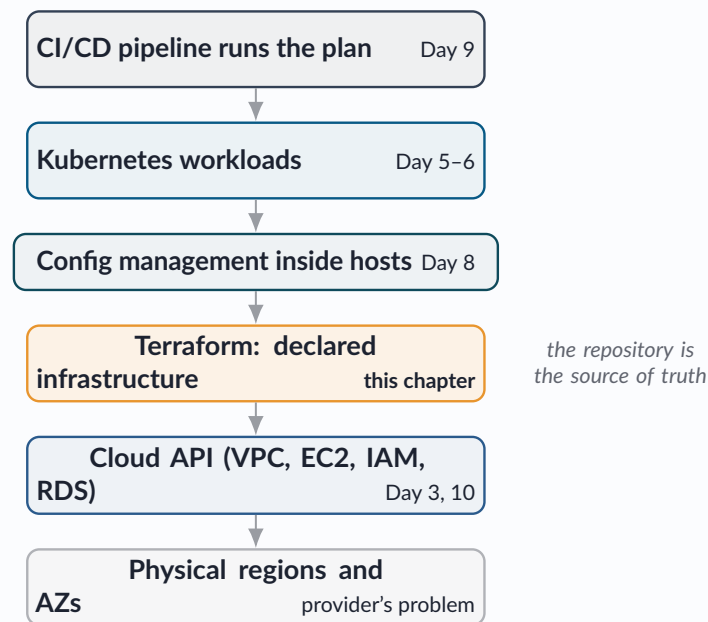


Fig 7.0 — Where today sits. Terraform creates the substrate; Ansible configures inside it; Kubernetes schedules on top of it.

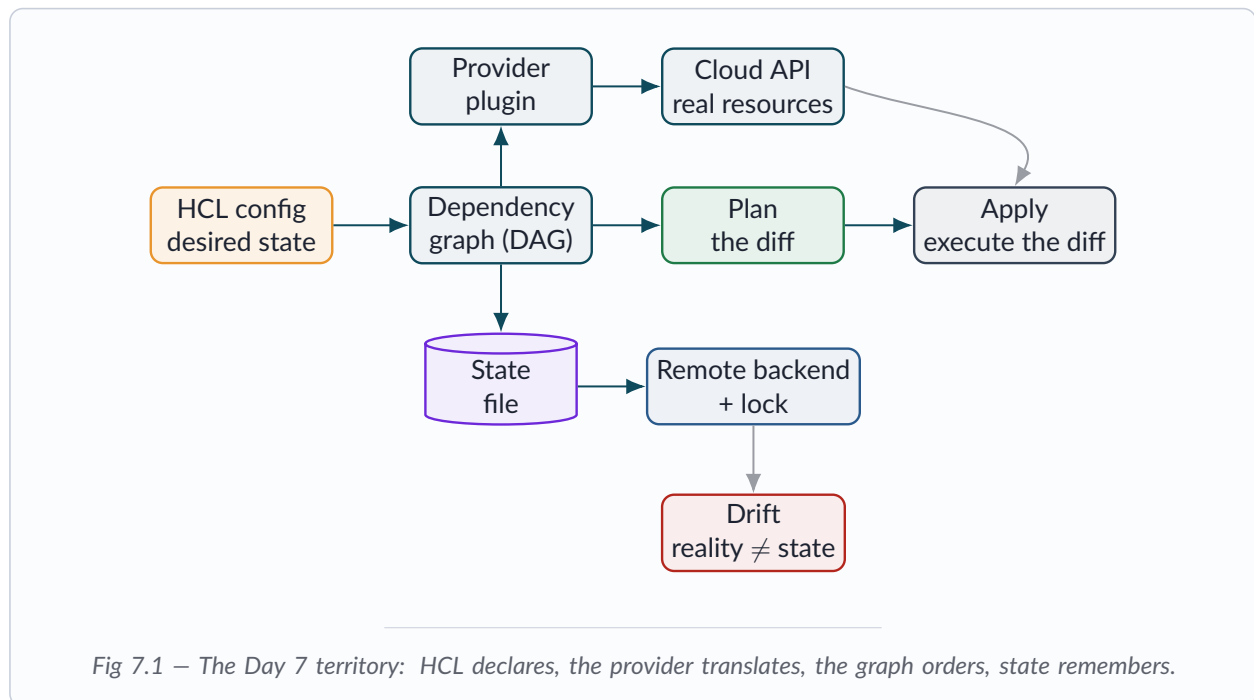
How it connects to previous chapters. Day 1 ended on **idempotency** — a script that is safe to rerun. Terraform is that idea taken to its conclusion: you do not write the rerun logic at all, you declare the outcome. Day 2's pull-request review now applies to networks and IAM roles, not only application code. Day 3's CIDR plan becomes literal HCL today. Day 5's reconciliation loop is the same algorithm. And Day 8's Ansible is the complement, not the competitor: Terraform decides that a machine exists, Ansible decides what is installed on it.

Where companies use it in production. Slack manages close to 1,400 Terraform state files across AWS, GCP, DigitalOcean and NS1, deliberately splitting state per region and per child account so that any single apply touches a small blast radius. They store state in a version-enabled S3 bucket with locking, run every apply from dedicated Jenkins workers holding the privileged IAM roles, and give engineers read-only "Ops" boxes so they can `plan` but never `apply` by hand. They built an internal "Smart Planner" that works out which of those 1,400 state files a pull request touches, plans each one, and posts the results back onto the PR — turning `terraform plan` into a code-review artifact. Shopify's Conversations team went further in a different direction: they wrote their own Terraform provider for Twilio TaskRouter so that contact-centre routing rules, previously edited by hand in a vendor GUI, became reviewable HCL with an Atlantis-driven plan on every pull request. Their

reported win was not speed; it was `git blame` on a routing rule.

Common misconceptions. That Terraform "manages your infrastructure" continuously (it does nothing between applies); that state is a cache you can delete (it is the only record linking your code to real resource IDs); that a plan is a preview of the code (it is a preview of the difference between code, recorded state and live reality); and that Terraform and Ansible compete (they answer different questions).

2 Concept Map



Terminology to anchor: **declarative** → **provider** → **resource** → **dependency graph** → **refresh** → **plan** → **apply** → **state** → **backend** → **lock** → **module** → **drift** → **import**.

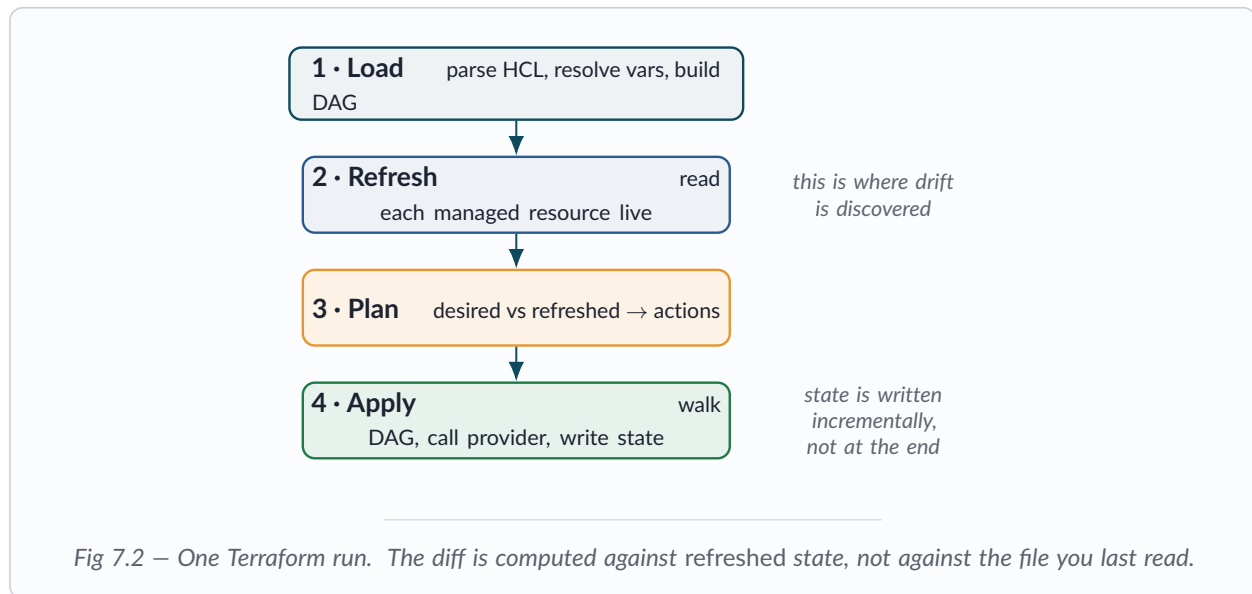
3 Guided Learning

3.1 Declarative Infrastructure and the Reconciliation Model

What is it? You write files describing the infrastructure you want to exist. You never write "create", "update" or "delete". The tool compares your description with what it believes exists, and derives the verbs itself.

Why does it exist? Because imperative provisioning has no safe second run. A script that says "create a VPC, then create a subnet in it" is correct on an empty account and wrong everywhere else — on rerun it either fails, duplicates, or silently skips because someone added an `if` that was never tested. The engineering problem is that the *starting state is unknown*. Declarative tools solve it by making the starting state an input to be measured rather than an assumption to be guessed.

How does it work internally? A run has four phases, and confusing them is the source of most beginner surprises.



When should I use it? For anything that outlives a single afternoon and that more than one person depends on: networks, IAM, clusters, databases, DNS, queues, buckets.

When should I not? For a genuinely throwaway experiment you will destroy in an hour, and — more importantly — for things that change many times per day on their own. Autoscaling group sizes, Kubernetes Deployment replica counts, and anything a runtime controller adjusts should not be pinned in Terraform, or every plan will fight the controller. Declare the resource; let `lifecycle.ignore_changes` concede the fields something else owns.

Production example. Shopify's Conversations team applied this to a domain most people would never call "infrastructure": Twilio TaskRouter workflows for their contact centre. Previously an operations person edited routing rules in a vendor GUI — no tests, no rollback, no record of intent. They built a custom Terraform provider, put the rules in HCL, and wired Atlantis so every change arrived as a pull request with a plan attached. The declarative model turned a GUI into a reviewable artifact, and non-engineers began opening infrastructure PRs.

Design trade-off

Declarative buys reproducibility and costs expressiveness. There is no clean way to say "do X, then wait for a human, then do Y" in a declaration — multi-step operational choreography (a database cutover, a blue/green switch with a soak period) is genuinely awkward in HCL. Mature teams accept the split: Terraform owns the shape of the infrastructure, a pipeline or runbook owns the choreography.

3.2 Providers — the Plugin Boundary That Made Terraform Multi-Cloud

What is it? A provider is a separate executable that teaches Terraform about one API. The AWS provider knows what an `aws_vpc` is, which arguments it takes, which of them force replacement when changed, and how to create, read, update and delete one.

Why does it exist? CloudFormation is written by AWS, for AWS, and shipped with AWS. That is fine until your infrastructure includes a DNS vendor, a monitoring SaaS and a second cloud. Terraform's authors moved the vendor-specific knowledge out of the engine entirely, so the engine only knows about *resources with schemas and dependencies*. Everything vendor-shaped lives behind the plugin

boundary. That single design choice is why there are thousands of providers today and why you can express "create the VPC, then create the DNS record pointing at its NAT gateway's elastic IP" in one graph.

How does it work internally? When you run `terraform init`, Terraform downloads provider binaries into `.terraform/providers` and records their exact versions and checksums in `.terraform.lock.hcl`. During a plan, Terraform launches each provider as a child process and talks to it over a local gRPC connection. The engine asks structured questions — "validate this configuration", "read the current state of this object", "given prior state and desired config, what would the resource look like after apply?" — and the provider answers. The engine never speaks HTTP to AWS; the provider does.

Behind the scenes

The plan diff you read on screen is largely computed by the *provider*, not the engine. The engine sends prior state plus desired configuration and asks the provider to produce a planned new state. The provider fills in what it knows and marks anything it cannot predict — a generated ID, an ARN — as *(known after apply)*. That is why unknown values propagate: if a subnet's ID is unknown, everything referencing it is unknown too, and the plan output cascades question marks downstream.

`versions.tf` — pin the engine and every provider

```
terraform {
  required_version = "~> 1.9"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.60"    # allow 5.60.x -> 5.x, never 6.0
    }
  }
}

provider "aws" {
  region = var.region

  default_tags {
    tags = {
      Project    = "cloudbase"
      ManagedBy = "terraform"
      Env        = var.env
    }
  }
}
```

Security warning • unpinned providers are a supply-chain and a stability problem

Without a version constraint, `terraform init` on a fresh CI runner resolves to whatever is newest that morning. A provider major version can change a default, tighten validation, or alter which arguments force replacement — so a plan that was empty yesterday proposes destroying your NAT gateway today, and the person who typed `yes` did nothing wrong. Commit `.terraform.lock.hcl`: it records the exact versions *and their checksums*, so CI resolves the same bytes you tested.

Senior engineer insight

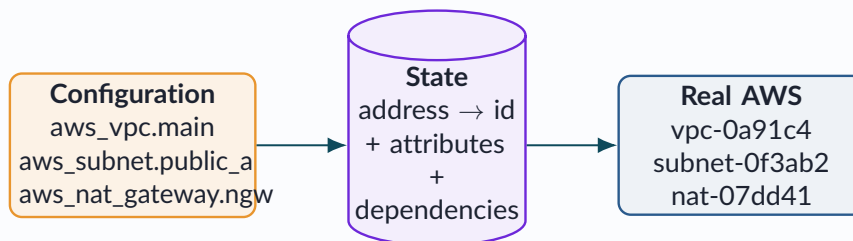
Slack supports several AWS provider versions simultaneously rather than forcing a big-bang upgrade. Their Terraform wrapper reads each state directory's `versions.tf` and selects the matching binary and plugins, so provider 3.x and 4.x configurations coexist while teams migrate at their own pace,

with a purpose-built pre-flight tool checking a bulk upgrade before it runs. The lesson generalises: at scale, "upgrade everything at once" is not an option, so your tooling must make version heterogeneity survivable.

3.3 State — the Map From Your Code to the Real World

What is it? A JSON document recording, for every resource you have created, the address in your configuration (`aws_subnet.public["a"]`), the real object's ID (`subnet-0f3a...`), every attribute as last observed, and the dependency edges that were in effect. It is Terraform's memory.

Why does it exist? Because there is no reliable way to ask a cloud "which of these objects did I create from this configuration?" Tags are a convention, not a guarantee; two resources can look identical; some resources have no queryable name at all. Without state, Terraform could not tell "this subnet is mine and needs updating" from "this subnet is someone else's, leave it alone" or "the subnet I made is gone, recreate it". State also stores the dependency graph as it was at last apply, which is how Terraform can still destroy resources in the right order after you have deleted their configuration blocks entirely.



no state = Terraform sees an empty account and proposes creating everything a second time

Fig 7.3 — State is the join table between configuration addresses and cloud resource IDs. Lose it and both sides become orphans.

How does it work internally? The file carries a `version`, a `lineage` (a UUID identifying this state's ancestry) and a `serial` that increments on every write. A backend uses those to refuse a write from a stale reader, and to notice if you point a workspace at a state from a different lineage. Attributes are stored verbatim as the provider returned them — which is exactly why an RDS password, an IAM access key or a generated TLS private key ends up in the file as plaintext even if you were careful never to write it in HCL. Terraform marks such values `sensitive` for `display` purposes; that redaction is a console courtesy, not encryption.

Security warning • state is the crown jewel

Treat the state file as a credential store, because in practice it is one. Never commit it to Git — not even to a private repository, because history is forever and clones are everywhere. Put it in a bucket with server-side encryption, versioning, public access fully blocked, and an IAM policy that grants read to almost nobody. If you must inspect it, use `terraform show` or `terraform state show`, which apply redaction, rather than opening the JSON.

Why remote, and why locked? With local state, two engineers have two divergent memories of the

same account, and whoever applies second silently recreates or destroys what the first one made. A remote backend gives the team one shared memory. Locking then solves the second problem: two applies running concurrently against the same state produce a lost update — both read serial 41, both write serial 42, one set of changes vanishes from the record while the resources it created remain in the cloud, unmanaged and unbilled to anyone's attention.

backend.tf – remote state with native S3 locking

```
terraform {
  backend "s3" {
    bucket      = "cloudbase-tfstate-prod"
    key         = "network/terraform.tfstate"
    region      = "eu-west-1"
    encrypt     = true
    kms_key_id  = "arn:aws:kms:eu-west-1:111122223333:key/abcd-1234"

    # Native S3 locking via a conditional-write .tflock object.
    # Replaces the old dynamodb_table argument, which is deprecated.
    use_lockfile = true
  }
}
```

Infrastructure insight • what changed recently, and why it matters

The S3 backend historically required a separate DynamoDB table for locking, because S3 offered no compare-and-swap primitive. Once S3 gained conditional writes, HashiCorp added `use_lockfile`, which takes the lock by conditionally creating a `<key>.tflock` object beside the state — one service instead of two. The `dynamodb_table` argument is now deprecated and slated for removal in a future minor version. For a migration you may set both at once so that older and newer CLI versions still exclude each other; once every operator is upgraded, drop `dynamodb_table` and delete the table. Note that `use_lockfile` still defaults to `false` — an unlocked backend is the silent default, so you must opt in.

Disaster recovery tip

Enable S3 object versioning on the state bucket before you store anything in it. Versioning turns "somebody ran `state rm` on the wrong address" from a reconstruction project into a thirty-second rollback to the previous object version. Slack relies on exactly this property. Also keep the bucket in a separate account or at minimum a separate blast radius from the infrastructure it describes — state that lives inside the environment it can destroy is not a backup.

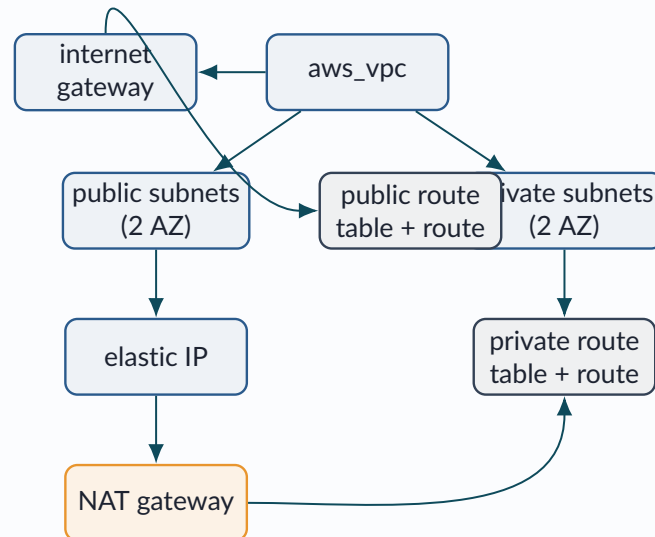
3.4 The Plan DAG — How Terraform Decides Order and Verbs

What is it? Terraform builds a directed acyclic graph whose nodes are resource instances and whose edges are dependencies. The plan is a walk over that graph; the apply is the same walk, executing.

Why does it exist? Because ordering is the hard part of provisioning, and humans get it wrong. A subnet needs its VPC's ID. A NAT gateway needs an elastic IP and a public subnet. A private route table's default route needs the NAT gateway's ID. Encoding that by hand as a numbered runbook is fragile and unparallelisable. Encoding it as a graph means order is *derived*, and independent branches run concurrently for free.

How does it work internally? Edges come from three places. Most are **implicit**: writing `vpc_id = aws_vpc.main.id` inside a subnet tells Terraform that the subnet reads an attribute of the VPC, so the VPC must exist first. Some are **explicit**, declared with `depends_on` when a real ordering requirement exists that no attribute reference expresses — an IAM policy that must be attached before a service

can assume the role, for instance. The rest are structural: providers, variables, outputs and module boundaries all become nodes. Terraform then topologically sorts and walks the graph, executing up to ten operations in parallel by default. **Destroy is the same graph with the edges reversed** – which is precisely why Terraform deletes the route table before the NAT gateway before the subnet before the VPC without being told to.



destroy reverses every arrow

Fig 7.4 – The CloudBase network graph. Terraform derives this from attribute references; you never write the order.

How does it decide the verb? For each instance the provider compares prior state with desired configuration attribute by attribute. Four outcomes exist, and the third is the one that ends careers.

Symbol in the plan	What it means and when it happens
+ create	No prior state for this address. New resource.
~ update in-place	Only fields the API can modify were changed – a tag, a description.
-/+ replace	A changed field is marked <i>ForceNew</i> by the provider because the API cannot modify it. Destroy then create – data and IP addresses are lost.
- destroy	The configuration block was removed, or the resource was targeted for removal.

terraform plan – read the symbols, not the summary line

```

# aws_subnet.private["b"] must be replaced
-/+ resource "aws_subnet" "private" {
  ~ cidr_block = "10.0.20.0/24" -> "10.0.24.0/24" # forces replacement
  ~ id         = "subnet-0f3ab2c1" -> (known after apply)
    vpc_id      = "vpc-0a91c4de"
}

# aws_route_table.private will be updated in-place
~ resource "aws_route_table" "private" {
  ~ tags = { "Name" = "cb-private" -> "cloudbase-private" }
}

Plan: 1 to add, 1 to change, 1 to destroy.

```

⚠ Common misconception

"1 to add, 1 to destroy" at the bottom of a plan reads like two independent, harmless operations. It is frequently one resource being *replaced*, and for a stateful resource that is total data loss. Read upward for `-/+` and for the comment `# forces replacement`; the provider tells you exactly which attribute triggered it. If a replacement is genuinely required but must not be an outage, add `lifecycle { create_before_destroy = true }` so the new object exists before the old one goes away — and verify the API allows both to coexist.

💡 Production tip

Always save the plan and apply *that file*: `terraform plan -out=tfplan` then `terraform apply tfplan`. An interactive `apply` re-plans at the moment you type `yes`, so what you approved and what runs can differ if someone else changed reality in between. Applying a saved plan file also makes the pipeline's approval gate meaningful — the artifact a human approved is the artifact that executes.

3.5 Modules — Composition Without Copy-Paste

What is it? A module is a directory of `.tf` files with declared inputs (`variable`) and declared outputs (`output`). Calling it instantiates every resource inside it under a namespaced address. Every configuration is already a module — the one you run in is the **root module**.

Why does it exist? Because the alternative is three near-identical copies of your network code in `dev/`, `staging/` and `prod/`, and by month four they differ in ways nobody intended. The problem is not typing; it is that copies drift independently and there is no single place to fix a security-group mistake.

How does it work internally? At `init`, Terraform downloads or copies each module source into `.terraform/modules` and records it in a manifest. During graph construction the module boundary is flattened: `module.vpc.aws_subnet.private["a"]` is an ordinary graph node, and dependencies cross module boundaries transparently through inputs and outputs. Modules provide *naming and interface* discipline; they are not runtime isolation, and they do not get their own state.

`modules/network/variables.tf` — a module is defined by its interface

```
variable "env"      { type = string }
variable "cidr"     { type = string }
variable "az_count" {
  type      = number
  default   = 2
  validation {
    condition     = var.az_count >= 2 && var.az_count <= 3
    error_message = "Two AZs minimum for availability; three maximum for cost."
  }
}

variable "single_nat_gateway" {
  type      = bool
  default   = false
  description = "One NAT for all AZs. Cheaper, but a per-AZ failure domain."
}

output "private_subnet_ids" { value = [for s in aws_subnet.private : s.id] }
output "vpc_id"             { value = aws_vpc.this.id }
```

`envs/prod/main.tf` – the root module composes, it does not implement

```
module "network" {
  source = "git::ssh://git@github.com:cloudbase/tf-modules.git//network?ref=v1.4.0"

  env          = "prod"
  cidr         = "10.0.0.0/16"
  az_count     = 2
  single_nat_gateway = false # prod pays for HA; dev does not
}
```

💡 Senior engineer insight

The hardest part of module design is not the code, it is the interface. A module with thirty input variables is a leaky abstraction that has re-exposed the provider it was meant to hide; a module with two is usually too rigid to reuse. Aim for inputs that express *intent* (`single_nat_gateway`, `env`) rather than plumbing (`route_table_id`), and let the module derive the plumbing. If a caller needs a raw resource ID from inside, that is a signal your boundary is drawn in the wrong place.

📈 Scalability note

Slack's module story is a good map of the road ahead. They began with relative-path modules – fastest to change, but a single edit could silently break dozens of state files. They moved to an internal catalog where a pipeline versions each module into a tarball in S3 with a `versions.json` history, plus a browser tool so teams can find modules. The cost is honest: publishing before consuming slows iteration, and there is still no way to force every consumer to test an upgrade. Version pinning does not eliminate breakage; it converts unplanned breakage into scheduled work.

💰 Cloud cost optimization

A NAT gateway is billed per hour *and* per gigabyte processed, in every AZ you put one in. Making `single_nat_gateway` a module input rather than a hardcoded choice lets development and staging run one gateway while production runs one per AZ – a decision the module makes cheap to express and cheap to revisit. The general principle: parameterise the knobs that trade cost against availability, and set the default to the safe option so a careless caller overspends rather than under-protects.

3.6 Drift, Import, and Moving Things Without Destroying Them

What is it? **Drift** is the divergence between what state records and what the cloud actually contains, caused by anything that changes a managed resource outside Terraform: a console click, an incident fix at 3am, another automation, or the cloud provider itself changing a default.

Why does it matter? Because Terraform's next plan does not know the change was deliberate. It sees a discrepancy and proposes to close it – by reverting your emergency fix. The scariest version is silent: someone widens a security group during an incident, nobody records it, and the next unrelated apply quietly narrows it again three weeks later, causing a second incident whose cause looks unrelated to its trigger.

How does it work internally? The refresh phase reads every managed resource from the API and updates the in-memory state before diffing. So drift always surfaces as an unexplained ~ line in a plan you expected to be empty. `terraform plan -refresh-only` performs the refresh and shows you what changed in the world without proposing to fix it – the correct first move when you suspect drift. From there you have exactly two honest choices: revert the manual change (apply the plan), or update the code to match reality (the change was right, the method was wrong).

```
terraform plan -refresh-only - drift, discovered
```

Note: Objects have changed outside of Terraform

```
# aws_security_group.api has changed
~ resource "aws_security_group" "api" {
  ~ ingress = [
    + { from_port = 22, to_port = 22, cidr_blocks = ["0.0.0.0/0"] },
  ]
}
```

This is a refresh-only plan, so Terraform will not take any actions.

Bringing existing resources under management. Days 5 and 6 built the CloudBase cluster by hand. You do not have to destroy and rebuild it to codify it — `import` adopts an existing object into state. Since Terraform 1.5 this is a *declarative* `import` block rather than a state-mutating CLI command, which means importing is now a plannable operation you can review in a pull request. Running `plan` with `-generate-config-out` even writes a first draft of the resource block from the live object.

```
import.tf - adopt, do not recreate
```

```
import {
  to = aws_vpc.main
  id = "vpc-0a91c4de"
}

# terraform plan -generate-config-out=generated.tf
# -> writes a draft resource block matching the live object
# then review, fold into your real files, and apply.
```

☰ Operational excellence

Two blocks prevent the most avoidable destructive plans. `moved` tells Terraform that a resource was *renamed or moved into a module*, so it updates the state address instead of destroying the old address and creating a new one. `removed` tells Terraform to forget a resource without deleting the real object — the reviewable replacement for `terraform state rm`. Refactoring a working configuration should never show a destroy line; if it does, you are missing a `moved` block.

Workspaces versus directories. Workspaces let one configuration hold several states, selected by name. They are convenient for short-lived, structurally identical copies. For long-lived environments most serious teams use separate directories with separate backend keys instead, for two reasons: environments always eventually differ structurally (prod has a replica, dev does not), and workspace selection is invisible ambient state — an operator who forgets `terraform workspace select` applies a dev change to production with no visual cue. Separate directories make the target explicit in the path.

♥ Reliability principle

Make the blast radius of a single apply small on purpose. Slack splits state per region and per child account, and isolates large services into their own state files, precisely so no single `apply` can touch everything. Splitting has a real cost — you need `terraform_remote_state` data sources or explicit inputs to pass values across boundaries — but the cost is paid once, and the alternative is a monolithic state where one bad plan is an outage.

✓ Best practices — what experienced engineers do

- Remote state, encrypted, versioned, locked. Commit `.terraform.lock.hcl`; never commit `.tfstate`.
- Pin `required_version` and every provider; upgrade deliberately, not by accident of timing.
- `plan -out=tfplan` then `apply tfplan` — approve and execute the same artifact.
- Read plans upward for `-/+` and `forces replacement` before trusting the summary line.
- Split state by environment and by blast radius; make the target explicit in the directory path.
- Use `import` to adopt existing resources and `moved` to refactor without destroying.
- Tag everything with `default_tags` so cost reports and orphan hunts are possible later.

✗ Common mistakes beginners make

- Committing a state file — it contains passwords and keys in plaintext, and Git history is forever.
- Applying without reading the plan, then discovering `-/+` meant the database was replaced.
- Fixing production in the console and not telling anyone, leaving drift for a future stranger.
- Running with no lock, so two concurrent applies produce a lost update and orphaned resources.
- Hand-editing state JSON instead of using `import`, `moved` or `state mv`.
- Renaming a resource in code and shipping the resulting destroy-and-create as if it were a rename.
- One giant state file for the whole company, where every plan is slow and every apply is risky.

4 Visualizations to Internalize

Redraw these from memory. If you can draw the graph, you can predict the plan — and predicting the plan is the whole skill.

 Redraw: the CloudBase network DAG — VPC, IGW, public and private subnets, EIP, NAT gateway, both route tables — with every arrow pointing from dependency to dependent. Then mark what destroy order would be.

 *Redraw: the four phases of a run, and mark exactly where drift is discovered, where the lock is taken and released, and where state is written*

5 Guided Hands-On Lab — Codify the CloudBase Network

Goal. Turn the Day 3 network design into Terraform with shared, locked, encrypted remote state — then prove it is reproducible by destroying and recreating a non-production copy. This is the foundation Day 10 builds compute and data on top of.

1 Create the state backend before you create anything else

By hand (or with a tiny separate configuration), create the S3 bucket: versioning on, default encryption on, public access fully blocked, a restrictive bucket policy.

 *Why: The backend is the one bootstrap paradox in Terraform — state has to live somewhere before there is state. Creating it deliberately, once, and never touching it again is cleaner than the recursive trickery people attempt.*

2 Declare the backend and pin your versions

Write `backend.tf` with `use_lockfile = true` and `encrypt = true`; write `versions.tf` with `required_version` and a provider constraint. Run `init` and `commit` `.terraform.lock.hcl`.

 *Why: You are deciding two things that are painful to change later: where the team's shared memory lives, and which provider version the team's plans are computed against.*

3 Express the CIDR plan as data, not as literals

Derive subnet CIDRs from the VPC CIDR with `cidrsubnet()`, and take the AZ list from a data source rather than hardcoding `eu-west-1a`.

 *Why: Hardcoded AZ names are the single most common reason a configuration cannot be reused in another region. Deriving the plan keeps the Day 3 arithmetic in one place, where it can be reviewed.*

4 Build the public path first, and plan without applying

VPC, internet gateway, public subnets with `map_public_ip_on_launch`, a public route table with a default route to the IGW, and associations. Run `plan -out` and read every line.

 *Why: Reading a plan of resources you fully understand is how you learn to read a plan of resources you do not. Note which values are (known after apply) and ask yourself why.*

5 Add the private path and watch the graph deepen

Private subnets, an elastic IP, a NAT gateway in a public subnet, a private route table whose default route targets the NAT gateway.

💡 *Why:* This is where the dependency chain becomes non-obvious: the NAT gateway lives in a public subnet but serves private ones. Terraform derives that ordering from your attribute references — you never state it.

6 Apply, then run plan again and demand an empty diff

A second plan must report no changes.

💡 *Why:* An idempotent, empty second plan is the proof that your configuration fully describes reality. A perpetual diff — a field the API normalises, a tag the cloud adds — is a real bug that will hide genuine changes forever if you tolerate it.

7 Cause drift on purpose, then diagnose it properly

In the console, add an inbound rule or retag a subnet. Run `plan -refresh-only` first, then a normal plan.

💡 *Why:* You are practising the incident response, not the vandalism. `-refresh-only` tells you what the world did; the normal plan tells you what Terraform intends to do about it. Deciding which of the two is right is a judgement call, and you should make it consciously.

8 Extract it into a module and instantiate it twice

Move the resources into `modules/network`, expose `env`, `cidr`, `az_count`, `single_nat_gateway`. Add `moved` blocks for every address that changed. Then instantiate a second, non-overlapping copy for `dev`.

💡 *Why:* Refactoring must not be destructive. If your plan after extraction shows destroys, your `moved` blocks are incomplete — fix them until the plan is empty, and you will have learned the single most valuable Terraform refactoring skill.

💡 Production tip

Never grant apply permissions to a human laptop in production. Slack's engineers plan from read-only "Ops" boxes and apply only through Jenkins workers that hold the privileged IAM roles. The pattern to copy: humans propose and review; a machine identity with a narrow role executes; every apply leaves a log with a commit SHA attached. Day 9 wires exactly this into your pipeline.

6 Thinking Exercises

🧠 Think it through

Kubernetes reconciles continuously and needs no state file — it queries the API server, which is the record. Terraform reconciles once and needs state. Explain why the difference in *when* the loop runs forces the difference in *whether* a state file is required. What would Terraform have to become to stop needing one?

 Think it through

Your team is deciding between one state file for the whole platform and thirty state files split by service. Enumerate what gets harder in each direction — not just risk, but cross-boundary references, plan duration, onboarding, and who has to be in the room to ship a change. Where would you draw the first boundary, and what evidence would tell you it was wrong?

 Think it through

An engineer widened a security group in the console at 3am to stop an outage, and it worked. Twelve days later an unrelated apply reverts it. Whose failure is this, and which of the following would actually have prevented it: alerting on drift, denying console write access, requiring a follow-up PR within 24 hours, or refresh-only checks in CI? Rank them by cost and by effectiveness — they are not the same ordering.

7 Debugging Practice

 Debugging Lab • "Error acquiring the state lock" — and nobody else is running Terraform

Symptom. Every plan fails immediately. The engineer who ran the last apply says their laptop lost Wi-Fi during it.

```
terraform plan
```

```
Error: Error acquiring the state lock

Error message: operation error S3: PutObject, https response error
StatusCode: 412, PreconditionFailed: At least one of the pre-conditions
you specified did not hold
Lock Info:
ID:          9f4c2a1e-77b3-4a90-b0c1-2d5e6f7a8b90
Path:        cloudbase-tfstate-prod/network/terraform.tfstate.tflock
Operation:   OperationTypeApply
Who:         amine@ops-box-3
Created:     2026-07-19 02:14:51.229 +0000 UTC
```

✓ **Observe.** A `.tflock` object exists in the bucket and the conditional write is being refused with 412 — the lock is held, not the state corrupted.

✓ **Hypothesise.** The apply process died before releasing the lock, so the lock object was

never deleted. This is a *stale* lock, not a concurrent one.

- ✓ **Investigate.** Confirm the holder in `Who` and `Created`; ask that person directly. Then check whether resources were partially created — the state object's `serial` and its S3 version history tell you whether any writes landed.
- ✓ **Root cause.** A crashed or network-interrupted apply. Locks are released by the process that took them; if it is killed, nothing cleans up.
- ✓ **Fix.** Once you have *verified with a human* that no apply is running, `terraform force-unlock 9f4c2a1e-...` using the exact ID. Then run `plan -refresh-only` to see what actually got created before the crash before doing anything else.
- ✓ **Prevent.** Run applies from CI, not laptops, so a dropped connection cannot orphan a lock. Alert on `.tflock` objects older than the longest plausible apply. Never `force-unlock` on a hunch — unlocking a genuinely running apply is how you get two writers and a lost update.

🔧 Debugging Lab • A tag change that proposes to destroy the NAT gateway

Symptom. A junior engineer opened a PR that only renames resources for consistency. CI attaches a plan showing 6 to destroy.

```
plan attached to the pull request

# aws_nat_gateway.ngw will be destroyed
# (because aws_nat_gateway.ngw is not in configuration)
- resource "aws_nat_gateway" "ngw" {
  - id           = "nat-07dd4193ce8f2b01a"
  - allocation_id = "eipalloc-0b3c9e12"
}

# aws_nat_gateway.this will be created
+ resource "aws_nat_gateway" "this" {
  + id           = (known after apply)
}

Plan: 6 to add, 0 to change, 6 to destroy.
```

- ✓ **Observe.** Add and destroy counts are equal and the destroy reason is "not in configuration" — the classic signature of a rename, not a deletion.
- ✓ **Hypothesise.** The resources still exist; only their *configuration addresses* changed. State still maps the old addresses, so Terraform sees six orphans and six new declarations.
- ✓ **Investigate.** Diff the PR: are the resource bodies identical and only the labels different? Cross-check `terraform state list` against the new addresses.
- ✓ **Root cause.** A refactor without `moved` blocks. Terraform has no way to know `ngw` and `this` are the same object — an address is an identity.
- ✓ **Fix.** Add a `moved` block per renamed address (`from = aws_nat_gateway.ngw, to = aws_nat_gateway.this`) and re-plan. The correct outcome is *no changes at all*.
- ✓ **Prevent.** Make "a refactor-only PR must plan to zero changes" a review rule, and fail CI on any destroy in a PR that carries a `refactor` label. Had this applied, the NAT gateway's elastic IP would have changed — breaking every downstream allowlist that trusted it.

8 Mini-Labs

Mini-Lab • Beginner

40 min

Goal. Build a VPC with two public subnets and observe every plan symbol at least once.

Business context. You are proving to your team that the console can be replaced.

Architecture. One root module, local state, one region, no NAT.

Requirements. Create the VPC and subnets; then change a tag (expect ~); then change a subnet CIDR (expect -/+); then delete a subnet block (expect -).

Constraints. Predict the symbol in writing *before* running plan each time.

Hints. `cidrsubnet("10.0.0.0/16", 8, 1)` yields `10.0.1.0/24`.

Expected outcome. A four-row table: change made, symbol predicted, symbol observed, why.

Production considerations. Note which attributes were ForceNew and how the provider signalled it.

Common pitfalls. Reading only the summary line; assuming a CIDR change is an in-place edit.

Mini-Lab • Intermediate

70 min

Goal. Migrate local state to a locked, encrypted S3 backend without losing anything.

Business context. A second engineer is joining the project on Monday.

Architecture. Existing local state → versioned, encrypted bucket with `use_lockfile`.

Requirements. Create the bucket with versioning, encryption and public access blocked; add the backend block; run `init -migrate-state`; verify with `state list`; then hold a lock in one terminal and prove a second run is refused.

Constraints. Back up the local state first. Do not skip the concurrency test — it is the point.

Hints. `terraform state list` before and after must be identical.

Expected outcome. Identical state lists, a state object in S3, and a captured lock error.

Production considerations. Write the IAM policy the CI role needs, including `s3:DeleteObject` on the `.tflock` key, and no more.

Common pitfalls. Forgetting versioning before the first write; leaving `use_lockfile` at its default of false and believing you are locked.

Mini-Lab • Production-style

2 hours

Goal. Extract a reusable network module and stand up dev and prod from it, non-destructively.

Business context. Dev and prod must be structurally identical but differently sized and priced.

Architecture. `modules/network` consumed by `envs/dev` and `envs/prod`, separate backend keys.

Requirements. Inputs `env`, `cidr`, `az_count`, `single_nat_gateway` with validation; outputs `vpc_id`, `public_subnet_ids`, `private_subnet_ids`; moved blocks so the prod refactor plans to zero changes; dev on a non-overlapping CIDR with one NAT gateway.

Constraints. The prod plan after extraction must show *no changes*. No resource may be destroyed.

Hints. `terraform state list` gives you the exact old addresses to write `moved` blocks from.

Expected outcome. Two environments from one module, and a prod plan that is empty.

Production considerations. Estimate the monthly NAT cost difference between the two settings and record it in the module README.

Common pitfalls. Overlapping CIDRs that block future peering; a module with twenty pass-through variables; forgetting that a module rename changes every address beneath it.

9

Engineering Notes

 Infrastructure insight

Infrastructure as Code is not really about automation — provisioning was already scriptable. It is about *auditability*. The value is that `git log` answers "why does this security group allow that", that a reviewer saw the change before it happened, and that the difference between two environments is a diff rather than an archaeology project. Teams that adopt Terraform but apply from laptops without review keep the syntax and throw away the benefit.

 Security warning • the identity that runs Terraform is the most powerful thing you own

A CI role with permission to apply your network configuration can also delete it. Scope it: separate roles per environment, no wildcard `*` on delete actions for stateful resources, and `prevent_destroy` on the resources whose loss would be unrecoverable (the state bucket, production databases). Day 12's least-privilege IAM work applies to your automation before it applies to your applications — the pipeline is the highest-value target in the estate.

 Automation tip

Put three cheap gates in CI, in this order: `terraform fmt -check` (style, no argument), `terraform validate` (syntax and type errors, no cloud credentials needed), then `plan` with the output posted to the PR. Slack's Smart Planner shows where this leads at scale — it determines which state files a change affects, plans each, and attaches the results for reviewers. The generalisable idea: the plan is a review artifact, so a reviewer should never have to run Terraform to know what a PR does.

 Cloud cost optimization

Terraform makes waste visible and therefore fixable. Because every managed resource has an address and a tag set, you can answer "what does this environment cost" and "which resources have no owner". The corollary is that unmanaged resources are invisible to that accounting — so a partially-adopted estate has a permanent blind spot. Importing the last 10% is usually worth more in cost clarity than in reproducibility.

 Interview insight

Expect: "Walk me through what happens if someone changes a Terraform-managed resource in the console." The weak answer is "it causes drift". The answer that lands walks the mechanism: the next run's refresh phase reads the live object and updates in-memory state, the plan diffs desired configuration against that refreshed state, and Terraform proposes to revert the manual change — silently, if nobody reads the plan. Then give the remedy: `plan -refresh-only` to see the drift on its own, then a conscious decision to either revert it or codify it, and `import` if the object was created outside Terraform entirely.

10

Build-Along Project — CloudBase

Day 1 gave CloudBase a hand-built host. Day 2 put it under version control. Day 3 designed the network on paper. Days 4 to 6 containerised the API and ran it on a cluster you created *by hand* — which means today the entire platform exists only because somebody remembered the right clicks. Today that stops: the network becomes code, and the code becomes the source of truth.

 Build-Along • Day 07 — the network, declared and reproducible

- Create the state backend by hand, once: an S3 bucket with versioning, default encryption and all public access blocked, in a blast radius separate from the workloads it will describe.
- Add `versions.tf` pinning `required_version` and the AWS provider, and `backend.tf` using `use_lockfile = true` and `encrypt = true`. Commit `.terraform.lock.hcl`; add `*.tfstate*` to `.gitignore`.
- Codify the Day 3 design: VPC, internet gateway, two public and two private subnets across two AZs, an elastic IP, a NAT gateway, and both route tables with their associations — CIDRs derived with `cidrsubnet()`, AZs from a data source.
- Apply, then run `plan` again and do not stop until the second plan is empty.
- Extract everything into `modules/network` with inputs `env`, `cidr`, `az_count`, `single_nat_gateway` and outputs `vpc_id`, `public_subnet_ids`, `private_subnet_ids`; use `moved` blocks so the refactor plans to zero changes.
- Instantiate a `dev` environment from the same module on a non-overlapping CIDR with a single NAT gateway, under its own backend key.
- Adopt one hand-made resource from Days 5–6 with an `import` block, generating a draft with `-generate-config-out` and folding it into the real files.
- Cause drift deliberately in the console, capture the `-refresh-only` output, and write `docs/drift-runbook.md` describing how to decide between reverting and codifying.

Definition of done: `terraform apply` from a clean clone reproduces the entire CloudBase network in an empty account with no console clicks; a second `plan` reports no changes in both `dev` and `prod`; state is remote, encrypted, versioned and locked; no `.tfstate` file exists anywhere in Git history; and the drift runbook is committed.

11 Chapter Summary**Key takeaways**

- ✓ Declarative means you describe the end state; the tool derives create, update, replace and destroy.
- ✓ A provider is a plugin process with a CRUD schema — it, not the engine, computes most of the diff.
- ✓ State is the map from configuration addresses to real resource IDs, and it holds secrets in plaintext.
- ✓ Remote state gives the team one memory; locking prevents two concurrent applies losing an update.
- ✓ Plan walks a DAG built from attribute references; destroy is the same graph with edges reversed.
- ✓ `-/+` is a replacement — destroy then create — and for stateful resources that is data loss.
- ✓ Drift surfaces in the refresh phase; `-refresh-only` shows it without proposing to fix it.
- ✓ `import` adopts existing resources; `moved` refactors without destroying.

Things to remember

- An address is an identity. Renaming a resource without a `moved` block means destroy-and-create.
- `use_lockfile` defaults to `false`. Unlocked is the silent default; you must opt in.
- DynamoDB-based locking for the S3 backend is deprecated in favour of native S3 lock files.
- A second plan that is not empty is a bug, not a quirk — it will hide real changes forever.
- Terraform provisions; Ansible (Day 8) configures. Neither replaces the other.

Common pitfalls (last look)

- Committed state • unread plans • unpinned providers • no lock
- Console fixes that nobody codifies • one monolithic state • hand-edited state JSON

📄 One-Page Cheat Sheet

Declarative — describe the end state; the engine derives the verbs and the order.

Provider — a plugin process with a CRUD schema; pin it, and commit `.terraform.lock.hcl`.

State — address → real ID, plus attributes and dependencies. Secret. Remote. Versioned. Locked.

Plan — refresh, then diff desired against refreshed state, then order by the DAG.

Symbols — `+` create • `~` in-place • `-/+` replace (destructive) • `-` destroy.

Module — a directory with inputs and outputs; expose intent, not plumbing; version it.

Drift — reality diverged from state. `plan -refresh-only` to see it; then revert or codify.

Refactor — `import` to adopt • `moved` to rename • `removed` to forget without deleting.

CLI quick reference

Day 7 commands worth memorising

```
# lifecycle
terraform init
terraform init -migrate-state
terraform fmt -check -recursive
terraform validate
terraform plan -out=tfplan
terraform apply tfplan

# download providers + configure backend
# move local state into a remote backend
# style gate for CI
# syntax and types, no credentials needed
# save the exact diff you will approve
# execute THAT plan, not a fresh one

# state inspection (never open the JSON by hand)
terraform state list
terraform state show aws_vpc.main
terraform show -json tfplan
# every managed address
# attributes, with sensitive values redacted
# machine-readable plan, for policy checks

# drift and adoption
terraform plan -refresh-only
terraform plan -generate-config-out=generated.tf
# what changed in the world, no proposed fix
# draft HCL for import blocks

# locks and graph
terraform force-unlock <LOCK_ID>
terraform graph
# ONLY after a human confirms nothing is running
# emit the dependency DAG in DOT format
```

Interview questions to master

- › What is Terraform state, and why can't Terraform work correctly without it?
- › Why should state live in a remote backend with locking instead of locally?
- › Walk me through what happens if someone manually changes a Terraform-managed resource in the console.
- › What's the difference between a resource and a module?
- › How would you safely bring an existing, manually-created resource under Terraform management?
- › Why pin provider versions — and what does `.terraform.lock.hcl` add on top of a version constraint?
- › In a plan, what is the difference between `~` and `-/+`, and why does it matter?
- › How does Terraform know what order to create and destroy resources in?

Further reading & official docs

- HashiCorp, "State" — developer.hashicorp.com/terraform/language/state — purpose of state, and the explicit warning about secrets in it.
- HashiCorp, "Backend Type: s3" — developer.hashicorp.com/terraform/language/backend/s3 — `use_lockfile`, the `dynamodb_table` deprecation, encryption and required IAM permissions.
- HashiCorp, "Import resources overview" and the `import` block reference — developer.hashicorp.com/terraform/language/import — config-driven import and `-generate-config-out`.
- HashiCorp blog, "Terraform 1.5 brings config-driven import and checks" — why import became a plannable operation.
- Slack Engineering, "How We Use Terraform At Slack" — slack.engineering/how-we-use-terraform-at-slack — 1,400 state files, the module catalog, and the Smart Planner.
- Shopify Engineering, "Using Terraform to Manage Infrastructure" — shopify.engineering/manage-infrastructure-with-terraform — writing a custom provider and reviewing infrastructure via Atlantis.
- AWS Prescriptive Guidance, "Terraform AWS provider best practices — Backend" — state bucket hardening.

Production checklist

- ☐ State is remote, encrypted at rest, versioned, and locking is explicitly enabled.
- ☐ No `.tfstate` file exists in any branch or any commit of any repository.
- ☐ `required_version` and every provider are pinned; `.terraform.lock.hcl` is committed.
- ☐ CI runs `fmt -check`, `validate` and `plan`, and posts the plan onto the pull request.
- ☐ Applies run from a machine identity with a scoped role — never from a human laptop in production.
- ☐ A saved plan file is what gets approved and what gets applied.
- ☐ State is split by environment and by blast radius, with the target explicit in the path.
- ☐ Irreplaceable resources carry `prevent_destroy`; every resource carries ownership tags.

- ☐ A drift runbook exists, and drift is checked on a schedule — not discovered by accident.

Key idea

What you will learn next (Day 8). Terraform now guarantees that the *machine exists* and that the network around it is exactly what the repository says. It has nothing to say about what is installed on that machine, which packages are patched, or which config files are correct — the moment you SSH in and edit something, you are back to Day 1's invisible drift. Tomorrow, Ansible closes that gap: agentless configuration management, idempotent modules, inventories and roles, and the reason the industry keeps both tools instead of picking one.

DAY 08 • CONFIGURATION MANAGEMENT

Configuration Management with Ansible08

Idempotency, playbooks, inventory, roles and Vault

🕒 Estimated study time: 7 hours

🎯 By the end of this day, you will be able to...

- State precisely where Terraform's job ends and Ansible's begins — and why owning both with one tool creates two sources of truth.
- Explain idempotency as a *module-level* property, and say why a `shell` task is not idempotent even when re-running it is harmless.
- Describe what actually happens on the wire when a task runs: SSH multiplexing, the packaged module, the JSON result.
- Structure configuration as roles with environment-separated inventory, and make restarts fire only on real change.
- Keep credentials in a Git-tracked repository safely with Ansible Vault — and know what Vault does *not* give you.

1 The Big Picture

What problem this solves. Yesterday Terraform created a bastion host, a VPC and a security group. Terraform is now finished: the instance exists, it has an IP, it is tagged. But the machine is a bare Ubuntu image. It has no service user, no hardened SSH configuration, no packages, no `systemd` unit, no log rotation. Something must put those things *inside* the host — and must do so identically on the next host, and the next fifty, and again in six months when the AMI changes. That is configuration management, and Ansible is today's default answer.

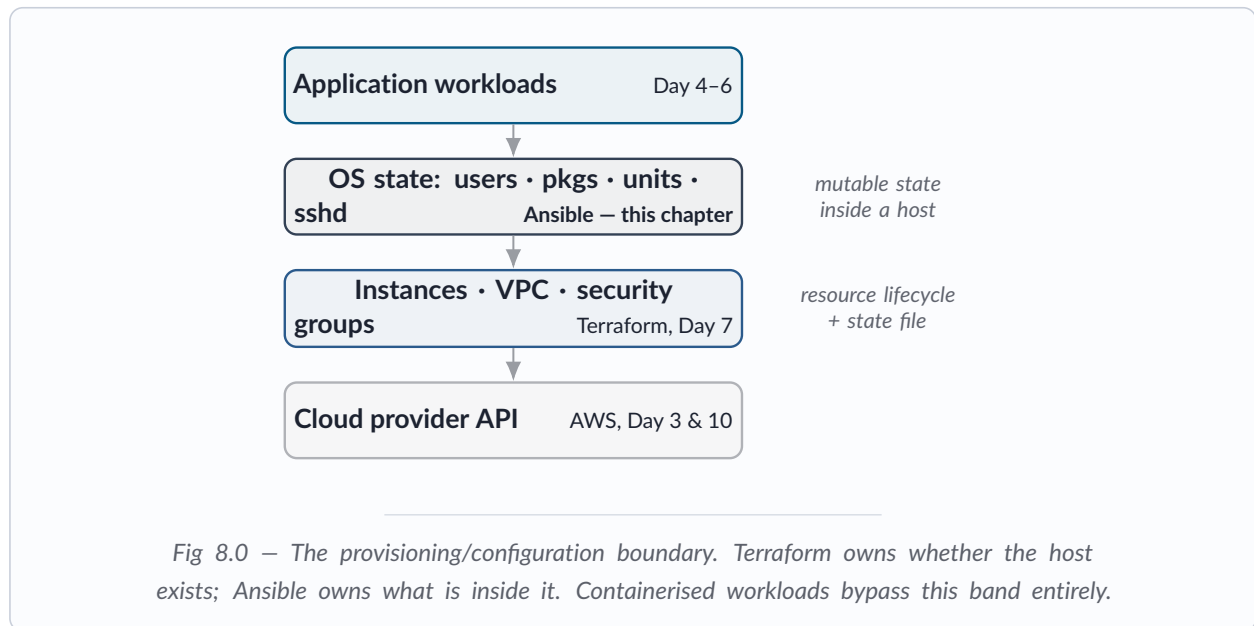
Why DevOps engineers need it. The failure mode this prevents has a name: **configuration drift**. Two hosts provisioned from the same image diverge because someone SSH'd into one at 2am to "just quickly" install a debugging tool, edit `/etc/nginx/nginx.conf`, or bump a kernel parameter. Six months later one host serves traffic fine and its twin returns 502s, and nobody can say what differs. Drift is invisible, it compounds, and it is the reason the phrase "works on that box" exists. Configuration management makes the desired state of a host a reviewed file in Git rather than an accumulation of undocumented human actions.

🔑 Key idea

Day 1 ended with `bootstrap.sh` — a shell script you had to write carefully so that running it twice was safe. Ansible inverts that burden. Instead of you writing "check if the user exists, and if not create it," you *declare* `state: present` and a purpose-built module does the checking. Idempotency stops being your discipline and becomes the tool's contract. That single shift is what this chapter is about.

Where it fits in a modern cloud architecture. Ansible occupies a narrow, precise band of the stack:

above the cloud API (Terraform's territory) and below the container runtime (Docker's territory). It configures the operating system of machines that are not disposable.



How it connects to previous chapters. Day 1 gave you the material Ansible manipulates — users, permission bits, `systemd` units, the journal. Day 2 gave you the review workflow that makes a playbook change auditable. Day 7 gave you Terraform and the idea of declaring desired state; Ansible is the same idea applied one layer up, with a crucial difference in how state is tracked. Day 12 will generalise the secrets problem that Vault only partially solves here.

Where companies use it in production. NASA's public case study describes migrating roughly 65 applications out of a hardware data centre into AWS with no centralised management and a highly heterogeneous estate. They standardised on Ansible plus Ansible Tower (the control-plane product, whose open-source upstream is AWX) specifically because it needed no agent installed on the managed machines and because non-specialists could read a play and understand it. The published outcomes are operational, not glamorous: baselining a standard AMI moved from an hour of manual configuration to a background process, and provisioning OS accounts across the whole estate to under ten minutes. The architectural lesson is that the win came from *centralising* execution behind an inventory and an audit trail, not from any single clever playbook.

Common misconceptions. That Ansible and Terraform overlap and you should pick one (they answer different questions); that "idempotent" means "safe to run twice" (it means the tool can *tell* whether anything changed); that Ansible tracks state like Terraform does (it does not — it re-derives current state from the host on every run); and that Vault is a secrets manager (it is file encryption, and it has no rotation, no audit log and no dynamic credentials).

2

Concept Map

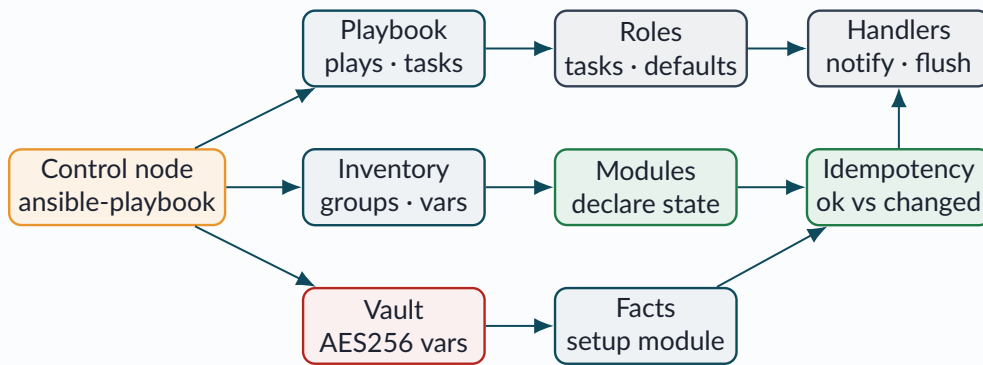


Fig 8.1 — The Day 8 territory. Inventory says where, plays say what, modules decide whether to act, handlers react to real change.

Terminology to anchor: **control node** → **inventory** → **play** → **task** → **module** → **idempotency** → **handler** → **role** → **Vault**.

3 Guided Learning

3.1 Provisioning versus Configuration — Two Different Questions

What is it? Terraform answers "does this resource exist, with these properties, in this account?" Ansible answers "given that the machine exists and I can reach it over SSH, what is installed on it and what is running?" The boundary is the machine's network interface: outside it is cloud API territory, inside it is operating-system territory.

Why does it exist? Because the two problems have genuinely different shapes. Cloud resources have a *lifecycle*: they are created, updated, and destroyed, and something must remember that the `aws_instance` named `bastion` in your code corresponds to instance `i-0abc123` in the account. That memory is Terraform state. Operating-system configuration has no lifecycle to remember: a package is either installed or it is not, and you can find out by asking the host. Ansible therefore keeps *no state file at all*. It re-derives the current state on every run, which is why an Ansible run can be aborted, resumed, or run from a different laptop with no reconciliation ceremony — and also why Ansible can never tell you "this thing exists and is no longer in your code, shall I delete it?" Terraform can; Ansible structurally cannot.

Design trade-off

No state file is both Ansible's greatest simplification and its hardest limit. You gain: no state locking (Day 7's `DynamoDB` table), no drift between state and reality, no `terraform import` ritual. You lose: no ability to detect removal. Delete a task that installed a package and the package stays installed on every host forever. Deletion in Ansible must be *explicitly declared* (`state: absent`) and kept in the repo, sometimes for years. Teams call these "tombstone tasks", and forgetting them is the most common way an Ansible codebase quietly stops describing reality.

How does it work internally? Ansible is **agentless** and **push**-based. The control node — your laptop, or more usually a CI runner — opens an SSH connection to each target, ships code over it, and

executes it. Nothing is installed on the target beyond a Python interpreter, which every server distribution already has.

Contrast that with the previous generation. CFEngine (1993), Puppet (2005) and Chef (2009) are **agent**-based and **pull**-based: a daemon on every managed node wakes on a timer, fetches its catalogue from a central server, and converges itself. GitLab runs exactly this model on GitLab.com and documents it publicly: `chef-client` runs on every VM every 30 minutes, cookbook versions are pinned per Chef environment (`gstg` for staging, `gprd` for production) mirroring the same infrastructure split their Terraform uses, and a cookbook change is promoted by merging a version bump into the non-production environment first, verifying convergence, then merging the production bump. Rollback is reverting the version number.

Property	Push / agentless (Ansible)	Pull / agent (Chef, Puppet)
Who initiates	An operator or CI job, deliberately	The node itself, on a timer
Install footprint	SSH plus Python on the target	A daemon, its runtime, and certs
Drift correction	Only when someone runs it	Continuously, every interval
New host joins	Must appear in inventory first	Bootstraps itself on first check-in
Blast radius	Bounded by the hosts you targeted	A bad catalogue reaches everything
Best fit	Orchestrated, ordered change	Large steady-state fleets

💡 Senior engineer insight

The push/pull distinction matters most for *ordered* change. Rolling a database schema, draining a load balancer, upgrading node by node — these need a coordinator that knows the sequence. A pull agent waking up independently on each host cannot orchestrate anything; it can only converge its own machine. This is why many organisations run both: an agent for continuous baseline compliance, and Ansible for the deliberate, sequenced operations. Choosing "one tool for everything" is usually a decision to do one of those two jobs badly.

When should I use it? Any host you cannot rebuild from scratch on demand: bastions, build agents, database VMs, network appliances, legacy application servers, on-premises hardware. Also for orchestrated operational procedures — rolling restarts, certificate rotation, emergency patching — where the ordering is the point.

When should I not? For anything running in a container. A container image is already the declarative artefact; configuring a running container with Ansible reintroduces the mutable state that images exist to eliminate. Also do not use Ansible to *create* cloud resources. Ansible has `amazon.aws` modules that can launch an EC2 instance, and they work, but now two tools claim ownership of that instance and neither can be trusted. Let Terraform own resource lifecycle. Let Ansible own the inside.

⚠️ Common misconception

"Ansible replaces Terraform because it can also create EC2 instances." It can, and the result is an instance with no recorded lifecycle: nothing knows to destroy it when the code changes, nothing detects that someone resized it by hand, nothing can plan a change before applying it. The capability exists mainly for environments where Terraform is not available. Duplicating the source of truth is a more expensive mistake than learning the second tool.

3.2 Inventory — Telling Ansible What Exists and Where

What is it? The inventory is the list of managed hosts, organised into groups, with variables attached at host and group level. Everything Ansible does is scoped by it: `hosts: web` in a play means "every host in the `web` group of the inventory I was given."

Why does it exist? Because the alternative — passing hostnames on the command line — makes environment separation a matter of human care. With inventories as files, "run this against production" and "run this against staging" are two different paths, and the wrong one is a visible mistake in a command, not an invisible typo in an IP address.

inventories/prod/hosts.ini

```
[bastion]
bastion-prod-1.cloudbase.internal

[app]
app-prod-1.cloudbase.internal
app-prod-2.cloudbase.internal

[db]
db-prod-1.cloudbase.internal ansible_user=dbadmin

[prod:children]
bastion
app
db

[prod:vars]
ansible_python_interpreter=/usr/bin/python3
```

How does it work internally? Group membership is a graph, not a list: `[prod:children]` makes `prod` a parent of three groups, and every host inherits variables from every group it belongs to, transitively. Where two groups set the same variable, Ansible resolves the conflict by group depth (a child overrides its parent), then by alphabetical name at equal depth, and a host variable overrides them all. The full precedence chain has more than twenty levels; the two that matter daily are that a role's `defaults/main.yml` sits near the *bottom* (designed to be overridden by anyone), while `vars/main.yml` in the same role sits near the *top* (designed to be hard to override). Variables belong in `group_vars/` and `host_vars/` directories beside the inventory, never inline in the host line except for genuinely per-host connection details.

Path (under <code>inventories/</code>)	Applies to
<code>prod/hosts.ini</code>	Which hosts exist in production
<code>prod/group_vars/all.yml</code>	Every production host
<code>prod/group_vars/app.yml</code>	Only the <code>app</code> group in production
<code>prod/group_vars/app/vault.yml</code>	Encrypted secrets for that group
<code>staging/...</code>	The same shape, different values
<code>(roles/base/defaults/main.yml)</code>	Lowest precedence — the role's opinion

Dynamic inventory. A static file is a lie the moment an autoscaling group replaces an instance. A **dynamic inventory plugin** queries the cloud provider at run time and builds the host list from tags. With the `amazon.aws.aws_ec2` plugin, an instance tagged `Environment=prod` and `Role=app` lands automatically in groups derived from those tags. The inventory then has exactly one source of truth: the tags Terraform already set when it created the instance.

inventories/prod/aws_ec2.yml

```
plugin: amazon.aws.aws_ec2
regions:
  - eu-west-1
filters:
  tag:Environment: prod
  instance-state-name: running
```

```
keyed_groups:
  - key: tags.Role          # tag Role=app -> group "role_app"
    prefix: role
  - key: placement.availability_zone
    prefix: az
hostnames:
  - private-ip-address      # never public IP; reach via the bastion
compose:
  ansible_host: private_ip_address
```

Infrastructure insight

Notice the handoff this creates. Terraform sets the tags as part of resource creation; Ansible reads them as its inventory. Neither tool calls the other, there is no shared state file, and yet the two stay consistent because both derive from the same authoritative record — the cloud provider's own metadata. Designing integrations so that both sides read from a third source, rather than one pushing to the other, is a general pattern worth stealing.

Security warning • production is not a flag

Keep production in a separate inventory *directory*, and never make it the default. `ansible-playbook site.yml` with no `-i` should either fail or target a harmless environment. Every incident report that begins "I meant to run it against staging" ends with a recommendation that could have been written in advance: make the dangerous target require an explicit argument, and make the safe one the default.

3.3 The Execution Model — What Actually Happens on the Wire

What is it? When you run a play, Ansible does not "send commands" to the host. For each task it *builds a self-contained Python program*, ships it over SSH, runs it once, reads a single JSON document back, and discards it.

Why does it exist? Because the alternative — streaming shell commands and parsing their human-readable output — is exactly what makes bespoke scripts fragile. Parsing `apt-get` output to decide whether something changed is guesswork; a program that inspects the package database and returns `{"changed": false}` is a fact.

How does it work internally? The packaging step is called **AnsiballZ**. Ansible takes the module's Python source, the module utilities it imports, and the task's parameters, and zips them into one executable Python file. For each host and task, the sequence is roughly:

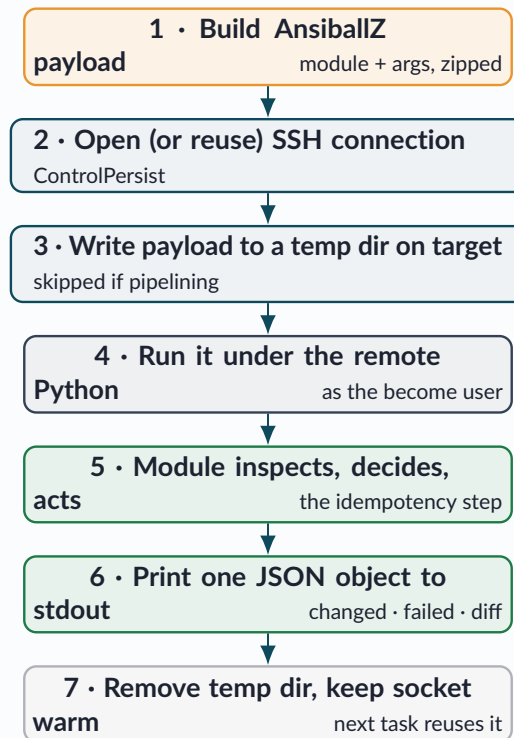


Fig 8.2 — One task, one host. Everything above the dashed line is the module's own decision; Ansible only reads the JSON it returns.

Three configuration knobs follow directly from that picture, and knowing the mechanism tells you what each one does.

Setting	Default	What the mechanism explains
<code>forks</code>	5	How many hosts run a task in parallel. Five is a laptop-safe default, not a fleet-safe one.
<code>pipelining</code>	off	Removes step 3: the payload is piped straight into the remote Python. Requires <code>requiretty</code> to be off in <code>sudoers</code> .
<code>ControlPersist</code>	60s	Keeps the SSH master socket open so later tasks skip the handshake entirely.
<code>strategy</code>	linear	Every host finishes task n before any starts $n+1$. <code>free</code> lets hosts race ahead independently.
<code>gather_facts</code>	true	An implicit <code>setup</code> task on every host before task one. Turn it off when no fact is used.

⚡ Performance note

A slow playbook is almost never slow because the modules are slow. It is slow because `forks: 5` serialises a hundred hosts into twenty batches, because pipelining is off and every task pays two extra round trips, or because fact gathering runs on every host for a play that reads no facts. Measure before tuning: run with `ANSIBLE_CALLBACKS_ENABLED=profile_tasks` and let the per-task timings tell you where the time went. Raising `forks` to 50 on a control node that then exhausts its file descriptors is a common self-inflicted wound.

👁 Behind the scenes

The `setup` module is why a play appears to "pause" before its first task. It is a real module run on every host that collects hundreds of facts — distribution, kernel, network interfaces, mounts, CPU count — and returns them as `ansible_facts`. Those facts are what make one role work on both Debian and RHEL: the role branches on `ansible_facts['os_family']` rather than assuming. Disabling fact gathering to speed things up will silently break any such branch.

3.4 Idempotency — A Property of Modules, Not of Playbooks

What is it? An operation is **idempotent** when applying it to a system already in the desired state produces no further change — and, critically for Ansible, when the operation can *report* whether it changed anything. The second half is what most definitions omit and what makes the concept useful.

Why does it exist? Automation must run against an unknown starting state. Someone rebuilt one host last week, someone patched another by hand, a third is brand new from an autoscaling event. A procedure that assumes a clean slate breaks on two of the three. A procedure that declares the end state works on all three.

How does it work internally? Idempotency is implemented *inside each module*, not by the playbook engine. A well-written module follows the same four steps regardless of what it manages:

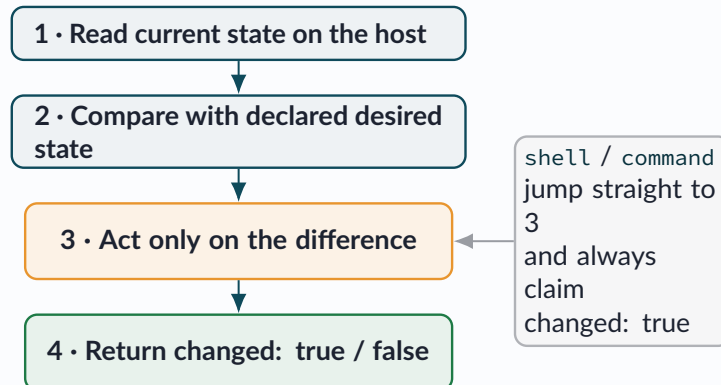


Fig 8.3 — Inside any state-aware module. The engine contributes nothing here; *ok* versus *changed* is the module's own verdict.

This is why `shell: apt-get install -y nginx` is a bug even though running it twice is harmless. The module has no idea whether `nginx` was already installed; it cannot skip work, cannot report accurately, and — most damagingly — will fire every handler that depends on it, restarting a healthy service on every single run. Your "no-op" playbook becomes a fleet-wide restart.

roles/base/tasks/main.yml — the same intent, three ways

```

# WRONG: not idempotent. Always "changed", always notifies.
- name: Install nginx
  ansible.builtin.shell: apt-get install -y nginx

# RIGHT: the module reads dpkg state and decides.
- name: Install nginx
  ansible.builtin.apt:
    name: nginx
    state: present
  
```

```
update_cache: true
cache_valid_time: 3600

# ACCEPTABLE: no module exists, so guard the shell task yourself.
- name: Compile the custom auth plugin
  ansible.builtin.command: /opt/cloudbase/build-plugin.sh
  args:
    creates: /opt/cloudbase/lib/auth.so # skip entirely if it exists
  register: plugin_build
  changed_when: plugin_build.rc == 0
```

The three guards. When no purpose-built module exists, you supply the missing logic explicitly. `creates:` and `removes:` make the task skip if a path already does (or does not) exist — this is the cheapest guard and should be the first reach. `changed_when:` overrides the module's verdict with an expression over the registered result, letting you say "this only counts as a change if the output contained the word `applied`". `failed_when:` does the same for failure, which matters for commands that use non-zero exit codes to mean something other than failure.

Automation tip

`-check` runs the playbook in **check mode**: modules report what they *would* change without changing it, and combined with `-diff` you get a line-level preview of every file that would be modified. This is Ansible's equivalent of `terraform plan`, with an important caveat — `command` and `shell` tasks cannot be previewed, so they are skipped by default. A playbook full of shell tasks has a useless check mode, which is another reason to prefer modules. Run `-check -diff` against production on a schedule and treat any reported change as drift that needs explaining.

Common misconception

"My playbook is idempotent because running it twice does not break anything." Not the same claim. Idempotency in Ansible means the second run reports `changed=0`. If it reports `changed=8` every time, you have lost the ability to distinguish "nothing drifted" from "eight things drifted" — which is precisely the signal configuration management exists to give you. A green run that always says `changed` is a broken smoke detector.

3.5 Roles and Handlers — Structure and Reaction

What is it? A **role** is a directory with a fixed layout bundling everything needed to configure one concern. A **handler** is a task that runs only if some other task actually reported a change, and only once, at the end of the play.

Why does it exist? A single `site.yml` that configures everything becomes unreadable at roughly two hundred lines and unreviewable shortly after. More importantly it cannot be reused: the nginx configuration for the bastion and for the app tier are ninety percent identical, and copy-paste guarantees they diverge. Roles are to Ansible what modules are to Terraform — the unit of reuse and the unit of review.

How does it work internally? Role directories are discovered by convention. When a play includes a role, Ansible looks for seven standard subdirectories and loads `main.yml` from each if present. Nothing is registered or imported; the layout is the interface.

Directory	Role in the design
tasks/	The work. Entry point is <code>tasks/main.yml</code> .
handlers/	Reactions, referenced by name from <code>notify</code> .
templates/	Jinja2 files rendered with host variables (<code>.j2</code>).
files/	Static files copied verbatim.
defaults/	Lowest-precedence variables — the role's overridable opinion.
vars/	High-precedence variables — internal constants, rarely overridden.
meta/	Dependencies on other roles, and Galaxy metadata.

💡 Senior engineer insight

The `defaults` versus `vars` distinction is a public-API decision, not a filing preference. Anything in `defaults/` is you saying "callers may override this" — it is your role's parameter list. Anything in `vars/` is you saying "this is internal; overriding it will break assumptions." A role whose every variable lives in `vars/` cannot be reused; a role with no `defaults/` has no documented interface. Write `defaults/main.yml` first, with a comment per variable, and treat it as the role's README.

Handler semantics. A handler is notified by name. Ansible collects notifications during the play, deduplicates them, and runs each notified handler exactly once, in the order the handlers are *defined* — not the order they were notified. They run at the end of the play, after every task. This ordering is deliberate: if three tasks each modify part of the nginx configuration, you want one restart at the end, not three restarts mid-configuration leaving the service running on a half-written config.

roles/base/handlers/main.yml and the tasks that notify them

```
# --- tasks/main.yml (excerpt) ---
- name: Deploy hardened sshd configuration
  ansible.builtin.template:
    src: sshd_config.j2
    dest: /etc/ssh/sshd_config
    owner: root
    group: root
    mode: "0600"
    validate: /usr/sbin/sshd -t -f %s # refuse to install a broken config
  notify: Restart sshd

- name: Install the CloudBase systemd unit
  ansible.builtin.template:
    src: cloudbase-api.service.j2
    dest: /etc/systemd/system/cloudbase-api.service
    mode: "0644"
  notify:
    - Reload systemd
    - Restart cloudbase-api

# --- handlers/main.yml ---
- name: Reload systemd # defined first -> runs first
  ansible.builtin.systemd_service:
    daemon_reload: true

- name: Restart cloudbase-api
  ansible.builtin.systemd_service:
    name: cloudbase-api
    state: restarted

- name: Restart sshd
  ansible.builtin.service:
    name: sshd
```

```
state: restarted
```

♥ Reliability principle

The `validate:` parameter on `template` and `copy` is the single most underused safety feature in Ansible. It runs a validation command against the rendered file in a temporary location and installs it only if the command exits zero. Without it, a typo in `sshd_config.j2` deploys a file that stops `sshd` from starting — and the handler that restarts `sshd` then locks you out of every host in the play simultaneously. With `validate`, the task fails loudly and the old, working config stays in place. Use it for `sshd`, `nginx` (`nginx -t`), `sudoers` (`visudo -cf %s`) and every config format that ships a checker.

⚠ Common misconception

"My handler did not run, so the task must have failed." More often the play failed *before* reaching the end, and handlers are discarded when a play aborts on a host. If a mid-play restart is genuinely required — for example the service must be running before a later task talks to it — force it with a `meta: flush_handlers` task at that point. Relying on end-of-play handlers for something a later task depends on is an ordering bug waiting for a slow day to surface.

When should I *not* use a role? A five-line playbook that runs once during an incident does not need a role. Extraction is justified by the *second* caller, not the first. Building a role library before you have two consumers produces abstractions shaped by imagination rather than by use, and those are harder to fix than duplication.

3.6 Ansible Vault — Secrets That Can Live in Git

What is it? `ansible-vault` encrypts a file, or a single variable inside a plaintext file, with a passphrase. The encrypted form is ASCII-armoured text that diffs and merges tolerably in Git, and Ansible decrypts it in memory at run time.

Why does it exist? Playbooks need real credentials — a database password, an API token, a TLS private key — and those cannot sit in plaintext in a repository that every engineer clones. Before Vault, teams either kept a secrets file outside version control (so it drifted, and onboarding meant "ask someone to send you the file") or committed plaintext and hoped. Day 2 covered what happens after a secret is pushed; Vault is how you avoid needing that chapter.

How does it work internally? A vaulted payload is AES-256 encrypted with a key derived from your passphrase, wrapped in a header line that records the format version, the cipher, and optionally a **vault ID** label:

```
head -2 inventories/prod/group_vars/app/vault.yml
```

```
$ANSIBLE_VAULT;1.2;AES256;prod
39373530666432616465383966393261333866626434366533373062303538616...
```

The vault ID (`prod` above) is a label, not a secret. It lets one run carry several passphrases — staging encrypted with one, production with another — so a developer holding only the staging passphrase can decrypt staging files and simply cannot read production ones. That is the mechanism you use to give the whole team the ability to run staging without giving anyone production credentials.

```
working with vault
```

```
# encrypt a whole file (preferred: rekeyable, greppable filename)
ansible-vault encrypt --vault-id prod@prompt \
```

```
inventories/prod/group_vars/app/vault.yml

# encrypt one value to paste into an otherwise plaintext vars file
ansible-vault encrypt_string --vault-id prod@prompt 's3cr3t' \
  --name 'cloudbase_db_password'

# edit in place: decrypts to a temp file, re-encrypts on save
ansible-vault edit inventories/prod/group_vars/app/vault.yml

# rotate the passphrase across a file
ansible-vault rekey inventories/prod/group_vars/app/vault.yml

# run: in CI, read the passphrase from a file the runner injects
ansible-playbook -i inventories/prod site.yml \
  --vault-id prod@run/secrets/vault-pass
```

Production tip

Adopt the two-file convention: `group_vars/app/vars.yml` holds plaintext variable names that reference vaulted values (`cloudbase_db_password: "{{ vault_cloudbase_db_password }}"`), and `group_vars/app/vault.yml` holds the encrypted `vault_`-prefixed originals. You get a readable, grep-able, reviewable list of which secrets exist and where they are used, without decrypting anything. Encrypting the whole variables file instead makes every review of a non-secret change opaque.

Security warning • what Vault does not do

Vault is file encryption, not secret management. There is no rotation workflow, no per-user access log, no expiry, no dynamic credentials, and no revocation — if someone leaves with the passphrase, every secret it ever protected is compromised and must be rotated by hand. It also cannot stop a leak at run time: a vaulted value passed to a task is printed in the output unless the task carries `no_log: true`, and it appears in plaintext in `-vvv` debug output regardless. Use Vault for the bootstrap secret and low-churn credentials; for anything that must rotate, reach for AWS Secrets Manager or HashiCorp Vault, which Day 12 covers.

Cloud cost optimization

There is an unglamorous economic argument for configuration management that rarely gets made. Snowflake hosts cannot be rebuilt, so they cannot be right-sized, moved to cheaper instance families, or replaced by spot capacity — nobody dares, because nobody knows what is on them. A fleet that is fully described by roles can be destroyed and recreated on demand, which is the precondition for every meaningful cloud cost lever. Ansible's payoff is often measured in the migrations it makes possible rather than in hours saved.

Best practices — what experienced engineers do

- Prefer a purpose-built module over `shell/command`; when you must use one, guard it with `creates:` or `changed_when:`.
- Use fully qualified module names (`ansible.builtin apt`) — short names resolve differently depending on installed collections.
- Put restarts in handlers so they fire on real change, and add `validate:` to every config template that has a checker.
- Separate inventories per environment, with `group_vars/` beside each; never let production be the default target.

- Keep the role's public interface in `defaults/main.yml`, one commented variable per line.
- Run `-check -diff` on a schedule against production; treat any reported change as drift to explain.
- Lint before merge with `ansible-lint`, and add `no_log: true` to any task handling a secret.

✖ Common mistakes beginners make

- Writing tasks that report `changed` on every run, destroying drift detection and triggering pointless restarts.
- Hardcoding IP addresses in a static inventory that an autoscaling group invalidates within the hour.
- Committing plaintext secrets because Vault "felt like overhead" for one password.
- Using Ansible to create infrastructure Terraform should own, leaving two tools claiming the same resource.
- Assuming a handler ran, when the play aborted before reaching the end.
- Putting everything in `vars/` so the role can never be reused with different values.

4 Visualizations to Internalize

Redraw these from memory. Being able to reproduce the mechanism is what turns "I use Ansible" into "I can debug Ansible."

 Redraw: the seven steps of one task on one host — *payload, SSH, temp dir, execute, decide, JSON, cleanup*

 Redraw: the four steps inside a state-aware module, and mark exactly where `shell` enters the diagram

5

Guided Hands-On Lab — Replace `bootstrap.sh` With a Role

Goal. Take the Day 1 shell script and re-express it as an Ansible role that configures the Terraform-provisioned bastion: service user, packages, hardened SSH, the `systemd` unit, and a vaulted secret. The finish line is not "it ran" — it is "the second run reports `changed=0`."

1 Reach the host at all, before writing any task

Set up the control node, an SSH key, and a `prod` inventory containing exactly one host. Verify with the `ping` module — which is not ICMP; it connects, runs Python, and returns `pong`.

 *Why: Connectivity, privilege escalation and the Python interpreter are three separate failure modes. Proving all three work before writing logic means every later failure is about your code, not your plumbing.*

2 Scaffold the role and write `defaults/main.yml` first

Create `roles/base/` with the seven standard directories. Before any task, list every value the role should accept — service user name, app directory, SSH port, allowed SSH groups — with a comment each.

 *Why: Writing the interface first forces you to decide what varies per environment. Discovering that later means retrofitting variables through tasks you have already written and tested.*

3 Convert the script's package and user steps to modules

Replace `useradd` and `apt-get install` with `ansible.builtin.user` and `ansible.builtin.apt`. Run the play, then run it again immediately.

 *Why: The second run is the whole lesson. The script needed hand-written guards to survive it; the modules needed none, because state checking lives inside them.*

4 Template the `systemd` unit instead of copying it

Use `ansible.builtin.template` with a `.j2` file that interpolates the service user and app directory from your defaults. Notify a `Reload systemd` handler and a `restart` handler.

 *Why: A template turns "the unit file" into "the unit file for this environment". This is where the Day 1 unit stops being a static artefact and becomes a function of the inventory.*

5 Harden `sshd` — with `validate:` in place before you touch it

Deploy an `sshd_config` template disabling password authentication and root login, with `validate: /usr/sbin/sshd -t -f %s`.

 *Why: This is the task most likely to lock you out of every host at once. Adding the validator before the change, not after the first incident, is the habit worth building.*

6 Introduce a deliberate non-idempotent task, then fix it

Add a `shell` task that appends a line to a file. Run twice, observe the duplicate line and the permanent `changed`. Then replace it with `lineinfile`, or guard it with `creates:`.

 *Why: You will recognise this failure instantly in someone else's codebase afterwards — and you will understand why the handler was firing on every run.*

7 Move one credential into Vault

Encrypt a `vault.yml` with a `prod` vault ID, reference it from a plaintext `vars.yml`, and mark the consuming task `no_log: true`. Confirm the value is unreadable in the repository and absent from

normal output.

 *Why: Secrets handling that is added at the end is always retrofitted badly. Doing it while the role is small teaches the two-file convention when it costs nothing.*

8 Prove idempotency and preview drift

Run the play until the summary reads `changed=0`. Then hand-edit a managed file on the host and run with `-check -diff`.

 *Why: You have just built a drift detector. The same command in a scheduled CI job turns "someone changed something" from a mystery into an alert.*

Production tip

Put the idempotency proof in CI. A pipeline stage that runs the play twice against a throwaway VM and fails if the second run reports any change catches non-idempotent tasks at review time, when they cost minutes, instead of in production, when they cost a fleet-wide restart. Molecule is the standard tool for this, but a two-run shell check plus a `grep` for `changed=0` gets you most of the value on day one.

6 Thinking Exercises

Think it through

Terraform keeps a state file and can therefore tell you that a resource exists which is no longer in your code. Ansible keeps none and cannot. Design a practical way for a team to detect "this package is installed on our fleet but no role installs it" — and then argue whether the mechanism you invented is worth its maintenance cost.

Think it through

Ansible is push-based and runs when a human or CI job decides; Chef is pull-based and converges every thirty minutes. Consider a security patch that must reach every host within an hour, and a database configuration change that must be applied one node at a time. Which model handles each better, and what does that suggest about organisations that run both?

 Think it through

Check mode cannot preview `shell` tasks, so a playbook built from shell tasks has a useless dry run. Notice that this is the same structural problem as a test suite full of mocks: the safety mechanism silently degrades in proportion to how much you bypass the abstraction. Where else in your stack does a safety net quietly weaken exactly when it is needed most?

7 Debugging Practice

 Debugging Lab • The playbook that restarts the whole fleet on every run

Symptom. A nightly compliance run of `site.yml` is meant to be a no-op, but every night the API tier drops requests for a few seconds. The playbook always finishes green.

```
ansible-playbook site.yml -- PLAY RECAP, run 2 of 2

TASK [base : Ensure log directory exists] *****
changed: [app-prod-1]

TASK [base : Register host with the metrics agent] *****
changed: [app-prod-1]

RUNNING HANDLER [base : Restart cloudbase-api] *****
changed: [app-prod-1]

PLAY RECAP *****
app-prod-1 : ok=17 changed=4 unreachable=0 failed=0
```

- ✓ **Observe.** The second consecutive run reports `changed=4`. On an idempotent playbook it should report `changed=0`; the handler firing is a downstream consequence, not the cause.
- ✓ **Hypothesise.** One or more tasks report `changed` unconditionally, and at least one of them notifies the restart handler.
- ✓ **Investigate.** Run with `-diff` to see which tasks claim a change and what they think differs. Inspect the four culprits: two are `shell` tasks, one is a `file` task setting `mode` on a directory whose mode is already correct but written as `0755` unquoted (YAML parses it as the integer 493), and one is a `copy` of a file whose content is rendered with a timestamp.
- ✓ **Root cause.** `shell/command` always report `changed` because they cannot inspect state; the unquoted octal mode is a YAML typing bug; the timestamped template guarantees new content every run.
- ✓ **Fix.** Replace the shell tasks with `ansible.builtin.file` and the metrics-agent module (or guard them with `creates:/changed_when:`); quote the mode as `"0755"`; remove the timestamp from the template, or move it to a file the service does not depend on.
- ✓ **Prevent.** Add a CI stage that runs the play twice against a throwaway host and fails unless

the second run reports `changed=0`. Add `ansible-lint`, which flags unquoted octal modes and unguarded shell tasks before review.

Debugging Lab • Works on my laptop, fails in the CI runner with a decryption error

Symptom. A deploy pipeline that has run for months starts failing at the first task that reads a variable. The same playbook runs fine from an engineer's machine.

```
GitLab CI job log - deploy:prod

TASK [Gathering Facts] *****
ok: [app-prod-1]

TASK [base : Configure database connection] *****
fatal: [app-prod-1]: FAILED! => {"msg": "Attempting to decrypt
but no vault secrets found"}

ERROR! the field 'vault_password_file' has an invalid value
('/run/secrets/vault-pass'), and could not be converted to
an AnsibleUnicode. No such file or directory
```

- ✓ **Observe.** Two messages: the playbook found vaulted content it cannot decrypt, and the configured passphrase file does not exist in the runner.
- ✓ **Hypothesise.** The passphrase reaches the local machine through a developer's `ansible.cfg` or an interactive prompt, and reaches CI through an injected file — and that injection stopped happening.
- ✓ **Investigate.** Check whether the CI variable holding the passphrase is still defined and still exposed to this branch. Protected CI variables are only injected on protected branches: a release cut from a new, unprotected branch silently receives nothing, and the file is never written.
- ✓ **Root cause.** A configuration change, not a code change — the pipeline ran from a branch that does not match the protected-branch pattern, so the masked variable was absent and the `before_script` wrote an empty (or no) file.
- ✓ **Fix.** Restore the branch protection pattern, or scope the variable to the environment rather than the branch. Have the `before_script` assert the variable is non-empty and exit with a clear message if not.
- ✓ **Prevent.** Never let a missing secret surface as a decryption error three tasks in. Validate every required secret at the start of the job with an explicit check, so the failure names the missing variable instead of describing a symptom. Keep vault IDs per environment so a staging passphrase can never accidentally satisfy a production run.

8 Mini-Labs

Mini-Lab • Beginner

35 min

Goal. Feel idempotency by breaking it deliberately.

Business context. Your team's playbook reports changes on every run and nobody trusts the

output any more.

Architecture. One local VM or container, one playbook, no roles.

Requirements. Write two tasks that install nginx — one with `shell`, one with `ansible.builtin.apt` — plus one `lineinfile` and one equivalent `shell` append. Run twice and record the recap.

Constraints. Do not use handlers yet; observe raw `ok/changed` counts only.

Hints. `ansible-playbook p.yml -diff` shows why a task believes it changed something.

Expected outcome. A short table: task, run 1 result, run 2 result, and a sentence explaining each.

Production considerations. Note which tasks would have fired a handler on run 2, and what that would have restarted.

Common pitfalls. Writing `mode: 0644` unquoted and getting a spurious change; concluding `shell` is "fine because nothing broke."

Mini-Lab • Intermediate

60 min

Goal. Extract a flat playbook into a reusable role with a real public interface.

Business context. The bastion and the app tier need the same baseline, and the copy-pasted versions have already diverged.

Architecture. `roles/base/` with tasks, handlers, templates, defaults; two host groups consuming it with different values.

Requirements. Every value that differs between the two groups lives in `defaults/main.yml` and is overridden in `group_vars/`. One templated config with `validate:`, one handler.

Constraints. `vars/main.yml` may hold only values a caller must never change; justify each in a comment.

Hints. Run against one group, then the other, and diff the resulting files on each host.

Expected outcome. One role, two host groups, two correct and different configurations, second run `changed=0` on both.

Production considerations. Write the three-line `README` a colleague would need to consume your role.

Common pitfalls. Leaking a host-specific value into the role; notifying a handler from a task that always reports changed.

Mini-Lab • Production-style

110 min

Goal. Configure a freshly Terraform-provisioned host end to end, with secrets and drift detection.

Business context. New CloudBase hosts must be production-ready without anyone SSHing in.

Architecture. Terraform creates and tags the instance → dynamic inventory reads the tags → the `base` role configures it → a scheduled `-check` job reports drift.

Requirements. Dynamic inventory from cloud tags; service user, packages, hardened `sshd` with `validate`, templated `systemd` unit; database password in a Vault file with a `prod` vault ID; `no_log` on the consuming task.

Constraints. No IP address anywhere in the repository. No plaintext secret. Ansible must create no cloud resources.

Hints. Reach private hosts through the bastion with an SSH `ProxyCommand` in the inventory's group vars.

Expected outcome. `terraform apply` then `ansible-playbook site.yml` yields a working host; a second play run reports `changed=0`; `-check -diff` after a hand-edit reports exactly that edit.

Production considerations. What breaks when the fleet reaches 500 hosts and `forks: 5`? Where would the vault passphrase live if the control node were a CI runner nobody owns?

Common pitfalls. Committing the dynamic inventory cache; letting the role `apt upgrade` everything and calling it hardening; forgetting that removing a task does not uninstall anything.

9 Engineering Notes

☰ Operational excellence

Ansible's real product is not the playbook — it is the answer to "what is supposed to be on this host, and who decided that?" A role in Git with a merge request history answers both questions in thirty seconds. A host configured by hand answers neither, ever. When you evaluate whether a piece of automation is worth writing, weigh the review trail as heavily as the time saved.

📈 Scalability note

The scaling wall is the control node, not the fleet. At a few hundred hosts you will hit `forks`, file-descriptor limits and a single point of failure in whoever's laptop is running the play. The architectural answer is a centralised execution layer — AWX, its supported form Ansible Automation Platform, or simply a dedicated CI runner — that owns the inventory, holds the credentials, serialises concurrent runs and logs who ran what against which hosts. NASA's published case study attributes its results specifically to centralising execution this way rather than to any individual playbook.

🚨 Disaster recovery tip

Configuration management is a recovery tool before it is a convenience. The question "can we rebuild this host from scratch in an hour?" has an honest answer only if a role describes it completely. Test it the only way that means anything: destroy a non-production host and rebuild it from Terraform plus the role, on the clock, with the runbook you actually have. Whatever manual step you discover is missing is the same step that will be missing at 3am.

👤 Interview insight

Expect: "Where is the line between Terraform and Ansible?" The weak answer names the tools. The strong one names the property: Terraform owns resource *lifecycle* and keeps state so it can detect additions and removals; Ansible owns in-host state and keeps none, re-deriving reality every run. Then land the consequence — that is why deleting an Ansible task uninstalls nothing, and why letting both tools own the same resource gives you two sources of truth and no way to reconcile them.

❓ Why does this exist?

Ansible succeeded against technically mature competitors largely because it removed the bootstrap problem. Agent-based tools require you to install and certificate-enroll an agent before you can manage a host — which means you need configuration management to set up configuration management. Building on SSH and an interpreter that every server already has made the first useful run possible in minutes. Reducing time-to-first-value beats feature completeness more often than engineers expect.

10 Build-Along Project — CloudBase

Day 1 gave CloudBase a hand-written `bootstrap.sh`; Day 2 put it under review; Day 7 had Terraform provision the VPC, the bastion and the app subnet. The script is now the weakest link: it lives in the repository but nothing verifies it was run, what version was run, or whether anyone has edited a host since. Today it retires.

🔧 Build-Along • Day 08 — from bootstrap script to declared host state

- Add `ansible/` to the CloudBase repository: `ansible.cfg`, `site.yml`, `roles/`, and `inventories/staging/` and `inventories/prod/` as separate directories with their own `group_vars/`.
- Build `roles/base/` that reproduces everything `bootstrap.sh` did — the `cloudbase` service user, `/opt/cloudbase` with mode `0750`, the JRE and base packages, log rotation — using purpose-built modules only.
- Template the Day 1 `systemd` unit as `cloudbase-api.service.j2`, interpolating the service user, app directory and environment name from `defaults/main.yml`.
- Add hardened `sshd` configuration — no password authentication, no root login, group-restricted access — deployed with `validate: /usr/sbin/sshd -t -f %s` and notifying a restart handler.
- Wire dynamic inventory to the tags Terraform already sets, so no IP address or hostname is committed; reach private hosts through the bastion via `ProxyCommand` in group vars.
- Move the database password into `inventories/prod/group_vars/app/vault.yml` encrypted with a `prod` vault ID, referenced from a plaintext `vars.yml`, with `no_log: true` on the consuming task.
- Delete `bootstrap.sh` and record in `docs/host-baseline.md` that the role is now the sole description of host state.
- Add a CI stage that runs the play twice against a throwaway host and fails unless the second run reports `changed=0`; add a scheduled `-check -diff` run against production that reports drift.

Definition of done: `terraform apply` followed by `ansible-playbook -i inventories/prod site.yml` takes a bare instance to a fully configured CloudBase host with no manual step; an immediate second run reports `changed=0` on every host; no IP address and no plaintext secret exists anywhere in the repository; and `bootstrap.sh` is gone.

11 Chapter Summary

Key takeaways

- ✓ Terraform owns resource lifecycle and keeps state; Ansible owns in-host state and keeps none.
- ✓ Because there is no state, Ansible cannot detect removal — deletion must be declared with `state: absent`.
- ✓ Idempotency lives inside modules: read current state, compare, act on the difference, report `changed`.
- ✓ `shell` and `command` always report `changed`, so they break drift detection, check mode and handlers.
- ✓ Handlers deduplicate, run once, at the end of the play, in definition order — not notification

order.

- ✓ Roles are the unit of reuse; `defaults/` is the public interface, `vars/` is internal.
- ✓ Vault is AES-256 file encryption with per-environment vault IDs — not rotation, audit or revocation.

Things to remember

- A second run reporting `changed=0` is the only proof of idempotency that counts.
- `mode: 0644` unquoted is an integer in YAML; always write `"0644"`.
- `validate:` on a template is what stops a typo from locking you out of an entire fleet at once.
- `-check -diff` is Ansible's `plan`, and it is worthless in a playbook made of shell tasks.

Common pitfalls (last look)

- Always-changed tasks • hardcoded IPs • plaintext secrets in Git
- Two tools owning one resource • assuming a handler ran after a failed play

One-Page Cheat Sheet

Boundary — Terraform provisions and owns lifecycle; Ansible configures what is inside a host.

Agentless — SSH plus remote Python; no daemon, no certificates, no state file.

Idempotency — A module property: compare declared state to actual, act on the difference, report `changed`.

Guards — `creates:` / `removes:` to skip, `changed_when:` / `failed_when:` to correct a verdict.

Handlers — Notified by name, deduplicated, run once at end of play in definition order.

Roles — `tasks` • `handlers` • `templates` • `files` • `defaults` (low) • `vars` (high) • `meta`.

Inventory — Static file or dynamic plugin over cloud tags; `group_vars/` beside it, one directory per environment.

Vault — `$ANSIBLE_VAULT;1.2;AES256;prod` — file or single value, vault IDs per environment.

CLI quick reference

Day 8 commands worth memorising

```
# connectivity and inventory
ansible -i inventories/prod all -m ping          # SSH + python check
ansible-inventory -i inventories/prod --graph    # resolved group tree
ansible-inventory -i inventories/prod --host app-prod-1 # all vars

# running
ansible-playbook -i inventories/prod site.yml
ansible-playbook -i inventories/prod site.yml --check --diff # dry run
ansible-playbook -i inventories/prod site.yml --limit app --tags sshd
ansible-playbook -i inventories/prod site.yml --start-at-task "Deploy unit"

# introspection
ansible-doc ansible.builtin.apt                 # params + check-mode support
ansible-lint roles/base                         # catch non-idempotent patterns
ANSIBLE_CALLBACKS_ENABLED=profile_tasks ansible-playbook site.yml

# vault
```

```
ansible-vault encrypt --vault-id prod@prompt group_vars/app/vault.yml
ansible-vault edit --vault-id prod@prompt group_vars/app/vault.yml
ansible-vault rekey --vault-id prod@prompt group_vars/app/vault.yml
ansible-playbook site.yml --vault-id prod@run/secrets/vault-pass
```

Interview questions to master

- › What does idempotency mean in Ansible, concretely, and why does it matter?
- › Why is Ansible agentless, and what is the trade-off versus an agent-based tool?
- › What is the difference between a task and a handler, and when do handlers actually run?
- › How would you structure configuration for reuse across many playbooks and environments?
- › Where does the line sit between what Terraform should manage and what Ansible should manage?
- › How does Ansible Vault protect secrets in a Git-tracked repo — and what does it not protect against?
- › Your playbook reports `changed=6` on every run. Walk me through diagnosing it.

Further reading & official docs

- Ansible playbook guide, at [docs.ansible.com](https://docs.ansible.com/docs.ansible.com/playbook_guide/) under `playbook_guide/` → `playbooks_intro` — the authoritative introduction to plays, tasks and idempotency.
- Reusing playbooks with roles — `playbook_guide/` → `playbooks_reuse_roles`: the seven standard directories, and `defaults` versus `vars` precedence.
- Playbook execution strategies — `playbook_guide/` → `playbooks_strategies`: linear versus free, forks and serial execution.
- Protecting sensitive data with Ansible Vault — the `vault_guide/` section, including vault IDs and payload format 1.1 and 1.2.
- Ansible-core porting guides (2.19 and 2.20) — required reading before an upgrade; 2.19 overhauled templating and introduced Data Tagging, which broke brittle Jinja patterns.
- Red Hat, "8 ways to speed up your Ansible playbooks" — forks, pipelining, `ControlPersist` and fact gathering, with the mechanism behind each.
- GitLab public runbooks, runbooks.gitlab.com — their Chef cookbook process, for a real pull-based, agent-driven contrast to everything in this chapter.

Production checklist

- ☐ A second consecutive run of every playbook reports `changed=0`.
- ☐ No `shell` or `command` task exists without `creates:`, `removes:` or `changed_when:`.
- ☐ Every config template that has a validator uses `validate:`.
- ☐ Production and staging are separate inventory directories; production is never the default.
- ☐ No IP address is committed — inventory derives from cloud tags.
- ☐ Every secret in the repository is vaulted, with per-environment vault IDs, and consumed under `no_log: true`.
- ☐ `ansible-lint` runs in CI, and a scheduled `-check -diff` job reports drift against production.

- ☐ Removal is declared explicitly, not implied by deleting a task.

Key idea

What you will learn next (Day 09). Your hosts are now declared, your infrastructure is declared, and both live in Git — but a human still decides when to run them. Tomorrow you build the pipeline that decides: CI/CD, from a commit through build, test and artefact to an automated, reversible deployment, with the deployment strategies that let you change production without holding your breath.

DAY 09 • DELIVERY

CI/CD Pipeline Engineering

Stages, artifact promotion, secrets and the gates that actually matter

09

🕒 Estimated study time: 8 hours

🎯 By the end of this day, you will be able to...

- Design a pipeline as a sequence of *gates* with an explicit failure policy, not as a build script that happens to run on a server.
- Build a GitHub Actions workflow that tests, scans, builds, pushes and deploys — and explain what the runner is doing underneath each step.
- Promote one immutable, SHA-tagged artifact through every environment, and say precisely which class of bug that prevents.
- Handle pipeline credentials correctly: short-lived OIDC identities, scoped tokens, pinned third-party actions, and no secret in a log.
- Make feedback fast with caching and job parallelism, and articulate honestly when Jenkins is still the right answer.

1 The Big Picture

What problem this solves. By Day 8 you can build an image (Day 4), run it on a cluster (Days 5–6), provision the infrastructure under it (Day 7) and configure the hosts around it (Day 8). Every one of those is still triggered by a human deciding to run something. That human is the bottleneck, the source of variance, and — during an incident at 2am — the least reliable component in the system. A delivery pipeline replaces "someone runs the steps" with "the repository runs the steps, identically, every time, and refuses to proceed when a check fails."

Why DevOps engineers need it. CI/CD is where every other discipline in this book becomes enforceable. A test suite nobody runs is documentation. A vulnerability scanner run quarterly by hand is theatre. A rollback procedure written in a wiki page is fiction. The pipeline is the one place where those things become mandatory, because it is the only path code takes to production. Engineers who understand this stop asking "how do I write a workflow file?" and start asking "what must be true before this artifact is allowed to move forward?"

Where it fits in a modern cloud architecture. The pipeline sits between the source of truth (the repository, Day 2) and every runtime you have built so far. It is the seam — and therefore the highest-value target in your whole estate.

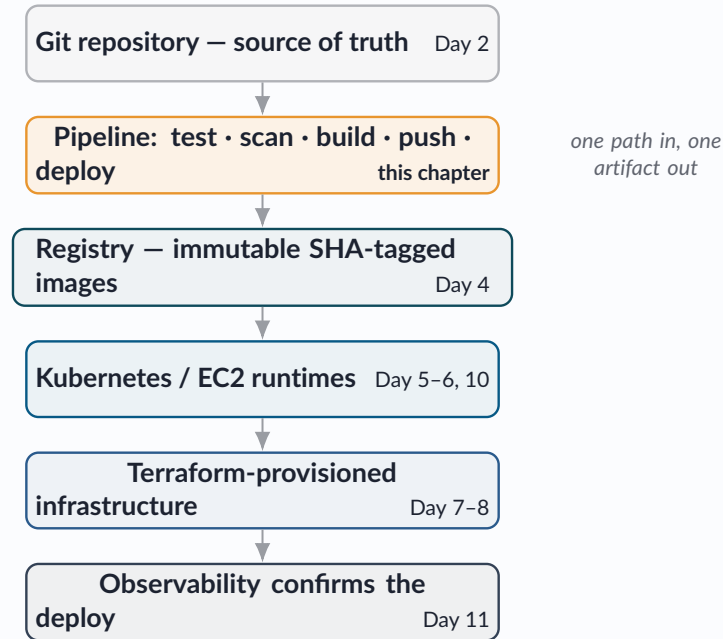


Fig 9.0 — Delivery is the seam between version control and every runtime. Everything that ships passes through this one path.

Key idea

A pipeline is not automation of your deploy steps. It is a *policy engine* that happens to run commands. The interesting question is never "what does stage 3 do?" but "what is stage 3 refusing to let through, and can a human under deadline pressure bypass it?" If the answer to the second half is yes, that gate does not exist.

How it connects to previous chapters. Day 2 gave you branch protection — the mechanism that makes a pipeline's verdict binding rather than advisory. Day 4 gave you the artifact this pipeline produces and the environment-variable configuration model that makes one image runnable in every environment. Day 7 gave you the idea that a plan is reviewed before it is applied; today's manual approval gate is the same idea with a different noun. Day 8's idempotency is why re-running a pipeline is safe.

Where companies use it in production. Shopify built and open-sourced `Shipit`, a deployment coordination engine that gives each project a version-controlled `shipit.yml` describing its own deploy recipe, so dozens of teams shipping many times a day share one control plane instead of one shared script. Their published release flow is a chain of gates — pull request, CI, canary, then production — with deliberate engineering effort spent on keeping the end-to-end path fast, because a slow pipeline is a pipeline engineers route around. The architectural lesson is not the tool: it is that deploy recipes live *in the repository they deploy*, versioned with the code, while the orchestration and the gates live centrally.

Common misconceptions. That CI means "we have a build server" (it means every change is integrated and verified continuously, on a trunk, in minutes); that green means safe (it means the gates you wrote passed); that the pipeline is infrastructure plumbing rather than the most security-sensitive system you own; and that pipeline speed is a nice-to-have rather than the single largest determinant of whether the gates get respected.

2 Concept Map

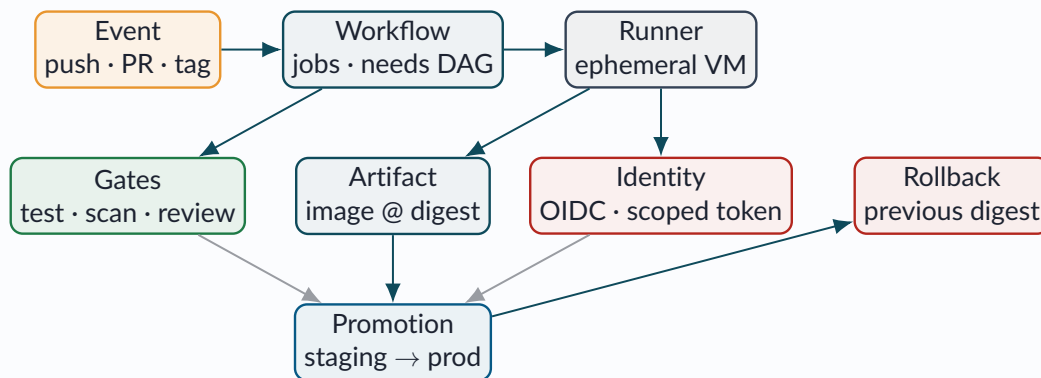


Fig 9.1 — The Day 9 territory: an event triggers a run, the run executes gated jobs on a runner, and one artifact is promoted through environments.

Terminology to anchor: **event** → **workflow** → **job** → **runner** → **gate** → **artifact** → **digest** → **promotion** → **rollback**.

3 Guided Learning

3.1 Integration, Delivery, Deployment — and What a Gate Really Is

What is it? **Continuous Integration** means every change is merged into the shared trunk frequently and automatically verified there. **Continuous Delivery** means the trunk is always in a releasable state — production is one deliberate decision away. **Continuous Deployment** removes that decision: every green build goes to production automatically.

Why does it exist? Before CI, teams worked on long-lived branches and merged near a release date. The result had a name: *integration hell* — weeks of work colliding at exactly the moment there was no schedule left to absorb it. The first fix was the nightly build: a machine compiled the trunk at 2am and emailed whoever broke it. That collapsed the feedback loop from weeks to a day, and every improvement since — Hudson and Jenkins, hosted runners, per-PR checks — has been the same move repeated: shorten the interval between "I wrote a mistake" and "a machine told me."

How does it work internally? A pipeline is a directed acyclic graph of jobs plus a failure policy. Each job declares its dependencies; the scheduler runs whatever has no unmet dependencies, in parallel when it can. A **gate** is simply a job whose non-zero exit blocks its dependents — but a gate only has teeth when something outside the pipeline honours the verdict. That something is Day 2's branch protection: a *required status check* means the merge button is disabled until the check reports success, enforced by the forge, not by discipline.

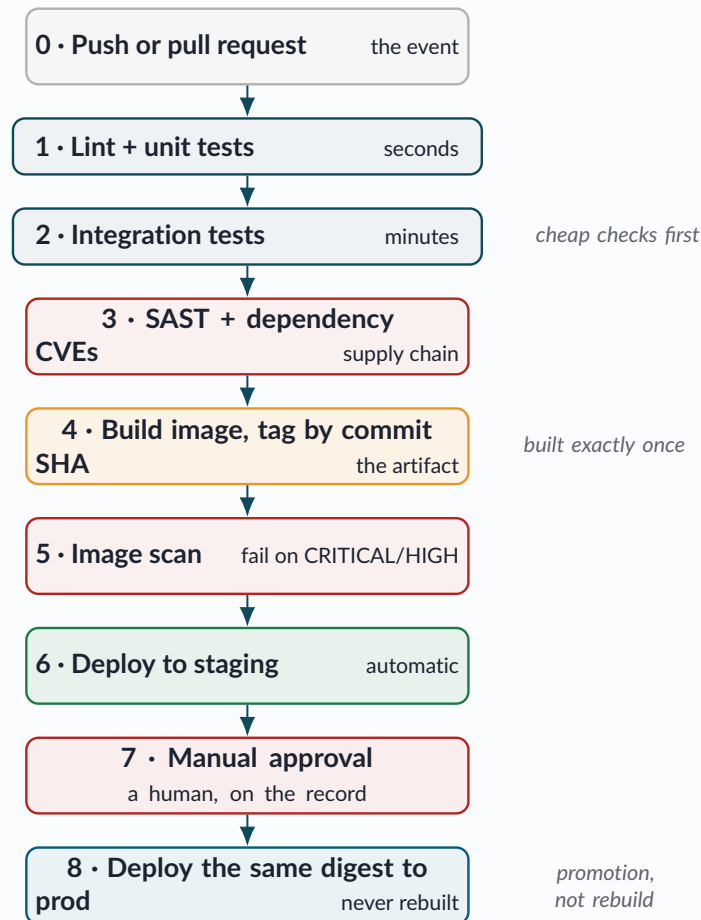


Fig 9.2 — The gate chain. Each stage is cheap and fast before it is expensive and slow, so failures surface at the leftmost possible point.

? Why does this exist?

Ordering stages by cost is not an optimisation detail — it is the whole design. If a lint error is only discovered after a twelve-minute integration suite and a four-minute image build, you have taught your engineers that the pipeline wastes their time, and they will start merging on faith and watching the deploy instead. Fast, cheap gates first is how you keep the slow, expensive gates credible.

When should I use it? CI on every repository, from the first commit, without exception. Continuous Delivery as soon as you have a staging environment worth deploying to.

When should I not? Continuous *Deployment* — fully automatic to production — is not a maturity badge. It requires strong automated test coverage, feature flags, fast rollback and real-time observability (Day 11). Turning it on without those means you have automated the propagation of defects. Plenty of excellent teams deliberately keep a human gate on production forever, because the cost of a bad deploy in their domain (payments, healthcare, industrial control) dwarfs the cost of a five-minute wait.

Production example. Shopify's flow is instructive precisely because it is not fully automatic: a change moves pull request → CI → canary → production, with the canary step exposing the new build to a slice of real traffic before the fleet. That is a gate whose evidence is production telemetry rather than

a test assertion — the most honest gate there is, and one you can only afford once your observability is good enough to read it quickly.

3.2 What a Hosted Runner Is Actually Doing

What is it? A **runner** is a machine (or container, or ephemeral VM) that executes the steps of one job. GitHub-hosted runners are freshly created per job and destroyed afterwards; self-hosted runners are machines you own that register themselves and wait for work.

Why does it exist? Jenkins' original model was a long-lived controller with long-lived agents, and it produced a specific class of misery: agent number 7 has a different JDK, a leftover `node_modules`, a cached Maven artifact from a deleted branch. Builds passed or failed depending on where they landed. Ephemeral runners exist to make the execution environment a *function of the workflow definition* rather than a property of a machine's history — the same immutability argument that produced containers on Day 4 and Terraform on Day 7, applied to the build host.

How does it work internally? The mechanism is worth knowing because almost every pipeline mystery lives in it:

- An event (push, pull request, tag, schedule, manual dispatch) is written to the forge's event stream. Workflow files in `.github/workflows/` are evaluated against it; matching workflows produce a *run*.
- The run's jobs are placed on a queue. A runner acquires a job over an outbound long-poll HTTPS connection — the runner dials out, nothing dials in. This is why self-hosted runners need no inbound firewall rule, and why a runner inside a private subnet still works.
- The runner materialises the job: it checks out nothing by default (`actions/checkout` is an ordinary step), sets up a workspace directory, and writes the job's environment.
- Each *run:* step is written to a temporary script file and executed by a shell as a child process. Its exit status is the step's result. Each step is a *separate process*, which is why `cd` or an exported variable in one step is invisible to the next — state crosses steps only through the filesystem or the environment files the runner exposes.
- An *uses:* step is not a plugin. It is a git repository checked out into a temp directory and executed — a Node entrypoint, a Docker container, or a composite of further steps. *Third-party actions run with the same privileges as your own code*. Hold that thought for §3.4.
- Secrets referenced by the job are injected as environment variables or step inputs. The runner also registers each secret value with a log-scrubbing filter that replaces exact matches with asterisks on the way to the log stream.
- When the job ends, a hosted runner's VM is destroyed. Nothing survives except what you uploaded as an artifact, pushed to a registry, or wrote to a cache.

👁 Behind the scenes

Log masking is string matching, not a security boundary. The runner knows the literal secret value and redacts occurrences of it. Transform the value — base64 it, print it one character per line, pipe it through `jq` — and the filter no longer matches. This is exactly the technique the March 2025 `tj-actions/changed-files` compromise used: the injected payload dumped the runner process's memory and wrote it, encoded, into the workflow log, from which secrets could be read. Treat masking as a guard against accidents, never against an adversary.

🔧 Design trade-off • hosted versus self-hosted runners

Hosted runners give you a clean machine per job, zero maintenance and no capacity planning; you pay per minute and you accept the provider's images and network egress. Self-hosted runners give you private-network access to internal registries and databases, big machines, custom hardware and predictable cost at scale; you inherit patching, isolation and — critically — the risk that a job leaves residue for the next one. The rule that resolves most arguments: **self-hosted runners must be ephemeral too**. A self-hosted runner that persists between jobs, and accepts jobs from forked pull requests, is a remote code execution service you have voluntarily attached to your VPC.

3.3 Build Once, Promote the Same Artifact

What is it? The artifact — for us, a container image — is built exactly once, in one job, and the *identical bytes* are what runs in staging, then in production. Environments differ only by injected configuration.

Why does it exist? Because the alternative silently reintroduces the drift the whole pipeline exists to eliminate. If you run `docker build` again to produce the "production" image, you have created a second artifact that was never tested. Between the two builds, a base image tag may have been re-pushed with a patched OS layer; a transitive dependency may have published a new patch version that your lockfile permits; a build-time download may resolve to different bytes. Each of those is individually harmless and collectively the reason "it passed in staging" stops meaning anything.

How does it work internally? A registry stores an image as a **manifest** — a JSON document listing layer digests and a config blob — and the image's true identity is the SHA-256 **digest** of that manifest. A *tag* is only a mutable pointer to a digest, exactly like a git branch pointing at a commit. `app:latest` can mean different bytes tomorrow; `app@sha256:9f2c...` cannot, ever. Tagging by commit SHA gives you a human-traceable, unique, immutable-by-convention name; referencing by digest at deploy time gives you immutability enforced by cryptography.

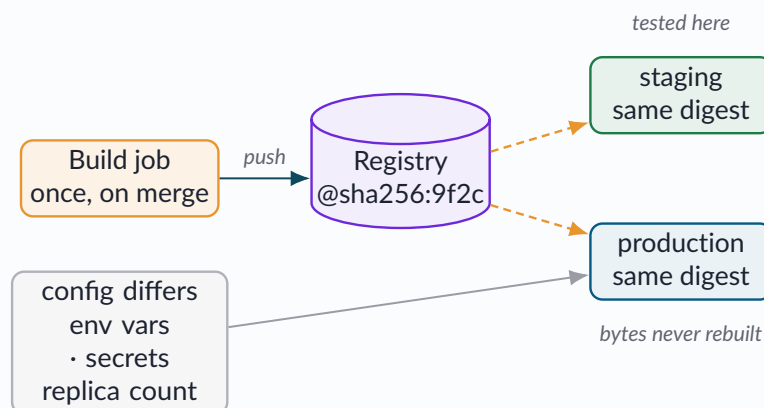


Fig 9.3 — One build, many environments. The digest is constant; only injected configuration changes.

💚 Reliability principle

Rollback is a consequence of this design, not a separate feature. Because the previous release is still in the registry under its own digest, reverting is "deploy that digest again" — a thirty-second operation with a known outcome. The alternative, rebuilding from the previous commit under incident pressure, asks your build to reproduce a state from days ago at the exact moment reproducibility is least likely

and the cost of being wrong is highest. Write the rollback command into the runbook *before* you need it, and rehearse it (Day 14).

⚠ Common misconception

"We tag with `latest` and the SHA, so we are fine." You are not: whichever reference the deployment manifest uses is the one that matters. If the Kubernetes Deployment says `app:latest`, then a pod rescheduled at 3am pulls whatever `latest` points to at 3am — which may be a build nobody approved. Worse, with `imagePullPolicy: IfNotPresent`, some nodes hold an old `latest` and others pull a new one, and you get two different versions serving traffic from one Deployment. Pin the deployed reference. Always.

3.4 Pipeline Security — A Compromised Pipeline Is Compromised Production

What is it? The set of controls ensuring that only intended code, built by an intended process, using narrowly scoped credentials, can produce an artifact that reaches production.

Why does it exist? Think about what your pipeline holds: registry push credentials, cloud deploy roles, database migration access, signing keys. It executes code from your repository, from every pull request, and from every third-party action you referenced. It is, in effect, a machine with production credentials that runs arbitrary code from the internet. Attackers worked this out well before most engineering teams did. In March 2025 the widely used `tj-actions/changed-files` action was compromised (CVE-2025-30066): existing version tags were rewritten to point at malicious code, so even pipelines pinned to a "stable" tag executed the payload, which dumped runner memory into workflow logs. Public repositories leaked their secrets into publicly readable logs.

How does it work internally? Four mechanisms carry most of the weight.

1. Short-lived identity instead of stored credentials. With OpenID Connect, the job requests a signed JWT from the forge's OIDC provider. That token carries claims describing exactly what is running — notably `sub`, which encodes the repository, and optionally the branch or the deployment environment, for example `repo:acme/cloudbase:environment:production`. The cloud provider has a pre-established trust relationship with that issuer and a role whose trust policy asserts which `sub` and `aud` values it will accept. It validates the token's signature and claims, then returns a cloud access token valid only for the life of the job. No long-lived key exists to leak, rotate or forget. The job must declare `permissions: id-token: write` to be allowed to request the token at all.

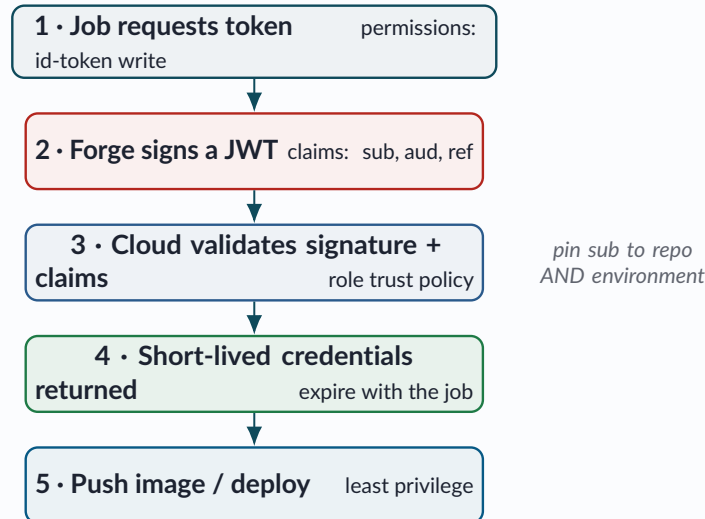


Fig 9.4 — OIDC token exchange. No long-lived cloud key is ever stored in the pipeline.

2. Pin third-party actions to a full commit SHA. A tag is mutable; a commit SHA is not. `uses: someone/action@v4` trusts whoever can move v4. `uses: someone/action@a1b2c3...` trusts a specific tree you could have reviewed. This single change would have neutralised the tj-actions incident for every repository that had adopted it.

3. Least-privilege job tokens. The automatically provided repository token defaults to broad permissions in older configurations. Declare `permissions:` explicitly at workflow level — start from `contents: read` and add only what a job needs. A test job has no business being able to write packages.

4. Untrusted input never reaches a privileged context. Pull requests from forks must not run with secrets. The dangerous pattern is a trigger that runs `merged` untrusted code with repository secrets available; the safe pattern is to run untrusted code in a low-privilege job that produces only a report, and to perform anything privileged in a separate job triggered by trusted events. Related: never interpolate attacker-controllable text (a PR title, a branch name, an issue body) directly into a `run:` script — the interpolation happens before the shell sees the line, so a crafted title becomes a shell command. Pass it through an environment variable instead.

Security warning • the blast radius nobody sizes until afterwards

A stolen deploy credential is not "a CI problem." It is production write access, registry push access (so an attacker can publish an image your cluster will happily pull), and often cloud IAM broad enough to read data stores. Size it deliberately: write down what a fully compromised pipeline could do in your account today. If that list is uncomfortable, the fix is scoping the role and the OIDC trust policy — not adding another approval step in front of the same over-privileged identity.

Senior engineer insight

The mature framing is **provenance**: can you prove which source commit, built by which process, produced the bytes now running in production? The SLSA specification formalises this as build levels — level 1 is "provenance metadata exists," and the higher levels add signed, non-falsifiable provenance produced by a build platform the build script itself cannot tamper with. You do not need a compliance programme to benefit. Emitting a signed attestation that binds image digest to commit SHA to workflow run turns "I think that's the right build" into a verifiable claim, and it is the natural extension of everything in §3.3.

3.5 Feedback Speed: Caching and Parallelism as Engineering Decisions

What is it? Two levers that shorten wall-clock time to a verdict: reusing expensive work between runs (caching), and running independent work simultaneously (parallelism).

Why does it exist? Feedback latency is not a comfort metric — it changes behaviour. A three-minute pipeline is consulted; a forty-minute pipeline is worked around. Engineers batch changes into bigger, riskier merges, or push and context-switch and return an hour later having forgotten the change. Shopify treated end-to-end deploy speed as an explicit engineering target for exactly this reason. Pipeline duration is a product decision about how your team writes software.

How does it work internally? A cache step computes a key — typically a hash of a lockfile plus the OS and language version — and asks the cache service for an archive stored under that key. On a hit it extracts the archive into a path (the dependency directory). On a miss it may fall back to *restore-keys*, prefix matches that give a slightly stale but still useful starting point, and then uploads a fresh archive under the exact key at job end. Two properties follow directly from the mechanism:

- **The key must include everything that invalidates the cache.** Hash the lockfile, not the manifest: a manifest with version ranges resolves differently over time, so a manifest-keyed cache serves stale dependencies indefinitely.
- **A cache must never be trusted for correctness.** It is a performance artifact, and on many systems it can be written by less-trusted contexts. Never cache secrets or built release artifacts you intend to ship; build outputs belong in an artifact store or a registry, where they are addressed by digest.

Docker layer caching is the same idea one level down: each instruction produces a layer keyed by the instruction plus the digest of its inputs, so ordering your Dockerfile to copy the lockfile and install dependencies *before* copying source code (Day 4) is what makes a code-only change reuse the dependency layer instead of rebuilding it.

Parallelism is expressed by the job dependency graph. Jobs with no *needs*: relationship start together; a matrix expands one job definition into N parallel jobs across dimensions (language version, OS, shard index). Test sharding — splitting a suite across five runners — converts a ten-minute serial suite into roughly two minutes of wall clock at five times the compute cost.

Performance note

There is a real ceiling. Beyond a certain fan-out you pay a fixed per-job cost — runner acquisition, checkout, dependency restore — on every shard, so 20 shards of a 10-minute suite is not 30 seconds; it is 30 seconds of tests plus 20 checkouts. Measure the per-job overhead first, then choose the shard count where marginal savings stop exceeding it. And parallelise the *critical path*: shaving a job that is not the longest chain in the DAG changes the wall clock by exactly zero.

Cloud cost optimization

Hosted runner minutes are billed, and matrix builds multiply them silently. A five-way matrix on every push to every branch, with a ten-minute suite, is fifty compute-minutes per push. Two cheap controls recover most of it: cancel superseded runs on the same branch (concurrency groups), and run the full matrix only on the trunk and on release tags, keeping pull requests on the single most representative configuration. Cost discipline here is also feedback discipline — the same change makes the queue shorter.

3.6 GitHub Actions and Jenkins — An Honest Comparison

What is it? Jenkins is a self-hosted, plugin-extensible automation server, originally Hudson, that

predates hosted CI by roughly a decade. Its pipelines are `Jenkinsfile` definitions in Groovy (declarative or scripted) executed by a controller that dispatches to agents you operate.

Why does it exist — and why is it still here? Jenkins solved build automation when there was no credible hosted option, and it accumulated an enormous plugin ecosystem covering hardware, legacy systems and integrations no SaaS vendor will ever ship. Teams still choose or keep it for concrete reasons, not inertia alone: fully on-premises or air-gapped environments; builds that must touch internal networks, licensed toolchains or specialised hardware; approval and audit workflows encoded over years; and a cost model where owning big build machines is dramatically cheaper than per-minute billing at high volume.

Dimension	GitHub Actions	Jenkins
Execution env.	Ephemeral runner per job; clean by construction	Agents you operate; clean only if you make them so
Definition	YAML in the repo, evaluated per event	Groovy <code>Jenkinsfile</code> ; scripted mode is a full language
Extension model	Actions = git repos executed in your job	Plugins = JVM code in the controller process
Operational load	Provider runs the control plane	You run, patch, back up and upgrade it
Coupling	Tight to the forge; events and PR checks are native	Forge-agnostic; integrates with anything
Main risk	Third-party action supply chain	Plugin sprawl, upgrade drift, a stateful controller

Design trade-off • the real trade being made

You are choosing where complexity lives. Hosted CI pushes it outward: less to operate, but the extension surface is untrusted code from the internet and you inherit the provider's model. Jenkins pulls it inward: total control and no per-minute bill, but the controller becomes a stateful, security-sensitive service that must be patched, whose plugins can conflict on upgrade, and whose agents drift. Neither is wrong. What is wrong is choosing without naming which cost you are accepting — and the most common failure mode in both worlds is identical: pipeline logic accumulating in one enormous unversioned blob that nobody dares change.

Automation tip

Whichever tool you use, keep the pipeline thin. A workflow step should call a script that lives in your repository and runs identically on a laptop — `make test`, `./scripts/build-image.sh`. Logic embedded in YAML or Groovy can only be tested by pushing a commit and waiting, which is the slowest debugging loop in software. Thin pipelines are also portable: migrating from Jenkins to Actions becomes rewriting fifty lines of orchestration rather than reverse-engineering two thousand lines of Groovy.

`.github/workflows/delivery.yml` — the shape that matters

```
name: delivery
on:
  push:
    branches: [main]
  pull_request:

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
# cancel superseded runs on the same ref

permissions:
  contents: read
# least privilege by default
```

```

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683 # v4.2.2
      - uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: temurin
          cache: maven # keyed on the lockfile, not the manifest
      - run: ./mvnw --batch-mode verify

  build-push:
    needs: test # gate: no artifact without a green suite
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write # only this job may push
      id-token: write # only this job may mint an OIDC token
    outputs:
      digest: ${ steps.push.outputs.digest }
    steps:
      - uses: actions/checkout@v4
      - name: Build image tagged by commit SHA
        run: docker build -t ghcr.io/acme/cloudbase:${ github.sha } .
      - name: Scan before publishing
        run: trivy image --exit-code 1 --severity CRITICAL,HIGH \
          ghcr.io/acme/cloudbase:${ github.sha }
      - name: Push and capture the immutable digest
        id: push
        run: |
          docker push ghcr.io/acme/cloudbase:${ github.sha }
          echo "digest=$(docker inspect --format='{{index .RepoDigests 0}}' \
            ghcr.io/acme/cloudbase:${ github.sha })" >> "$GITHUB_OUTPUT"

  deploy-staging:
    needs: build-push
    environment: staging # no approval rule -> deploys automatically
    runs-on: ubuntu-latest
    steps:
      - run: ./scripts/deploy.sh staging "${ needs.build-push.outputs.digest }"

  deploy-production:
    needs: deploy-staging
    environment: production # protection rule -> waits for a reviewer
    runs-on: ubuntu-latest
    steps:
      - run: ./scripts/deploy.sh production "${ needs.build-push.outputs.digest }"

```

Read that file as a policy document. `needs:` is the gate chain. `permissions:` is the blast radius. The `digest` output threaded from `build-push` into both deploy jobs is the "build once, promote" rule expressed mechanically — production physically cannot receive a different artifact, because no second build exists to produce one.

How the approval gate actually works. A deployment **environment** is a named object on the repository carrying protection rules: required reviewers (a small set of people or teams, one approval sufficient, with optional prevention of self-review), an optional wait timer measured in

minutes, and deployment branch policies restricting which refs may deploy to it at all. A job that declares `environment: production` pauses before its first step until those rules are satisfied, and the environment's secrets are only readable by jobs that passed them. That last property is the important one: the approval does not merely delay the job, it is what unlocks the production credentials.

☰ Operational excellence

An approval gate is only meaningful if the approver is shown something decidable. "Approve deploy to production" with no context produces reflexive clicking within a week. Put the commit range, the artifact digest, the diff of configuration and the staging soak result in front of them. A gate that always gets approved is a latency tax pretending to be a control.

✓ Best practices — what experienced engineers do

- Build the artifact once; promote the same digest through every environment; never rebuild for production.
- Make gates binding via branch protection required status checks — a check nobody must satisfy is a suggestion.
- Pin third-party actions to a full commit SHA, and declare `permissions:` explicitly, starting from `contents: read`.
- Use OIDC short-lived cloud credentials instead of stored access keys wherever the provider supports it.
- Order stages cheapest-and-fastest first, and treat total pipeline duration as a tracked metric with a target.
- Keep pipeline logic in repository scripts that run identically on a laptop; keep the YAML thin.

✗ Common mistakes beginners make

- Deploying a mutable tag such as `latest`, then being unable to say what is running.
- Rebuilding the image per environment, so the tested bytes and the shipped bytes are not the same bytes.
- Committing a `.env` with real credentials, or echoing a secret to debug it (the log outlives the incident).
- Giving one pipeline identity broad, permanent cloud permissions and calling the approval step the control.
- Skipping or marking tests `continue-on-error` to make the pipeline green under deadline pressure.
- Having a forward-deploy path only — no rehearsed rollback, discovered during the outage.

4 Visualizations to Internalize

Redraw these from memory. If you can draw the gate chain and the OIDC exchange without notes, you can design a pipeline in a whiteboard interview.

 Redraw: the gate chain from push to production — name each gate and what it refuses to let through

 Redraw: one build job feeding staging and production from a single digest — and mark exactly what differs between the two environments

5

Guided Hands-On Lab — A Pipeline With Teeth

Goal. Take the CloudBase repository from a repo with tests to a repository that ships: automatic verification on every pull request, one artifact built on merge, automatic staging, and a human gate in front of production — with no credential stored anywhere in the code.

1 Write the verification job first, and only that

Create a workflow triggered on `pull_request` and on push to trunk that checks out, sets up the toolchain with dependency caching, and runs the full test suite. Nothing else yet.

 **Why:** Every later stage depends on this one. Get the trigger set, the cache key and the runtime honest before adding anything that pushes bytes anywhere. A pipeline built stage-by-stage stays debuggable; one written all at once fails for six overlapping reasons.

2 Make the check binding

In branch protection (Day 2), mark the job a required status check on the trunk and forbid direct pushes.

 **Why:** This is the moment the workflow stops being a script and becomes a gate. Until the forge refuses the merge, you have automation, not policy — and under deadline pressure the difference is everything.

3 Add the build job, gated on tests, tagged by commit SHA

Add a second job with `needs: test` and a condition restricting it to the trunk. Build the image, tag it

<repo>:\${{ github.sha }}, push it, and capture the resulting digest as a job output.

 **Why:** *needs: encodes "no artifact exists unless the suite passed." Capturing the digest — not just the tag — gives downstream jobs a reference that cannot be repointed later.*

4 Replace stored cloud keys with an OIDC trust relationship

Create a cloud role whose trust policy accepts tokens from the forge's issuer with a `sub` claim scoped to this repository (and, for the production role, to the production environment). Grant the job `id-token: write` and delete the long-lived access key.

 **Why:** *You are removing the single most valuable stealable object in the pipeline. Scoping `sub` to the environment as well as the repository means a token minted by a test job on a feature branch cannot assume the production role even if an attacker gets code execution in it.*

5 Insert the scan gate before the push, not after

Run an image scan configured to exit non-zero on critical and high findings, positioned between build and push.

 **Why:** *Scanning after publishing means a vulnerable image already exists in the registry where something can pull it. The gate belongs upstream of the artifact becoming reachable — the same reasoning as validating input before writing it to a database.*

6 Deploy to staging automatically, parameterised by digest

Add a staging deploy job that takes the digest as input and calls a script in your repository. No secrets in the YAML; configuration comes from the environment.

 **Why:** *This proves the "one image, config differs" model from Day 4 end to end. If your app needs a rebuild to run in staging, the problem is the application's configuration model, and it will resurface as a production incident later.*

7 Put a real approval gate in front of production

Create a `production` environment with required reviewers and a deployment branch policy limiting it to the trunk. Store production-only secrets on that environment. Add the production deploy job using the same digest output.

 **Why:** *The environment is doing two jobs: pausing for a human, and withholding production credentials from every job that has not passed the gate. That second job is the security control; the pause is merely the visible part.*

8 Break it deliberately, twice

Push a failing test and confirm the merge button is disabled. Then merge a change, deploy it, and roll back by re-deploying the previous digest. Time the rollback.

 **Why:** *An untested gate and an untested rollback are both assumptions. Knowing the rollback takes forty seconds changes what you are willing to ship on a Friday; discovering it during an incident changes your evening.*

Production tip

Add a `concurrency` group keyed on the workflow and the ref with `cancel-in-progress: true` for pull requests — but leave it off for deploy jobs. Cancelling a superseded test run saves money; cancelling a half-finished deploy leaves the cluster in a state no one designed. Concurrency control on the deploy path should serialise, never cancel.

6 Thinking Exercises

 Think it through

Your pipeline takes 45 minutes. Leadership wants Continuous Deployment to production. You believe the right first move is to make the pipeline take 8 minutes rather than to remove the approval step. Construct the argument in terms of risk and behaviour — not developer convenience — that persuades them.

 Think it through

A third-party action you use was compromised by rewriting its version tags, so pinning to `v4` would not have saved you. Pinning to a commit SHA would have. But now hundreds of pinned SHAs across your repositories need updating for genuine security patches. Design a workable policy, and be explicit about what you are trading away.

 Think it through

"Build once, promote the artifact" is nearly universal advice — yet many organisations still rebuild per environment. What legitimate constraint might make rebuilding feel necessary (think compile-time configuration, per-region assets, or licensing), and how would you restructure the application so the constraint disappears rather than being worked around?

7

Debugging Practice

 Debugging Lab • Staging is green, production is serving a different build

Symptom. A release is approved and deployed. Half the production pods behave like the new version, half like the previous one. The pipeline is green and shows one deploy.

kubectl get pods -o custom-columns and describe

```

NAME                                IMAGE                                NODE
cloudbase-api-7d4c9-2x8lq          ghcr.io/acme/cloudbase:latest      ip-10-0-1-41
cloudbase-api-7d4c9-9wq4t          ghcr.io/acme/cloudbase:latest      ip-10-0-2-17

Events:
  Normal  Pulled   6m   kubelet  Container image "ghcr.io/acme/cloudbase:latest"
              already present on machine
  Normal  Pulled   2m   kubelet  Successfully pulled image
              "ghcr.io/acme/cloudbase:latest" in 4.2s

```

- ✓ **Observe.** Both pods report the same image *string*, yet one node reused a cached copy and the other pulled fresh. Same name, different bytes.
- ✓ **Hypothesise.** The deployment references a mutable tag. `latest` was re-pushed, so the tag now resolves to a new digest — but nodes that already held an image under that name did not re-pull.
- ✓ **Investigate.** Compare digests: `kubectl get pod <name> -o json-path='{.status.containerStatuses[*].imageID}'` on each pod. Two distinct sha256 values confirm it.
- ✓ **Root cause.** The manifest pins a tag, not a digest, and `imagePullPolicy` is `IfNotPresent`, so image identity depends on each node's cache history.
- ✓ **Fix.** Redeploy referencing the digest the pipeline captured (`ghcr.io/acme/cloudbase@sha256:...`), which forces every node to converge on identical bytes.
- ✓ **Prevent.** Never deploy a mutable tag. Have the pipeline emit the digest and template it into the manifest; enable registry tag immutability so a published tag cannot be overwritten at all.

🔧 Debugging Lab • The deploy job cannot assume its cloud role after an OIDC migration

Symptom. Tests and build pass. The production deploy job fails immediately at the credential step. It worked from the staging job an hour earlier.

deploy job log

```

Run aws-actions/configure-aws-credentials@v4
Error: Could not assume role with OIDC: Not authorized to perform
sts:AssumeRoleWithWebIdentity

# and, earlier in the same run, from a job that was NOT migrated:
Error: Unable to get ACTIONS_ID_TOKEN_REQUEST_URL env variable

```

- ✓ **Observe.** Two distinct failures. One job cannot even request a token; the other requests one and is refused by the cloud.
- ✓ **Hypothesise.** The first job is missing `permissions: id-token: write`. The second gets a valid token whose `sub` claim does not match what the role's trust policy allows.
- ✓ **Investigate.** Print the token's claims (never the token itself) — the `sub` reads `repo:acme/cloudbase:environment:production`, while the trust policy was written for `repo:acme/cloudbase:ref:refs/heads/main` when only the staging path existed.

- ✓ **Root cause.** Adding the `environment:` key to the job changed the `sub` claim's shape. The claim is a function of the trigger context, not a constant per repository.
- ✓ **Fix.** Update the production role's trust policy condition to the environment-scoped subject, and add the missing `id-token: write` permission to the other job.
- ✓ **Prevent.** Document the exact `sub` value each role accepts next to the Terraform that creates it (Day 7), and treat "which claim shape does this deploy path produce?" as part of the review whenever a job's trigger or environment changes.

8 Mini-Labs

Mini-Lab • Beginner

40 min

Goal. Turn a repository with a test suite into one where a red suite blocks merge.

Business context. Broken code reached the trunk twice last month because reviewers approved without running tests.

Architecture. One repository, one workflow, one job, branch protection on the trunk.

Requirements. Trigger on pull request; check out; set up the toolchain with dependency caching; run the suite; register the job as a required status check.

Constraints. No secrets, no deploy, no image build — verification only.

Hints. Key the cache on a hash of the lockfile plus the runner OS. Open a PR with a deliberately failing test to prove the gate.

Expected outcome. A pull request with a failing test cannot be merged, and the failure is visible within a few minutes.

Production considerations. Record the pipeline's median duration now; you will optimise against it later.

Common pitfalls. Keying the cache on the manifest instead of the lockfile; forgetting that a required check must actually run on the PR to ever report.

Mini-Lab • Intermediate

70 min

Goal. Build one image on merge, tag it by commit SHA, scan it, push it, and capture its digest.

Business context. Nobody can currently answer "which commit is running in production?" without guessing.

Architecture. Test job → build/scan/push job (`needs:`) → registry.

Requirements. Restrict the build job to the trunk; fail the job on critical or high image findings; expose the pushed digest as a job output; declare minimal `permissions:` per job.

Constraints. Exactly one `docker build` may exist in the entire workflow. No `latest` tag anywhere.

Hints. Order the Dockerfile so dependency layers cache (Day 4). Read the digest back from the push rather than assuming it.

Expected outcome. A registry containing one image per merged commit, each traceable to a commit and a workflow run.

Production considerations. Add a retention policy — untagged and superseded images are a quiet, growing storage bill.

Common pitfalls. Scanning after the push; granting `packages: write` at workflow level instead

of on the one job that needs it.

Mini-Lab • Production-style

2 h

Goal. A complete promotion pipeline: automatic staging, gated production, keyless cloud auth, and a rehearsed rollback.

Business context. The team ships daily and needs an auditable production path that does not depend on who is on call.

Architecture. PR checks → build once → staging deploy (auto) → approval → production deploy, both deploys consuming the identical digest.

Requirements. OIDC-based cloud credentials with the production role scoped to the production environment; required reviewers and a branch policy on that environment; environment-scoped secrets; a documented one-command rollback.

Constraints. No long-lived cloud key may exist in the repository's secret store. All third-party actions pinned to commit SHAs.

Hints. Thread the digest through job outputs. Keep deploy logic in `scripts/deploy.sh` so it is testable locally.

Expected outcome. A merge reaches staging unattended; production waits for a named approver; rolling back is redeploying the previous digest, and you have timed it.

Production considerations. Write the runbook: how to identify the last known-good digest during an incident, in under a minute, without the pipeline.

Common pitfalls. Reusing one over-privileged role for both environments; letting the production job rebuild "just to be safe"; an approval gate with no context for the approver.

9 Engineering Notes

Infrastructure insight

Your pipeline is production infrastructure and deserves the same treatment: version-controlled definition, code review on changes, an owner, and monitoring. A team that would never accept an unreviewed change to a Terraform module routinely accepts unreviewed changes to the workflow that *applies* that module. Review the pipeline with the same seriousness as the thing it deploys.

Scalability note

Pipeline design changes shape with repository count. At five repositories, copy-pasted workflows are fine. At fifty, they are a fleet you cannot patch — when a pinned action needs an urgent security bump you must touch fifty files. The answer is reusable workflows or a shared pipeline library called by a thin per-repo stub, so a fix lands once. Plan this migration before you need it urgently; doing it during an active incident is how mistakes get shipped.

Disaster recovery tip

Ask the uncomfortable question: if your CI provider is unavailable for six hours, can you ship an emergency fix? For most teams the honest answer is no, and that is often an acceptable, deliberate risk. What is not acceptable is discovering it during an outage. Document the manual break-glass path — build locally, push to the registry, deploy with the same script the pipeline calls — and require that the same script is used, so the emergency path is a subset of the normal path rather than a parallel universe.

 Interview insight

Expect: "Why promote the same artifact instead of rebuilding per environment?" A weak answer says "it is faster." A strong answer names the failure: rebuilding produces bytes that were never tested, because base images, transitive dependencies and build-time downloads can all change between two builds of the same commit — so "it passed in staging" stops being evidence. Then land the consequence: immutable artifacts are also what make rollback a thirty-second redeploy of a known digest instead of a rebuild under pressure.

 Reliability principle

Deploy frequency and change failure rate are not in tension — small, frequent, automatically verified changes are easier to diagnose and revert than large, rare ones, because the diff between working and broken is small. A pipeline that makes shipping cheap and reverting instant is a reliability control, not merely a productivity tool. That reframing is what separates a build engineer from an SRE.

10 Build-Along Project — CloudBase

CloudBase already has a containerised API (Day 4) running on Kubernetes (Days 5–6), infrastructure defined in Terraform (Day 7) and hosts configured by Ansible (Day 8). Everything is reproducible — and every single piece of it is still applied by a human running a command. Today you remove the human from the path, and put one back deliberately, in exactly the right place.

 Build-Along • Day 09 — the delivery pipeline

- Add `.github/workflows/delivery.yml` with a `test` job on every pull request and push to trunk, using dependency caching keyed on the lockfile.
- Make that job a required status check on the trunk in branch protection (Day 2), and forbid direct pushes.
- Add a `build-push` job gated on `needs: test` and restricted to the trunk: build the image once, tag it with `github.sha`, scan it (failing on critical/high), push it, and expose the pushed digest as a job output.
- Replace any stored cloud access key with an OIDC trust relationship: a role per environment whose trust policy pins the repository and, for production, the environment claim. Declare `id-token: write` only on the jobs that need it.
- Pin every third-party action to a full commit SHA, and set workflow-level `permissions: contents: read`, widening only per job.
- Add `deploy-staging` (automatic) and `deploy-production` (behind a `production` environment with required reviewers and a trunk-only branch policy). Both call `scripts/deploy.sh` with the same digest.
- Move deploy logic out of YAML into `scripts/deploy.sh` so it runs identically from a laptop — this is also your break-glass path.
- Write `docs/rollback.md`: how to find the last known-good digest, the one command to redeploy it, and the measured time it took when you rehearsed it.

Definition of done: a pull request with a failing test cannot be merged; merging to trunk

produces exactly one image, tagged by commit SHA and scanned before publication, which deploys itself to staging and then waits for a named approver before the identical digest reaches production; no long-lived cloud credential exists anywhere in the repository; and you have rolled production back to the previous digest at least once and recorded how long it took.

11 Chapter Summary

Key takeaways

- ✓ A pipeline is a policy engine. Gates only exist when the forge refuses the merge or the deploy — otherwise they are suggestions.
- ✓ Order stages cheapest-first; total duration determines whether engineers respect the pipeline or route around it.
- ✓ Build once. A tag is a mutable pointer; the digest is the artifact's true, immutable identity.
- ✓ Rebuilding per environment ships bytes that were never tested — base images, transitive deps and downloads all drift between builds.
- ✓ Prefer short-lived OIDC identities to stored keys; scope the trust policy by repository *and* environment.
- ✓ Pin third-party actions to commit SHAs — tags can be rewritten, as the March 2025 tj-actions compromise demonstrated.
- ✓ Log masking is string matching, not a security boundary. Assume anything a job can read, a compromised job can exfiltrate.
- ✓ Rollback is redeploying a known digest, not rebuilding from source during an incident.

Things to remember

- Each `run:` step is a separate process — state crosses steps only via the filesystem or the environment files.
- An `uses:` step is a git repository executed with your job's privileges. Treat it as a dependency, because it is one.
- A deployment environment gates *and* withholds credentials; the pause is the visible half, the secret scoping is the control.
- Cache keys must hash the lockfile, never the manifest, or the cache serves stale dependencies indefinitely.

Common pitfalls (last look)

- Deploying `latest` • rebuilding per environment • scanning after the push
- Echoing a secret to debug • one over-privileged role for all environments • no rehearsed rollback

📄 One-Page Cheat Sheet

CI / CD / CD — integrate and verify continuously → always releasable → release automatically.
Gate — a job whose failure blocks dependents, made binding by branch protection or an environment rule.

Runner — ephemeral machine that long-polls for a job; each step is a separate process; nothing survives the job.

Digest — SHA-256 of the image manifest — the immutable identity. A tag is only a movable pointer to it.

Promotion — deploy the same digest to the next environment; only injected configuration differs.

OIDC — job requests a signed JWT; the cloud validates `sub/aud` and returns job-lifetime credentials.

Cache vs artifact — cache = untrusted speed optimisation keyed on a lockfile; artifact/registry = what you ship.

Rollback — redeploy the previous known-good digest — written down, and timed, before the incident.

CLI quick reference

Day 9 commands worth memorising

```
# what exactly is running right now?
kubectll get pods -o jsonpath='{..imageID}' | tr ' ' '\n' | sort -u
docker inspect --format='{{index .RepoDigests 0}}' ghcr.io/acme/app:$SHA

# resolve a tag to its immutable digest before deploying it
docker buildx imagetools inspect ghcr.io/acme/app:1.4.2

# scan as a gate: non-zero exit fails the job
trivy image --exit-code 1 --severity CRITICAL,HIGH ghcr.io/acme/app:$SHA

# inspect and rerun pipeline runs from the terminal
gh run list --branch main --limit 10
gh run view <run-id> --log-failed           # only the failing steps
gh run rerun <run-id> --failed              # retry just what failed

# roll back: redeploy the previous digest, do not rebuild
kubectll set image deploy/cloudbase-api api=ghcr.io/acme/app@sha256:<prev>
kubectll rollout status deploy/cloudbase-api --timeout=120s
```

👤 Interview questions to master

- › What is the difference between Continuous Delivery and Continuous Deployment, and what must be true before you enable the latter?
- › Why should the same built artifact be promoted across environments instead of rebuilt for each one?
- › How do you manage secrets safely across pipeline stages, and why is log masking not a security control?
- › What would you make a mandatory gate before a production deploy, and how do you stop it being bypassed?

- › How would you roll back a bad deploy quickly and safely?
- › Why tag images by commit SHA rather than `latest` — and why is a digest stronger still?
- › Explain OIDC-based cloud authentication from CI. What claim does the cloud check, and why does that matter?
- › When would you still choose Jenkins over a hosted CI system?

Further reading & official docs

- GitHub Docs — *OpenID Connect*: the token flow, the `sub` and `aud` claims, and the `id-token: write` permission (docs.github.com, Actions security concepts).
- GitHub Docs — *Managing environments for deployment*: required reviewers, wait timers and deployment branch policies (docs.github.com, Actions deployment how-tos).
- CISA alert, 18 March 2025 — supply-chain compromise of `tj-actions/changed-files` (CVE-2025-30066) and `reviewdog/action-setup` (CVE-2025-30154).
- SLSA v1.0 specification (slsa.dev) — build levels, provenance requirements, and what non-falsifiable provenance means.
- Shopify Engineering — *Introducing Shipit and Software Release Culture at Shopify*: a real deploy control plane and the PR → CI → canary → production gate chain.
- OCI Image Specification — the manifest and descriptor format that makes a digest an artifact's identity.

Production checklist

- ☐ Tests and security scans run automatically and block merge on failure, enforced by branch protection.
- ☐ Exactly one image build exists per commit; nothing is rebuilt for a later environment.
- ☐ Deployments reference an immutable digest, never a mutable tag.
- ☐ No long-lived cloud credential is stored in the CI secret store; OIDC is used where available.
- ☐ Third-party actions are pinned to commit SHAs and job `permissions:` are declared explicitly.
- ☐ Production deploys require a named approver and are limited to the trunk by branch policy.
- ☐ A rollback command is documented, and has been executed at least once outside an incident.
- ☐ Pipeline duration is measured, has a target, and is treated as a defect when it regresses.

Key idea

What you will learn next (Day 10). Your pipeline now produces a trustworthy artifact and can place it into an environment on demand. Tomorrow you look hard at the environment itself: AWS in production shape — VPCs, subnets and routing, load balancers, managed data stores, and the networking decisions that determine whether the thing this pipeline deploys is actually reachable, resilient and affordable.

DAY 10 • CLOUD

Cloud Infrastructure on AWS

VPC, IAM, compute, storage, load balancing and managed databases

10

 Estimated study time: 9 hours By the end of this day, you will be able to...

- Design a VPC whose public/private separation is enforced by *routing*, not by hope.
- Explain how AWS decides to allow or deny a request — explicit deny, implicit deny, and the order in between.
- Replace long-lived access keys with roles, and say precisely where the temporary credentials come from.
- Choose between an ALB and an NLB, and describe what a target group's health check actually changes.
- Distinguish Multi-AZ from a read replica, and read an AWS bill as an architecture review.

1

The Big Picture

What problem this solves. For nine days you have been building things that need somewhere to run. Day 3 gave you CIDR blocks and routing as theory. Day 7 gave you Terraform, which declares resources — but resources *in what*? Day 9 gave you a pipeline that builds an image and then has nowhere to push it. Today all of that lands on real infrastructure: a network you own, identities that can act inside it, machines that run your containers, a load balancer that is the only door in, storage that outlives every machine, and a database somebody else patches at 3am.

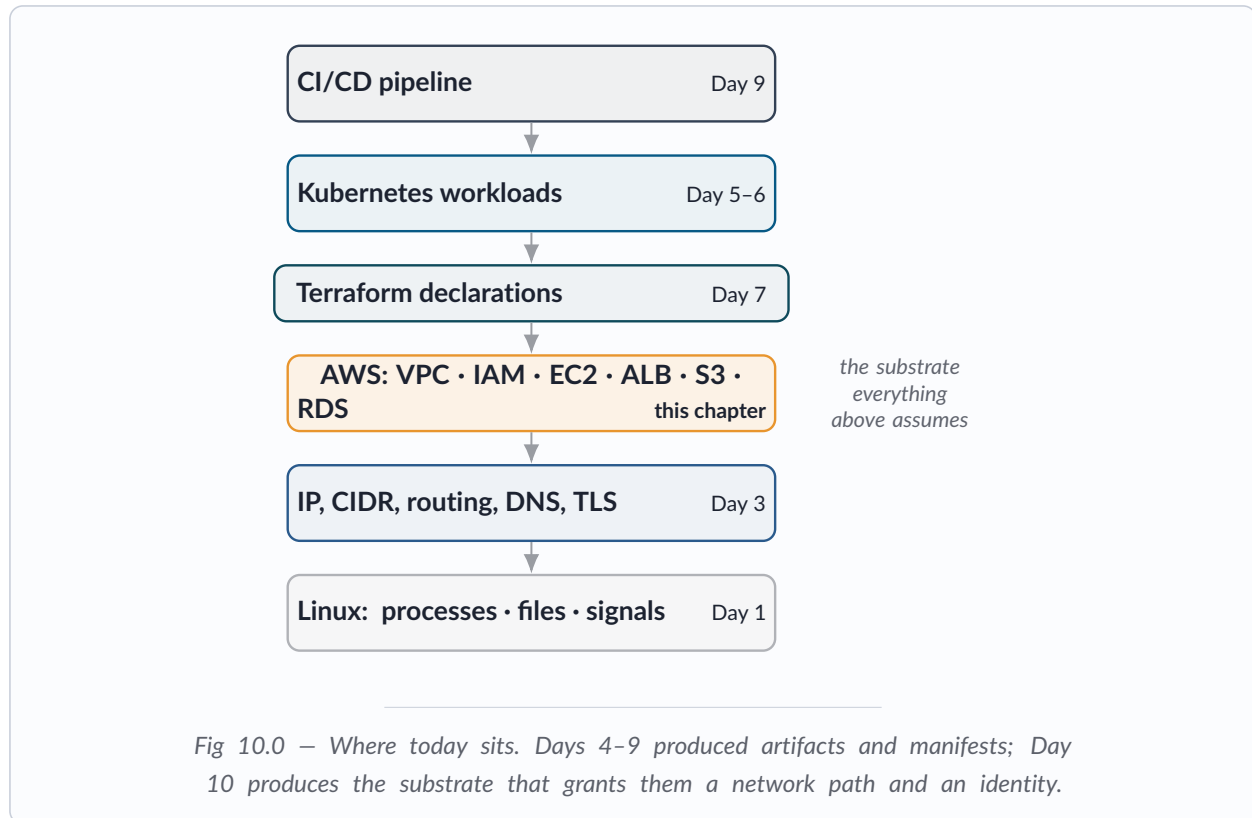
Why DevOps engineers need it. A cloud provider is not a hosting company with an API. It is a set of *primitives with enforcement semantics*, and almost every production incident on AWS comes from misunderstanding one of them: a route table that does not have the route you assumed, a security group that references a CIDR instead of another security group, an IAM policy that is allowed by one document and denied by another, a database in a subnet that quietly has a path to the internet. None of those are visible in the console at a glance. All of them are obvious once you know the mechanism.

 Key idea

On AWS there are only three questions that ever matter, and everything in this chapter is one of them. **Is there a network path?** (route tables, gateways, security groups). **Is the caller allowed?** (IAM evaluation). **Who pays for it, and how much?** (data transfer, idle resources, per-hour charges). An engineer who can answer those three about any architecture can debug and design on any cloud, because the other providers use the same shapes with different nouns.

Where it fits in a modern cloud architecture. Everything you built earlier resolves downward into this chapter's primitives. Read the diagram as a stack of *claims*: the layer above only works because

the layer below granted it a path and an identity.



How it connects to previous chapters. Day 3's `/16` and `/24` become a VPC CIDR and subnet CIDRs. Day 3's default gateway becomes a route table entry pointing at an internet gateway. Day 1's "grant to an identity, not to everyone" becomes an IAM policy. Day 4's image registry becomes ECR, backed by S3. Day 7's remote state becomes an S3 bucket with a lock. Day 6's Service of type `LoadBalancer` becomes an actual ALB with actual target groups. You are not learning a new mental model today; you are learning the AWS spelling of one you already have.

Where companies use it in production. Netflix runs its streaming control plane across multiple AWS regions in an active-active shape, and publishes engineering work on the network-visibility problem this creates: their TechBlog describes ingesting VPC Flow Logs from S3 at very high volume and *enriching* each flow with workload metadata, because raw flow logs contain IP addresses and IP addresses in an autoscaled cloud mean nothing an hour later. That single detail tells you more about operating a large VPC than any architecture poster: at scale, the hard problem is not building the network, it is knowing what is talking to what inside it. Smaller organisations hit the same wall in miniature the first time a security group audit asks "who actually uses this rule?"

Common misconceptions. That a "private subnet" is a checkbox (it is the *absence* of a route to an internet gateway). That security groups are firewalls with allow *and* deny rules (they only allow, and they are stateful). That an IAM role is a kind of user (it is a set of permissions with a trust policy, assumed to produce temporary credentials). That Multi-AZ is a scaling feature (it is availability; it serves no reads). That cloud cost is finance's problem (roughly half of a surprise AWS bill is architecture, and most of the rest is things nobody deleted).

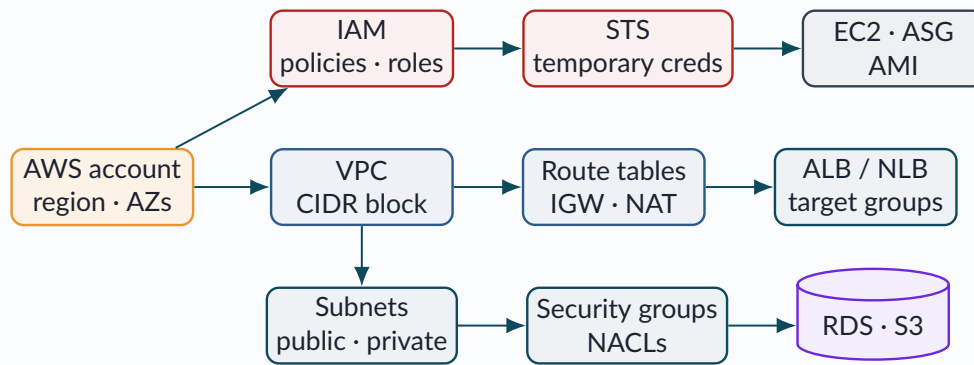


Fig 10.1 – The Day 10 territory. The account is the blast-radius boundary; the VPC is the network boundary; IAM is the identity boundary; everything else is a resource placed inside those three.

Terminology to anchor: **region** → **availability zone** → **VPC** → **subnet** → **route table** → **internet gateway** → **NAT gateway** → **security group** → **IAM role** → **STS** → **target group** → **Multi-AZ**.

3 Guided Learning

3.1 The VPC – A Network You Own, Defined in Software

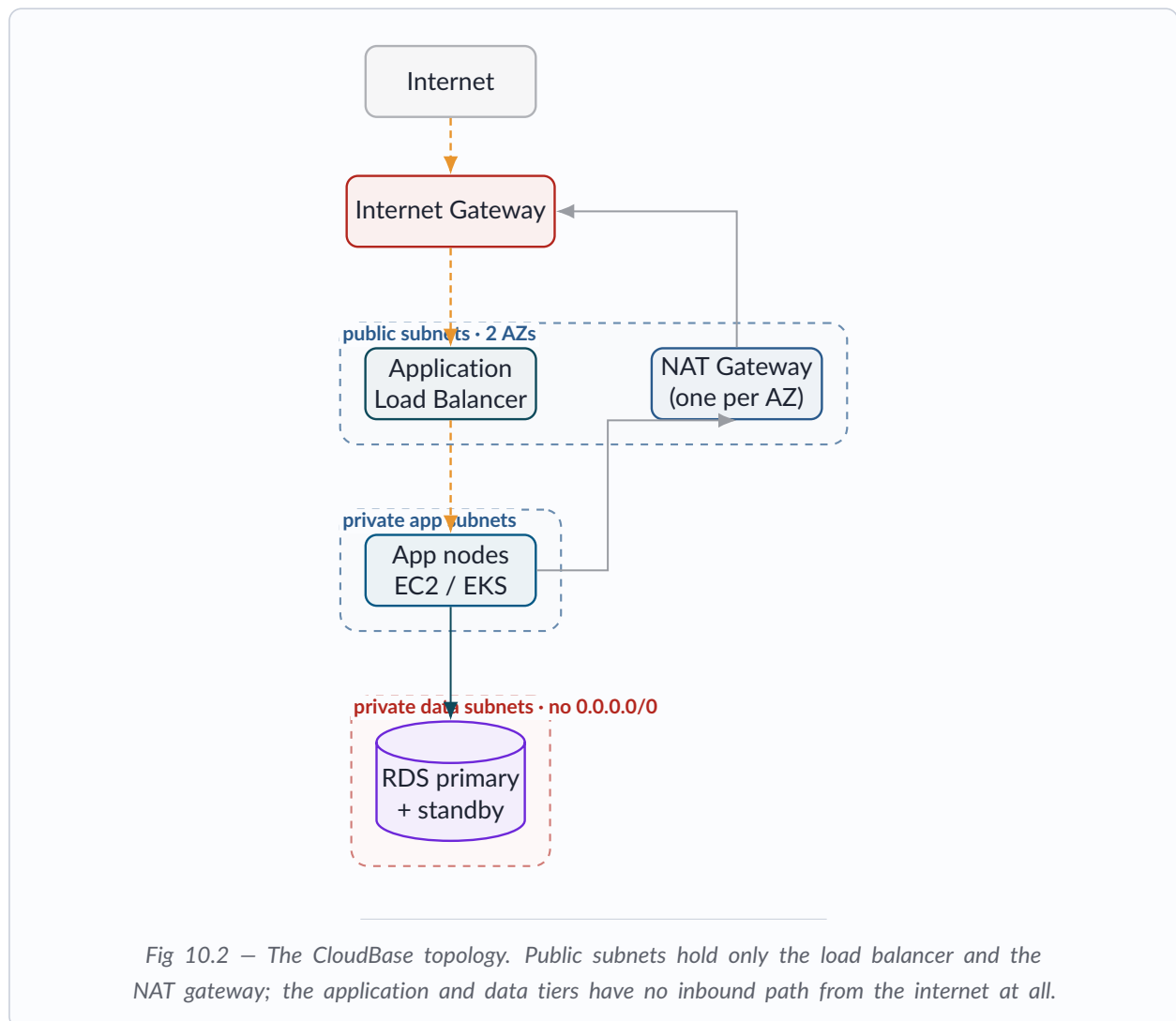
What is it? A **VPC** (Virtual Private Cloud) is a logically isolated IP network inside an AWS region. You choose its CIDR block – say `10.0.0.0/16` – and carve it into subnets, each of which lives in exactly one **availability zone**. An availability zone is one or more physically separate data centres with independent power and cooling, connected to its siblings by low-latency links.

Why does it exist? The original EC2 (retrospectively "EC2-Classic") put every customer's instances on a shared flat network. Isolation depended entirely on per-instance firewall rules, you could not choose your own address space, and you could not express "this machine has no route to the internet at all." Enterprises could not map that onto their existing network designs or their auditors' expectations. The VPC exists so that the network topology becomes a thing you *declare* – and so that "unreachable" is a structural property rather than a rule somebody might delete.

How does it work internally? There is no physical switch anywhere that belongs to you. AWS implements the VPC as a distributed **mapping service**: every packet leaving an instance's virtual NIC is encapsulated by the hypervisor (or, on Nitro instances, by a dedicated hardware card), the destination is looked up in a per-VPC mapping table, and the packet is tunnelled across the physical network to the host holding the destination. Route tables, security groups and NACLs are inputs to that lookup, evaluated in the data plane – which is why a security group change takes effect essentially immediately and why there is no "firewall appliance" to become a bottleneck.

The consequence that matters daily: **a subnet is not "public" or "private" by any attribute of its own**. It is public if and only if its associated route table contains a route to an **internet gateway**. Delete that route and the same subnet is private. Add it to your database subnet by accident and your database is exposed to the extent its security group allows.

Route table entry	What it means, operationally
10.0.0.0/16 → local	Always present, cannot be removed. Every subnet can reach every other subnet in the VPC by default.
0.0.0.0/0 → igw-...	This line is the definition of a public subnet. Two-way reachability, subject to security groups.
0.0.0.0/0 → nat-...	Outbound only. Return traffic for connections you started; nothing may initiate inbound.
0.0.0.0/0 absent	No internet path in either direction. The correct state for a database subnet.
s3 prefix list → vpce-...	A gateway VPC endpoint: S3 traffic leaves via AWS's network, not via NAT.



When should I use it? Always — there is no non-VPC option any more. The real decision is how many VPCs: one per environment (dev, staging, prod) is the common baseline, because it makes "staging cannot reach prod" a property of the network rather than a promise.

When should I not? Do not build a fifteen-subnet, four-tier, transit-gateway-connected topology for a service with one database and two containers. Every extra AZ multiplies NAT gateway cost; every extra VPC adds peering or transit-gateway complexity and a new set of route tables somebody must keep correct. Start with two AZs, three subnet tiers, and grow when a specific requirement forces it.

Infrastructure insight

Choose your VPC CIDR as if you will one day peer it with another network — because you will. Overlapping CIDRs cannot be peered, ever, and re-addressing a live VPC is a migration, not a change. Reserve a distinct /16 per environment per region from a written plan (for example `10.10.0.0/16` prod-eu, `10.20.0.0/16` staging-eu) before you create the first one. This is a ten-minute decision that either costs nothing or costs a quarter.

3.2 Security Groups, NACLs & the Reachability Chain

What is it? A **security group** is a stateful allow-list attached to a network interface. A **network ACL** is a stateless, ordered allow/deny list attached to a subnet. Both must permit a packet for it to arrive.

Why does it exist? Two filters exist because they answer two different questions. The NACL answers "what may cross this subnet boundary at all?" — a coarse, subnet-wide, deny-capable control that a network team can own. The security group answers "what may talk to *this workload*?" — fine-grained, identity-shaped, and owned by the team that runs the service. Before security groups, the equivalent was maintaining iptables rules per host with a configuration-management tool and hoping they converged.

How does it work internally? A security group has **no deny rules**. Its evaluation is a union of every rule across every group attached to the interface: if any rule matches, the packet is allowed; otherwise it is dropped. It is **stateful** — the data plane tracks connections, so if inbound TCP/443 is allowed, the response leaves without an outbound rule. A NACL is **stateless** and **numbered**: rules are evaluated in ascending order, the first match wins, and because it is stateless you must allow the ephemeral return ports (roughly 1024–65535) explicitly in the opposite direction. Forgetting that is the single most common NACL bug.

The most powerful and most under-used feature: a security group rule's source can be *another security group* rather than a CIDR. "Allow TCP/5432 from `sg-app`" means "from whatever instances currently carry the app security group", which stays correct through every autoscaling event, AMI replacement and IP change. A CIDR-based rule encodes a fact about addresses; a group-based rule encodes a fact about roles.


EC2 Console — CloudBase database security group
illustration

Group ID `sg-0db1cloudbase`

Inbound `tcp/5432 from sg-0app1cloudbase (NOT 10.0.0.0/16)`

Inbound - no other rules -

Outbound none required (stateful return traffic)

Property	Security group	Network ACL
Attached to	Network interface	Subnet
State	Stateful	Stateless
Rules	Allow only	Allow and deny, numbered
Evaluation	Union of all rules	First match wins
Best used for	Workload-to-workload access	Coarse subnet-wide guardrails

⚠ Common misconception

"I opened the security group and it still does not connect." A security group is only one link in the chain. A packet reaches your process only if *all* of these hold: a route exists in the source subnet's route table; the source security group allows egress; the source NACL allows out; the destination NACL allows in *and* allows the return; the destination security group allows in; the host firewall (Day 1) allows it; and

the process is actually listening on that interface, not on `127.0.0.1`. Debug the chain in order rather than repeatedly widening the group.

3.3 IAM – How AWS Actually Decides to Allow or Deny

What is it? IAM is the authorisation engine in front of every AWS API call. Every request carries a request context – principal, action, resource, and condition keys such as source IP or MFA state – and IAM evaluates the applicable policy documents against it to return exactly one answer: allow or deny.

Why does it exist? Early cloud access control was a shared root credential, which makes "who did this?" unanswerable and "revoke one person" impossible. IAM exists to make permission a first-class, auditable, machine-readable object – and, critically, to let a *machine* have an identity, so that credentials never need to be written down anywhere.

How does it work internally? This is the part beginners memorise as a slogan and seniors can actually walk through. AWS gathers every policy that applies – identity-based policies on the principal, resource-based policies on the target, permissions boundaries, session policies, and organisation-level service control policies (SCPs) and resource control policies (RCPs) – and evaluates them with these rules:

- **Deny by default.** If nothing explicitly allows the request, it is denied. This is the *implicit deny* and it is the resting state of every permission in AWS.
- **An explicit deny in any applicable policy wins, always.** It cannot be overridden by an Allow anywhere else. This is what makes guardrails possible.
- Identity-based and resource-based policies *union*: an allow in either is enough within one account.
- Permissions boundaries, SCPs and RCPs *intersect*: they can only ever reduce what an identity policy grants. An SCP that allows something grants nothing by itself.

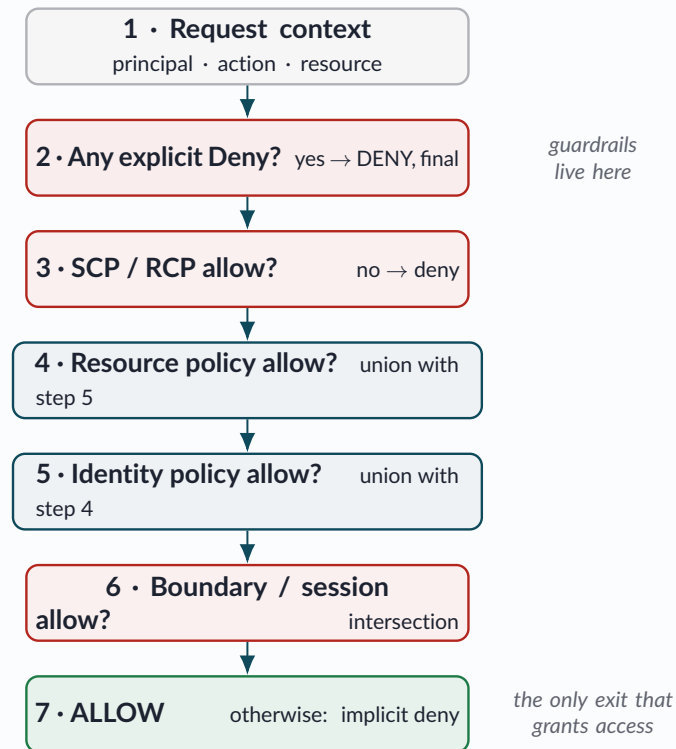


Fig 10.3 – IAM evaluation as a decision chain. Only the bottom-left path ends in Allow; everything else falls through to a deny.

Roles, and where the credentials come from. An **IAM role** is not a user. It is a permission set plus a *trust policy* saying which principals may assume it. Assuming it calls `sts:AssumeRole`, and **STS** returns a short-lived triple: access key ID, secret, and a session token that expires. On EC2 the instance profile makes this invisible – the AWS SDK fetches credentials from the **instance metadata service** at `169.254.169.254` and refreshes them before expiry, so nothing is ever stored on disk. In EKS the equivalents are IAM Roles for Service Accounts (a projected OIDC token the pod exchanges with STS) and the newer EKS Pod Identity, where a node-local agent supplies credentials and the association lives in EKS rather than in service-account annotations.

`cloudbase-uploads-policy.json`

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadWriteOwnPrefixOnly",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::cloudbase-uploads-prod/tenant/*"
    },
    {
      "Sid": "ListOnlyThatBucket",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::cloudbase-uploads-prod",
      "Condition": { "StringLike": { "s3:prefix": "tenant/*" } }
    }
  ]
}
```

```

{
  "Sid": "NeverOutsideTLS",
  "Effect": "Deny",
  "Action": "s3:*",
  "Resource": "arn:aws:s3:::cloudbase-uploads-prod/*",
  "Condition": { "Bool": { "aws:SecureTransport": "false" } }
}
]
}

```

Read that document as three separate decisions. Object actions are scoped to a prefix, not the bucket. `ListBucket` is a *bucket-level* action, so its resource is the bucket ARN without `/*` — getting this wrong is why "I granted `s3:GetObject` and listing still fails" is such a common ticket. And the explicit `Deny` is a guardrail: no future Allow, from any policy, can permit plaintext access.

Security warning • why a static access key is a different class of risk

A long-lived key pair does not expire, works from any machine on earth, appears in `git` history forever once committed, and is harvested by scanners crawling public repositories continuously. A role-derived credential expires in hours, is bound to a session, is never written to disk, and is revocable by editing one policy. The engineering point is not "keys are bad" but that a role removes the *secret* entirely: there is nothing to leak because nothing was ever stored. If you must issue a static key — a legacy on-prem job, say — treat it as an incident waiting to happen: scope it to one action, add a condition on source IP, and put an expiry date in the calendar.

Behind the scenes

The metadata service is why a compromised web application can sometimes escalate to full AWS access: a server-side request forgery that makes your app fetch `169.254.169.254` returns the instance role's live credentials. IMDSv2 closes this by requiring a session token obtained via a `PUT` with a hop limit that stops the response crossing a proxy or a container boundary — and since 2024 AWS lets you make IMDSv2 the account-wide default for new launches, with newer instance types IMDSv2-only. Turn it on, then set the hop limit to 1 for anything not running containers.

3.4 Compute & the Load-Balancing Tier

What is it? `EC2` rents you a virtual machine by the second, booted from an **AMI** (a machine image). An **Auto Scaling Group** maintains a desired count of healthy instances from a launch template, replacing any that fail. An **Application Load Balancer** terminates HTTP and TLS, applies routing rules, and forwards to a **target group** of registered targets.

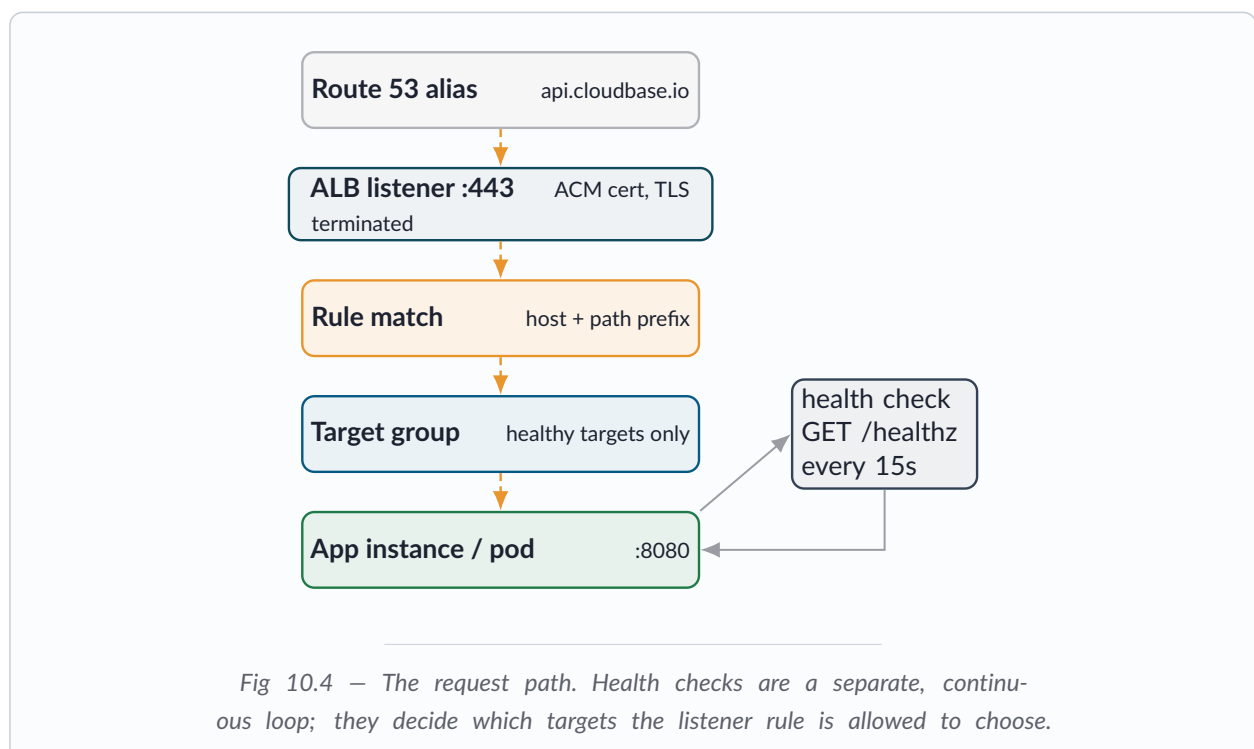
Why does it exist? Before autoscaling groups, capacity was a spreadsheet and a purchase order, and a failed machine was a pager at 3am followed by manual reprovisioning. The ASG turns "we have five healthy web servers" from an observation into a *declared state* the platform enforces — exactly the same shift Kubernetes makes with a `ReplicaSet`, one layer down. Before managed load balancers, teams ran HAProxy or nginx pairs with keepalived and a floating IP, and the load balancer itself was the most fragile component in the system.

How does it work internally? An ALB is not a machine; it is a fleet of nodes, one or more per enabled subnet, fronted by a DNS name that resolves to changing IP addresses — which is why you point a Route 53 alias record at it and never hardcode an IP. Traffic arriving at a listener is matched against ordered rules (host header, path, HTTP method, header, source IP) and forwarded to a target group. The target group runs an independent health check against each target on a configured path and interval; a target must pass a threshold to be considered healthy, and unhealthy targets stop receiving

new requests without being terminated. Because the ALB terminates the connection, it can also do sticky sessions, request tracing headers, and per-rule authentication — all things a Layer 4 balancer structurally cannot.

An NLB operates at Layer 4. It does not parse HTTP, preserves the client source IP by default, supports static IPs and Elastic IPs per AZ, and handles protocols an ALB cannot (arbitrary TCP, UDP, TLS passthrough). Its lower latency comes from doing less.

Need	Choose	Because
Path or host routing	ALB	It parses HTTP; an NLB never sees a URL.
Terminate TLS with ACM	Either	Both integrate with ACM; the ALB can also re-encrypt to targets.
Static IP for a firewall allow-list	NLB	An ALB's addresses change.
Raw TCP, UDP, or mTLS passthrough	NLB	The ALB only speaks HTTP/HTTPS/gRPC.
Extreme connection rates, low latency	NLB	Less per-packet work.
WebSockets, gRPC, redirect rules	ALB	Protocol-aware features.



Reliability principle

Your health check endpoint must reflect the thing the load balancer is deciding: *can this instance serve a request right now?* A `/healthz` that returns 200 unconditionally hides a broken instance and keeps sending it traffic. A `/healthz` that queries the database on every check turns a database blip into all targets going unhealthy simultaneously — an outage manufactured by the health check itself. The usual

resolution is two endpoints: a shallow liveness check for the balancer, and a deeper readiness check that gates startup and deployment. This is the same distinction Kubernetes draws on Day 6, for the same reason.

Design trade-off

Scaling on CPU is the default and is usually wrong for a request-serving service. CPU is a lagging, indirect signal — an I/O-bound API can be saturated at 30% CPU. Target-tracking on *requests per target* or on p99 latency reacts to the thing users experience. The trade-off is noise: a request-based metric is spikier, so pair it with a cooldown and a sensible minimum capacity, or you will pay for a fleet that oscillates.

3.5 S3 — Object Storage as the Platform's Substrate

What is it? S3 stores immutable **objects** in flat **buckets**, addressed by key. There is no filesystem, no in-place edit, no rename: writing to an existing key replaces the whole object. Keys containing slashes look like directories in the console, but the prefix is just part of the name.

Why does it exist? Shared filesystems do not scale across data centres, and POSIX semantics — in-place partial writes, locking, atomic rename, directory metadata — are exactly what makes them hard to distribute. S3 dropped all of it. Because objects are immutable and namespaced by key, S3 can replicate and shard aggressively, which is why it can be effectively unbounded in capacity and durability while a NAS cannot.

How does it work internally? The bucket namespace is partitioned by key prefix; S3 splits and rebalances partitions automatically as request rates grow, which is why highly sequential key prefixes (a timestamp at the front of every key) can hot-spot a partition while high-entropy prefixes spread naturally. Durability comes from erasure-coding each object's shards across multiple facilities within the region. Since 2020 all GET/PUT/LIST operations are strongly read-after-write consistent — older tutorials warning about eventual consistency describe a system that no longer exists.

When should I use it? Build artifacts, container-adjacent blobs, Terraform state, backups, logs, static assets, data-lake storage, and anything a user uploads. In CloudBase, S3 is the Day 7 state backend and the Day 9 artifact store; both were already S3 without you looking at it closely.

When should I not? As a database, or as a filesystem for a running application. Objects have no partial update and no locking; "read, modify, write the whole object" from two processes silently loses one of them. If you need POSIX semantics, use EFS and accept its cost; if you need transactions, use RDS.

Security warning • the default changed, and you should still be explicit

Publicly readable buckets remain one of the most common causes of real-world data exposure. Since April 2023 AWS enables S3 Block Public Access and disables ACLs on all *newly created* buckets, which removes the classic accidental-`public-read` footgun. That is a floor, not a strategy: existing buckets were not changed, and any engineer with the right permission can still turn it off. Set Block Public Access at the *account* level, enforce it with an SCP, and serve genuinely public content through CloudFront with an origin access control rather than by opening the bucket.

Cloud cost optimization

Storage class and lifecycle are the whole cost story for S3. A bucket collecting build artifacts, CI logs and old backups grows monotonically and nobody ever deletes anything, because deleting feels risky and costs nothing to postpone. Write the lifecycle rule when you create the bucket: transition to Infrequent

Access or Glacier at a defined age, expire non-current versions, and abort incomplete multipart uploads after seven days. That last one is the invisible leak — failed uploads leave parts you are charged for and cannot see in the object listing.

storage.tf – artifacts bucket with lifecycle and lock-down

```
resource "aws_s3_bucket" "artifacts" {
  bucket = "cloudbase-artifacts-prod"
}

resource "aws_s3_bucket_public_access_block" "artifacts" {
  bucket                = aws_s3_bucket.artifacts.id
  block_public_acls      = true
  block_public_policy    = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_versioning" "artifacts" {
  bucket = aws_s3_bucket.artifacts.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_lifecycle_configuration" "artifacts" {
  bucket = aws_s3_bucket.artifacts.id

  rule {
    id      = "expire-old-artifacts"
    status  = "Enabled"
    filter { prefix = "builds/" }
    noncurrent_version_expiration { noncurrent_days = 30 }
    abort_incomplete_multipart_upload { days_after_initiation = 7 }
  }
}
```

3.6 RDS – Managed Databases, Multi-AZ vs Read Replicas

What is it? RDS runs a standard engine (PostgreSQL, MySQL, and others) on instances AWS operates: it handles patching, automated backups with point-in-time recovery, parameter groups, and failover. You get a hostname and a port; you do not get the operating system.

Why does it exist? Running a production relational database means backups you have actually tested, replication that is monitored, failover that is rehearsed, and minor-version upgrades that do not lose data. Teams that ran their own Postgres spent a meaningful fraction of an engineer on those four things forever. RDS trades some control for the removal of that standing cost.

How does it work internally? The two features people conflate work completely differently.

Multi-AZ provisions a standby in a second availability zone kept in sync *synchronously* at the storage layer. The standby serves no traffic — it accepts no connections and answers no reads. On failure, AWS repoints the database's DNS record at the standby; your application reconnects to the same hostname and continues. It buys *availability*, and it costs roughly double, and it slightly increases write latency because a commit must be acknowledged in two AZs.

A **read replica** is an *asynchronous* copy that does serve reads. It has its own endpoint, so the application must deliberately route read-only queries to it, and it lags the primary by an amount that varies with write load. It buys *read capacity*. It is not failover: promoting a replica is a manual (or scripted)

operation, and any transactions not yet replicated at the moment of failure are lost.

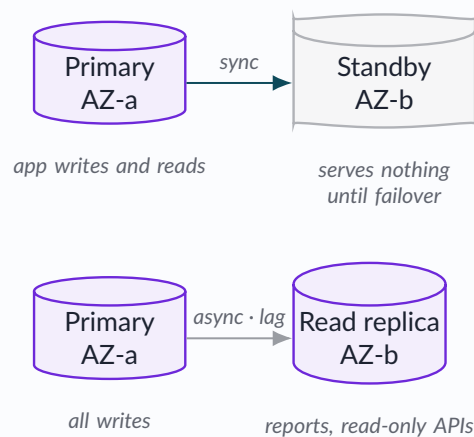


Fig 10.5 – Two different problems. The standby answers nothing and exists for failover; the replica answers reads and lags.

🚨 Disaster recovery tip

Multi-AZ is not a backup. It replicates faithfully — including the `DELETE` that a bad migration just ran, and including corruption. Availability protects you from infrastructure failure; backups protect you from *you*. Keep automated backups with a retention window that matches how long a logical error can go unnoticed (often longer than you think), enable deletion protection, and restore a snapshot into a scratch instance on a schedule. A backup you have never restored is a hypothesis.

📈 Scalability note

Read replicas move read load, not write load, and they do not remove the primary's bottleneck — they add to it, because replication itself is work. When writes are the constraint the answers are different in kind: connection pooling in front of the database (RDS Proxy or PgBouncer), caching, batching, then partitioning or sharding. Reaching for a read replica when the primary is write-saturated makes the problem slightly worse and buys nothing.

3.7 Cost as an Engineering Concern

Why does it exist as a topic? On-premises, capacity was capital expenditure decided once a year; a bad architecture wasted hardware you had already bought. In the cloud every architectural decision has a recurring price, charged hourly, forever, whether or not anyone uses the resource. Cost is therefore a property of the design — and reviewing it is an engineering activity, not an accounting one.

How does it work internally? Almost all surprising AWS bills come from three mechanisms rather than from compute:

- **Per-hour charges for provisioned things.** A NAT gateway is billed for every hour it exists plus a per-GB data-processing charge, and production guidance is one per AZ for availability — so a three-AZ VPC pays three hourly charges before a single byte moves. Load balancers, Elastic IPs and provisioned IOPS bill the same way.
- **Data transfer, especially across boundaries.** Traffic between availability zones is charged in both directions; traffic out to the internet is charged again. A chatty service placed in one AZ talking to a database in another pays for every round trip, permanently, invisibly.

- **Orphans.** Unattached EBS volumes after an instance is terminated, snapshots of deleted volumes, idle load balancers left behind by a deleted environment, old AMIs and their backing snapshots. Each is small; together they are the steady-state waste in most accounts.

Cloud cost optimization • the NAT gateway is usually the first thing worth fixing

If your private subnets pull container images, packages and S3 objects through a NAT gateway, you are paying a per-GB processing fee for traffic that never needed to leave AWS. A *gateway VPC endpoint* for S3 and DynamoDB costs nothing and routes that traffic off the NAT path entirely — one route-table entry. Interface endpoints for ECR, CloudWatch Logs and Secrets Manager do carry an hourly charge, so compare them against the NAT data volume they displace rather than adding them reflexively. Measure first: VPC Flow Logs will tell you exactly what is crossing the NAT gateway.

Senior engineer insight

Tag everything at creation, in Terraform, with `owner`, `environment` and `service` — and make untagged resources fail the pipeline. The reason is not the bill; it is that six months from now somebody will find a load balancer costing money and nobody will know whether deleting it takes down a customer. Tags are how an infrastructure estate stays legible. A cost report grouped by service tag is one of the most honest architecture reviews you will ever read: it shows what you actually built, weighted by what it actually costs.

Best practices — what experienced engineers do

- Only load balancers and NAT gateways live in public subnets. Application and data tiers have no `0.0.0.0/0` route to an internet gateway.
- Security group rules reference other security groups, not CIDR ranges, wherever both sides are yours.
- Compute gets an IAM role; static access keys are reserved for things that genuinely cannot assume one, and are scoped and conditioned when they are.
- Scope policies to specific actions and specific ARNs, and add explicit `Deny` guardrails for the things that must never happen.
- Block public access on S3 at the account level, enforce it with an SCP, and write the lifecycle rule when you create the bucket.
- Spread across at least two AZs, and check that both the ASG and the load balancer are actually enabled in both.
- Tag every resource at creation with `owner`, `environment` and `service`, from code.

Common mistakes beginners make

- Putting the database in a public subnet "so it is easier to connect" — then adding `0.0.0.0/0` on port 5432 to make it work.
- Granting `s3:*` on `*` because a scoped policy returned `AccessDenied` once.
- Hardcoding an access key in a container image, a Terraform variable file, or CI — and assuming a private repository is a control.
- Assuming a read replica will fail over automatically the way a Multi-AZ standby does.
- Health checks that always return 200, or that call the database and take the whole fleet down together.
- Running one NAT gateway shared across AZs to save money, creating a single point of failure and cross-AZ transfer charges.

- Never deleting anything, then being surprised by the bill and blaming "the cloud".

4 Visualizations to Internalize

Redraw these from memory. If you can draw the reachability chain and the IAM decision chain on a whiteboard, you can debug most AWS incidents without opening the console.

 Redraw: the CloudBase VPC – two AZs, three subnet tiers, IGW, NAT, ALB, RDS. Label which route table entry makes each subnet public or private

 Redraw: the IAM evaluation chain from request context to Allow, marking where an explicit Deny short-circuits and where policies intersect rather than union

5 Guided Hands-On Lab – Stand Up the CloudBase AWS Environment

Goal. Build – in Terraform, extending Day 7 – a two-AZ VPC with public, private-app and private-data tiers, an ALB as the only public entry point, an autoscaled compute tier, an RDS instance with no internet route, and an S3 bucket for artifacts. You are practising the *decisions*, not typing HCL from a tutorial.

- 1 Choose the address plan before you write any code

Pick a /16 for this environment from a written allocation table, then split it: /20 per tier per AZ leaves room to grow, and keeps subnet boundaries readable at a glance.

💡 *Why: CIDR choice is the one decision here that is expensive to reverse. Overlapping ranges cannot be peered, and re-addressing a live VPC is a migration project. Ten minutes now, or a quarter later.*

2 Create the VPC, subnets and route tables as three explicit tiers

Public, private-app and private-data — each with its own route table, even where two tables start identical.

💡 *Why: Sharing a route table between tiers means a future edit intended for one tier silently applies to another. Separate tables make "the data tier has no internet route" a structural fact that a diff will show.*

3 Attach an internet gateway and add the default route to the public tier only

Then verify by reading the private tiers' route tables and confirming no 0.0.0.0/0 entry exists.

💡 *Why: This is the moment the words "public subnet" acquire meaning. Reading the table back is the habit that catches the mistake; trusting the module name is what causes it.*

4 Place one NAT gateway per AZ and point each private-app route table at its own

Note the cost of what you just did, and note what a single shared NAT would have cost you in availability.

💡 *Why: A shared NAT gateway is a cheaper single point of failure that also generates cross-AZ transfer charges. Making the trade-off consciously — and writing it in the README — is the senior behaviour.*

5 Add a gateway VPC endpoint for S3

Route S3 traffic from the private subnets through the endpoint instead of the NAT gateway.

💡 *Why: Gateway endpoints are free and remove a per-GB charge on traffic that never needed to leave AWS. It is the highest ratio of saving to effort in this entire chapter.*

6 Define security groups that reference each other

ALB accepts 443 from the internet; app accepts its port from the ALB's group; database accepts 5432 from the app's group. No CIDR ranges between your own tiers.

💡 *Why: Group-to-group rules survive every autoscaling event and AMI swap. They also read as intent — "the app may reach the database" — rather than as a list of addresses somebody must maintain.*

7 Give the compute tier an instance profile, and remove every credential

Attach a role granting exactly the S3 prefix and Secrets Manager entries the app needs. Confirm no key exists in the AMI, the user data, or the environment.

💡 *Why: You are converting a secret into an identity. Verify by looking for what should be absent — that inversion is the whole discipline of credential hygiene.*

8 Launch RDS into the data subnets, Multi-AZ, and prove the isolation

Then attempt to connect from outside the VPC and watch it time out rather than refuse.

💡 *Why: A timeout means no route; a refusal means a route existed and something rejected you. Learning to read that difference will save you hours later.*

 Production tip

Before you call it done, force a failover on the RDS instance (there is an explicit reboot-with-failover option) while a client is connected, and watch what your application does. Most applications discover for the first time in a real incident that their connection pool caches DNS forever and never reconnects. A rehearsed failover during working hours is a ten-minute exercise; an unrehearsed one is an outage with an audience.

6 Thinking Exercises **Think it through**

AWS made explicit `Deny` unconditionally win over any `Allow`, rather than using a "most specific rule wins" model like many firewalls. What class of mistake does that choice prevent, and what does it make harder or more frustrating in day-to-day work?

 Think it through

A private subnet has no inbound path from the internet, yet a machine in one can still be compromised — through a poisoned dependency, a vulnerable outbound HTTP client, or a stolen credential used from inside. Given that, what is the private subnet actually buying you, and what should you add so that it is not the only thing standing between an attacker and your data?

 Think it through

Cross-AZ data transfer is charged in both directions, yet spreading across AZs is the standard availability advice. Describe a concrete workload where the correct engineering answer is to keep chatty components in a single AZ, and what you would put in place to make that choice safe.

7

Debugging Practice

🔧 Debugging Lab • Pods in the new private subnet cannot pull images

Symptom. After adding a third availability zone, workloads scheduled onto nodes in the new subnet sit in `ImagePullBackOff`. The same image pulls fine on nodes in the two original AZs.

route table for the new private subnet, and a pod event

```
$ aws ec2 describe-route-tables --route-table-ids rtb-0c3newprivate
DestinationCidrBlock  GatewayId / NatGatewayId  State
10.0.0.0/16           local                     active
# ...no other routes...

Events:
Warning Failed kubelet Failed to pull image "1234.dkr.ecr.eu-west-1.amazonaws.com/cloudbase-api:9f21c":
rpc error: code = Unknown desc = failed to resolve reference: dial tcp 52.x.x.x:443: i/o timeout
```

- ✓ **Observe.** A timeout, not a refusal or an authentication error — and it is scoped to one subnet. That combination points at routing, not at credentials or the registry.
- ✓ **Hypothesise.** The new private subnet was created but never associated with a route table containing a NAT route, so it fell back to the VPC main route table, which only has `local`.
- ✓ **Investigate.** Compare the three private subnets' route table associations. Confirm the NAT gateway for the new AZ exists and is in the *public* subnet of that AZ, not the private one.
- ✓ **Root cause.** A missing subnet-to-route-table association. The Terraform module created the subnet and the NAT gateway but the association resource was not extended to the third AZ.
- ✓ **Fix.** Add the association and the `0.0.0.0/0 → NAT` route for the new AZ, apply, and let the kubelet retry.
- ✓ **Prevent.** Drive subnets, NAT gateways, route tables and associations from a single `for_each` over the AZ list so a new AZ cannot be half-created — and add an ECR interface endpoint so image pulls stop depending on NAT at all.

🔧 Debugging Lab • AccessDenied on S3 despite an Allow policy that looks correct

Symptom. The application's role has an `s3:PutObject` Allow on the bucket, but uploads fail. The same call succeeds from an administrator's session.

application log and a policy simulation

```
2026-07-19T11:04:22Z ERROR upload failed: AccessDenied: User:
arn:aws:sts::1234:assumed-role/cloudbase-app/i-0ab12 is not authorized to
perform: s3:PutObject on resource: arn:aws:s3:::cloudbase-uploads-prod/tenant/42/a.png
with an explicit deny in a service control policy
```

- ✓ **Observe.** The error text names the deciding factor: *an explicit deny in a service control policy*. AWS usually tells you which layer denied the request — read the whole message before theorising.
- ✓ **Hypothesise.** An organisation-level SCP denies S3 writes that do not meet a condition —

commonly a required encryption header, a TLS-only condition, or a region restriction.

- ✓ **Investigate.** Read the SCPs attached to the account and its parent OUs, then re-run the call with the condition satisfied. The IAM policy simulator can evaluate with organisation policies included.
- ✓ **Root cause.** The SCP denies `s3:PutObject` unless `s3:x-amz-server-side-encryption` is present; the SDK call omitted it because the bucket relies on default encryption, which does not set the request header.
- ✓ **Fix.** Set the encryption header in the client, or narrow the SCP condition to match how default bucket encryption actually behaves.
- ✓ **Prevent.** Test guardrails against a real workload in a sandbox account before attaching them to production OUs, and remember the rule: an SCP can only subtract. Widening the role's policy would never have fixed this.

8 Mini-Labs

Mini-Lab • Beginner

45 min

Goal. Write a least-privilege IAM policy and prove it is minimal.

Business context. An image-processing service must read and write one prefix of one bucket, nothing else.

Architecture. One role, one policy document, one bucket with two prefixes.

Requirements. Scope object actions to `tenant/*`; add a separate `ListBucket` statement with a prefix condition; add an explicit `Deny` for non-TLS requests.

Constraints. No wildcard action, no wildcard resource, no managed policy shortcuts.

Hints. Object actions take the bucket ARN plus `/*`; bucket-level actions take the bucket ARN alone.

Expected outcome. Reading the permitted prefix succeeds; reading a sibling prefix and any plaintext call fail.

Production considerations. How would you discover the true minimum set of actions for an existing service? (CloudTrail, then IAM Access Analyzer policy generation.)

Common pitfalls. Putting `/*` on a `ListBucket` resource; assuming default bucket encryption satisfies a request-header condition.

Mini-Lab • Intermediate

75 min

Goal. Build the two-AZ VPC in Terraform and prove the isolation properties empirically.

Business context. A security review asks you to demonstrate — not assert — that the data tier is unreachable from the internet.

Architecture. VPC, three subnet tiers across two AZs, IGW, one NAT per AZ, S3 gateway endpoint, group-to-group security rules.

Requirements. Every subnet, route table and association driven from a single AZ list; an output listing each subnet's effective default route.

Constraints. No default VPC, no `0.0.0.0/0` ingress except on the ALB's group, port 443 only.

Hints. Use `for_each` over a map of AZ to CIDR so adding an AZ is a one-line change.

Expected outcome. A short written proof: for each tier, the route table contents and a connection test whose result matches.

Production considerations. Enable VPC Flow Logs and identify what is crossing the NAT gateway; estimate the monthly cost of that traffic.

Common pitfalls. Creating a NAT gateway in a private subnet; forgetting the route-table association; enabling the ALB in only one AZ.

Mini-Lab • Production-style

2.5 hours

Goal. Deliver the complete CloudBase AWS environment, with a failover rehearsal and a cost review.

Business context. This environment will receive the Day 9 pipeline's deployments tomorrow; it must be reproducible from an empty account.

Architecture. VPC (2 AZs, 3 tiers) → ALB → autoscaled compute → RDS Multi-AZ → S3 for artifacts and state, roles throughout.

Requirements. No static credentials anywhere; account-level S3 public access block; lifecycle rules; deletion protection and backups on RDS; every resource tagged with owner, environment and service.

Constraints. `terraform destroy` then `apply` must reproduce the environment exactly; no console clicks.

Hints. Split into modules — network, compute, data — with explicit outputs, so the network module can be reused per environment.

Expected outcome. A working environment, a forced RDS failover with the observed application behaviour written down, and a one-page cost estimate naming the three largest line items.

Production considerations. Which resources would you protect with `prevent_destroy`, and what is your recovery story if the state file is lost?

Common pitfalls. Hardcoding AZ names; forgetting that a security group cannot be deleted while an ENI references it; discovering the NAT gateway cost only after the first bill.

9

Engineering Notes

Operational excellence

The AWS console is a debugging tool, not a change interface. Anything you click is invisible to your Terraform state, unreviewed, undocumented, and gone the next time somebody applies. The discipline that keeps a cloud account comprehensible is simple and unpopular: read in the console, write in code. When an emergency genuinely forces a manual change, open the pull request that codifies it before you close the incident, not "later".

Infrastructure insight

Availability zone names are not stable across accounts. `eu-west-1a` in your account may be different physical infrastructure from `eu-west-1a` in a colleague's — AWS randomises the mapping per account to spread load. Pin placement by AZ ID when it matters (for example when co-locating with a partner's resources), and never assume two accounts' "same" AZ are the same building.

 Automation tip

Guardrails belong in policy, not in review comments. An SCP that denies creating an IAM user, or denies disabling S3 public access blocks, or denies launching outside approved regions, enforces itself at 3am on a Sunday when nobody is reading pull requests. Combine that with IAM Access Analyzer to find resources shared outside the account, and with Config rules for drift you cannot express as a deny.

 Interview insight

Expect: *"How does AWS decide whether to allow a request?"* The answer that lands is the chain: deny by default, an explicit Deny anywhere is final, identity and resource policies union within an account, and boundaries and SCPs intersect — they can only subtract. Then close it with the practical consequence: when you get AccessDenied, read which layer denied you before widening any policy, because if an SCP denied it, no amount of granting will ever help.

 Disaster recovery tip

Your disaster-recovery story is only as good as the thing that is hardest to rebuild. Machines are disposable and defined in code; data is not. Rank your recovery work by what cannot be regenerated: the database, the object store, and the Terraform state. Everything else can be re-created from the repository in an afternoon — provided the repository actually describes it, which is precisely why the console discipline above matters.

10 Build-Along Project — CloudBase

Day 3 gave CloudBase an address plan on paper. Day 7 declared infrastructure in Terraform with remote state. Day 9 built and pushed an image through a pipeline that, until now, had nowhere to deploy to. Today all three become one running environment in a real AWS account.

 Build-Along • Day 10 — the real network, identity and data tier

- Realise the Day 3 address plan: a VPC with public, private-app and private-data subnets across two AZs, each tier with its own route table.
- Attach an internet gateway to the public tier only; place one NAT gateway per AZ and route the private-app tier through it. Add a free S3 gateway endpoint and confirm S3 traffic no longer crosses NAT.
- Deploy an Application Load Balancer as the *only* public entry point — HTTPS listener with an ACM certificate, health-checked target group, enabled in both AZs.
- Run the compute tier (EC2 autoscaling group or EKS nodes, matching Day 5–6) in the private-app subnets, registered with the target group.
- Launch RDS Multi-AZ into the private-data subnets, with a subnet group spanning both AZs, deletion protection on, automated backups on, and a security group that accepts 5432 only from the application's security group.
- Replace every credential with an identity: instance profile or Pod Identity for the app, an OIDC-federated role for the Day 9 pipeline. Grep the repository, the image and the launch template to prove no access key survives.
- Move Terraform state and build artifacts into S3 buckets with versioning, lifecycle rules

and public access blocked at the account level.

- Tag every resource with `owner`, `environment` and `service`; produce a one-page cost estimate naming the three largest recurring line items and one change that would reduce each.

Definition of done: from an empty account, `terraform apply` produces a reachable HTTPS endpoint served by instances in private subnets; the database is unreachable from outside the VPC (a timeout, not a refusal); no static AWS credential exists anywhere in the repository, image or instance configuration; a forced RDS failover is survived by the application with the behaviour written down; and the cost estimate is committed alongside the code.

11 Chapter Summary

Key takeaways

- ✓ A subnet is private because its route table lacks a route to an internet gateway — nothing else makes it private.
- ✓ Security groups are stateful and allow-only; NACLs are stateless and ordered, and need explicit return-traffic rules.
- ✓ Reference security groups from other security groups, not CIDRs, so rules survive autoscaling.
- ✓ IAM denies by default; an explicit `Deny` anywhere is final; boundaries and SCPs only ever subtract.
- ✓ A role plus STS removes the secret entirely — there is nothing to leak because nothing is stored.
- ✓ ALB is Layer 7 (host and path routing, HTTP awareness); NLB is Layer 4 (static IPs, raw TCP/UDP, lower latency).
- ✓ Multi-AZ is availability and serves nothing; a read replica is read capacity and does not fail over.
- ✓ Cost is architecture: NAT hours, cross-AZ transfer, and resources nobody deleted.

Things to remember

- A connection *timeout* means no route; a *refusal* means a route existed and something said no. Different bugs.
- Bucket-level actions like `s3:ListBucket` take the bucket ARN *without* `/*`; object actions take it *with*.
- AZ names are randomised per account; AZ IDs are not.
- Read in the console, write in code. Every console change is future drift.

Common pitfalls (last look)

- Database in a public subnet • `s3:*` on `*` • hardcoded access keys
- NAT gateway in a private subnet • ALB enabled in one AZ only • health checks that always pass
- Expecting a read replica to fail over • no lifecycle rules • untagged resources

📄 One-Page Cheat Sheet

VPC — your CIDR in one region; subnets each live in exactly one AZ.

Public subnet — a route table with `0.0.0.0/0` → internet gateway. That line is the definition.

NAT gateway — outbound-only egress for private subnets; per-hour plus per-GB; one per AZ in production.

Security group — stateful, allow-only, union of rules; source may be another security group.

IAM — deny by default → explicit Deny wins → identity/resource union → boundary and SCP intersect.

Role + STS — temporary credentials from the instance profile or Pod Identity; no secret to leak.

ALB vs NLB — L7 host/path routing versus L4 static IP and raw TCP/UDP.

Multi-AZ vs replica — synchronous standby for failover (serves nothing) versus async copy for reads (no failover).

CLI quick reference

Day 10 commands worth memorising

```
# who am I, really? (the single most useful AWS command)
aws sts get-caller-identity

# reachability: read the route table before widening any security group
aws ec2 describe-route-tables --filters Name=association,subnet-id,Values=subnet-0ab
aws ec2 describe-security-groups --group-ids sg-0ab --query 'SecurityGroups[].IpPermissions'

# who can reach what, end to end, without sending a packet
aws ec2 describe-network-insights-analyses

# load balancer health: the answer to "why is traffic not arriving?"
aws elbv2 describe-target-health --target-group-arn <arn>

# storage hygiene
aws s3api get-public-access-block --bucket cloudbase-artifacts-prod
aws s3api get-bucket-lifecycle-configuration --bucket cloudbase-artifacts-prod

# database posture
aws rds describe-db-instances --query \
  'DBInstances[].{id:DBInstanceIdentifier,az:MultiAZ,pub:PubliclyAccessible}'

# orphans that quietly cost money every hour
aws ec2 describe-volumes --filters Name=status,Values=available
aws ec2 describe-addresses --query 'Addresses[?AssociationId==null]'
```

👤 Interview questions to master

- › Why can a private subnet not be reached directly from the internet, by design — and what exactly enforces that?
- › What is the practical difference between an ALB and an NLB, and when would you be forced to choose the NLB?
- › Why is a static IAM access key a bigger long-term risk than an assumed role?
- › Multi-AZ versus read replica — what problem does each actually solve, and which one is a backup?

- › How would you scope an IAM policy to the minimum permissions a service needs, and how would you discover that minimum?
- › Walk me through IAM's evaluation order. Where does an explicit Deny fit, and can an SCP ever grant access?
- › What is the blast-radius difference between compromising something in a public subnet and something in a private one?
- › Name three line items that make an AWS bill larger than people expect, and what you would do about each.

Further reading & official docs

- AWS IAM User Guide — "Policy evaluation logic", on docs.aws.amazon.com under [IAM/latest/User-Guide](#): the authoritative evaluation order, including SCPs, RCPs, permissions boundaries and the explicit-deny rule.
- Amazon VPC User Guide — "Pricing for NAT gateways" and the VPC pricing page: read these before you design your AZ layout, not after.
- Amazon S3 User Guide — "Blocking public access to your Amazon S3 storage", plus the AWS announcement that Block Public Access is enabled and ACLs disabled by default for new buckets from April 2023.
- Amazon EKS User Guide — "IAM roles for service accounts" and "EKS Pod Identity": the two ways a pod gets an AWS identity, and why the newer one exists.
- AWS News Blog — "Amazon EC2 Instance Metadata Service IMDSv2 by default": what changed, and how to enforce it account-wide.
- Netflix TechBlog — "How Netflix is able to enrich VPC Flow Logs at Hyper Scale to provide Network Insight": what network observability looks like when the VPC is very large and IP addresses are ephemeral.
- AWS Well-Architected Framework — the Security and Cost Optimization pillars, read as a review checklist rather than as prose.

Production checklist

- ☐ Application and data tiers sit in subnets with no `0.0.0.0/0` route to an internet gateway.
- ☐ The load balancer is the only resource accepting traffic from the internet, on 443 only.
- ☐ Security group rules between your own tiers reference groups, not CIDR ranges.
- ☐ Every compute resource uses an IAM role; a repository-wide search finds no access key.
- ☐ S3 public access is blocked at the account level and enforced by an SCP.
- ☐ Every bucket has versioning and a lifecycle rule, including multipart-upload cleanup.
- ☐ RDS has Multi-AZ, automated backups, deletion protection — and a restore that has been tested.
- ☐ Resources are spread across at least two AZs, and both the ASG and the load balancer are enabled in both.
- ☐ Every resource is tagged with owner, environment and service, from code.
- ☐ Someone has looked at the cost report this month and can explain the top three line items.

 Key idea

What you will learn next (Day 11). You now have infrastructure that is reachable, authorised and paid for — and almost entirely opaque. Tomorrow you make it observable: metrics that answer whether users are being served, the difference between a metric, a log and a trace, Prometheus's pull model and why it works the way it does, and alerts that fire on symptoms rather than on causes. The ALB target-health check you wrote today is the first monitor in that system; Day 11 turns it into a signal you can actually act on.

DAY 11 • OBSERVABILITY

Monitoring, Logging & Observability

11

Metrics, logs, traces, SLOs and alerts a human will actually act on

🕒 Estimated study time: 8 hours

🎯 By the end of this day, you will be able to...

- Explain what metrics, logs and traces each answer that the other two cannot – and why you need all three.
- Describe how Prometheus really works: pull-based scraping, service discovery, and the on-disk time-series database.
- Choose the right metric type, write correct PromQL, and read percentile latency without fooling yourself.
- Emit structured JSON logs carrying a correlation ID that survives a multi-service incident.
- Define an SLI, an SLO and an error budget, and build a burn-rate alert that pages a human only when it matters.

1 The Big Picture

What problem this solves. Ten days of this book have been about *building* a system: a hardened host, containers, a Kubernetes cluster, a CI/CD pipeline, Terraform-managed cloud infrastructure. None of it tells you whether the system is currently doing its job. Observability is the discipline of answering, from the outside and without redeploying anything, three escalating questions: **is it broken?**, **how badly**, and **for whom?**, and **where exactly?** A system you cannot ask those questions of is not production-ready – it is a demo that happens to be running.

Why DevOps engineers need it. You will be on call for systems you did not write. At 03:14 the only inputs you have are the signals the system emits. If those signals were designed by someone who understood distributed systems, you will find the cause in ten minutes. If they were not, you will spend two hours SSH-ing into pods reading unstructured text. The difference is entirely decided *before* the incident, by whoever instrumented the code and wrote the alerts. Today you become that person.

🔑 Key idea

Monitoring answers questions you thought of in advance. Observability is the property that lets you answer questions you did *not* think of in advance, without shipping new code. The practical test: when a novel failure happens, can you form and check a new hypothesis using data you already have? If every new question needs a new deploy, you have monitoring, not observability.

Where it fits in a modern cloud architecture. Observability is not a layer on top of the stack – it is a sideways plane that reads from every layer at once. The application exposes metrics and logs, the container runtime and kubelet expose their own, the node exposes hardware counters, and the

cloud provider exposes load-balancer and database metrics. A collector pulls them together so one query can span all of them.

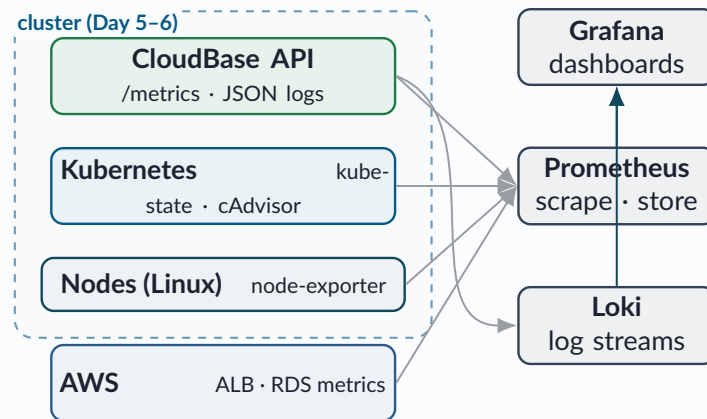


Fig 11.0 — The observability plane reads sideways across every layer you have built since Day 1.

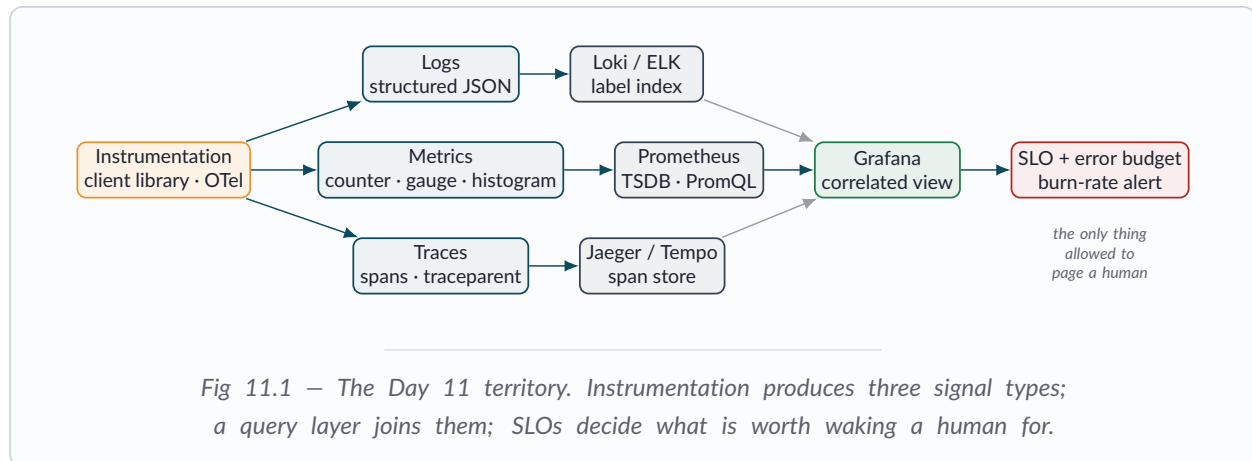
How it connects to previous chapters. Day 1 taught you that a process writes to file descriptor 1 and that `journalctl` reads it — structured logging is just deciding what bytes go into fd 1. Day 4's container gave every process an ephemeral filesystem, which is exactly why logs must leave the container. Day 5-6's Kubernetes gave you pods that come and go with new IPs every deploy, which is exactly why a monitoring system must *discover* its targets instead of being handed a static list. Day 9's pipeline is what will ship the instrumentation. Day 10's cloud infrastructure is what you are now going to watch.

Where companies use it in production. Cloudflare published that it runs approximately 916 Prometheus instances holding around 4.9 billion active time series — roughly 5 million per instance, with the largest near 30 million — deliberately as *many small* Prometheus servers rather than one giant one, then joins them behind a single query layer (Thanos) for global questions. Uber built Jaeger and deployed a `jaeger-agent` on every host so application libraries only ever speak to a local UDP port, which removed the service-discovery dependency that was making tracing adoption painful for product teams. Both are the same architectural instinct: push the hard distributed-systems problem into the observability infrastructure so that application engineers can instrument their code in two lines.

Common misconceptions. That collecting more data makes you more observable (it makes you poorer — cardinality, not volume, is the cost driver); that a dashboard is monitoring (a dashboard nobody opens is a screensaver); that alerting on CPU is useful (users do not experience CPU); that average latency describes user experience (it describes nobody); and that tracing replaces logging (a trace tells you *which* hop was slow, a log tells you *what the code was doing* there).

2

Concept Map



Terminology to anchor: **instrumentation** → **time series** → **label / cardinality** → **scrape** → **PromQL** → **histogram bucket** → **correlation ID** → **span** → **SLI** → **SLO** → **error budget** → **burn rate**.

3 Guided Learning

3.1 The Three Pillars – What Each One Actually Answers

What is it? Three different shapes of telemetry, each a lossy compression of reality chosen for a different question.

Signal	What it is, and what it can answer	Cost shape
Metrics	Numeric time series, aggregated at collection. Answers <i>is it broken, how much, since when, trending which way</i> . Cannot answer "what happened to this request".	Cheap per event, expensive per unique label combination.
Logs	Discrete events with full context. Answers <i>what exactly did the code do for one request</i> . Terrible at "what is the 99th percentile" — you would have to read everything.	Expensive per event (bytes), cheap per dimension.
Traces	A causally-linked tree of spans across services. Answers <i>which hop was slow / which service failed</i> in a request that touched six services.	Expensive; nearly always sampled.

Why does it exist? Before this vocabulary existed, operations meant Nagios-style *check scripts*: a central server SSH-ing to a box every five minutes to run a script that returned OK, WARNING or CRITICAL. That model works when a service is a named machine. It collapses when a service is forty ephemeral pods that are replaced on every deploy: there is no stable host to check, five-minute resolution hides a two-minute outage entirely, and a boolean verdict cannot express "7% of requests are failing". Metrics replaced the boolean with a number over time; structured logs replaced `grep` over free text with queryable fields; tracing replaced "which of these six services is at fault?" guesswork with a causal tree.

How does it work internally? The three are not independent — they are joined by *shared identifiers*. A metric carries labels (`service`, `route`, `status`); a log line carries the same labels plus a `correlation_id`; a span carries a `trace_id` that the log line also carries. That shared vocabulary is what turns three tools into one investigation: a latency spike on a graph, click through to the logs for that exact time window and service, find a correlation ID, pivot to the trace, see the 4-second database span. Getting

those identifiers consistent across services is 80% of the engineering work in observability, and it is why OpenTelemetry's *semantic conventions* exist at all.

? Why does this exist?

Prometheus was written at SoundCloud in 2012 by engineers who had worked on Google's Borgmon. Borgmon's key insight, carried into Prometheus, was to stop treating a monitored thing as a *host with checks* and start treating it as a *multi-dimensional time series with labels you can query*. Everything else — the pull model, service discovery, PromQL — follows from that one reframing.

When should I use them? Metrics: always, for everything, as the default signal. Logs: always, but structured and rate-limited. Traces: as soon as a single user request crosses more than two services — and not before, because the instrumentation cost is real.

When should I not? Do not add tracing to a monolith to look modern; a well-instrumented histogram per endpoint tells you the same thing for a fraction of the effort. Do not ship every debug log to a hosted log platform "just in case" — log ingestion bills are one of the most common surprise line items in cloud spend, and nobody ever reads `DEBUG` from three weeks ago.

3.2 Prometheus Internals — Pull, Discovery and the TSDB

What is it? A server that periodically fetches a plain-text page of numbers over HTTP from every target it knows about, stores those numbers as time series in a local database, and lets you query them with PromQL. That is genuinely all it is. An application "supports Prometheus" by serving text like this:

GET /metrics - the entire exposition format

```
# HELP http_requests_total Total HTTP requests.
# TYPE http_requests_total counter
http_requests_total{route="/orders",method="POST",status="200"} 48123
http_requests_total{route="/orders",method="POST",status="500"} 97
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{route="/orders",le="0.1"} 41022
http_request_duration_seconds_bucket{route="/orders",le="0.3"} 47510
http_request_duration_seconds_bucket{route="/orders",le="+Inf"} 48220
http_request_duration_seconds_sum{route="/orders"} 5121.4
http_request_duration_seconds_count{route="/orders"} 48220
```

Why does it exist? (the pull model) In a push model every application must know the address of the monitoring system, hold a connection to it, buffer on failure, and implement backoff — and a misbehaving app can flood the collector with no way for the collector to say no. Prometheus inverts it: the *server* decides when and how often to fetch, from a list of targets it builds itself. Three consequences matter operationally, and they are the real reason the model wins:

- **Target health is free.** If a scrape fails, that is itself a metric (`up` goes to 0). In a push model, silence is ambiguous — did the app die, or is it just idle?
- **Service discovery is the whole configuration.** Prometheus queries the Kubernetes API (or EC2, or Consul) for the current set of pods matching a selector, and rewrites that list into targets every few seconds. Pods created by a Day 6 rollout are scraped within seconds without anyone editing config.
- **You can reproduce a scrape by hand.** `curl` the `/metrics` endpoint and you see exactly what the server sees. Debugging a push pipeline means debugging someone else's buffer.

The Prometheus FAQ is honest about this: pulling is "slightly better" than pushing, not a religion. Short-lived batch jobs genuinely cannot be pulled, which is why the Pushgateway exists as a deliberate,

narrow exception.

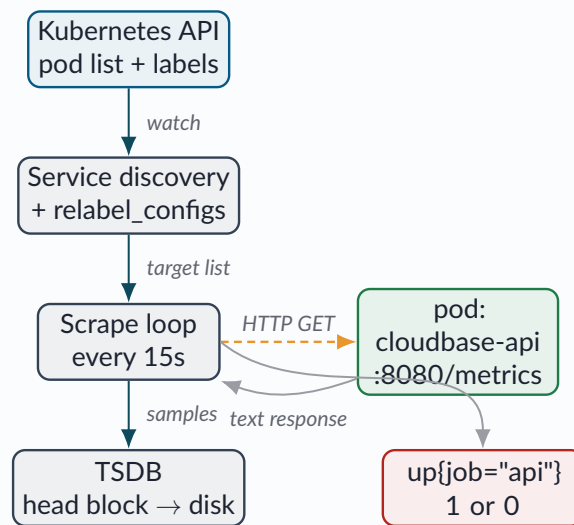


Fig 11.2 – The scrape loop. Discovery rewrites the target list continuously; the scrape itself is an ordinary HTTP GET.

How does it work internally? (the TSDB) Every unique combination of metric name and label values is one **time series**, held in memory as a `memSeries` structure: a copy of its labels plus a chain of compressed chunks. Samples are appended to the active chunk; chunks are cut on two-hour wall-clock boundaries and eventually flushed to an immutable on-disk block alongside an inverted index mapping label pairs to series IDs. Two properties follow directly, and both bite people in production:

- **Memory scales with active series, not with sample volume.** Scraping the same 10,000 series twice as often roughly doubles disk, but barely moves memory. Adding one new label with 10,000 possible values multiplies your series count by 10,000 and can kill the server.
- **Series that appear once are still expensive.** A series stays resident for the lifetime of its head chunk – on the order of one to three hours – even if it received exactly one sample and will never be seen again.

👁 Behind the scenes

This is precisely why Cloudflare had to patch Prometheus rather than just add capacity. With 916 servers and 4.9 billion series, a single team labelling a metric with a raw URL path or an error string could take a server down. Their fix is layered and worth copying at any scale: per-scrape limits on sample and label counts (their published defaults are on the order of 200 samples and 64 labels per application), CI checks that estimate the cardinality impact of a change *before* merge, and a global cap that degrades gracefully – dropping the excess series rather than failing the entire scrape. Note the design philosophy: an application team that gets a metric wrong should get a warning, not an outage of the monitoring system.

👤 Security warning • cardinality is the one thing that will take your monitoring down

Never put an unbounded value in a label: user ID, email, session ID, full URL path, raw error message, timestamp, container ID. A metric labelled with the request path lets *any anonymous internet user* create unlimited time series by requesting random URLs – a denial-of-service against your own monitoring. Always use the *route template* (`/orders/{id}`), never the resolved path. High-cardinality identifiers

belong in logs and traces, which are indexed for exactly that.

When should I *not* use Prometheus? It is not a log store — the project's own FAQ says so and points you at Loki or OpenSearch. It is not durable long-term storage: a single server is local-disk, non-replicated, and losing it loses the data, which is why Thanos or Mimir exist for multi-year retention and global query. And it is not a billing system: samples are lossy and scrape-aligned, so never derive invoices from a counter.

3.3 Metric Types, PromQL and Why p99 Beats the Average

What is it? Four instrument types, of which you will use three constantly.

Type	Semantics	Use it for
Counter	Monotonically increasing; resets to 0 on process restart. Never read the raw value — always <code>rate()</code> .	Requests served, errors, bytes sent, retries.
Gauge	Goes up and down freely. Read directly.	Queue depth, in-flight requests, memory in use, replica count.
Histogram	Pre-defined cumulative buckets plus <code>_sum</code> and <code>_count</code> . Percentiles computed at query time.	Latency, payload size — anything you need a percentile of.
Summary	Quantiles computed <i>in the client</i> . Cheap to read, but mathematically un-aggregatable across instances.	Rare. Prefer histograms.

How does it work internally? (histograms) A histogram does not store observations. It stores a set of counters, one per *cumulative* bucket boundary: "how many requests took at most 0.1s", "at most 0.3s", and so on, ending at `+Inf`. At query time `histogram_quantile(0.99, ...)` finds the bucket where the 99th percentile falls and linearly interpolates within it. Three implications you must internalise:

- **Your bucket boundaries decide your accuracy.** If your buckets jump from 1s to 10s, every p99 between them reports as an interpolation artefact. Choose buckets around your SLO threshold — if the SLO is 300 ms, you want boundaries at 0.1, 0.2, 0.3, 0.5, 1.
- **Buckets are cheap in bytes and expensive in cardinality.** Ten buckets $\times$ twenty routes $\times$ five status codes is 1,000 series from one histogram. This is the single most common cause of accidental cardinality growth.
- **Because buckets are counters, they aggregate correctly.** You can `sum` bucket counts across all pods and *then* compute the quantile. You cannot do that with pre-computed quantiles — and that is why summaries lose.

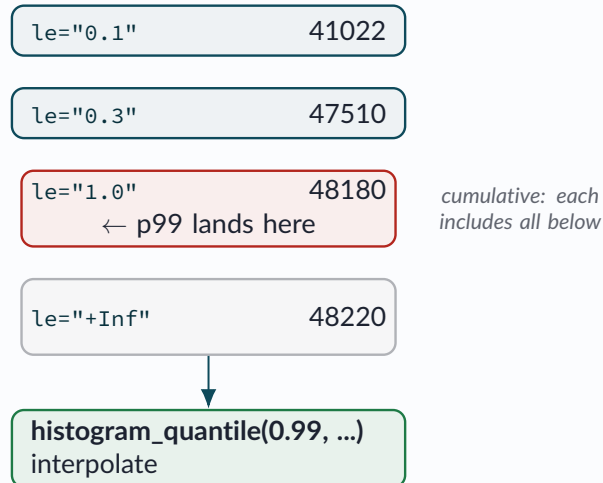


Fig 11.3 — A histogram stores cumulative bucket counters; the percentile is computed at query time, which is what makes it aggregatable across pods.

Why does p99 matter more than the average? Because an average is a statement about a fictional user. Consider a service where 99 requests take 50 ms and one takes 5 s. The mean is 99.5 ms — a number that describes precisely zero of those requests, and which looks perfectly healthy on a dashboard. Meanwhile one user in a hundred waited five seconds. Now scale it: a page that makes 20 backend calls will hit the 99th percentile of at least one of them roughly 18% of the time. **The tail is not an edge case; at scale the tail is the typical page load.** And the tail is systematically your most valuable users — the ones with the largest carts, the most data, the fullest inboxes — because their requests are the expensive ones.

⚠ Common misconception

"We report p99, so we're fine." Not if you are averaging p99 across pods, or across a long window. `avg(p99)` is not a percentile of anything — it is an average of maxima and it systematically understates the tail. Compute the quantile *after* summing the buckets: `histogram_quantile(0.99, sum by (le) (rate(..._bucket[5m])))`. The sum by (le) before the quantile is the entire correctness of the query.

the four PromQL queries you will write most often

```
# 1. Request rate (per second, averaged over 5 minutes) -- the RED "Rate"
sum by (route) (rate(http_requests_total[5m]))

# 2. Error ratio -- the RED "Errors". Note: ratio, not count.
sum(rate(http_requests_total{status=~"5.."}[5m]))
  / sum(rate(http_requests_total[5m]))

# 3. p99 latency, correctly aggregated across all pods -- the RED "Duration"
histogram_quantile(0.99,
  sum by (le, route) (rate(http_request_duration_seconds_bucket[5m])))

# 4. Saturation: is the app running out of its cgroup memory limit? (Day 4)
container_memory_working_set_bytes / container_spec_memory_limit_bytes
```

💡 Senior engineer insight

`rate()` is not "the derivative". It is a linear extrapolation over the window that is explicitly counter-reset aware: when it sees the value drop, it assumes a restart and treats the drop as a reset rather than a negative rate. This is why you must never apply `rate()` to a gauge (use `deriv()` or `delta()`) and never apply raw arithmetic to a counter. It is also why your range window must be at least 4x your scrape interval — with a 15 s scrape, `rate(x[20s])` may see only one sample in the window and return nothing, producing mysterious gaps in a dashboard that "worked yesterday" before someone raised the scrape interval.

🔧 Production tip

Standardise on two dashboard vocabularies and you will never argue about dashboard layout again. **RED** for request-driven services: Rate, Errors, Duration. **USE** for resources: Utilisation, Saturation, Errors — applied to CPU, memory, disk and network. Every CloudBase service dashboard should open with the same three RED panels in the same three positions, so an on-call engineer reads an unfamiliar service the way they read a familiar one.

⚡ Performance note

Prometheus 3.0 (November 2024, the first major release in seven years) shipped changes worth knowing: full UTF-8 support in metric and label names, so OpenTelemetry metrics no longer need dots rewritten to underscores; Remote-Write 2.0, which uses string interning and a symbol table to cut wire volume substantially; and *native histograms*, which replace fixed buckets with automatically-scaled exponential buckets at far lower cost. Native histograms are still stabilising — check their current status before adopting them for anything you alert on.

3.4 Structured Logging & Correlation IDs

What is it? Emitting each log event as a machine-parseable object with named fields, instead of a sentence a human composed. The single most important field is a **correlation ID** (equivalently, a trace ID) generated once at the edge and propagated to every downstream call.

one structured log event – every field is queryable

```
{
  "timestamp": "2026-07-19T10:22:01.481Z",
  "level": "ERROR",
  "service": "order-api",
  "version": "1.14.2",
  "pod": "order-api-7d9f-xk2p",
  "correlation_id": "a1b2c3d4e5f60718",
  "trace_id": "a1b2c3d4e5f60718",
  "route": "/orders",
  "status": 502,
  "duration_ms": 4021,
  "message": "payment gateway timeout",
  "order_id": 4821,
  "upstream": "payments-api",
  "attempt": 3
}
```

Why does it exist? Because free-text logging pushes the parsing cost from write time (once, cheaply, where the data is structured) to read time (repeatedly, expensively, with regexes, during an incident, at 3am). The regex approach also breaks silently: someone rewords a log message in a refactor, the parser stops matching, and the dashboard quietly shows zero errors forever. Structured logging makes the schema explicit and reviewable.

The correlation ID exists because of a specific distributed-systems failure. With one service, "show me the logs around 10:22" works. With six services and 200 requests per second, the logs of one failing request are interleaved with thousands of unrelated lines across six different log streams — and worse, the *symptom* appears in the edge service while the *cause* appears in a service three hops away. Without a shared identifier there is no join key, and reconstructing one request is genuinely impossible.

How does it work internally? The edge (ingress, API gateway, or the outermost service) checks

for an incoming `traceparent` header. If absent, it generates a new trace ID. That ID goes into a request-scoped context — a thread-local, an async context, or an explicit parameter — so the logging framework can inject it into every line automatically, and into an outbound HTTP header on every downstream call. The W3C Trace Context Recommendation standardises the header so that services written in different languages with different vendors interoperate:

W3C `traceparent` - version, trace-id, parent-id, flags

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
      ^   ^                                     ^               ^
      |   |                                     |               |
      |   | 16-byte trace id (whole request) 8-byte span id  sampled=1
      |   |
      |   | version
```

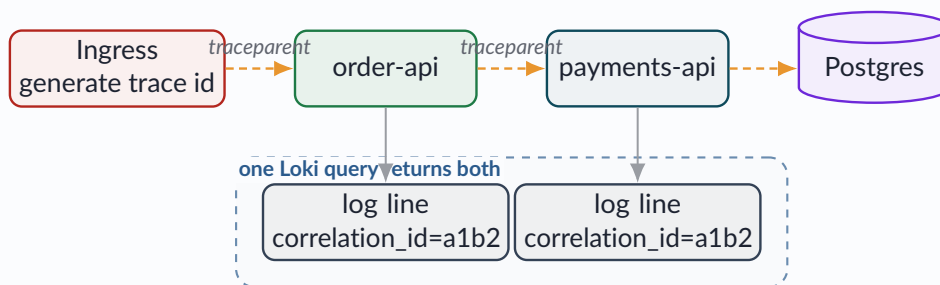


Fig 11.4 — One ID, generated at the edge, propagated by header, written into every log line. This is the join key that makes an incident tractable.

How does the log store work internally? Loki was designed on a deliberate bet: that you almost always know *roughly where* to look (which service, namespace, environment, time window) before you know *what* to look for. So Loki indexes only a small set of **labels** per log stream and stores the log lines themselves as compressed chunks that are never full-text indexed. A query first uses the label index to select a tiny subset of streams, then brute-force greps the compressed chunks in parallel. Elasticsearch takes the opposite bet: index every token, pay a large ingest and storage cost, get arbitrary full-text search back. Neither is wrong — but the label index gives Loki a cost profile close to object storage, which is why it dominates at high log volume with predictable query patterns.

🔧 Design trade-off

The Loki design flips the cardinality rule you just learned for Prometheus — but only halfway. Loki labels must stay *low-cardinality* for exactly the Prometheus reason: each unique label combination is a separate stream with its own index entry and chunk. So `namespace`, `app`, `level` are labels; `correlation_id` is emphatically *not* a label — it lives in the line body and is found by a filter expression after the label selector has narrowed the search. Making a correlation ID a Loki label is the classic first mistake, and it produces millions of one-line streams.

🔧 Operational excellence

Do not write logs to a file inside a container. Write JSON to stdout, let the container runtime capture it, and let a node-level agent (Promtail, Fluent Bit, Vector) ship it. This is a direct consequence of Day 4: a container filesystem is ephemeral, so a log file in a crashed pod is a log file you will never read — and the crash is precisely the event you needed the logs for.

3.5 Distributed Tracing & OpenTelemetry

What is it? A **trace** is a tree of **spans**. Each span records one unit of work — an inbound HTTP request, an outbound call, a database query — with a name, a start timestamp, a duration, key/value attributes, and a parent span ID. Assembling the spans that share a trace ID reconstructs the causal structure and the exact timing of a distributed request.

Why does it exist? A metric tells you p99 latency on `/checkout` rose from 200 ms to 3 s. In a six-service architecture that leaves six suspects, each of whom will look at their own dashboard and correctly report that their service is fine — because the latency is in the *aggregate path*, or in a retry loop, or in a service that is fast on average but occasionally serialises. Tracing turns a multi-team argument into a waterfall diagram with one obviously fat bar.

How does it work internally? Instrumentation (usually automatic, via an OpenTelemetry agent or language SDK) starts a span on request entry, attaches the current trace context, and ends it on exit. Spans are batched and exported asynchronously — never on the request's critical path. The volume is the problem: tracing every request at scale can generate more telemetry data than the application generates business data, so **sampling** is mandatory. Head-based sampling decides at the edge (cheap, but you will discard the interesting slow requests). Tail-based sampling buffers complete traces and keeps the anomalous ones (correct, but requires holding traces in memory until all spans arrive).

Infrastructure insight

Uber's Jaeger architecture is instructive because the interesting decision is not the storage backend — it is the `jaeger-agent` deployed on every host. Application libraries emit spans to a local UDP port and poll that same local agent for the current sampling strategy. Product engineers therefore never configure a collector address, never handle a network failure to the tracing backend, and never redeploy to change sampling rates. The infrastructure team can retune sampling globally without touching a single application. That pattern — a local sidecar or DaemonSet absorbing all discovery, buffering and policy — is the same reason you run a node-level log agent rather than a shipper in every container.

When should I use it? When requests cross service boundaries and you cannot attribute latency by inspection. Also when onboarding to an unfamiliar system: a handful of traces is the fastest architecture diagram you will ever get, and it is guaranteed current.

When should I not? On a single service with good histograms. Tracing has real costs — SDK integration, context propagation through async code and message queues (where it usually breaks first), a storage backend, and sampling policy. Earn it.

Senior engineer insight

OpenTelemetry's genuine contribution is not another tracing library — it is the *decoupling of instrumentation from vendor*. Before it, instrumenting for Datadog meant Datadog SDK calls compiled into your source, and migrating vendors meant touching every service. With OTel your code emits vendor-neutral spans and metrics to a Collector, and the Collector's configuration — not your code — decides where they go. Instrumentation becomes a durable asset instead of a vendor commitment. That is the property to argue for in a design review.

3.6 SLIs, SLOs, Error Budgets and Alerts a Human Will Act On

What is it? An **SLI** is a measured ratio of good events to valid events — e.g. the proportion of HTTP requests that returned non-5xx in under 300 ms. An **SLO** is the target for that ratio over a window: 99.5% over 30 rolling days. The **error budget** is the remainder: 0.5% of 30 days is approximately 3 hours 39 minutes of permitted badness per month. The **burn rate** is how fast you are consuming it: burn rate 1 means you will spend exactly the whole budget in the window; burn rate 14.4 means

you will spend it in about 50 hours.

SLO	Budget / 30 days	What it implies operationally
99%	approx. 7h 18m	A generous internal or batch service.
99.5%	approx. 3h 39m	Sensible starting point for a new user-facing API.
99.9%	approx. 43m	Needs redundancy and automated rollback (Day 9).
99.99%	approx. 4m 20s	Multi-AZ, no manual step in the recovery path, at real cost.

Why does it exist? Because "the site should be up" is not a decidable statement, and "100% uptime" is not a goal — it is a refusal to make a trade-off. An error budget converts reliability from an argument between engineering and product into a number both sides can read. Budget remaining → ship features. Budget exhausted → the next sprint is reliability work. The rule is agreed *before* the incident, when everyone is calm, which is the only time such rules can be agreed honestly.

How does it work internally? (burn-rate alerting) A naive alert — "page if the error rate exceeds 1% for 5 minutes" — has two failure modes at once: it fires on a harmless 6-minute blip that consumes 0.01% of the budget, and it stays silent through a slow 0.9% bleed that will exhaust the budget in a week. Google's SRE Workbook answers both with *multiwindow*, *multi-burn-rate* alerts. You require a high burn rate over a long window (proving it is a real, sustained problem) *and* over a short window (proving it is still happening right now, so the alert resets promptly when it stops). The canonical configuration uses long windows of 1 h, 6 h and 3 days with short windows one-twelfth as long, tuned to consumption of 2%, 5% and 10% of the budget respectively — the fast one pages, the slow ones open a ticket.

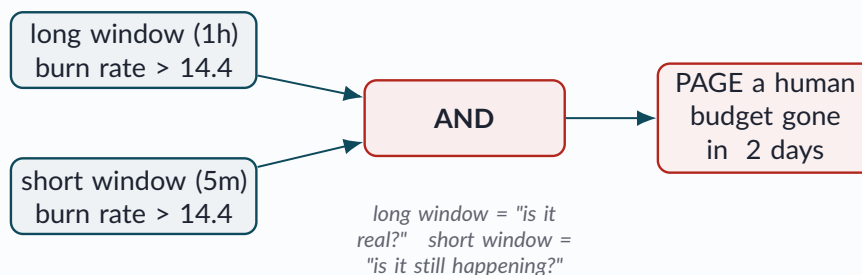


Fig 11.5 — A burn-rate alert fires only when a fast window and a slow window agree. That AND is what removes the noise.

prometheus/rules/cloudbase-slo.yaml — one SLO, one page

```

groups:
- name: cloudbase-slo
  rules:
    # Recording rule: the SLI error ratio, computed once, reused everywhere.
    - record: job:slo_errors:ratio_rate5m
      expr: |
        sum(rate(http_requests_total{job="cloudbase-api",status=~"5.."}[5m]))
        / sum(rate(http_requests_total{job="cloudbase-api"}[5m]))

    - alert: CloudBaseErrorBudgetBurningFast
      # 14.4x burn on a 99.5% SLO exhausts 30 days of budget in ~2 days.
      expr: |
        job:slo_errors:ratio_rate1h > (14.4 * 0.005)
        and
        job:slo_errors:ratio_rate5m > (14.4 * 0.005)
      for: 2m
  
```

```
labels:
  severity: page
annotations:
  summary: "CloudBase API burning error budget 14.4x"
  description: "Error ratio {{ $value | humanizePercentage }} over 1h and 5m."
  runbook_url: "https://runbooks.internal/cloudbase/error-budget"
```

How does it work internally? (why symptoms, not causes) There are hundreds of possible causes of a bad user experience and only a handful of symptoms. Alerting on causes — CPU at 90%, a full disk on one node, a pod restart, a slightly elevated GC pause — produces an alert per cause, most of which are individually harmless because the system is designed to tolerate them. That is how you get an on-call rotation with forty pages a week, of which two mattered. Alerting on symptoms — error rate, latency, saturation of a user-facing queue — produces one alert per *actual user impact*, and the cause is then a diagnosis you perform with the dashboards and logs you already built.

Reliability principle

Alert fatigue is not a personality flaw — it is a predictable systems failure with a measurable onset. Once an on-call engineer has learned that most pages are noise, their response time to *all* pages degrades, including the real one. Treat it as an SLO of its own: track pages per on-call shift, and the fraction of pages that resulted in an action. If more than roughly a quarter of your pages are closed with "no action needed", the alerting configuration is broken and fixing it is higher-priority production work than any feature. Every alert must satisfy three tests: it is *urgent* (cannot wait until morning), it is *actionable* (a human can do something), and it points to a *runbook*. Anything failing a test becomes a ticket or a dashboard panel, not a page.

Cloud cost optimization

Observability spend has a nasty shape: it grows with your fleet, not with your revenue, and it is invisible until the invoice. The three levers, in order of effect: drop unused high-cardinality labels at the collector (the biggest single win, usually 30–60% of metric volume); set log retention by level, keeping ERROR for 90 days and DEBUG for 3; and sample traces aggressively while always keeping the errors and the slow tail. A useful discipline is to review the top ten metrics by series count monthly — the list is almost always dominated by one or two accidental labels nobody has ever queried.

Best practices — what experienced engineers do

- Alert on symptoms tied to a user-facing SLO — never on every metric that crosses a threshold.
- Compute percentiles from summed histogram buckets; never average a p99 across pods or windows.
- Generate a correlation ID at the edge, propagate it via `traceparent`, and inject it into every log line automatically.
- Log JSON to stdout; let a node agent ship it. Never write log files inside a container.
- Use the route *template* in labels, never the resolved path, user ID or error string.
- Give every paging alert a runbook URL in its annotations before it is allowed to page anyone.
- Make `up == 0` and "no data" alert conditions too — a silent monitoring system looks exactly like a healthy one.

✖ Common mistakes beginners make

- Labelling a metric with something unbounded, then wondering why Prometheus was OOMKilled.
- Reporting average latency, hiding the tail where the actual user pain lives.
- Free-text logs with no correlation ID, making a cross-service incident unreconstructable.
- Building dashboards nobody opens — metrics collected but never consumed.
- Paging on CPU, memory or pod restarts, then ignoring the pager within a month.
- Setting an SLO of 100%, which makes the error budget zero and the whole framework meaningless.
- Applying `rate()` to a gauge, or using a range window shorter than 4x the scrape interval.

4 Visualizations to Internalize

Redraw these from memory. If you can draw the scrape loop and the burn-rate alert without notes, you can design an observability stack in a whiteboard interview.

 Redraw: service discovery → scrape loop → TSDB, and where the *up* metric comes from

 Redraw: a request crossing three services, showing where the trace ID is generated and every place it appears

 Write out: the SLI, SLO, error budget and burn-rate alert for a service you have worked on

5 Guided Hands-On Lab — Make CloudBase Observable

Goal. Take the CloudBase API running on the Day 10 infrastructure and give it the three signals, one dashboard and one alert that make it operable by someone who has never seen the code.

1 Expose a `/metrics` endpoint with the right instruments

Add a client library and register exactly three things: a counter `http_requests_total` labelled by route template, method and status; a histogram `http_request_duration_seconds` with buckets straddling your intended SLO; and a gauge for in-flight requests. Then `curl` the endpoint and read the raw text.

 **Why:** Reading the exposition format by hand demystifies the entire system — you see that "supporting Prometheus" means serving a text file, and you can immediately count how many series each metric will create.

2 Audit your own cardinality before deploying

Multiply out every label's possible values. Routes $\times$ methods $\times$ status codes $\times$ histogram buckets. If any label can take an unbounded value, remove it now.

 **Why:** Cardinality is the one mistake that takes down the monitoring system for everyone, not just you. Doing this arithmetic before merge is exactly the CI check Cloudflare built — you are doing it manually, once, to learn what it feels like.

3 Deploy Prometheus with Kubernetes service discovery

Configure the `kubernetes_sd_configs` pod role and use `relabel_configs` to keep only pods carrying a `prometheus.io/scrape` annotation. Delete a pod and watch the target list change on its own.

 **Why:** This is the moment the pull model justifies itself. You never enumerate a pod IP; the cluster's own API is the source of truth, so a Day 6 rolling update needs no monitoring change at all.

4 Verify the scrape, then break it deliberately

Check the Targets page, then change the container port so the scrape fails. Query `up{job="cloudbase-api"}` and confirm it returns 0.

 **Why:** You are proving that failure is observable as data. In a push system this failure is silence, and silence is indistinguishable from health — which is how outages go unnoticed for hours.

5 Build the RED dashboard in Grafana

Three panels, in this order: request rate by route, error ratio as a percentage, p99 latency computed with `sum by (le)` inside `histogram_quantile`. Add a fourth panel showing the average latency next to

the p99.

 **Why:** Putting the average beside the p99 under load is the single most convincing demonstration in this chapter. Generate a few slow requests and watch the average barely move while the p99 doubles.

6 Add structured JSON logging with a correlation ID

Switch the log encoder to JSON. Add middleware that reads `traceparent`, generates an ID when absent, stores it in request context, and injects it into every log line and every outbound HTTP header.

 **Why:** The value is entirely in the propagation, not the format. A correlation ID that stops at the first service boundary is decoration; one that survives three hops is a join key.

7 Ship logs to Loki and query one request end to end

Deploy Promtail (or Fluent Bit) as a DaemonSet with the pod's namespace, app and level as labels — and the correlation ID deliberately *not* as a label. Then find one request across both services with a single query.

 **Why:** You are internalising the Loki bet: labels narrow the search space cheaply, the filter expression greps the compressed chunks. Making the correlation ID a label would create one stream per request and defeat the entire design.

8 Define one SLO and one burn-rate alert

Write the SLI as a recording rule, pick a 99.5% target, and write the multiwindow burn-rate alert from section 3.6. Give it a `runbook_url` annotation pointing at a document you actually write.

 **Why:** Recording rules pre-compute the expensive expression so the alert evaluation is cheap and every dashboard uses the identical definition. And an alert without a runbook is a request for a stranger to improvise at 3am.

Production tip

Test the alert by causing the condition, not by editing the threshold until it fires. Inject a 20% error rate with a feature flag or a fault-injection sidecar and confirm the page arrives, the annotations render usefully, and the alert *resolves* on its own afterwards. An alert nobody has ever seen fire is an untested code path in the most safety-critical part of your system — and the classic failure is discovering during a real incident that the alert fires correctly but never clears, so the next incident is masked.

6 Thinking Exercises

Think it through

Prometheus stores data on local disk with no replication, and a server that dies loses its data. Yet it is the default choice at companies operating hundreds of thousands of machines. What property are they buying that outweighs durability — and what does that tell you about how the reliability of a monitoring system should differ from the reliability of the system it monitors?

 Think it through

Your team's SLO is 99.9% and you have consumed only 4% of the monthly error budget with a week to go. An engineer proposes shipping a risky migration *because* of the surplus. Is that a correct use of the error budget or an abuse of it? What does your answer imply about whether the SLO was set at the right level?

 Think it through

Tail-based trace sampling keeps the slow and failed traces — exactly the ones you want — but requires buffering every span of every request until the trace is complete. Head-based sampling decides at the edge and is nearly free, but discards most of the interesting requests. Where does that trade-off actually sit for a service doing 50,000 requests per second, and what hybrid would you build?

7 Debugging Practice Debugging Lab • Prometheus was OOMKilled an hour after a routine deploy

Symptom. The monitoring pod entered `CrashLoopBackOff` with exit code 137 (Day 1: $128 + 9 = \text{SIGKILL}$). Grafana shows a wall of "no data". The application deploy that preceded it was reviewed and looked harmless.

`kubectl describe pod prometheus-0` and a TSDB status query

```

Last State:   Terminated
Reason:       OOMKilled
Exit Code:    137
Memory Limits: 8Gi

# topk(5, count by (__name__)({"__name__=~".+"}))
http_requests_total          4812004
http_request_duration_seconds_bucket  291177
container_memory_working_set_bytes    8140
node_cpu_seconds_total          2304

```

- ✓ **Observe.** One metric holds 4.8 million series while everything else is in the thousands. Memory, not disk, is what failed — and Prometheus memory scales with *active series*.
- ✓ **Hypothesise.** A label on `http_requests_total` started taking unbounded values in the new

build.

- ✓ **Investigate.** Inspect the label sets: `count by (route) (http_requests_total)`. The `route` label now contains resolved paths — `/orders/4821`, `/orders/4822` — instead of the template `/orders/{id}`.
- ✓ **Root cause.** A refactor replaced the framework's route-template accessor with the raw request URI. Every distinct order ID now creates a permanent-for-three-hours time series, and any client requesting random paths can create more without limit.
- ✓ **Fix.** Restore the template accessor and redeploy. To recover immediately, add a `metric_relabel_configs` drop rule at the scrape config so the bad series are discarded at ingestion, then restart Prometheus to clear the head block.
- ✓ **Prevent.** Add a CI check that scrapes the built image's `/metrics` under synthetic load and fails the build if any metric exceeds a series budget. Set `sample_limit` and `label_limit` on the scrape config so a single misbehaving app degrades instead of taking down monitoring for every team — the graceful-degradation principle Cloudflare built into their fork.

🔧 Debugging Lab • The dashboard says p99 is 120 ms; users say the app is unusable

Symptom. Support tickets describe multi-second waits on checkout. The latency panel is flat and green all week. No alert has fired.

the panel query, and the same data queried correctly

```
# Panel query (as built by a well-meaning engineer)
avg(histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m])))
-> 0.118

# Same window, aggregating buckets BEFORE the quantile
histogram_quantile(0.99,
  sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
-> 3.940

# And per pod:
{pod="api-7d9f-xk2p"} 0.104
{pod="api-7d9f-m4kv"} 0.098
{pod="api-7d9f-q8zt"} 4.021 # <-- one pod
```

- ✓ **Observe.** Two queries over identical data differ by a factor of 33. The panel averages a per-pod quantile; the correct form sums buckets first.
- ✓ **Hypothesise.** `avg` of a percentile is not a percentile. One slow pod is being diluted by two fast ones, so its pain is arithmetically erased from the graph.
- ✓ **Investigate.** Break the query down per pod. One replica is at 4 s. Check it: it is scheduled on a node with a saturated disk, or it lost its connection-pool warm-up, or it is the only replica on an older instance type.
- ✓ **Root cause.** Two compounding faults — a genuinely degraded replica, and a dashboard query whose aggregation order guaranteed nobody would ever see it. Approximately a third of traffic was affected the whole time.
- ✓ **Fix.** Correct the query to `histogram_quantile(0.99, sum by (le) (rate(...)))`. Then address the replica: cordon the node, let the Day 6 scheduler reschedule, verify recovery.
- ✓ **Prevent.** Alert on the SLI ratio (requests under 300 ms as a fraction of all requests), which

is immune to aggregation-order mistakes because it counts events rather than averaging statistics. Add a per-pod latency heatmap so a single bad replica is visible as a stripe rather than dissolved into an average.

8 Mini-Labs

Mini-Lab • Beginner

40 min

Goal. Instrument a small HTTP service and scrape it with a local Prometheus.

Business context. Your team has no metrics at all and needs a first, credible dashboard this week.

Architecture. One app container exposing `/metrics`, one Prometheus container, a static scrape config.

Requirements. A counter labelled by route template and status; a latency histogram; both visible in the Prometheus expression browser.

Constraints. No label may take more than 20 distinct values — prove it by enumeration before you deploy.

Hints. Generate load with a loop of `curl`; set the scrape interval to 5 s so graphs move while you watch.

Expected outcome. `rate(http_requests_total[1m])` produces a sensible graph and `up` equals 1.

Production considerations. Decide now what your histogram buckets should be, given the latency you intend to promise.

Common pitfalls. A range window shorter than 4x the scrape interval, producing empty graphs; reading a counter's raw value instead of its rate.

Mini-Lab • Intermediate

70 min

Goal. Propagate one correlation ID across a two-service call chain and reconstruct a single request from logs.

Business context. An intermittent 502 appears at the edge; the downstream team insists their service is healthy.

Architecture. Service A calls service B; both log JSON to stdout; a log collector ships both streams.

Requirements. A generates the ID when `traceparent` is absent, forwards it downstream, and every log line in both services carries it. One query returns the full story of one failing request.

Constraints. The ID must be injected by middleware, not passed by hand to each log call.

Hints. Store it in request-scoped context and register a log-field hook; the correlation ID is a field, never a Loki label.

Expected outcome. A written incident narrative for one request, assembled purely from queries.

Production considerations. Where does propagation break — async work, message queues, thread pools, retries?

Common pitfalls. Losing the context across an async boundary; making the ID a stream label and creating millions of streams.

 Mini-Lab • Production-style

2 hours

Goal. Define an SLO for CloudBase and build a multiwindow burn-rate alert that survives a chaos test.

Business context. The team currently gets 30 pages a week and has started muting the channel.

Architecture. Recording rule for the SLI → burn-rate alert rules → Alertmanager routing → one notification channel.

Requirements. One SLI (availability or latency), one SLO with an explicit window, a fast burn-rate rule at `severity: page` and a slow one at `severity: ticket`, each with a runbook URL.

Constraints. Exactly one alert may page. Everything else routes to a ticket. Justify each removal from the existing set in writing.

Hints. Start from the Google SRE Workbook's 2/5/10% budget-consumption thresholds; use Alertmanager inhibition so a page suppresses its dependent tickets.

Expected outcome. Injected errors produce exactly one page; a 60-second blip produces none; the alert resolves automatically.

Production considerations. Who receives the page, what the escalation is after 15 minutes, and what happens when Prometheus itself is down (a dead-man's-switch alert that fires when it stops being sent).

Common pitfalls. Alerting on the raw error count instead of the ratio; forgetting that "no data" is not "no errors".

9 Engineering Notes

Operational excellence

The best measure of an observability stack is not coverage — it is **time to first useful hypothesis** during an incident. Time yourself in a game day: from page received to a stated, checkable hypothesis. If it exceeds five minutes, the problem is almost never missing data; it is that the data is not organised around the question a responder actually asks first, which is "who is affected and by how much?"

Scalability note

Cloudflare's architecture is the scaling lesson: they run 916 *small* Prometheus servers rather than a few large ones, then federate for global queries. A single Prometheus is vertically limited by memory, which is bounded by active series — so the scaling unit is a shard, usually per cluster or per region, with a query layer (Thanos, Mimir, Cortex) fanning out. Design for that shape from the start: never let a query depend on all your metrics living in one process, because one day they will not.

Disaster recovery tip

Your monitoring must not share a failure domain with what it monitors. Prometheus running inside the cluster it watches cannot alert you that the cluster is gone. The standard answer is a dead-man's switch: an alert rule that fires *continuously* into an external service configured to notify you when the stream *stops*. It is the only alert whose absence is the emergency, and it is usually the first thing missing from an otherwise excellent setup.

 Automation tip

Alert rules are code and belong in the Day 9 pipeline. Cloudflare wrote `pint` specifically to lint Prometheus rules in CI — catching queries that reference metrics which no longer exist, selectors that match nothing, and syntax that is valid but semantically dead. A rule referencing a renamed metric silently never fires, which is worse than a rule that errors, because it looks exactly like a system that is healthy. Lint your rules; unit-test them with `promtool test rules`.

 Interview insight

Expect: *"Walk me through how you would debug a latency spike in a microservice architecture."* A weak answer lists tools. A strong answer names the sequence and the reason for each step: check the SLO burn rate to size the impact; check the RED dashboard to find which service and route; check whether it is all requests or a tail; pull a trace for a slow request to find the hop; read the logs for that correlation ID at that hop; then form the hypothesis. You are demonstrating that you narrow the search space before you start reading text.

 Common misconception

"We should collect everything and figure out what we need later." Storage is cheap; *cardinality* is not, and neither is the human attention required to navigate a thousand dashboards. Teams that instrument everything reliably end up with worse observability than teams that instrument the four golden signals well — latency, traffic, errors, saturation — because the signal is buried. Start narrow, and add a metric only when you can name the question it answers.

10 Build-Along Project — CloudBase

Day 1 gave CloudBase a hardened host; Day 4 containerised it; Days 5–6 put it on Kubernetes with rolling updates and probes; Day 7–8 declared its infrastructure and configuration; Day 9 built the pipeline that ships it; Day 10 put it on real cloud infrastructure. Today CloudBase stops being a black box.

 Build-Along • Day 11 — make CloudBase observable

- Instrument the API: `/metrics` exposing a request counter (route template, method, status), a latency histogram with buckets straddling 300 ms, and an in-flight gauge. Record your cardinality arithmetic in `docs/metrics.md`.
- Deploy Prometheus into the Day 6 cluster using Kubernetes service discovery, with `sample_limit` and `label_limit` set so no single app can take monitoring down.
- Scrape the cluster too: `node-exporter` as a DaemonSet and `kube-state-metrics`, so CloudBase has both application and infrastructure signals in one query language.
- Build one Grafana dashboard, `CloudBase / API — RED`, with request rate, error ratio and correctly-aggregated p99 — plus an average-latency panel beside the p99 as a permanent reminder of why you use percentiles.
- Switch logging to JSON on stdout, add correlation-ID middleware reading and emitting `traceparent`, and ship logs with a Promtail DaemonSet labelled by namespace, app and level.
- Define exactly one SLO — 99.5% of requests non-5xx and under 300 ms over 30 days — as a

recording rule, and one multiwindow burn-rate alert at `severity: page` with a `runbook_url`.

- Write `docs/runbooks/error-budget-burn.md`: what the alert means, the first three queries to run, and the two most likely causes.
- Add a dead-man's-switch alert so the failure of monitoring itself is detectable, and commit every rule file to the Day 9 pipeline with `promtool check rules` in CI.

Definition of done: injecting a sustained 20% error rate into CloudBase produces exactly one page within five minutes, carrying a runbook link; a 60-second blip produces none; the alert resolves by itself; and using only the dashboard and a Loki query you can name the affected route and produce the full log trail of one failing request by its correlation ID — without SSH-ing into anything.

11 Chapter Summary

Key takeaways

- ✓ Metrics say *that* something is wrong; traces say *where*; logs say *what*. You need all three, joined by shared identifiers.
- ✓ Prometheus pulls over HTTP, discovers targets from the Kubernetes API, and stores series in a local TSDB whose memory scales with *active series count*.
- ✓ Cardinality — not sample volume — is what kills a monitoring system. Never label with an unbounded value.
- ✓ Counters need `rate()`; gauges are read directly; histograms store cumulative buckets so percentiles can be computed *after* aggregation.
- ✓ `avg(p99)` is not a percentile. Always `sum by (le)` before `histogram_quantile`.
- ✓ An SLO plus an error budget turns reliability from an opinion into a number, and a burn-rate alert is how that number reaches a human.
- ✓ Alert on symptoms with runbooks; everything else is a ticket or a dashboard panel.

Things to remember

- 99.5% over 30 days is approximately 3h 39m of error budget; 99.9% is approximately 43 minutes.
- Burn rate 14.4 exhausts a 30-day budget in about two days — the canonical fast-page threshold.
- The range window in `rate()` must be at least 4x the scrape interval.
- `up == 0` is a metric. "No data" is not "no errors" — alert on both.
- A correlation ID is a log *field*, never a Loki label.

Common pitfalls (last look)

- Unbounded labels • `avg` of a percentile • alerting on CPU
- Log files inside containers • an SLO of 100% • an alert with no runbook

📄 One-Page Cheat Sheet

Three pillars — metrics = is it broken; traces = which hop; logs = what happened. Joined by trace/correlation ID.

Pull model — server scrapes targets it discovered itself; failure is visible as `up == 0`.

Cardinality — series = unique label-value combination; memory scales with active series, not samples.

Metric types — counter (rate it) · gauge (read it) · histogram (bucket it, quantile at query time).

RED / USE — Rate-Errors-Duration for services; Utilisation-Saturation-Errors for resources.

Percentiles — `histogram_quantile(0.99, sum by (le) (rate(x_bucket[5m])))` — sum before quantile.

SLO maths — budget = $(1 - \text{SLO}) \times \text{window}$; burn rate = actual error ratio / budgeted error ratio.

Alerting — symptom + urgent + actionable + runbook. Anything else is a ticket.

CLI quick reference

Day 11 commands and queries worth memorising

```
# see exactly what Prometheus sees
curl -s localhost:8080/metrics | head -40
curl -s localhost:8080/metrics | grep -c '^http_requests_total'

# validate and unit-test rules before they ever reach production
promtool check config prometheus.yml
promtool check rules rules/*.yaml
promtool test rules tests/slo_test.yaml

# find your worst cardinality offenders (expression browser)
topk(10, count by (__name__)({__name__=~".+"}))
count(count by (route) (http_requests_total))

# the RED three
sum by (route) (rate(http_requests_total[5m]))
sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))
histogram_quantile(0.99, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))

# is anything even being scraped?
up{job="cloudbase-api"}
sum(scrape_samples_scraped) by (job)

# Loki: narrow by label, then filter the line body
{namespace="cloudbase",app="order-api"} |= "a1b2c3d4" | json | level="ERROR"
```

👤 Interview questions to master

- › What is the practical difference between metrics, logs and traces — what can each answer that the others cannot?
- › Why is p99 latency more useful than average latency for finding real problems?
- › What is an error budget, and how should it change a team's priorities?
- › Why does a correlation ID matter more as you add services?
- › Counter vs gauge vs histogram — when would you use each?
- › How would you design alerting that does not cause fatigue?

- › Why does Prometheus pull rather than push, and when does that model break down?
- › What is cardinality explosion, how does it happen, and how would you prevent it in CI?
- › Why can you not average a p99 across pods?

Further reading & official docs

- Prometheus documentation — prometheus.io/docs, especially the FAQ's honest treatment of pull vs push and its statement that Prometheus is *not* a log aggregation system.
- "Announcing Prometheus 3.0" (prometheus.io blog, November 2024) — UTF-8 metric names, Remote-Write 2.0, native histograms.
- Google SRE Workbook, "Alerting on SLOs" — sre.google/workbook/alerting-on-slos — the source of multiwindow, multi-burn-rate alerting.
- Google SRE Book, "Monitoring Distributed Systems" — the four golden signals.
- "How Cloudflare runs Prometheus at scale" (blog.cloudflare.com) — 916 instances, 4.9 billion series, and the cardinality controls that make it survivable.
- Grafana Loki architecture — grafana.com/docs/loki/latest/get-started/architecture — why indexing labels instead of full text changes the cost profile.
- W3C Trace Context Recommendation — w3.org/TR/trace-context — the `traceparent` header format.
- OpenTelemetry specification and semantic conventions — opentelemetry.io/docs — vendor-neutral instrumentation.
- "Evolving Distributed Tracing at Uber Engineering" (uber.com blog) — the local-agent architecture behind Jaeger.

Production checklist

- ☐ Every service exposes `/metrics` with a request counter and a latency histogram.
- ☐ No metric label can take an unbounded value; cardinality is checked in CI.
- ☐ Scrape configs set `sample_limit` and `label_limit` so one app cannot break monitoring for all.
- ☐ Every service has a RED dashboard with the same panels in the same order.
- ☐ Logs are JSON on stdout, shipped by a node agent, carrying a correlation ID from the edge.
- ☐ At least one SLO exists, is written down, and has an agreed error-budget policy.
- ☐ Paging alerts are symptom-based, burn-rate driven, and every one has a runbook URL.
- ☐ A dead-man's-switch alert detects the failure of monitoring itself.
- ☐ Alert rules are version-controlled, linted and unit-tested in the pipeline.

Key idea

What you will learn next (Day 12). You can now see what your system is doing. Tomorrow you decide who is allowed to do it: least-privilege IAM, secrets that never touch a Git repository, image and dependency scanning in the pipeline, and the security controls that turn a working platform into one you would let a stranger audit. The instinct is the one you met with `chmod 777` on Day 1 — grant to an identity, not to everyone — moved up to the cloud control plane.

DAY 12 • SECURITY

Security & Secrets Management

Least privilege, secrets, image scanning and pipeline supply chain

12

🕒 Estimated study time: 8 hours

🎯 By the end of this day, you will be able to...

- Apply least privilege as a mechanism — IAM policy evaluation, Kubernetes RBAC, pipeline tokens — not as a slogan.
- Explain precisely what a secret is at rest, in transit and in memory, and why base64 in a Kubernetes Secret is encoding rather than encryption.
- Issue short-lived dynamic credentials from a secrets manager instead of shipping long-lived static ones.
- Gate a pipeline with three distinct scans — SAST, dependency (SCA) and image — and say what each one alone cannot see.
- Contain lateral movement in a cluster with default-deny NetworkPolicies, and run a leaked-credential incident correctly.

1 The Big Picture

What problem this solves. Everything you built in the previous eleven days assumed the participants were honest. The Terraform state, the registry, the CI runner, the flat pod network, the database password in a `ConfigMap` — each is a good design right up to the moment one credential leaks or one dependency turns malicious. Today's controls decide whether that moment is an inconvenience or a breach. The goal is narrow: *shrink what any single compromised identity can reach, and shorten how long any credential stays valid.*

Why DevOps engineers need it. Security is not a separate team's deliverable arriving after the platform is built. Every control today is an infrastructure decision — an IAM policy document, a Kubernetes object, a pipeline stage, a Vault role. If the platform engineer does not encode them, nobody else can. The industry name is **shifting left**: the cheapest place to stop a vulnerable image is the build, not the runtime.

🔑 Key idea

There is no such thing as "secure". There is only *blast radius* and *time-to-invalid*. Assume every credential you issue will leak, every image you build will contain a CVE, and every pod will be compromised — then design so each of those costs a rotation and a redeploy, not a breach. That assumption, not any single tool, is what separates a senior architecture from a junior one.

Where it fits in a modern cloud architecture. Security is not a layer in the stack; it is a set of enforcement points *distributed through* the stack you already built. Read the diagram as "one control per boundary you have already crossed in this book".

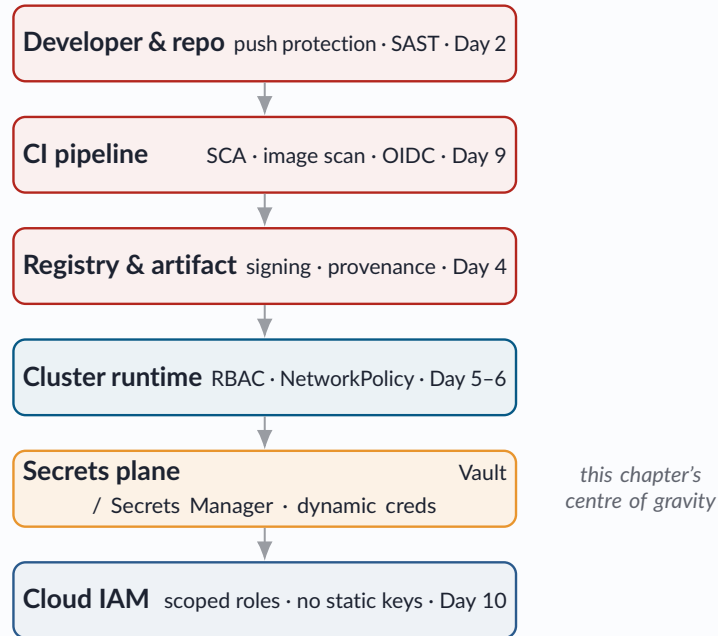


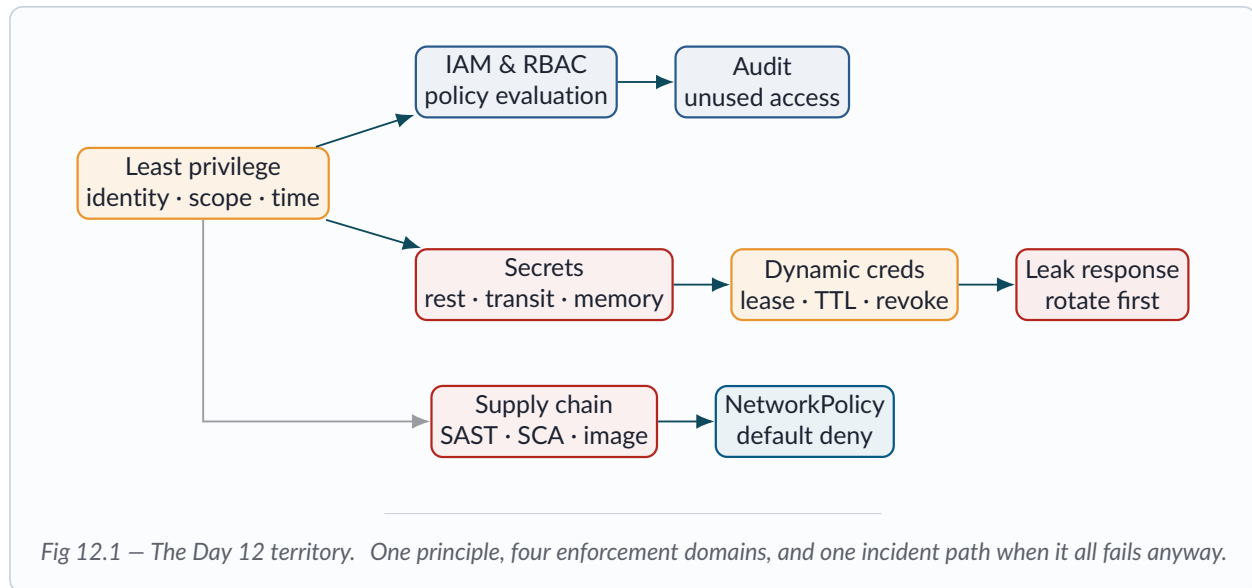
Fig 12.0 — Today's controls are enforcement points on boundaries you already built. Each one limits a different blast radius.

How it connects to previous chapters. Day 1's `chmod 777` warning was least privilege on one host; the enforcement point now moves to the cloud control plane. Day 2's immutable Git object graph is exactly why a deleted secret is still there. Day 4's image layers are why a base image ships CVEs you never wrote. Day 6's RBAC and Day 10's IAM are the policy engines you will audit, Day 9's pipeline is where the gates go, and Day 11's observability is how you *detect* what these controls contain.

Where companies use it in production. Netflix describes an internal "paved road" for application security: rather than asking every team to solve authentication, secret storage and TLS certificates individually, the security organisation ships centralised, secure-by-default platforms so product engineers inherit the correct behaviour without choosing it. Its edge architecture takes the same shape — external tokens terminate at the Zuul gateway, which converts them into an internal, cryptographically verifiable identity object propagated onward, so no downstream service ever handles a customer credential. GitHub runs secret scanning with **push protection**, blocking a push containing a recognised credential pattern before it lands in history. Shopify's hybrid-operations guidance is to centralise secrets behind one retrieval interface with different back ends per environment, so credentials stop being scattered across environment files and deploy scripts.

Common misconceptions. That a Kubernetes Secret is encrypted (it is base64-encoded, and by default stored in plaintext in etcd); that deleting a committed secret in a follow-up commit removes it (it does not, and the secret is already burned); that "we scan our dependencies" covers the supply chain (it misses everything in the base image); that a private cluster network is a security boundary (by default every pod can reach every other pod).

2 Concept Map



Terminology to anchor: *least privilege* → *blast radius* → *static vs dynamic credential* → *lease / TTL* → *envelope encryption* → *SBOM* → *provenance* → *lateral movement* → *defense in depth*.

3

Guided Learning

3.1 Least Privilege as a Mechanism, Not a Slogan

What is it? Every identity — human, service account, CI job, pod — holds exactly the permissions it needs, on exactly the resources it needs, for no longer than it needs them. Three axes: *who*, *what*, *how long*. Most teams remember the first, approximate the second, and ignore the third.

Why does it exist? Before identity systems, access control was the Unix model of Day 1: a handful of users and groups, and `root` for anything awkward. That scaled to one machine, not to a thousand ephemeral ones, so operators did the obvious thing — they created one admin credential and shared it. Every large breach post-mortem of the last two decades contains a sentence of the form "the attacker obtained credentials for X and, because those credentials were broadly scoped, was able to reach Y". Least privilege exists to make that second clause impossible to write.

How does it work internally? You cannot audit a policy engine you cannot simulate in your head. Both engines you use are *deny by default with explicit allow*, but they differ crucially.

AWS IAM evaluates a request against every applicable policy — identity, resource, permission boundaries, SCPs, session policies — with fixed precedence: an **explicit deny anywhere wins immediately**; otherwise the request needs an **explicit allow** in every applicable category; otherwise the implicit deny stands. This is why an organisation-level SCP makes an account-level `Allow` irrelevant, and why "but I attached `AdministratorAccess`" does not always work.

Kubernetes RBAC is smaller and purely additive: there is *no deny rule* at all. A request is allowed if *any* `RoleBinding` or `ClusterRoleBinding` grants the (verb, resource, namespace) triple to the subject. Because there are no denials, you cannot carve out an exception — the only way to reduce a subject's power is to stop granting it. That single fact is why `cluster-admin` bound to a service account is unrecoverable without editing the binding.

Property	AWS IAM	Kubernetes RBAC
Explicit deny	Yes — always wins	Does not exist
Combination	Intersection across policy types	Pure union of all bindings
Scope unit	ARN + condition keys	namespace + resource + verb
Reducing access	Add a deny, boundary or SCP	Remove or narrow the binding

Behind the scenes

Kubernetes has a second, quieter authoriser in front of RBAC: the **Node authoriser**, which restricts what a kubelet may read — roughly, only the Secrets and ConfigMaps mounted by pods scheduled on *that node*. Without it, one compromised node could read every Secret in the cluster. This is also why bound, audience-scoped ServiceAccount tokens replaced the old forever-valid tokens that used to be auto-mounted into every pod.

When should I use it? Always, but the practical entry point is *new grants*. Retro-fitting least privilege across an existing account is a multi-month project; making every new role start from an empty policy and grow by observed need costs nothing.

When should I not? Do not hand-craft resource-level policies for a three-person prototype in a sandbox with no customer data — you will spend the week debugging `AccessDenied`. Use a broad role there, then let tooling generate the tight policy from the record of what the workload actually did. Observe, then constrain.

Production example. AWS's own recommended path is mechanical rather than heroic. IAM Access Analyzer runs two distinct analyses: *policy generation*, which reads CloudTrail logs for a principal over a chosen window and emits a policy containing only the actions actually used; and *unused access findings*, which continuously flag unused roles, unused access keys and, for active principals, untouched services and actions. Large organisations run the second as a scheduled report and treat "unused for 90 days" as an automatic ticket. The insight worth stealing: least privilege becomes tractable the moment it is driven by *measured* access rather than by a design meeting.

Security warning • the wildcard that looks harmless

`iam:PassRole` with `Resource: "*"` is one of the most consequential grants in AWS, and it appears in countless "CI deploy" policies. It lets the principal hand *any* role to *any* service — so a pipeline that can create a Lambda function and pass an administrator role has, in effect, administrator. Always scope `PassRole` to specific role ARNs and add an `iam:PassedToService` condition. The same class of bug in Kubernetes is granting `create` on `pods` in a namespace whose service account is powerful: pod creation is arbitrary code execution as that identity.

3.2 What a Secret Actually Is: at Rest, in Transit, in Memory

What is it? A secret is any value whose disclosure grants access — a password, an API token, a TLS private key. The engineering question is never "is it secret?" but "in which of its three states is it exposed, and to whom?"

Why does it exist as a separate discipline? Configuration and secrets look identical — both are strings in a file — but have opposite lifecycles. Configuration should be versioned, diffed, reviewed and copied freely; a secret must be none of those. Teams that treat them as one category end up with credentials in Git, because Git is where configuration correctly belongs.

How does it work internally? — the three states.

State	Where it lives, and the real threat	The control that helps
At rest	Disk, etcd, a backup, an S3 object, a laptop. Threat: anyone who reads the storage or restores the backup.	Envelope encryption with an external KMS; short TTLs so old copies are useless.
In transit	Hops between the store, the API server, the kubelet and the pod. Threat: interception, a rogue proxy.	TLS everywhere; mTLS between control-plane components.
In memory	The heap, environment variables, <code>/proc</code> . Threat: a crash dump, a log line, an exception trace, any child process.	Files with <code>0400</code> over env vars; never log the value.

The base64 point, precisely. A Kubernetes Secret's `data` field is base64. Base64 is a *transport encoding* that maps arbitrary bytes to a safe ASCII alphabet — it takes no key, and `base64 -d` reverses it in one command. It exists so binary values (a TLS key, a certificate) survive YAML. It provides zero confidentiality. What a Secret really buys you over a ConfigMap is that it is a distinct RBAC resource type (you can grant `get` on ConfigMaps without granting it on Secrets), that the kubelet only delivers it to nodes running pods that reference it, and that it can be selected for encryption at rest.

What encryption at rest actually requires. By default the API server writes Secret objects into etcd unencrypted. To change that you supply an `EncryptionConfiguration` via `--encryption-provider-config`, listing providers for the `secrets` resource. The serious option is the **KMS provider**, which uses **envelope encryption**: the API server generates a unique **data encryption key** (DEK) per resource, encrypts the object with it, and sends the DEK to an external KMS to be wrapped by a **key encryption key** (KEK) that never leaves the KMS. etcd stores the ciphertext plus the wrapped DEK; on read the API server has the KMS unwrap it. The property this buys: an attacker with a raw etcd snapshot holds ciphertext and wrapped keys, and no way to unwrap them without also compromising the KMS and the IAM permission to call it.

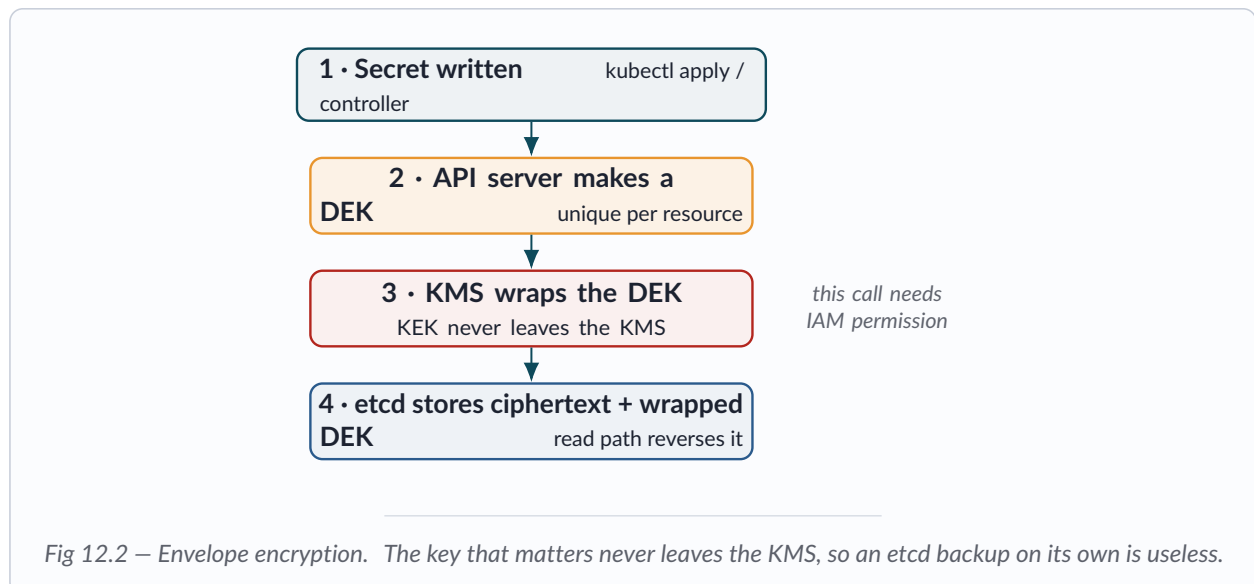


Fig 12.2 — Envelope encryption. The key that matters never leaves the KMS, so an etcd backup on its own is useless.

⚠ Common misconception

"We enabled encryption at rest, so Secrets are safe." Encryption at rest defends against *offline* access to storage — a stolen disk, a leaked backup, a misconfigured snapshot. It does nothing against an attacker who can call the API server, because the API server decrypts on read by design. If a principal has `get secrets` in a namespace, encryption at rest is invisible to them. RBAC, not encryption, is the control for that threat.

secret-vs-reality.yaml

```

apiVersion: v1
kind: Secret
metadata:
  name: cloudbase-db
type: Opaque
data:
  # base64 is ENCODING, not encryption. Anyone with `get secrets` here
  # reads the plaintext: kubectl get secret cloudbase-db \
  #   -o jsonpath='{.data.password}' | base64 -d
  password: c3VwZXJzZWNyZXQxMjM=

```

Security warning • env vars leak in more places than you think

Injecting a secret as an environment variable exposes it to every child process, to `/proc/<pid>/environ`, to crash dumps, to APM agents that snapshot the environment, and — most commonly — to an exception handler that helpfully prints the whole environment into your Day 11 log pipeline. Mounting it as a file with mode `0400` narrows that to one path one process reads once.

3.3 Dynamic Secrets: Making Credentials Expire by Construction

What is it? Instead of storing one long-lived database password and distributing it, the application asks a broker for credentials at startup. The broker *creates a brand-new database user on the spot*, returns it with a lease, and destroys that user when the lease expires. Every workload instance gets a distinct, short-lived identity.

Why does it exist? Static credentials have an unbounded exposure window. A password created in 2021 and stored in a config repo, a CI variable, three laptops and a Slack thread cannot meaningfully be rotated, because nobody knows the distribution list — so rotation never happens and the credential's value to an attacker grows monotonically. Dynamic secrets invert the economics: the useful lifetime is an hour, a leaked credential is usually already dead, and rotation stops being an operation anyone performs.

How does it work internally? HashiCorp Vault's database secrets engine is the canonical implementation and worth understanding mechanically.

- **Connection.** Vault holds one privileged *root* connection. Standard practice is to hand it the root credential once and immediately call `rotate-root`, so from then on no human knows it.
- **Role.** A role maps a name to `creation_statements` — literal SQL with `{{username}}`, `{{password}}` and `{{expiration}}` placeholders — plus a default and maximum TTL.
- **Request.** A client authenticates (in Kubernetes, by presenting its projected ServiceAccount token, which Vault verifies via the TokenReview API) and reads `database/creds/<role>`. Vault runs the creation statements and returns a unique username and password.
- **Lease.** Vault's expiration manager tracks every outstanding lease and, on expiry or revoke, runs the revocation statements to drop the user. The dynamic-role default TTL is one hour.
- **Renew or re-read.** A long-running app renews within `max_ttl`, then requests fresh credentials and reconnects — which is why connection pools need a "credentials changed" path. That is the single most common integration bug.

vault-database-engine.sh

```
# 1. Point the engine at Postgres, then burn the root password
vault write database/config/cloudbase-pg \
  plugin_name=postgresql-database-plugin allowed_roles="orders-ro" \
  connection_url="postgresql://{{username}}:{{password}}@pg:5432/orders"
vault write -f database/rotate-root/cloudbase-pg # now nobody knows it

# 2. A role = the exact SQL Vault runs, plus how long the user may live
vault write database/roles/orders-ro db_name=cloudbase-pg \
  default_ttl=1h max_ttl=8h \
  creation_statements="CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD \
  '{{password}}' VALID UNTIL '{{expiration}}'; GRANT SELECT ON ALL \
  TABLES IN SCHEMA public TO \"{{name}}\";"

# 3. What the workload does - a different database user every single time
vault read database/creds/orders-ro
```

What Vault is doing beneath that. Vault's storage holds only ciphertext. On start it is *sealed*: the key protecting the data is itself encrypted by a master key Vault does not have. Unsealing reconstructs it — classically from Shamir shares held by different people, in production via auto-unseal delegated to a cloud KMS. Same envelope pattern as Kubernetes KMS encryption, one level up: the root of trust sits deliberately outside the system that needs it.

When should I use it? Anything with a database, and any credential whose blast radius exceeds one service. Dynamic credentials also give audit teams something no other approach can: because every workload has a distinct database user, a slow query or a destructive statement is attributable to a specific pod and lease.

When should I not? Self-hosting Vault is a real operational commitment — an HA quorum, unseal key custody, upgrade discipline, and a hard dependency that can stop every deploy when unavailable. For a small AWS-only team, Secrets Manager with rotation Lambdas or SSM Parameter Store with KMS gives most of the benefit with none of the on-call burden. And some third-party APIs simply issue one static key with no rotation API: accept it, store it centrally, scope it as tightly as the vendor allows, and monitor its use.

Approach	How it actually works	Choose it when
Env vars from a store	CI or orchestrator injects at deploy; nothing committed	Baseline; the minimum bar
External Secrets Operator	Controller syncs an external store into native K8s Secrets	Existing charts must work unchanged
Secrets Store CSI driver	Mounts secrets as a volume at pod start, as the pod's ServiceAccount	The value must never become an etcd object
Vault dynamic secrets	Creates and destroys a per-workload DB user under a lease	Rotation must be automatic and attributable
AWS Secrets Manager / SSM	Managed, IAM-integrated, rotation via Lambda	AWS-centric team, low ops overhead

Design trade-off

The Secrets Store CSI driver and the External Secrets Operator solve the same problem with opposite trade-offs. ESO writes a real Kubernetes Secret, so everything that already consumes Secrets keeps working — but the value now lives in etcd, visible to anyone with `get secrets`. The CSI driver mounts it straight into the pod's filesystem, so it never becomes an API object — but the app must read a file, and nothing expecting `envFrom` works unchanged. Compatibility versus exposure surface: pick deliberately, and write down why.

3.4 The Supply Chain: Three Scans That See Three Different Things

What is it? Your artefact is not just your code. A container image is a few hundred lines you wrote, several hundred transitive dependencies you did not read, and an entire OS userland from the base image. Supply-chain security means knowing and gating all three.

Why does it exist? Because attackers moved upstream: compromising one popular build tool or library reaches thousands of organisations at once, where attacking each individually does not scale. Once that arithmetic was obvious, "we review our own code" stopped being a defence.

How does it work internally? The three scan types read three different inputs — which is the whole point.

Scan	Input it reads	What it alone can never see
SAST	Your source, as a syntax tree / data-flow graph	Anything in a dependency or in the OS layer; also blind to runtime config
Dependency (SCA)	The lockfile / dependency graph, matched against CVE databases	OS packages baked into the base image; logic flaws in your own code
Image scanning	The image's package databases across all layers	Your source logic; a vulnerable dependency vendored without a lockfile entry

An image scanner such as Trivy or Gype does not "look for malware". It unpacks each layer, reads the package metadata it finds — the `dpkg/rpm/apk` databases plus language manifests like `package-lock.json` — builds an inventory (a **Software Bill of Materials**, or SBOM), and joins it against vulnerability feeds keyed by package name and version. Two consequences follow. A result is only as fresh as the feed, so *the same image scanned tomorrow can fail* — which is why you re-scan the registry. And a package installed by unpacking a tarball rather than through a package manager is invisible, because it left no metadata.

`ci-security-gates.sh`

```
set -euo pipefail
semgrep ci --config auto # gate 1 - your own source
trivy fs --scanners vuln --severity CRITICAL,HIGH --exit-code 1 . # gate 2

# Gate 3 - the built image, including all the base image dragged in.
# --ignore-unfixed matters: a CVE with no upstream fix cannot be actioned
# today, and blocking on it only teaches the team to bypass the gate.
trivy image --severity CRITICAL,HIGH --ignore-unfixed --exit-code 1 \
  --format sarif --output trivy.sarif "ghcr.io/cloudbase/api:${GIT_SHA}"

# Keep the SBOM as a build artefact: it answers "are we affected?" the day
# a new CVE lands, without rebuilding anything.
```

```
trivy image --format cyclonedx --output sbom.json "ghcr.io/cloudbase/api:${GIT_SHA}"
```

Production example. GitHub's build attestations follow the SLSA provenance model and sign through Sigstore: rather than long-lived signing keys, the CI job's OIDC identity is exchanged at the Fulcio CA for a short-lived certificate, the signature is recorded in the Rekor transparency log, and the private key is discarded. The verification question changes shape — from "do I trust this key?" to "was this artefact built by *that* workflow, in *that* repository, at *that* commit?" It is dynamic credentials applied to signing.

💡 Senior engineer insight

The scanner is not the hard part; installing Trivy takes ten minutes. The hard part is the day it reports 340 findings on an image built from a base you did not choose. Junior response: raise the severity threshold and ship. Senior response: change the input. Moving from a general-purpose base to a slim or distroless one often drops the OS package count — and therefore the CVE count — by an order of magnitude, because most of those packages were a shell, a package manager and a locale database you never used. Fix the supply, not the report.

⚠️ Common misconception

"The build passed the scan, so the image is safe." A scan is a statement about *known* CVEs at *that instant* — nothing about zero-days, your own logic bugs, or tomorrow's disclosures. A passing scan is evidence you are not shipping a *known* problem: genuinely valuable, genuinely limited.

3.5 NetworkPolicies: Containing the Pod You Have Already Lost

What is it? A Kubernetes object restricting which pods may talk to which pods, on which ports, selected by labels — the same mechanism Services use from Day 5.

Why does it exist? Kubernetes' network model mandates that every pod reach every other pod without NAT. That flatness makes discovery and scheduling simple — and lets a compromised front-end pod open a TCP connection to the database, the metrics backend, the cloud metadata endpoint and every other namespace. The pre-Kubernetes control, a security group per subnet tier, does not work when every workload shares nodes and IP space. NetworkPolicy restores segmentation at the identity level rather than the IP level.

How does it work internally? Three mechanics, and beginners get all three wrong at least once.

- **The API object does nothing on its own.** Enforcement lives in the CNI plugin — Calico, Cilium, Antrea. On a cluster whose CNI does not implement NetworkPolicy, the object is accepted and silently has no effect. Calico programs iptables (or eBPF) rules per node; Cilium compiles eBPF programs onto the pod's virtual interface. Either way enforcement is per-node and per-packet, never centralised.
- **Isolation is opt-in per pod, then deny-by-default.** A pod is non-isolated until *some* policy selects it. The moment one selects it with `Ingress` in `policyTypes`, all ingress not explicitly allowed is dropped. Hence the correct pattern: a default-deny with an empty `podSelector`: `{}` first, then narrow allow policies.
- **Policies are additive and unordered.** No deny rule, no precedence — exactly like RBAC. What is permitted is the union of all matching allow rules. You cannot write "allow everything except X".

A fourth mechanic causes more incidents than the other three combined: a default-deny *egress* policy also blocks DNS. CoreDNS is a pod like any other, so every workload instantly fails to resolve

anything, and the symptom looks like a total application outage rather than a policy problem. Every default-deny egress policy needs a companion rule allowing UDP and TCP 53 to the DNS pods.

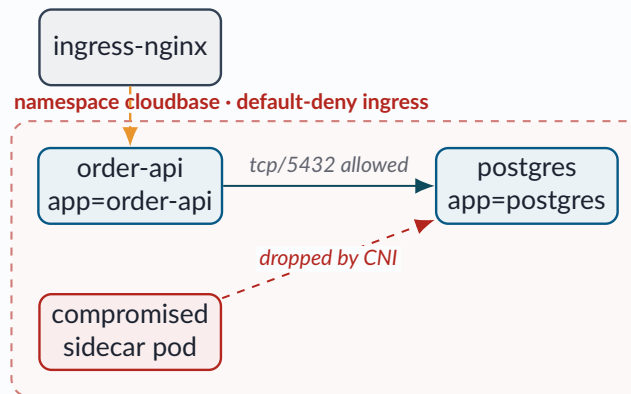


Fig 12.4 — Default-deny then explicit allow. The dashed boundary is enforced by the CNI on each node, not by a firewall appliance.

netpol-default-deny-then-allow.yaml

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: cloudbase
spec:
  podSelector: {}          # every pod becomes ingress-isolated;
  policyTypes: [Ingress]   # no rules at all means allow nothing
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-allow-order-api
  namespace: cloudbase
spec:
  podSelector:
    matchLabels: { app: postgres }
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels: { app: order-api }
      ports:
        - protocol: TCP
          port: 5432
```

Security warning • the two-space bug that opens the whole cluster

In a `from` list, a dash before `namespaceSelector` starts a new element; without it, `namespaceSelector` and `podSelector` combine into one. Two elements mean OR — "pods labelled X here, OR every pod in namespaces labelled Y". One combined element means AND. The wrong indentation silently widens the policy from one deployment to a whole set of namespaces, and `kubectl apply` reports success either way. Verify a policy by *testing a connection that must fail*, never by reading the YAML.

When should I not? Do not hand-write policies for forty microservices on day one; you will produce forty policies nobody can reason about and an on-call engineer who deletes them all at 3am. Start with the boundaries that matter: default-deny in the namespace holding data, an explicit allow for the app tier, and an egress rule blocking pods from reaching the cloud metadata endpoint. Those three cover most real lateral-movement paths. Note the honest limits too: NetworkPolicy is layer 3/4 only — no HTTP paths, no methods — and `hostNetwork: true` pods bypass it entirely. Layer-7 policy is a service-mesh concern.

3.6 When a Secret Leaks: the Only Correct Order of Operations

What is it? An incident procedure. Someone commits a credential and pushes; a token appears in a CI log; a key lands in a Jira attachment. A team is measured by the next fifteen minutes.

How does it work internally? — why deletion does not help. From Day 2 you know Git stores immutable objects addressed by content hash, and a commit merely points at a tree. A follow-up commit that deletes the file creates a *new* tree; the old blob is untouched and remains reachable via the old commit, any tag, every clone, every fork, and every CI cache. On a hosted platform, unreferenced objects can still be fetched by hash until garbage collection runs — and forks may never garbage-collect at all. Therefore: **once pushed, the secret is public. Treat the value as burned.**

The correct order. Rotate first — before rewriting history, before discussing it, before confirming whether anyone actually saw it. Rewriting history is cleanup, and cleanup is worthless while the credential is still valid. Then check the audit log (CloudTrail, Vault audit device, database logs) for use from unexpected sources; then rewrite history with `git filter-repo` and force-push, coordinating with everyone holding a clone; then fix the cause — push protection, a pre-commit hook, or removing the need for a static secret at all.

leaked-credential-response.sh

```
# STEP 1 - ROTATE. Nothing else matters until this is done.
aws secretsmanager rotate-secret --secret-id cloudbase/db/orders
vault lease revoke -prefix database/creds/orders-rw # if it was dynamic

# STEP 2 - Look for abuse between commit and rotation.
aws cloudtrail lookup-events --start-time 2026-07-15T00:00:00Z \
  --lookup-attributes AttributeKey=AccessKeyId,AttributeValue=AKIAEXAMPLE

# STEP 3 - Only now clean history (rewrites every later commit hash).
git filter-repo --path config/prod.env --invert-paths
git push --force-with-lease --all

# STEP 4 - Prevent: make the next one impossible to push at all.
gitleaks protect --staged --redact
```

☰ Operational excellence

Write the runbook before you need it. Under time pressure the on-call engineer does the first thing that feels productive — almost always deleting the file, because it is visible and satisfying. A one-page runbook whose first line is "ROTATE FIRST" in bold is worth more than any tool you could buy: like Day 11's alert runbooks, its value is removing decision-making from the worst possible moment.

✓ Best practices — what experienced engineers do

- Default-deny everywhere: no IAM allow without a resource, no namespace without a default-deny NetworkPolicy.

- Prefer short-lived, dynamically issued credentials; make rotation the system's normal behaviour, not a quarterly project.
- Mount secrets as files with restrictive modes rather than injecting them as environment variables.
- Run SAST, dependency and image scanning as three separate gates, and re-scan images already in the registry.
- Pin base images by digest and shrink the base to shrink the CVE surface — do not raise the severity threshold.
- Use CI OIDC federation to assume a cloud role, never a long-lived access key. Rotate first, investigate second, rewrite history third.

✖ Common mistakes beginners make

- Believing base64 is encryption, or that encryption at rest protects against a principal holding `get secrets`.
- Deleting a leaked secret in a follow-up commit and considering the incident closed.
- Running only dependency scanning and calling the supply chain covered.
- Leaving cluster networking flat, so one compromised pod reaches the database and the metadata endpoint.
- Applying default-deny egress without allowing DNS, then debugging it as an application outage.
- Granting a broad IAM policy under deadline pressure with a `TODO` nobody revisits.

4 Visualizations to Internalize

Redraw these from memory. If you can draw the mechanism, you can defend the design decision in a review.

 Redraw: envelope encryption — write and read path, where the DEK lives, where the KEK lives, and what an attacker with only an etcd snapshot holds

 *Redraw: a namespace with default-deny ingress plus one allow policy. Mark which arrows the CNI drops, and add the egress rule DNS needs*

5 Guided Hands-On Lab — Harden CloudBase End to End

Goal. Take the running, observable CloudBase of Day 11 and remove its three standing weaknesses: static secrets, an unguarded supply chain, a flat pod network. Practise the *decisions*, not the typing.

1 Inventory every secret CloudBase currently holds

Grep the repository, Helm values, CI variables and running manifests. For each secret record: what it grants, who can read it, when it was last rotated, how many copies exist.

 *Why:* You cannot apply least privilege to an inventory you do not have. This step almost always finds a credential nobody knew was live — and "last rotated: never" justifies the rest of the day.

2 Enable encryption at rest, and prove it changes nothing you can see

Configure a KMS provider for the `secrets` resource, then read a Secret through the API as a user holding `get secrets`.

 *Why:* You will see plaintext, exactly as before. Encryption at rest defends storage; RBAC defends the API. Confusing the two is the most common security misconception in Kubernetes.

3 Replace the database password with a dynamic credential, then delete the old one

Configure the secrets engine against CloudBase's Postgres, define a role with a one-hour TTL, have the API obtain credentials at startup via its ServiceAccount identity — then drop the original database user entirely and watch what breaks.

 *Why:* The workload's identity becomes its ServiceAccount token rather than a shared string, which forces you to design what your connection pool does at lease expiry. And the dependency graph of a shared credential is invisible until you cut it: better discovered deliberately at 2pm than during an incident.

4 Add the three scanning gates, then fail the build on purpose

SAST advisory, dependency scanning blocking on CRITICAL and HIGH, image scanning blocking with `--ignore-unfixed`, SBOM retained. Then pin a deliberately old base image, watch the gate block, and fix it by switching to a slim or distroless base.

 *Why:* Blocking on findings with no upstream fix teaches the team to bypass the gate, which is worse than no gate. And the fix for a vulnerability report is usually a change of input, not a change of threshold.

5 Apply default-deny, then the minimum allow set

Default-deny ingress in the CloudBase namespace, then allow ingress-controller to API, API to Postgres on 5432, and everything to DNS.

 **Why:** Building the allow list from an empty set is what makes the policy readable. Starting from allow-all and subtracting is how you get rules nobody dares delete.

6 Prove the isolation, do not assume it

From a throwaway debug pod, open a TCP connection to Postgres — it must fail. From the API pod — it must succeed.

 **Why:** A NetworkPolicy that silently does nothing (wrong labels, wrong namespace, a CNI that does not enforce policy) looks identical to one that works. The only evidence is a connection that fails.

Production tip

Do the final step twice: once with your CNI enforcing policy, once on a cluster that does not enforce NetworkPolicy at all (a plain kind cluster will do). Both accept the object; only one drops the packet. Having seen that failure mode once, you will check for it first every time a policy "does not work".

6 Thinking Exercises

Think it through

Dynamic credentials with a one-hour TTL are strictly better than a static password — until Vault is unavailable, and now no pod can start or reconnect. You have traded a confidentiality risk for an availability risk. How would you quantify that trade, and what would you actually build to make the availability side acceptable?

Think it through

Kubernetes RBAC deliberately has no deny rule; AWS IAM's explicit deny always wins. Both designs are defensible. What does each choice make easy, and what does each make dangerous? Which would you rather audit at 3am, and why?

Think it through

Your image scanner reports 340 vulnerabilities: 12 CRITICAL, 9 of them with no available fix. The release is due tomorrow. What decision do you make, what evidence supports it, and —

most importantly — what do you put in place so this conversation does not recur next month?

7

Debugging Practice

🔧 Debugging Lab • Every pod suddenly cannot resolve anything after a "security improvement"

Symptom. A default-deny egress NetworkPolicy was applied to the `cloudbase` namespace at 14:02. By 14:03 every service is failing with name-resolution errors, not connection refusals. The cluster looks healthy: nodes Ready, pods Running, no restarts.

`kubectl logs deploy/order-api` and `kubectl get netpol`

```
2026-07-19T14:03:11Z ERROR db.pool: dial tcp: lookup pg.cloudbase.svc
.cluster.local on 10.96.0.10:53: read udp 10.244.2.17:41233->10.96.0.10:53:
i/o timeout
```

NAME	POD-SELECTOR	AGE
default-deny-egress	<none>	67s
db-allow-order-api	app=postgres	9d

- ✓ **Observe.** The failure is at DNS resolution and began exactly when an egress policy was applied. Pods are Running — not a scheduling or image problem.
- ✓ **Hypothesise.** The policy made every pod egress-isolated. CoreDNS is an ordinary pod, so traffic to it on port 53 is now dropped like any other egress.
- ✓ **Investigate.** Confirm the policy has `Egress` in `policyTypes` and no `egress` rules at all; from a pod try `nc -zvu 10.96.0.10 53`.
- ✓ **Root cause.** Default-deny egress with no companion DNS allow rule — every lookup is dropped by the CNI before it leaves the pod's interface.
- ✓ **Fix.** Add an egress rule allowing UDP and TCP 53 to the DNS pods, selected by a `namespaceSelector` for kube-system plus the DNS pod labels. Verify resolution from a debug pod.
- ✓ **Prevent.** Ship default-deny egress and the DNS allow rule as one atomic change, and add a smoke test that resolves a service name and fails the rollout if it cannot.

🔧 Debugging Lab • The deployment works in staging and gets AccessDenied in production

Symptom. The CI job assumes a role and calls S3. Identical pipeline, identical policy JSON, works in staging, denied in production. The engineer has already attached a second, broader policy — still denied, which is the clue.

CI step output, production account

```
Run: aws s3 cp ./dist s3://cloudbase-prod-artifacts/build-4831/ --recursive
upload failed: ./dist/app.tar.gz to s3://cloudbase-prod-artifacts/...
An error occurred (AccessDenied) when calling the PutObject operation:
User: arn:aws:sts::4455:assumed-role/cloudbase-ci/gha-4831 is not authorized
to perform: s3:PutObject on resource: "arn:aws:s3:::cloudbase-prod-artifacts
/build-4831/app.tar.gz" with an explicit deny in a service control policy
```

- ✓ **Observe.** The message names the deciding policy type: *an explicit deny in a service control policy*. Adding allows cannot help — that is what "explicit deny" means in IAM evaluation.
- ✓ **Hypothesise.** The production account sits under an OU with an SCP constraining S3 — to approved buckets, prefixes, or a required condition such as encryption or region.
- ✓ **Investigate.** Read the SCPs on the account's OU and the root; re-run the IAM policy simulator against the production principal; compare with staging's OU.
- ✓ **Root cause.** A guardrail SCP requires `s3:x-amz-server-side-encryption` on `PutObject`; the upload does not set it, so the SCP denies regardless of identity policy.
- ✓ **Fix.** Set the encryption header (or enable bucket default encryption and match the condition). Then remove the broader policy attached while debugging — it grants real access and fixed nothing.
- ✓ **Prevent.** Make guardrails discoverable: document them, and run `simulate-principal-policy` as a pre-merge check so the denial surfaces in review, not in a production deploy.

8**Mini-Labs****Mini-Lab • Beginner**

40 min

Goal. Prove a Kubernetes Secret is not encrypted, and that RBAC is the control that matters.

Business context. A teammate argued in review that "Secrets are encrypted, so a ConfigMap and a Secret are equivalent". Settle it with evidence.

Architecture. One local cluster, one Secret, two ServiceAccounts with different RBAC.

Requirements. Create a Secret; decode it in one command; create a ServiceAccount with `get` on ConfigMaps but not Secrets and show the difference.

Constraints. No external secret store — this lab is about the primitive itself.

Hints. `kubecttl auth can-i get secrets --as=system:serviceaccount:ns:sa.`

Expected outcome. A note stating what base64 gives you (nothing) and what the Secret type gives you (separate RBAC resource, node-scoped delivery, encryption eligibility).

Production considerations. Which of your namespaces grant `get secrets` more broadly than intended?

Common pitfalls. Concluding "Secrets are useless" — the type matters, just not for the reason people assume.

 Mini-Lab • Intermediate

75 min

Goal. Add three scanning gates to the Day 9 pipeline and tune them until they are actionable rather than ignored.

Business context. A dependency CVE reached production last quarter; leadership wants a gate, engineering does not want one that blocks every release.

Architecture. CI job → SAST → dependency scan → build → image scan → push.

Requirements. Three separate steps; blocking on CRITICAL and HIGH with unfixed excluded; SBOM retained; SARIF uploaded so findings appear in review.

Constraints. The security stage must add under two minutes — cache the vulnerability database between runs.

Hints. Pin a deliberately old base image first, so you see a real failure before tuning anything.

Expected outcome. A pipeline that blocks a bad image, passes a good one, and a written exception policy.

Production considerations. Who grants an exception, for how long, recorded where? A gate with no exception path gets disabled the first time it is inconvenient.

Common pitfalls. Blocking on unfixed CVEs; never re-scanning the registry; treating a passing scan as proof of safety.

 Mini-Lab • Production-style

2.5 hours

Goal. Eliminate every static long-lived credential from CloudBase, and prove it.

Business context. An audit requires no credential lifetime over 24 hours, and every database action attributable to a workload.

Architecture. Workload identity (ServiceAccount / IRSA) → secrets broker → leased database credentials; CI authenticates by OIDC federation, not an access key.

Requirements. Per-workload credentials with a one-hour TTL; no stored cloud key; clean reconnection on lease expiry; audit logging on every secret read.

Constraints. Zero downtime — old and new credential paths must both work during the cutover.

Hints. Run both behind a flag, migrate one deployment, confirm in the audit log that the dynamic path is really being used, then drop the static user.

Expected outcome. A repository-wide grep returning no credential material, plus an audit excerpt showing per-pod database users.

Production considerations. What happens if the broker is down for ten minutes during a rolling deploy? Write the answer down.

Common pitfalls. A pool that caches credentials for the process lifetime and fails silently at the one-hour mark; forgetting migration jobs and cron tasks hold the old credential too.

9 Engineering Notes Senior engineer insight

The most valuable security artefact a platform team produces is not a policy document — it is a *paved road*. If the easy way to get a database connection already issues a short-lived credential, and the easy way to build an image already scans and signs it, engineers get the secure behaviour by doing the convenient thing. Security that depends on people choosing the harder path erodes under every deadline. Netflix's application-security programme is built explicitly around this: centralise the hard

parts so teams inherit them by default.

Design trade-off

Every control here trades availability for confidentiality. Default-deny means a mislabelled pod cannot talk; short-lived credentials mean an unavailable broker stops deploys; blocking scans mean an unfixable CVE can stop a release. These are real costs, and pretending otherwise is how security teams lose credibility with engineers. Name the cost, measure it, and make the exception path explicit — a documented, time-boxed, logged override is far safer than a control quietly disabled.

Reliability principle

Treat the secrets broker as a tier-0 dependency: cache credentials in the workload for the lease duration so a brief outage restarts nothing; make lease-renewal failures a warning long before they are fatal; and rehearse "broker unavailable" as a game day. A security control that takes production down during its own outage will be removed — correctly — by the next on-call engineer.

Interview insight

Expect: "A secret was committed and pushed — what do you do?" The answer that fails is "remove it from history". The one that lands starts with **rotate or revoke immediately**, because Git objects are immutable and content-addressed, so the value is already public across every clone and fork. Then audit the exposure window, then rewrite history as cleanup, then prevent it structurally. The interviewer is testing whether you know that history rewriting is hygiene, not containment.

10 Build-Along Project — CloudBase

By Day 11 CloudBase runs in production shape: containerised on Kubernetes, provisioned by Terraform, deployed by a pipeline, observable through metrics, logs and traces. It is also, honestly assessed, wide open — a database password in a manifest, an unscanned image, a flat pod network, an IAM role grown by accretion. Today you close all four.

Build-Along • Day 12 — harden every boundary

- **Inventory and evict secrets.** Grep the Day 2 repository, the Day 7 Terraform, the Day 9 CI variables and the Day 6 manifests for credential material. Move every one into the secrets manager (or Vault) and reference it at deploy time — nothing secret remains in Git, in an image, or in a `ConfigMap`.
- **Go dynamic for the database.** Configure a database secrets engine or managed rotation so the API receives a per-workload credential with a one-hour TTL, authenticating with its `ServiceAccount` identity. Delete the original static database user and fix whatever breaks.
- **Encrypt at rest, and document the limit.** Enable a KMS provider for the `secrets` resource, then write one paragraph in `docs/security.md` on why this does *not* protect a `Secret` from a principal holding `get secrets` — and what does.
- **Gate the supply chain.** Extend the Day 9 pipeline with SAST (advisory), dependency scanning and image scanning, the last two blocking on CRITICAL and HIGH with unfixed excluded. Retain an SBOM per build. If the gate blocks, replace the base image — do not raise the threshold.

- **Segment the cluster.** Default-deny ingress in the CloudBase namespace, then the minimum allow set: ingress controller → API, API → Postgres on 5432, and DNS. Prove it with a debug pod whose connection to Postgres is refused.
- **Audit IAM and RBAC, then write the runbook.** Regenerate the CI role's policy from CloudTrail activity, remove unjustified wildcards, scope `iam:PassRole` to named roles, replace stored access keys with OIDC federation, and drop every `cluster-admin` binding that is not human break-glass. Then add `docs/runbooks/leaked-credential.md` — rotate, audit, rewrite, prevent — and enable push protection.

Definition of done: a repository-wide search for credential material returns nothing; the CloudBase API runs on a database credential that expires within one hour and is attributable to a single pod; the pipeline blocks a deliberately vulnerable image and passes the current one; a debug pod in the CloudBase namespace cannot open a TCP connection to Postgres while the API can; and the leaked-credential runbook has been walked end to end by someone who did not write it.

11 Chapter Summary

Key takeaways

- ✓ Least privilege has three axes — who, what, *how long*. The third is the one teams skip and the one dynamic credentials solve.
- ✓ IAM: explicit deny always wins, and an allow is needed in every applicable policy category. Kubernetes RBAC has no deny at all and is purely additive.
- ✓ Base64 is encoding. Encryption at rest needs an `EncryptionConfiguration`, ideally a KMS provider — and it protects storage, not the API.
- ✓ A secret exists at rest, in transit and in memory; env vars are the leakiest. SAST, dependency and image scanning read three different inputs; none substitutes for another.
- ✓ Pod networking is flat by default: a pod is non-isolated until some policy selects it, and then everything not explicitly allowed is dropped.
- ✓ A pushed secret is burned. Rotate first; history rewriting is cleanup, not containment.

Things to remember

- A NetworkPolicy on a cluster whose CNI does not enforce policy is accepted and does nothing — test with a connection that must fail. Default-deny egress breaks DNS: ship the DNS allow rule in the same change.
- `iam:PassRole` with a wildcard resource is effectively privilege escalation.
- The same image can pass a scan today and fail tomorrow — feeds change. Re-scan what is already in the registry.

Common pitfalls (last look)

- "Secrets are encrypted"
 - deleting a leaked secret instead of rotating it
 - one scan type
- flat pod networking
 - default-deny egress without DNS
 - wildcard IAM under deadline pressure

📋 One-Page Cheat Sheet

Least privilege — who · what · how long. Generate policies from observed access; alert on unused access.

IAM vs RBAC — IAM: explicit deny wins, intersection across policy types. RBAC: no deny, pure union of bindings.

Base64 — a transport encoding with no key. `base64 -d` reverses it. Not a security control.

Envelope encryption — per-resource DEK encrypts the object; a KMS-held KEK wraps the DEK; etcd stores both ciphertexts.

Dynamic secret — a per-workload DB user under a lease; expiry drops the user, so a leaked one is already dead.

Three scans — SAST = your source · SCA = your lockfile · image scan = the OS layers you inherited.

NetworkPolicy — label-selected, additive, no deny rule, CNI-enforced. Default-deny first, then allow.

Leak response — rotate → audit → rewrite → prevent. In that order, always.

CLI quick reference

Day 12 commands worth memorising

```
# --- audit what an identity can actually do ---
kubecttl auth can-i --list --as=system:serviceaccount:cloudbase:order-api
aws iam simulate-principal-policy --policy-source-arn <role-arn> \
  --action-names s3:PutObject --resource-arns <bucket-arn>

# --- prove base64 is not encryption ---
kubecttl get secret cloudbase-db -o jsonpath='{.data.password}' | base64 -d

# --- supply chain gates ---
trivy image --severity CRITICAL,HIGH --ignore-unfixed --exit-code 1 IMAGE
trivy image --format cyclonedx --output sbom.json IMAGE
cosign verify-attestation --type slsaprovenance IMAGE

# --- segmentation: PROVE it (must fail from an unauthorised pod) ---
kubecttl run probe --rm -it --image=nicolaka/netshoot --restart=Never \
  -- nc -zv postgres 5432

# --- dynamic credentials and leak response (rotate FIRST) ---
vault read database/creds/orders-rw
vault lease revoke -prefix database/creds/orders-rw
aws secretsmanager rotate-secret --secret-id cloudbase/db/orders
```

👤 Interview questions to master

- › What is the very first thing you do when a secret is accidentally committed and pushed — and why is deleting the file not it?
- › Why do you need SAST, dependency scanning and image scanning as three separate gates?
- › How does a Kubernetes NetworkPolicy limit blast radius if a pod is compromised, and what does it not cover?
- › What does "defense in depth" mean in practice, not as a buzzword?

- › Why prefer short-lived dynamically issued credentials over static long-lived ones, and what new failure mode does that introduce?
- › How would you audit whether an IAM policy actually follows least privilege?
- › A Kubernetes Secret is base64-encoded. What does that give you, and what does encryption at rest actually protect against?

Further reading & official docs

- Kubernetes docs — "Encrypting Confidential Data at Rest", "Using a KMS provider for data encryption" (both under kubernetes.io/docs/tasks/administer-cluster/) and "Network Policies": the `EncryptionConfiguration` format, the envelope mechanism, and the isolation model with its explicit list of what `NetworkPolicy` cannot do.
- HashiCorp Vault docs — "Database secrets engine" ([developer.hashicorp.com](https://developer.hashicorp.com/vault), under Vault → Secrets engines): roles, creation statements, leases, static roles, root rotation.
- AWS — "Using AWS IAM Access Analyzer" in the IAM User Guide, plus the AWS Security Blog post on generating IAM policies from CloudTrail access activity.
- GitHub docs — "About push protection" under Code security → Secret scanning; and Trivy's CI/CD documentation at trivy.dev for the `exit-code` and severity behaviour used above.
- The SLSA specification (slsa.dev) and the Sigstore project (Cosign, Fulcio, Rekor) for build provenance and keyless signing; CIS Kubernetes Benchmark and NIST SP 800-218 for the checklists auditors cite.

Production checklist

- ☐ No credential material exists in Git, in a container image, or in a `ConfigMap`.
- ☐ Database credentials are short-lived and attributable to a single workload.
- ☐ Encryption at rest is enabled for Secrets via a KMS provider — and the team knows what it does not protect.
- ☐ CI authenticates to the cloud by OIDC federation; no long-lived access key is stored anywhere.
- ☐ SAST, dependency and image scanning run as separate gates, with a documented, time-boxed exception path, and registry images are re-scanned on a schedule.
- ☐ Every namespace holding data has a default-deny `NetworkPolicy`, with DNS explicitly allowed.
- ☐ No `cluster-admin` binding exists except a documented human break-glass account, and unused IAM roles and keys are reviewed on a schedule.
- ☐ The leaked-credential runbook exists, starts with "rotate first", and has been rehearsed.

Key idea

What you will learn next (Day 13). You have now built, deployed, observed and hardened a real platform. Tomorrow you consolidate: the architecture-level thinking that ties the thirteen days together — reasoning about a system end to end, presenting the trade-offs you accepted, and answering the question every senior interview asks: *"walk me through something you built, and tell me what you would do differently."*

DAY 13 • OPERATIONS

GitOps & Deployment Strategies

Argo CD, blue-green, canary, rollback and progressive delivery

🕒 Estimated study time: 8 hours

13

🎯 By the end of this day, you will be able to...

- Explain GitOps as a continuous reconciliation loop, and why pulling beats pushing for production clusters.
- Describe what Argo CD actually does internally – render, diff, sync, self-heal – rather than which buttons it has.
- Compare rolling update, blue-green and canary honestly on blast radius, cost and rollback speed.
- Gate a canary promotion on real SLO metrics with an automated abort, not on a timer and a human's nerve.
- Rehearse a rollback so that the path exists before the incident that needs it.

1 The Big Picture

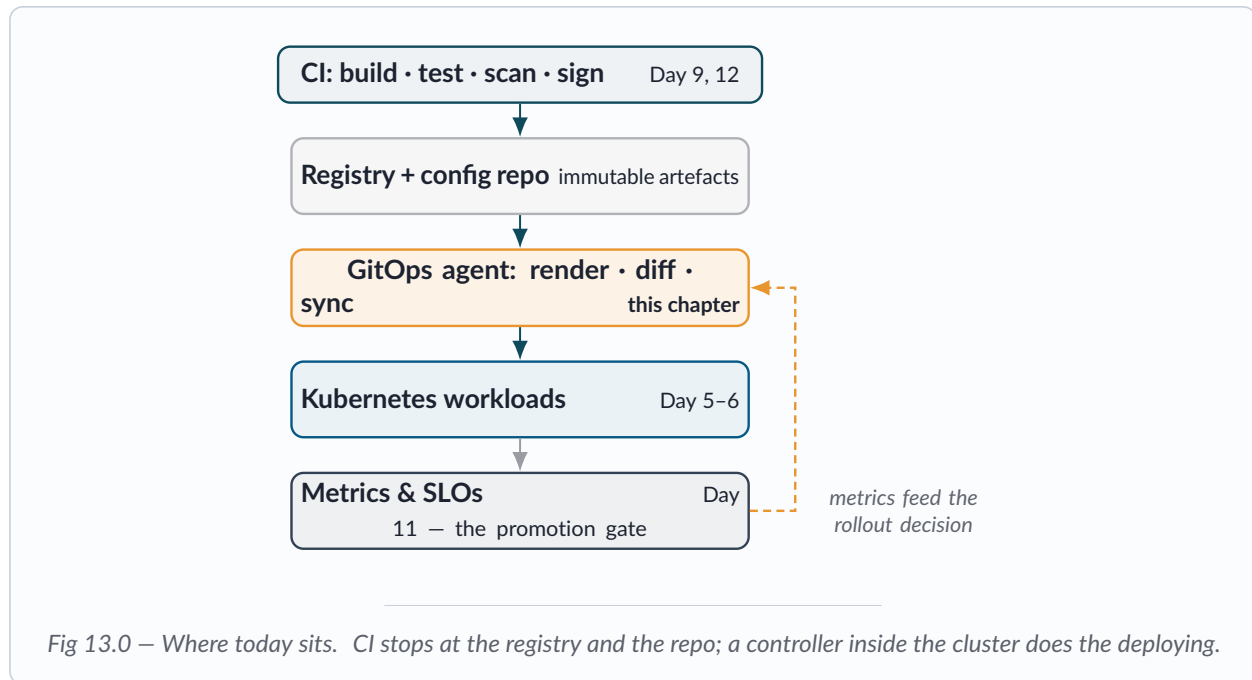
What problem this solves. By Day 12 you can build an image, test it, scan it, sign it and hand it to a cluster. What you cannot yet answer, in one sentence and without logging into anything, is: *what exactly is running in production right now, who put it there, and how do I put the previous thing back?* In a push-based pipeline that answer lives in the union of a CI job's history, whatever someone typed into a terminal at 02:40, and the cluster's own memory. GitOps collapses all three into one queryable artefact – the git history of a repository – and then makes a machine continuously enforce it.

Why DevOps engineers need it. Deployment is where every previous day's work is finally exposed to users. A perfect Terraform module and a beautiful dashboard do not help if a bad release reaches 100% of traffic in eleven seconds and the only rollback plan is "rebuild and redeploy, roughly nine minutes if the registry is fast." Senior engineers are distinguished less by how they ship and far more by how quickly and calmly they *un-ship*.

🔑 Key idea

Kubernetes is a reconciliation loop: you declare desired state, a controller works continuously to make reality match. GitOps is that same idea moved up one level – the desired state of the *whole cluster* is a git repository, and an in-cluster agent reconciles the cluster against it forever. Day 5 taught you the pattern. Today you apply it to delivery itself.

Where it fits in a modern cloud architecture. Today's layer sits between the pipeline you built on Day 9 and the cluster you built on Days 5–6, and it consumes the metrics you built on Day 11 to decide whether a release lives.



How it connects to previous chapters. Day 5's controller pattern is the mechanism. Day 7's declarative infrastructure is the mindset. Day 9's pipeline is the producer that now stops at the repository boundary instead of reaching into the cluster. Day 11's error-rate and latency queries stop being dashboards and become *control inputs*. Day 12's least-privilege thinking is why the deploy credential disappears from CI entirely.

Where companies use it in production. Adobe replaced its legacy internal container platform (Moonbeam) with **Flex**, a GitOps delivery platform built on the Argo ecosystem: Argo CD for continuous reconciliation and git-based delivery, Argo Workflows for pipeline orchestration, Argo Events for event-driven automation, and Argo Rollouts for progressive delivery. Published CNCF case-study figures put Flex at over 6,000 services across roughly 50,000 environments, of which approximately 19,000 are production, serving more than 3,000 developers, with over 10,400 legacy pipelines migrated or decommissioned. The architectural point is not the size. It is that at that size, "who changed production?" must be answerable by a query, not by asking people — and reconciliation is what makes the answer trustworthy. Argo CD itself was created at Intuit to manage large numbers of services across many clusters, and is now a CNCF Graduated project.

Common misconceptions. That GitOps means "we store YAML in git" (storing is trivial; *continuous reconciliation* is the idea). That a canary is a percentage (it is an experiment with a hypothesis and an abort criterion). That blue-green is strictly safer than canary (it exposes 100% of users at once, just reversibly). That rollback is a button (it is a rehearsed procedure, and the database is usually the part that cannot roll back).

2 Concept Map

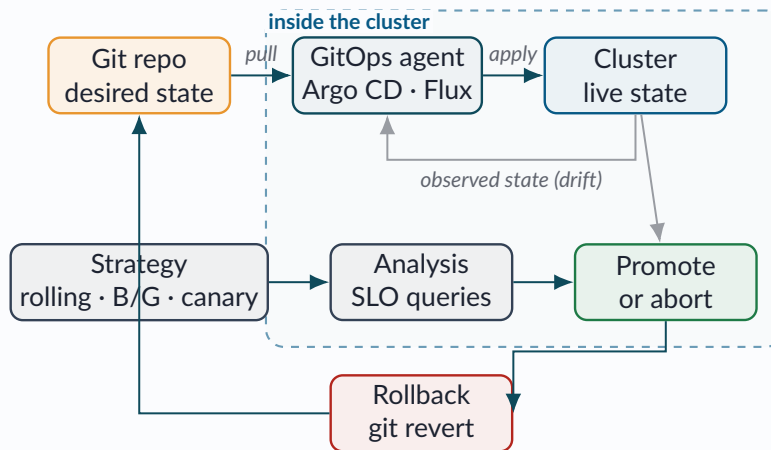


Fig 13.1 — The Day 13 territory: git holds desired state, an agent reconciles it, and a rollout controller decides how the change reaches users.

Terminology to anchor: **desired state** → **reconciliation** → **drift** → **self-heal** → **progressive delivery** → **analysis run** → **promotion gate** → **abort** → **feature flag**.

3

Guided Learning

3.1 GitOps — Reconciliation Applied to Delivery

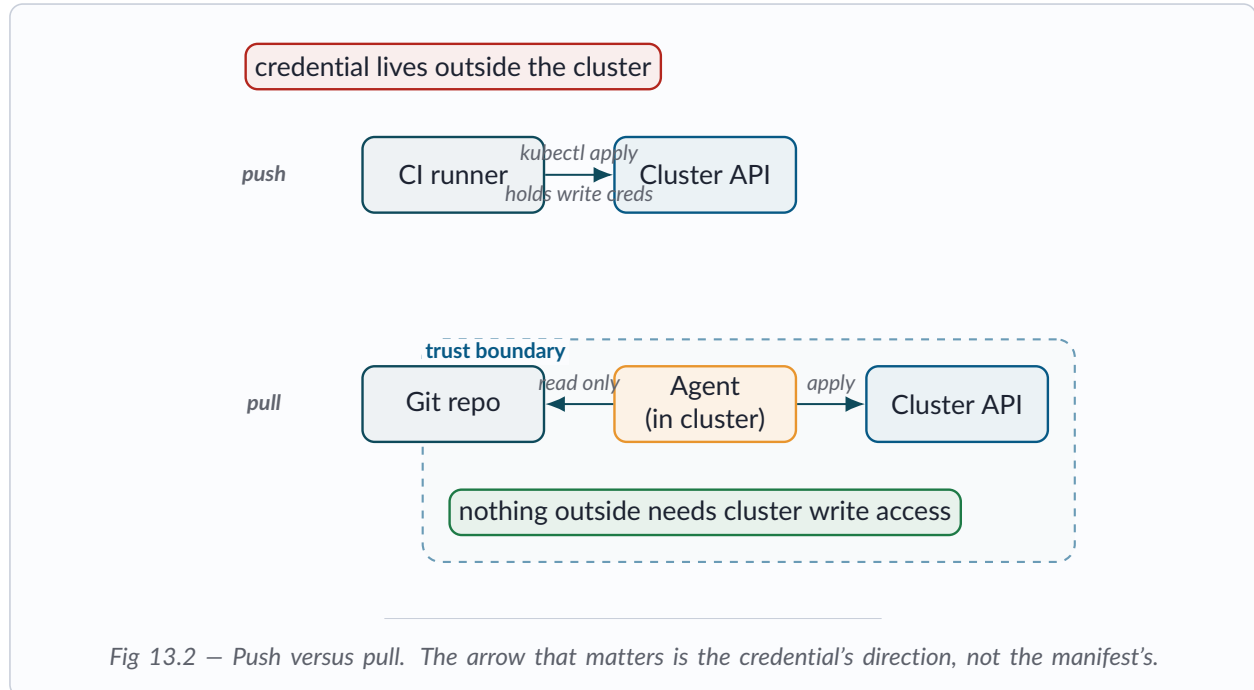
What is it? GitOps is an operating model with four properties, formalised by the OpenGitOps project: the system's desired state is **declarative**; that state is stored **versioned and immutably** with full history; software agents **pull** it automatically; and agents **continuously reconcile** actual state toward it. Notice what is absent from that list: git. The name is historical — the principles describe reconciliation from a versioned source of truth, and git is simply the store nearly everyone already has.

Why does it exist? Before GitOps, deployment was imperative and push-based. A CI job held cluster credentials and ran `kubectl apply` (or `helm upgrade`) at the end of a pipeline. Three problems compounded:

- **Credential sprawl.** Every pipeline that could deploy held a credential that could write to production. CI systems execute arbitrary code from branches, which makes them an attractive and well-trodden path to that credential.
- **No convergence.** `kubectl apply` is a one-shot event. If someone later ran `kubectl edit` or `scale`, nothing ever noticed. The cluster drifted, silently, and the drift was only discovered when the next deploy produced an unexplainable diff.
- **Unknowable state.** "What is deployed in staging?" required inspecting the cluster, because the pipeline's last successful run was not proof — someone might have changed it since.

How does it work internally? A controller inside the cluster holds three representations at all times: the **desired state** (manifests rendered from a specific git commit), the **live state** (objects read from the Kubernetes API, kept warm by watches rather than repeated list calls), and the **diff** between

them. The loop is: fetch the repo revision, render manifests, compare against live, report `Synced` or `OutOfSync`, and — if automated sync is enabled — apply the difference. Argo CD polls the repository roughly every three minutes by default (`timeout.reconciliation` of 120s plus up to 60s of jitter, so that a thousand applications do not all poll on the same second); a repository webhook makes that near-instant, and setting the interval to zero makes the system webhook-only.



Security warning • the credential argument, stated precisely

Pull-based GitOps does not make deployment "secure". It moves one specific credential. In push mode, a token with cluster write permission is stored in a system that runs untrusted code on every pull request. In pull mode, the agent lives inside the cluster, already has a ServiceAccount, and needs only *read* access to git and the registry. The blast radius of a compromised CI runner drops from "can deploy anything to production" to "can propose a commit" — which then meets code review and branch protection from Day 2. Combine it with signed commits and the deploy path is auditable end to end.

When should I use it? Any Kubernetes environment with more than one cluster, more than one environment, or more than one person who can deploy. The value grows superlinearly with the number of clusters, because reconciliation is the only approach whose operational cost per cluster stays flat.

When should I not? When you do not have a declarative target. GitOps agents reconcile Kubernetes objects; a pipeline that uploads a JAR to a VM or runs a database migration is not something Argo CD should own. Also resist it for a single hobby cluster where the agent itself becomes the most complicated component you operate — and note that the agent is a stateful production dependency: if Argo CD is down and your rollback plan is "revert the commit", you have no rollback plan.

Common misconception

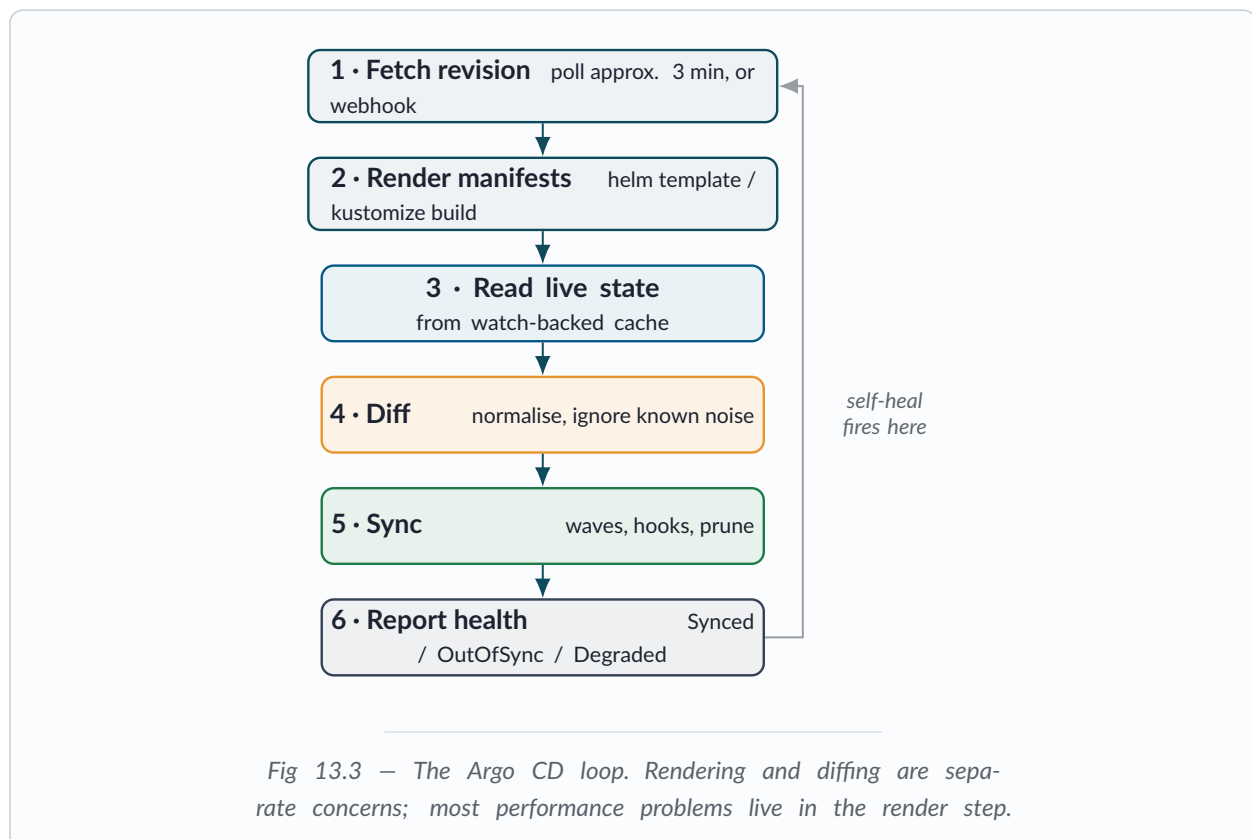
"We are doing GitOps — our pipeline commits the new image tag to a repo and then runs `kubectl apply`." That is still push-based delivery with a git-shaped audit log. The defining property is that *no external system applies anything*; an in-cluster agent observes divergence forever and corrects it. If disabling your CI system would stop drift from being corrected, you are not reconciling.

3.2 Inside Argo CD — Render, Diff, Sync, Self-Heal

What is it? Argo CD is a Kubernetes controller plus a set of CRDs. The unit of management is an `Application`: a custom resource naming a source (repo URL, revision, path, and any Helm or Kustomize settings) and a destination (cluster and namespace).

How does it work internally? Three components split the work, and knowing which one is struggling is most of Argo CD operations:

Component	Responsibility, and what its failure looks like
API server	gRPC/REST gateway for the UI, CLI and webhooks; authentication, RBAC, credential storage. If it is down, you cannot see or drive anything — but reconciliation continues.
Repo server	Clones and caches repositories, then <i>renders</i> manifests (Helm template, Kustomize build, plain YAML). CPU- and memory-hungry. Slow syncs and rendering timeouts are almost always here.
Application controller	Watches live cluster objects, computes the diff against rendered desired state, sets <code>Synced/OutOfSync</code> , executes syncs, runs hooks, and self-heals. Sharded across replicas when you manage many clusters.



The diff is harder than it looks. A naive comparison of your YAML against the live object always reports a difference, because the API server, admission webhooks and other controllers legitimately add fields: `status`, defaulted values, injected sidecars, autoscaler-managed replica counts, mutating-webhook annotations. Argo CD therefore diffs against the last-applied configuration and applies normalisation rules, and offers explicit `ignoreDifferences` for fields another controller owns. The classic symptom of getting this wrong is an application that flaps `OutOfSync` forever, or — worse — an app with self-heal enabled that fights the HorizontalPodAutoscaler, resetting `replicas` to the committed value every few minutes.

 Behind the scenes

`syncPolicy.automated.selfHeal` is what makes drift disappear rather than merely become visible. With it off, a manual `kubectl edit` shows up as `OutOfSync` and waits for a human. With it on, the controller re-applies the git state on the next reconciliation — so the manual edit survives for at most one interval. That is the desired behaviour in production and a genuinely infuriating one during a live incident, which is why teams keep a documented break-glass procedure: disable auto-sync on that one Application, act, then commit the fix and re-enable.

Ordering. Manifests are not applied in file order. Argo CD groups resources into **sync waves** (an annotation carrying an integer; lower waves complete and become healthy first) and runs **hooks** at the `PreSync`, `Sync` and `PostSync` phases. This is how a schema migration Job runs before the Deployment that needs the new column, and how a smoke test runs after.

`argocd/application-cloudbase-api.yaml`

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: cloudbase-api-prod
  namespace: argocd
spec:
  project: cloudbase
  source:
    repoURL: https://github.com/acme/cloudbase-config.git
    targetRevision: main           # never a mutable branch for prod? see note
    path: envs/prod/api
  destination:
    server: https://kubernetes.default.svc
    namespace: cloudbase-prod
  syncPolicy:
    automated:
      prune: true                 # delete objects removed from git
      selfHeal: true              # revert manual cluster edits
    syncOptions:
      - CreateNamespace=true
  ignoreDifferences:
    - group: apps
      kind: Deployment
      jsonPointers:
        - /spec/replicas          # owned by the HPA, not by git
```

 Design trade-off • tracking a branch versus a digest

Tracking `main` means "production is whatever the tip of main renders to," which is convenient and slightly terrifying: a merge deploys. Tracking an immutable tag or commit SHA means promotion is an explicit act, and `git log` of the config repo is literally the deploy log. Most mature setups track a branch for dev, and a promoted SHA or release tag for production. The same reasoning applies to image references: a `:latest` tag defeats reconciliation entirely, because the desired state is no longer deterministic. Pin image digests.

Scaling the pattern. Two idioms dominate. **App-of-apps**: one Application whose manifests are other Applications, so a single root object bootstraps a whole cluster. **ApplicationSet**: a generator (over a list, a git directory, a cluster label selector, or a pull request) that templates many Applications from one definition — the standard answer to "one app per environment per region per cluster" without copy-pasted YAML.

The alternative. Flux takes the same principles and decomposes them into small controllers on the GitOps Toolkit: `source-controller` fetches artefacts (git, OCI, Helm) and serves them internally as tarballs; `kustomize-controller` and `helm-controller` apply them; `notification-controller` routes events in and out. There is no bundled UI. Flux tends to appeal to platform teams who want composable controllers and Kubernetes-API-only interfaces; Argo CD appeals where a multi-tenant UI, RBAC and a visual diff for many teams are worth running a heavier component. Both are CNCF Graduated. The choice rarely matters as much as arguments about it suggest.

Scalability note

At a few hundred Applications the bottleneck is the repo server's rendering CPU and the controller's cache memory — one live-object cache per managed cluster. The scaling levers are: shard the application controller, split enormous mono-repos so a single commit does not invalidate every render, prefer webhooks to shorter polling intervals, and keep manifests cheap to render (deeply nested Helm charts with heavy templating are the usual culprit). Reconciling faster is rarely the answer; reconciling less redundantly is.

3.3 Deployment Strategies — Trading Cost for Blast Radius

What is it? A deployment strategy is a decision about *who sees the new version, when, and how fast you can undo it*. Three shapes cover almost everything.

How does it work internally? (rolling update) Kubernetes' default. The Deployment controller creates a new ReplicaSet and shifts replicas between the two, bounded by `maxSurge` (how many extra Pods may exist above the desired count) and `maxUnavailable` (how many may be missing). Pods only receive traffic once their readiness probe passes, which is the entire safety mechanism — a Deployment with no readiness probe rolls broken Pods straight into the Service's endpoint list. During the roll, both versions serve simultaneously, so your API contract must be backward compatible with itself. There is no traffic-percentage control: the split is whatever the replica ratio happens to be.

How does it work internally? (blue-green) Two complete environments exist. Blue serves; green is deployed and warmed with no production traffic. Cutover is a single atomic-ish change — flipping a Service's label selector, or an Ingress/listener rule — so all users move at once. Rollback is flipping it back, which is why blue-green's rollback time is measured in seconds and is by far its main selling point. The cost is running double capacity for the overlap, and the risk is that a defect only observable under real production load reaches 100% of users instantly.

How does it work internally? (canary) A small, controlled fraction of traffic goes to the new version while the rest stays on the old. The split is enforced either crudely by replica ratio (nine old Pods, one new Pod, behind one Service — approximately 10%, subject to connection reuse and keep-alive skew) or properly by a weighted router: an Ingress controller that supports weights, or a service mesh such as Istio or Linkerd manipulating routing rules. The mesh approach is the accurate one, because it splits *requests*, not endpoints.

Strategy	Blast radius of a bad release	Extra cost	Rollback
Rolling	Grows steadily to 100% as the roll proceeds	None (bounded by maxSurge)	Minutes — roll back through the same gradual process
Blue-green	100% instantly, but only from cutover	2× capacity during overlap	Seconds — flip the selector back
Canary	Bounded by the current weight (often 5–10%)	Small (a few extra Pods) plus a router	Seconds — set weight to zero

Design trade-off • the honest comparison

Blue-green optimises *recovery time*; canary optimises *exposure*. If your failure mode is "the new version is catastrophically broken and we know within thirty seconds", blue-green is excellent and simpler. If your failure mode is "the new version raises p99 latency by 40% under real traffic patterns we cannot reproduce in staging", canary is the only one of the three that finds it before everyone is affected. Canary is also the only strategy that requires you to have working SLO metrics — which is precisely why it is the one that fails when adopted by teams who skipped Day 11.

Cloud cost optimization

Blue-green's double capacity is often mispriced. The real bill is not "2× for five minutes"; it is 2× for however long you keep green warm as a rollback safety net — frequently hours, sometimes across a weekend. And if your cluster autoscaler must add nodes to fit the green environment, you also pay node startup latency inside your deploy window, plus whatever your cloud's minimum billing increment is. Canary's few extra Pods are almost free by comparison; its cost is the routing infrastructure and the engineering time to make the analysis trustworthy.

Common misconception

"Blue-green means zero downtime." It means zero downtime for *stateless request serving*. Shared state does not fork: both colours talk to the same database, the same queues, the same caches. If green writes a row shape blue cannot read, flipping back does not undo the row. This is why schema changes follow **expand–contract** — add the new column, deploy code writing both, backfill, deploy code reading new, and only then drop the old — so that every intermediate state is compatible with both colours.

3.4 Progressive Delivery — Promotion Gated on Metrics

What is it? **Progressive delivery** is canary plus automation: the traffic weight advances through defined steps, and at each step a machine evaluates real metrics against explicit success and failure conditions, then promotes or aborts without waiting for a human.

Why does it exist? The manual canary is theatre. An engineer sets 10%, stares at Grafana for ten minutes, sees nothing obviously red, and promotes. This fails in three ways: the human is not sampling anything statistically meaningful; the human is doing this at 23:00 and wants to go home; and there is no record of the criterion that was applied, so nobody can review or improve the decision. Encoding the criterion as a query with thresholds fixes all three at once.

How does it work internally? With Argo Rollouts, a `Rollout` CRD replaces the `Deployment`. Its canary strategy is a list of steps — set weight, pause, set weight, pause — and `AnalysisTemplate` objects define measurements. Each metric has an `interval` (sampling frequency), a `count` (how many

measurements; zero means run continuously), a `successCondition` and a `failureCondition` evaluated against the query result, and a `failureLimit` (how many failed measurements are tolerated before the analysis fails). A result matching neither condition is *inconclusive* — deliberately distinct from failure. Analysis runs in one of two modes: **inline**, as a step that blocks progression until it completes, and **background**, running in parallel with the steps and aborting the rollout retroactively if it fails. When analysis fails, the controller sets the canary weight back to zero and scales the canary ReplicaSet down — traffic returns to stable in seconds, without any git change.

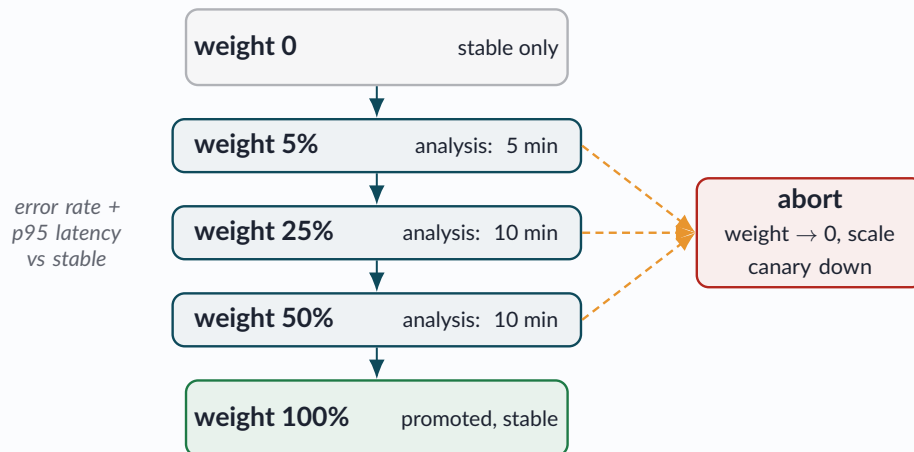


Fig 13.4 — A gated canary. Every arrow forward is a metric decision; every failure exits to the same place.

rollouts/analysis-error-rate.yaml

```

apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: cloudbase-slo
spec:
  args:
    - name: service
  metrics:
    - name: error-rate
      interval: 1m
      count: 5 # five samples, not one lucky reading
      successCondition: result[0] <= 0.01
      failureLimit: 1 # one bad sample is enough to abort
      provider:
        prometheus:
          address: http://prometheus.monitoring:9090
          query: |
            sum(rate(http_requests_total{
              service="{{args.service}}",version="canary",
              status=~"5.."}[2m]))
            /
            sum(rate(http_requests_total{
              service="{{args.service}}",version="canary"}[2m]))
    - name: latency-p95
      interval: 1m
      count: 5
      successCondition: result[0] < 0.400
      failureLimit: 2
      provider:

```

```

prometheus:
  address: http://prometheus.monitoring:9090
  query: |
    histogram_quantile(0.95, sum(rate(
      http_request_duration_seconds_bucket{
        service="{{args.service}}",version="canary"}[2m]
      )) by (le))

```

rollouts/rollout-cloudbase-api.yaml

```

apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: cloudbase-api
spec:
  replicas: 10
  strategy:
    canary:
      canaryService: cloudbase-api-canary
      stableService: cloudbase-api-stable
      trafficRouting:
        istio:
          virtualService:
            name: cloudbase-api
            routes: [primary]
      analysis: # background analysis
        templates:
          - templateName: cloudbase-slo
        args:
          - name: service
            value: cloudbase-api
      steps:
        - setWeight: 5
        - pause: {duration: 5m}
        - setWeight: 25
        - pause: {duration: 10m}
        - setWeight: 50
        - pause: {duration: 10m}

```

⚡ Performance note

The most common analytical error is a sample-size problem. At 5% of 200 requests per second you get 10 requests per second on the canary — roughly 600 in a one-minute window. If your baseline error rate is 0.1%, the expected number of errors in that window is *less than one*, so a single unlucky request pushes the measured rate to 0.17% and your threshold decides the release is broken. Two fixes, both used in practice: widen the query window so the denominator is large enough to be meaningful, and compare canary against the concurrently measured *stable* version rather than an absolute constant, so that a site-wide incident does not get blamed on your release.

💓 Reliability principle

Choose your failure conditions from the user's perspective, and make them cheap to evaluate. Error rate and latency are the reliable pair because they are the two symptoms every user notices. Avoid gating on CPU, memory or replica count — those are causes, not symptoms, and a new version that legitimately uses 15% more CPU will abort a perfectly good release. Gate on the SLIs from Day 11, and if a metric is not good enough to alert on at 03:00, it is not good enough to abort a deploy either.

💡 Senior engineer insight

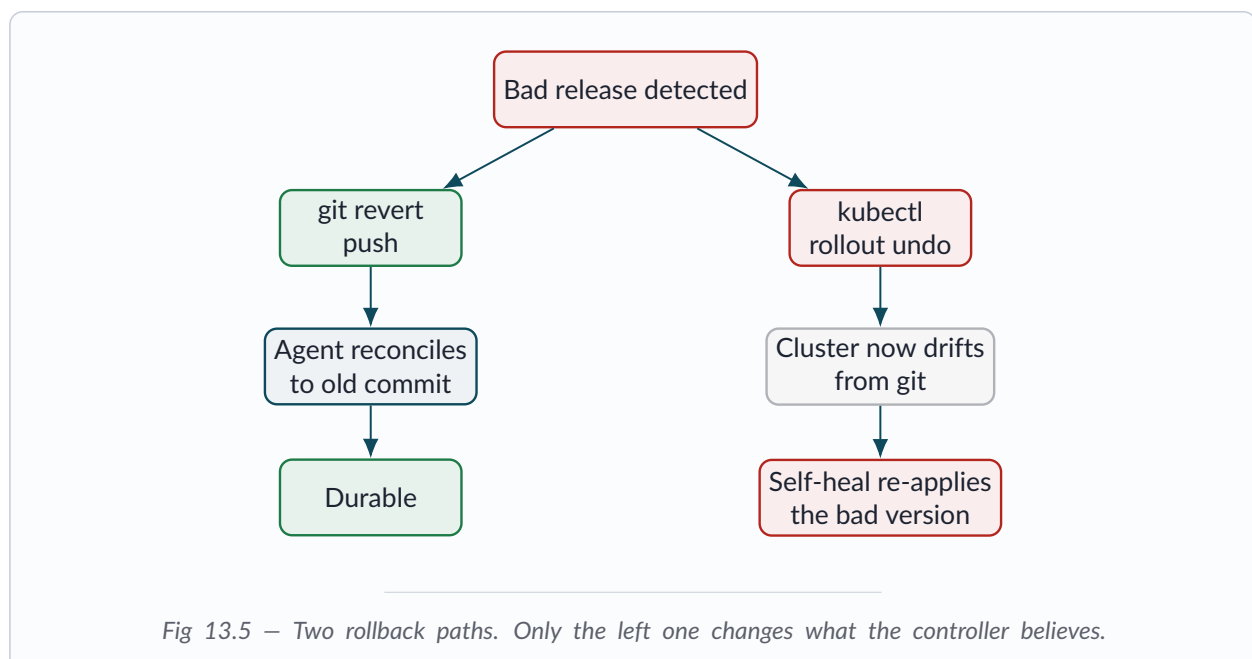
Progressive delivery only pays for itself when the analysis window is long enough to catch your actual failure modes and short enough that people do not route around it. A ninety-minute canary on a service that deploys eight times a day means the team will ship with the gate disabled by Friday. Start with the fastest-detecting signals, keep the total rollout under about twenty minutes for high-frequency services, and reserve long bake times for the one or two components where a subtle regression is genuinely expensive. Guardrails that get bypassed are worse than no guardrails, because they create false confidence.

3.5 Rollback as a First-Class, Rehearsed Operation

What is it? A rollback is a return to a known-good state. Under GitOps that means one thing: *make git describe the previous state*, and let reconciliation do the rest.

Why does it exist as a discipline? Because under pressure people do the fast thing, and in a self-healing system the fast thing is often undone thirty seconds later by the controller. This is the single most important operational consequence of GitOps and it surprises nearly everyone the first time.

Mechanism	What it does	Survives self-heal?
<code>git revert + push</code>	Changes desired state; agent reconciles the cluster back	Yes — this is the real rollback
<code>argocd app rollback</code>	Syncs the Application to an earlier deployed revision	Only with auto-sync disabled; it is an emergency lever
<code>kubectl rollout undo</code>	Switches the Deployment back to a previous ReplicaSet	No — git still says otherwise; drift is re-applied
Abort a canary	Sets weight to 0 and scales the canary down	Yes, and it is the fastest of all



 Disaster recovery tip

Rehearse the rollback in a scheduled exercise, with a stopwatch, on the production-shaped environment — not on a laptop. Measure three numbers and write them in the runbook: time to *detect* (alert fires), time to *decide* (who is allowed to call it, and can they do so without waking an architect), and time to *recover* (traffic actually back on the old version). Teams routinely discover in this exercise that the revert commit is blocked by a required-reviewers rule and there is no reviewer online at 03:00. That discovery is the entire point of the rehearsal.

rollback drill

```
# 1 - fastest: abort an in-flight canary (no git change needed)
kubectll argo rollouts abort cloudbase-api -n cloudbase-prod

# 2 - durable: make git describe the good state again
git revert --no-edit <bad-sha>
git push origin main
argocd app wait cloudbase-api-prod --sync --health --timeout 300

# 3 - emergency lever, only if git is unavailable
argocd app rollback cloudbase-api-prod 42 # 42 = a prior deployed revision
# then: re-enable auto-sync only AFTER git matches reality
```

 Security warning • what cannot be rolled back

Code rolls back. State does not. A migration that dropped a column, a message consumed off a queue with an incompatible schema, a cache populated with a new serialisation format, an outbound webhook already delivered to a customer — none of these are undone by reverting a commit. This is why forward-compatible, additive migrations are not a database nicety but a *deployment safety requirement*. Before every release, ask one question: if we revert this in ten minutes, what will be left behind?

3.6 Feature Flags — Decoupling Deploy from Release

What is it? A feature flag is a runtime condition around new behaviour, evaluated per request against a user, tenant or percentage. The code ships to production *dark* — deployed but inactive — and is enabled later, separately, without a new deploy.

Why does it exist? Deployment strategies control *which version of the binary* handles a request. They cannot control *which behaviour* that binary exhibits for a particular user. Bundling those two decisions makes every deploy a user-facing event, which pressures teams into large, infrequent, high-risk releases — exactly the dynamic continuous delivery is meant to eliminate. Flags separate two questions that fail for entirely different reasons: "did this code deploy safely?" and "does this feature behave correctly for users?"

How does it work internally? A flag SDK inside the process holds an in-memory rule set and evaluates locally, so the hot path costs a map lookup and not a network call. The rules are refreshed by streaming (server-sent events or a websocket) or polling from a control plane; percentage targeting is done by hashing a stable key — user ID plus flag key — into a bucket, so the same user gets a consistent answer across requests and services. Two properties follow directly from that design and both matter operationally: evaluation continues from cache if the control plane is unreachable, and the default value chosen for that case is a real availability decision.

🔧 Design trade-off • flags are not free

Every flag is a permanent branch in your code and a doubling of the state space to test. Ten live flags — a thousand combinations, most never exercised. Mature teams treat flags as inventory with an expiry date: every release flag has an owner and a removal ticket created when it is added, kill switches are labelled distinctly from experiments and are allowed to persist, and stale flags are audited monthly. Untended flag debt turns into behaviour nobody can explain — a service that responds differently for one legacy tenant because a flag was flipped in 2024 and never removed.

☰ Operational excellence

The pairing that works in production: deploy with a canary gated on technical SLIs (error rate, latency — is the binary healthy?), then release with a flag ramped on business metrics (conversion, engagement — is the feature good?). They answer different questions on different timescales. A canary that takes twenty minutes cannot tell you a checkout redesign reduces conversion; a flag ramp over a week can, and can be switched off in one second without any deploy at all.

✓ Best practices — what experienced engineers do

- Keep application code and deployment config in separate repositories — so a config commit does not trigger a rebuild, and the config repo's history is a clean deploy log.
- Pin image digests, never `:latest`; a non-deterministic desired state cannot be reconciled.
- Enable `selfHeal` in production and document the break-glass procedure for disabling it.
- Gate canary promotion on the same SLIs you alert on, compared against stable, not against a constant.
- Make every production change a reviewed commit — then branch protection and signing from Days 2 and 12 apply automatically to deploys.
- Rehearse rollback on a schedule and record detect/decide/recover times in the runbook.

✗ Common mistakes beginners make

- Leaving a long-lived, cluster-admin credential in CI after adopting an in-cluster agent — keeping the risk and adding a component.
- Promoting a canary on a fixed timer regardless of what the metrics show.
- Analysing a canary with too little traffic for the numbers to mean anything.
- Fixing an incident with `kubectl edit` in a self-healing cluster, and being confused when it reverts.
- Testing the rollback path for the first time during the incident that needs it.
- Treating "deployed" and "released" as the same event, so every deploy becomes an all-or-nothing user-facing change.

4

Visualizations to Internalize

Redraw these from memory. If you can draw the credential direction and the abort path without notes, you can explain this chapter in an interview.

 Redraw: push versus pull delivery, marking which side holds the cluster write credential and where the trust boundary sits

 Redraw: the canary ladder with its analysis gate at each step, and the single abort path all failures take

5**Guided Hands-On Lab — GitOps and a Gated Canary**

Goal. Move a running service from pipeline-pushed deploys to reconciliation, then put a metric-gated canary in front of it and prove that a bad release aborts itself. You are building the delivery path CloudBase uses for the rest of the book.

1 Split the config repository from the application repository

Create a second repository (or a clearly separated directory tree) holding only rendered Kubernetes manifests, structured per environment: `envs/dev`, `envs/staging`, `envs/prod`.

 **Why:** A commit to the config repo means "change what is running"; a commit to the app repo means "change the code". Merging them makes the deploy history unreadable and causes rebuild loops when the pipeline commits an image tag back into the repo it watches.

2 Install the agent and register the first Application

Install Argo CD into its own namespace, then declare one `Application` pointing at `envs/dev`. Sync it manually the first time and read the diff view before you approve it.

 **Why:** Reading the diff before the first sync is the habit that separates operators from button-pressers — it is also the only moment you will clearly see how much the API server defaults into your objects.

3 Cause drift deliberately, and watch it heal

With `auto-sync` and `selfHeal` on, run `kubectl scale` or `kubectl edit` against the live Deployment. Watch the application go `OutOfSync`, then watch it return.

 *Why:* This is the moment GitOps stops being an abstract idea. You must feel the loop, and you must know how long it takes, because that duration is how long a manual emergency change would survive.

4 Make the pipeline stop touching the cluster

Delete the cluster credential from CI. The pipeline's last step becomes: push the image, then open (or commit) a change to the config repo's image digest.

 *Why:* If the credential still exists "just in case", you have added a component without removing a risk. Deleting it is the change; everything else is plumbing.

5 Convert the Deployment to a Rollout with canary steps

Install Argo Rollouts, replace the Deployment with a `Rollout`, and configure weighted routing through your ingress controller or mesh with a stable and a canary Service.

 *Why:* Understand where the split is enforced. Replica-ratio canaries are approximations distorted by keep-alive connections; a weighted router splits actual requests, which is what your analysis assumes.

6 Write the AnalysisTemplate against your Day 11 metrics

Express error rate and p95 latency as Prometheus queries scoped to the canary version. Run each query by hand in the Prometheus UI first and confirm it returns a sensible number.

 *Why:* An analysis template whose query returns no data does not fail — it is inconclusive, and inconclusive behaves differently from failure. Verifying the query manually is the only way to know which one you have written.

7 Ship a deliberately broken version and watch the abort

Add an endpoint that returns HTTP 500 for a fraction of requests, deploy it, and watch the rollout abort and traffic return to stable. Record how long detection took.

 *Why:* A guardrail you have never seen fire is a guardrail you do not know the behaviour of. You need the measured detection time for your incident runbook, not an estimate.

8 Rehearse the full rollback, with a stopwatch

Promote the broken version to 100% on purpose in a non-production environment, then recover it via `git revert`. Time detect, decide and recover separately.

 *Why:* The three numbers are separately actionable: slow detection is a monitoring problem, slow decision is an org problem, slow recovery is a tooling problem. Averaging them hides which one you actually have.

Production tip

Give Argo CD its own repository and bootstrap itself with an app-of-apps: one root Application that manages Argo CD's own configuration, projects, RBAC and every other Application. It feels circular and it is — but it means the platform's configuration is reviewed and reconciled exactly like everything else, and rebuilding a lost cluster becomes "install the agent, point it at the root app, wait." Keep the bootstrap manifest in a second location, though: an agent cannot install itself.

6 Thinking Exercises

 Think it through

Self-heal reverts manual cluster changes within one reconciliation interval. During a live incident, that is exactly the behaviour you do not want. Design the break-glass procedure: who may invoke it, what it disables, how the system reminds you it is disabled, and what forces it back on. What does your answer imply about how much you trust processes versus mechanisms?

 Think it through

A service handles 40 requests per second. You want a canary at 5% weight to detect an error-rate regression from 0.1% to 1%. Roughly how many requests must the canary serve for that difference to be distinguishable from noise, and how long does that take? Now decide what you would change: the weight, the window, the metric – or the strategy entirely.

 Think it through

Blue-green gives you seconds-fast rollback but exposes 100% of users at cutover; canary bounds exposure but rolls back a partially-migrated population. For a payment API and for a recommendation widget, which do you choose, and what property of each system drove the choice rather than a general preference?

7

Debugging Practice

 Debugging Lab • The application that will not stay Synced

Symptom. `cloudbase-api-prod` flips between `Synced` and `OutOfSync` every few minutes. Nobody is deploying. Pods restart more often than expected during peak hours.

argocd app diff cloudbase-api-prod

```
==== apps/Deployment cloudbase-prod/cloudbase-api ====
spec:
  minReadySeconds: 0
- replicas: 12
+ replicas: 4
  revisionHistoryLimit: 10

# argocd app get cloudbase-api-prod
Health Status: Progressing
Sync Policy: Automated (Prune, SelfHeal)
CONDITION MESSAGE
SyncError one or more objects failed to apply (retried 3 times)
```

- ✓ **Observe.** The only differing field is `replicas`: git says 4, the cluster says 12. The flapping is periodic and correlates with load.
- ✓ **Hypothesise.** Two controllers own the same field. The HorizontalPodAutoscaler scales up under load; Argo CD's self-heal scales back down to the committed value; the HPA reacts again.
- ✓ **Investigate.** `kubectl get hpa -n cloudbase-prod` confirms an HPA targets this Deployment with min 4, max 20. Correlate the Argo CD sync timestamps with HPA scale events — they interleave.
- ✓ **Root cause.** Desired state was declared for a field that another controller legitimately owns at runtime. Reconciliation is working perfectly; the ownership model is wrong.
- ✓ **Fix.** Add `ignoreDifferences` for `/spec/replicas` on this Deployment (or remove `replicas` from the manifest entirely so the HPA is the sole owner).
- ✓ **Prevent.** Adopt a rule: any field a controller mutates at runtime — replica counts, injected sidecars, autoscaler and mesh annotations — is either excluded from git or explicitly ignored. Treat a permanently `OutOfSync` app as a design bug, never as background noise, or your team will learn to ignore the one that matters.

🔧 Debugging Lab • The canary that promoted a broken release

Symptom. A release with a clearly broken endpoint passed a five-minute canary at 5% weight and reached 100%. Error-rate alerts fired eight minutes after full promotion. The analysis run reported success.

kubectl argo rollouts get rollout cloudbase-api

```
Name: cloudbase-api
Status: Healthy
Strategy: Canary
Step: 6/6
SetWeight: 100
ActualWeight: 100

NAME                                KIND      STATUS      AGE  INFO
> analysisrun/cloudbase-api-5f8c-2 AnalysisRun Successful 14m
  measurement error-rate value: 0 phase: Successful # 5/5 samples
  measurement latency-p95 value: NaN phase: Inconclusive
```

- ✓ **Observe.** The error-rate metric measured exactly 0 for all five samples, and the latency

metric returned `NaN` and was recorded as inconclusive rather than failed.

- ✓ **Hypothesise.** The queries are not matching the canary's series. A metric that selects nothing returns an empty vector, which is not an error — so "no data" was read as "no errors".
- ✓ **Investigate.** Run the query in Prometheus with the canary label. The canary Pods carry `version=canary-abc123`, not the literal `version="canary"` the template hardcodes. No series match, so the numerator and denominator are both empty.
- ✓ **Root cause.** A label-selector mismatch between what the Rollout stamps onto canary Pods and what the AnalysisTemplate queries. The gate was measuring nothing and reporting success.
- ✓ **Fix.** Pass the canary pod-hash into the template as an argument from the Rollout's value-from reference, so the selector always matches the current canary ReplicaSet. Re-run against a known-bad build to confirm it now fails.
- ✓ **Prevent.** Add a *liveness assertion* to every analysis: a metric requiring the canary's total request count to exceed a floor, with a *failureCondition* when it does not. Absence of data must fail the gate, never pass it. Then verify the whole path periodically by deliberately shipping a broken build to staging — a fire drill for the guardrail itself.

8 Mini-Labs

Mini-Lab • Beginner

40 min

Goal. Experience reconciliation and drift correction first-hand.

Business context. Your team keeps finding hand-edits in staging nobody remembers making.

Architecture. One local cluster (kind or minikube), Argo CD, one git repo with a Deployment and a Service.

Requirements. Register the app with auto-sync and self-heal; then scale, edit an env var, and delete a Pod, observing each.

Constraints. Do not use the UI to fix anything — only git and observation.

Hints. Watch `argocd app get` in one terminal and `kubectl get deploy -w` in another; note the time between drift and correction.

Expected outcome. A written note stating your reconciliation interval and which of the three changes healed, which did not, and why.

Production considerations. Deleting a Pod is not drift — the Deployment controller owns that. Know which controller owns what.

Common pitfalls. Assuming self-heal is instant; forgetting that `prune` is what deletes objects removed from git.

Mini-Lab • Intermediate

70 min

Goal. Build a canary that aborts itself on a real metric.

Business context. A bad release last quarter reached all users before anyone noticed.

Architecture. Argo Rollouts plus a weighted ingress or mesh, Prometheus from Day 11, a load generator.

Requirements. Steps at 10%/50%/100%; an AnalysisTemplate on error rate; a deliberately broken build that must abort at the first step.

Constraints. Generate enough synthetic traffic that the canary sees a statistically meaningful request count.

Hints. Validate every PromQL query by hand first; add a request-count floor so empty results fail rather than pass.

Expected outcome. A recorded abort with the elapsed time from deploy to traffic fully back on stable.

Production considerations. Compare the canary against concurrently measured stable, so a site-wide incident does not abort an innocent release.

Common pitfalls. Label selectors that match nothing; thresholds tighter than your normal error-rate variance.

Mini-Lab • Production-style

2.5 hours

Goal. Run a rollback game day and produce a runbook with measured numbers.

Business context. Leadership asked "how fast can we undo a bad deploy?" and nobody has a defensible answer.

Architecture. GitOps-managed staging with a canary, alerting from Day 11, a colleague acting as incident commander.

Requirements. Have someone else deploy a defect without telling you what it is; detect, decide and recover; record all three times; repeat with a second, subtler defect.

Constraints. You may only use documented procedures. If a step is not written down, you may not do it — write it down first, then do it.

Hints. Include the awkward cases: branch protection blocking the revert, the agent itself unhealthy, a migration already applied.

Expected outcome. A one-page runbook: detection sources, decision authority, exact commands, and what to do when git is unavailable.

Production considerations. Decide in advance who may declare a rollback without escalation — ambiguity here costs more minutes than any tooling gap.

Common pitfalls. Rehearsing only the easy defect; assuming the person who deployed will be available.

9 Engineering Notes

Operational excellence

The DORA metrics land here. Deployment frequency and lead time are improved by making deploys boring; change failure rate is improved by canary gates; and time to restore service is improved almost entirely by rehearsed rollback. Notice that three of the four are properties of your *delivery mechanism*, not of your code — which is why this is the highest-leverage day in the operations track.

Infrastructure insight

Reconciliation makes cluster rebuild tractable. If every workload, namespace, quota and policy is declared in a repository and applied by an agent, recreating a lost cluster is: provision infrastructure (Day 7), install the agent, point it at the root Application, wait. Teams that can do this stop treating clusters as pets — and cluster upgrades stop being annual anxiety events, because you can stand up a new cluster

beside the old one and shift traffic.

Security warning

Secrets are the one thing that does not go into the config repo in plaintext. The three workable patterns: encrypt at rest in git with SOPS or Sealed Secrets so the agent decrypts in-cluster; use an External Secrets operator that pulls from Vault or a cloud secret manager at reconcile time; or use workload identity so no static secret exists. Whichever you choose, remember that git history is forever — a secret committed once and "removed" in the next commit is still in the repository, and the Day 12 rotation procedure applies.

Interview insight

Expect: "Why is pull-based GitOps more secure than push-based CI deploys?" The weak answer recites "no credentials in CI". The strong answer is precise about what moves: in push mode a token that can write to the production cluster sits in a system that executes untrusted code from branches, so compromising a runner compromises production. In pull mode the agent's identity never leaves the cluster and needs only read access to git and the registry, so the equivalent compromise yields the ability to *propose* a change — which then faces review, branch protection and signing. Then add the honest caveat: the agent itself becomes a highly privileged in-cluster component and must be treated as such.

Reliability principle

Every automated gate needs a manual override, and every manual override needs an audit trail. Argo Rollouts' `promote -full` skips remaining analysis; that is a legitimate incident tool for shipping a hotfix quickly. It is also how a team quietly disables its own safety net. Make the override loud: log it, alert on it, review it in the weekly ops meeting. A guardrail used ten times a month is either miscalibrated or ignored — both are worth knowing.

Cloud cost optimization

Progressive delivery has an underappreciated cost saving: aborted releases never reach the users who would have generated support tickets, refunds and incident hours. It also has a real running cost — a mesh or weighted ingress, a Prometheus that can answer analysis queries under load, and extra Pods during rollouts. Size it deliberately: a canary on your revenue-critical path is obviously worth it; a canary on an internal admin tool used by nine people is ceremony, and rolling update plus a fast revert is the correct engineering answer.

10 Build-Along Project — CloudBase

CloudBase is containerised (Day 4), orchestrated (Days 5–6), provisioned by Terraform (Day 7), built by a pipeline (Day 9), secured (Day 12) and observable (Day 11). Until today, that pipeline still ended by reaching into the cluster with a credential. Today you cut that arrow and replace it with a loop.

Build-Along • Day 13 — GitOps delivery with a gated canary

- Create `cloudbase-config`: a manifest-only repository with `envs/dev`, `envs/staging`, `envs/prod`, each pinning image *digests* rather than tags.
- Install Argo CD and bootstrap an app-of-apps root Application that owns every CloudBase Application, so the platform's own configuration is reconciled too.

- Enable automated sync with `prune` and `selfHeal` on staging and production; add `ignoreDifferences` for HPA-owned replicas.
- Rewrite the Day 9 pipeline's final stage: build, scan and sign the image, then commit the new digest to `cloudbase-config` — and **delete the cluster credential from CI**.
- Replace the API's Deployment with an Argo Rollouts `Rollout`: weights 5% → 25% → 50% → 100%, with stable and canary Services behind weighted routing.
- Write an `AnalysisTemplate` using the Day 11 SLIs — 5xx rate and p95 latency, canary compared against stable — plus a request-count floor so an empty result fails instead of passing.
- Prove the gate: deploy a build that returns 500s for 3% of requests and confirm it aborts at the first weight step. Record the elapsed detection time.
- Rehearse the rollback end to end: promote a bad build deliberately in staging, recover with `git revert`, and time detect / decide / recover separately.
- Add one kill-switch feature flag around a risky code path, and document who may flip it and how it is audited.
- Write `docs/deployment-runbook.md`: strategy per service and why, the exact rollback commands, the break-glass procedure for disabling self-heal, and your three measured recovery times.

Definition of done: no credential outside the cluster can write to production; a manual `kubectl edit` in staging is reverted automatically within one reconciliation interval; a deliberately broken release aborts at the first canary step without human intervention; and a rehearsed `git revert` restores the previous version with measured detect, decide and recover times recorded in the runbook.

11 Chapter Summary

Key takeaways

- ✓ GitOps is continuous reconciliation from a versioned, declarative source — not "YAML in git".
- ✓ Pull-based delivery moves the cluster write credential inside the cluster, shrinking the CI blast radius to "can propose a commit".
- ✓ Argo CD splits work across API server, repo server (rendering) and application controller (diff, sync, self-heal); most performance problems are rendering.
- ✓ Self-heal makes manual cluster edits temporary — plan a break-glass procedure before you need one.
- ✓ Blue-green optimises recovery time at double cost; canary optimises exposure but requires trustworthy metrics.
- ✓ Progressive delivery replaces a human staring at a dashboard with explicit success and failure conditions, and an automatic abort.
- ✓ Under GitOps the real rollback is `git revert`; `kubectl rollout undo` is drift that self-heal will overwrite.

- ✓ Code rolls back; state does not. Migrations must be additive and forward-compatible.
- ✓ Feature flags separate "deployed safely" from "released correctly" — two failures with different timescales.

Things to remember

- Argo CD's default reconciliation is approx. 3 minutes (120s plus up to 60s jitter); webhooks make it near-instant.
- An analysis metric that matches no series is *inconclusive*, not failed — always add a request-count floor.
- A canary at 5% of low traffic cannot measure a small error-rate change; widen the window or raise the weight.
- Never point a production Application at a mutable `:latest` image tag — desired state must be deterministic.

Common pitfalls (last look)

- Keeping the CI cluster credential "just in case" • timer-based promotion • gating on CPU instead of SLIs
- Fighting the HPA over `replicas` • an unrehearsed rollback • unbounded feature-flag debt

📋 One-Page Cheat Sheet

GitOps — declarative • versioned • pulled automatically • continuously reconciled (OpenGitOps v1).

Argo CD — API server (UI/CLI) • repo server (render) • application controller (diff, sync, self-heal).

Sync policy — `automated` + `prune` (delete removed objects) + `selfHeal` (revert drift).

Rolling — `maxSurge`/`maxUnavailable`; readiness probes are the only safety net; no traffic weights.

Blue-green — two full environments, selector flip; seconds to roll back, $2\times$ cost during overlap.

Canary — weighted routing (mesh/ingress); exposure bounded by weight; requires real metrics.

Analysis — `interval` • `count` • `successCondition` • `failureCondition` • `failureLimit`; no match = inconclusive.

Rollback — `git revert` is durable; `kubectl rollout undo` is drift; aborting a canary is fastest.

CLI quick reference

Day 13 commands worth memorising

```
# Argo CD - inspect before you act
argocd app list
argocd app get cloudbase-api-prod           # sync + health + conditions
argocd app diff cloudbase-api-prod         # git vs live, field by field
argocd app sync cloudbase-api-prod --prune
argocd app history cloudbase-api-prod      # the deploy log
argocd app rollback cloudbase-api-prod 42  # emergency lever only

# drift + break glass
kubectl -n argocd patch app cloudbase-api-prod --type merge \
  -p '{"spec":{"syncPolicy":{"automated":null}}}' # disable auto-sync
```



```
# Argo Rollouts - drive and observe a canary
kubectl argo rollouts get rollout cloudbase-api --watch
kubectl argo rollouts promote cloudbase-api           # next step
kubectl argo rollouts promote cloudbase-api --full # skip remaining analysis
kubectl argo rollouts abort cloudbase-api             # weight -> 0, back to stable
kubectl argo rollouts undo cloudbase-api --to-revision=3

# native fallbacks
kubectl rollout status deployment/cloudbase-api
kubectl rollout undo deployment/cloudbase-api         # drift under GitOps!
```

Interview questions to master

- › What is the practical security benefit of pull-based GitOps over push-based CI deploys — and what new risk does it introduce?
- › Blue-green versus canary: what does each actually protect you from, and at what cost?
- › How should a canary rollout decide automatically whether to proceed or roll back?
- › Why does GitOps make rollback simpler than an imperative deploy pipeline — and what does it make harder?
- › What problem do feature flags solve that a deployment strategy alone cannot?
- › What does "git as the source of truth for the cluster" actually mean in practice?
- › Your Argo CD application is permanently OutOfSync on one field. How do you diagnose it?
- › Which parts of a release cannot be rolled back, and how do you design around that?

Further reading & official docs

- OpenGitOps — opengitops.dev: the four GitOps principles (v1.0.0), the vendor-neutral definition.
- Argo CD documentation — argo-cd.readthedocs.io, especially "Operator Manual → Architectural Overview" and the FAQ entry on reconciliation timeouts.
- Argo Rollouts documentation — argo-rollouts.readthedocs.io, "Features → Analysis" for AnalysisTemplate semantics and background versus inline runs.
- Flux documentation — fluxcd.io/flux/components/: the GitOps Toolkit controllers (source, kustomize, helm, notification).
- Kubernetes documentation — "Performing a Rolling Update" and the Deployment strategy reference for `maxSurge` and `maxUnavailable`.
- CNCF case study: Adobe — the Flex platform built on Argo CD, Workflows, Events and Rollouts, and the scale it operates at.
- Google SRE Workbook, "Canarying Releases" — the statistical reasoning behind canary sizing and duration.

Production checklist

- ☐ No system outside the cluster holds a credential that can write to production.
- ☐ Application code and deployment config live in separate repositories.
- ☐ Every production image is referenced by digest, never by a mutable tag.
- ☐ Auto-sync with self-heal is enabled, and the break-glass procedure is written down and tested.

- ☐ Fields owned by other controllers are excluded or explicitly ignored; no app is permanently OutOfSync.
- ☐ Canary analysis uses the same SLIs you alert on, compares against stable, and fails on absent data.
- ☐ Rollback has been rehearsed within the last quarter, with detect/decide/recover times recorded.
- ☐ Schema changes are additive and forward-compatible for at least one release.
- ☐ Every feature flag has an owner and a removal date; kill switches are labelled separately.

Key idea

What you will learn next (Day 14). You can now build, secure, observe, deploy and safely undo a system. Tomorrow you assemble the whole fourteen days into one coherent platform story – the capstone review: what CloudBase looks like end to end, where its remaining weaknesses are, how to talk about these decisions in an interview, and how to keep learning once the roadmap ends.

DAY 14 • CAPSTONE

Final Project — CloudBase

14

Assembling, operating and presenting production infrastructure — built by you

🕒 Estimated study time: 10 hours

🎯 By the end of this day, you will be able to...

- State what "production-ready" means as a checklist of properties, not a feeling — and defend each item.
- Read a specification the way an engineer reads a ticket: separating functional from non-functional requirements and turning ambiguity into documented decisions.
- Design for failure explicitly — multi-AZ topology, RTO/RPO targets, and the failure modes each choice does and does not cover.
- Operate what you built: run a rollback under pressure, a DR restore drill, a cost audit and a game day.
- Present the system as a senior engineer would — architecture, trade-offs, known weaknesses, and what you would do at ten times the scale.

1 The Big Picture

What problem this solves. For thirteen days you learned mechanisms in isolation: a process and a signal, a commit graph, a subnet, an image layer, a scheduler, a state file, a playbook, a pipeline, a VPC, a time-series database, a policy engine, a reconciliation loop. Every one of them was correct on its own and useless on its own. The problem this chapter solves is *integration*: making thirteen independently-correct mechanisms behave as one system that a stranger can deploy, observe, break, and recover — without you in the room.

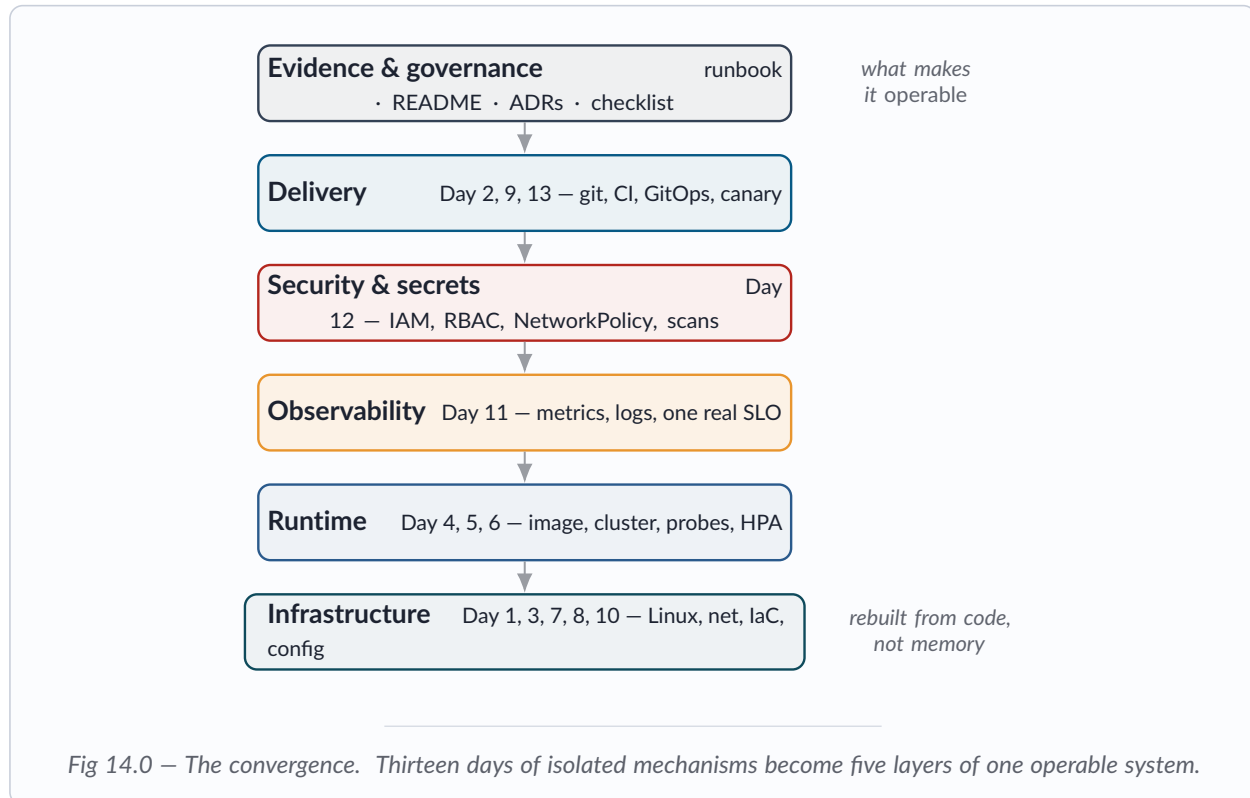
Why DevOps engineers need it. Nobody is paid to run Terraform. They are paid for the property that emerges when Terraform, a container registry, a scheduler, a secrets store and an alert route are wired together correctly: **a change can go from a commit to production safely, and come back**. That property is the job. It is also the thing an interview probes for, because it cannot be memorised — it can only be built and then explained.

🔑 Key idea

"Production-ready" is not a quality level; it is a set of *answered questions*. Who can deploy this? What happens when a node dies? Where does the data live and how far back can we restore it? Who is paged, on what signal, with what runbook? How do we undo the last change in ninety seconds? A system where those questions have written answers is production-ready — even if it is small. A system where they do not is a demo, no matter how many services it has.

Where it fits in a modern cloud architecture. CloudBase is deliberately a *small* application carrying a *large* amount of operational weight. Read the diagram as a convergence: each earlier day contributed

one layer, and today they stack.



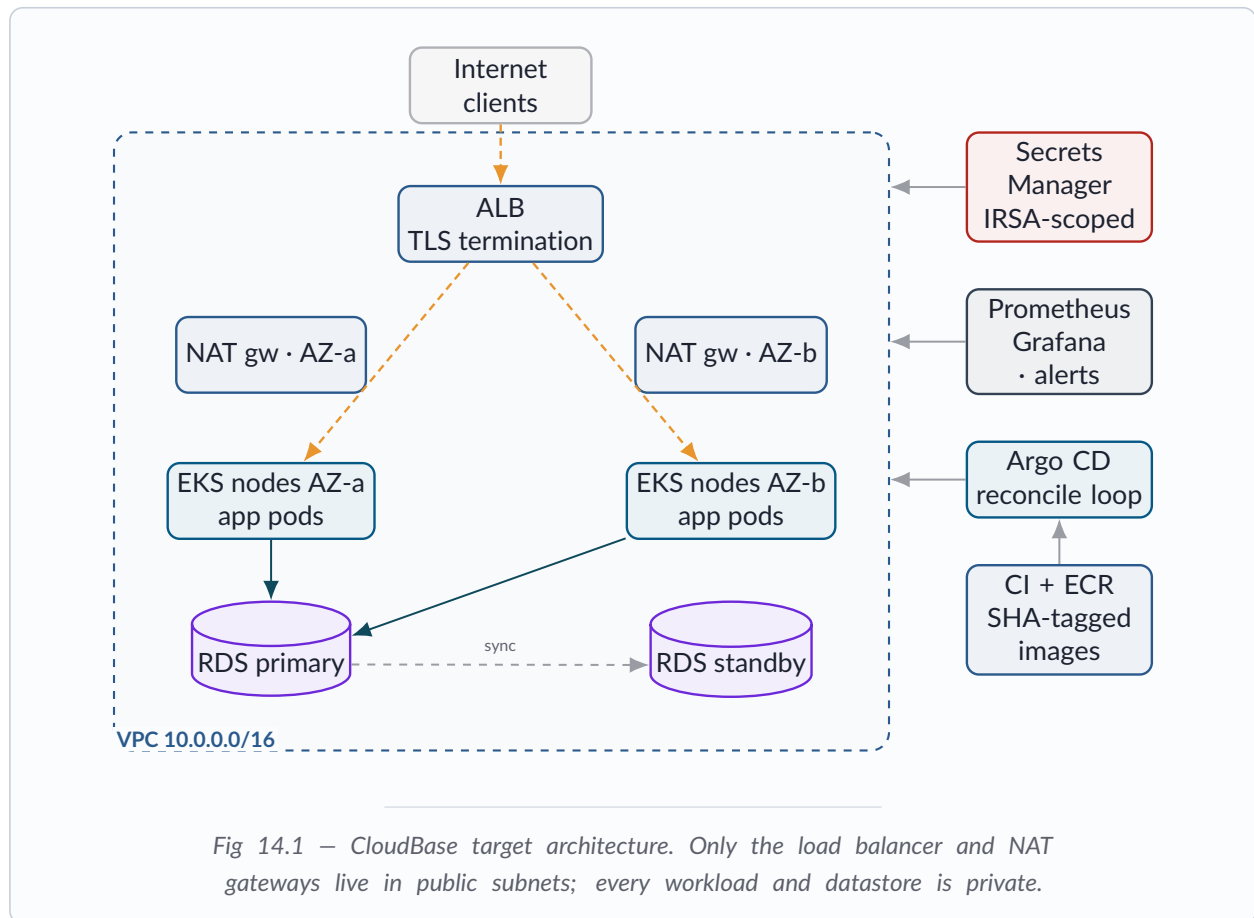
How it connects to previous chapters. Every earlier Build-Along instalment was a component of this one. Day 1 gave the host baseline and strict-mode scripting; Day 2 the repository and its protections; Day 3 and 10 the network and the cloud account; Day 4 the hardened image; Days 5–6 the cluster, probes and ingress; Day 7 the Terraform root modules and remote state; Day 8 idempotent host configuration; Day 9 the pipeline; Day 11 the metrics and the SLO; Day 12 least privilege and secret storage; Day 13 GitOps reconciliation and progressive delivery. Today adds no new mechanism. It adds *assembly, operation and evidence*.

Where companies use it in production. Monzo, a UK bank running on Kubernetes, published how it moved from a flat network — where any of roughly 1,500 microservices could call any other, giving about 9,300 live service-to-service connections — to a default-deny model enforced by Calico-backed NetworkPolicies. The engineering insight is not the policy syntax: it is that they *derived* the rules from code with an internal tool, shipped them first in a dry-run logging mode to measure what *would* have been dropped, and only then enforced. After the change the average service had roughly six allowed outbound destinations instead of the whole platform. That is the shape of every hardening project you will ever run: derive, measure in shadow mode, then enforce.

Netflix pushed the same idea to the infrastructure layer. Its Simian Army escalated from Chaos Monkey (kill one instance) to Chaos Gorilla (fail an entire Availability Zone) to Chaos Kong (evacuate a whole AWS region), because the only way to know a failover works is to perform it on purpose, on a weekday, with the team watching. Your Day 14 game day is the same exercise at a scale you can afford.

Common misconceptions. That a capstone should be *big* (a small system operated excellently is a far stronger signal than a sprawl of half-wired services); that "it works on my cluster" is a completed project (reproducibility from a clean state is the deliverable); that documentation is what you do afterwards if there is time (the README explaining *why* is the artefact a reviewer reads first); and that a rollback path counts if nobody has ever executed it.

2 Concept Map



Terminology to anchor: **requirement** → **architecture decision** → **blast radius** → **RTO/RPO** → **SLO** → **error budget** → **runbook** → **game day** → **post-mortem**.

3 Guided Learning

3.1 Reading the Specification: Functional Requirements

What is it? A functional requirement says what the system must *do*. It is testable by pointing at the system and observing behaviour. The CloudBase specification below is your contract with yourself; each row names the day whose mechanism satisfies it.

Capability	Requirement — and how you prove it	From
Network	VPC with public and private subnets across two Availability Zones; route tables prove the private subnets have no path to an internet gateway.	Day 3, 10
Container image	Multi-stage build, non-root user, pinned base digest, scanned in CI before push.	Day 4, 9, 12
Orchestration	Cluster runs the app via Helm with <i>separate</i> liveness and readiness probes, resource requests/limits, and a working HPA.	Day 5, 6
Edge	Ingress terminates TLS and routes to the Service; certificate renewal is automated, not diarised.	Day 6
Pipeline	test → scan → build → push → deploy, each stage a gate that can fail the build.	Day 9
GitOps	Cluster state reconciled from git by Argo CD or Flux; a canary strategy governs promotion.	Day 13
Observability	Prometheus metrics, a Grafana dashboard, structured logs with correlation IDs, one real SLO with an alert.	Day 11
Secrets	Sourced from a secrets manager at runtime; never in git, never baked into an image layer.	Day 12

Why does it exist? Before requirements were written down, "done" was whatever the last person to touch the system believed. Specifications exist so that *done* is falsifiable. Notice how each row above is phrased so a reviewer can disprove it in under a minute — that is the test of a well-written requirement.

How does it work internally? A requirement becomes real only when something automatic enforces it. "Images must be scanned" is a wish; a pipeline stage that fails on a HIGH-severity finding is a requirement. "Secrets must not be in git" is a wish; a pre-commit hook plus a repository secret scanner is a requirement. When you write your README, mark each requirement with the mechanism that enforces it — the ones you cannot name an enforcer for are the ones that will silently rot.

💡 Senior engineer insight

Juniors deliver features; seniors deliver *enforced invariants*. The interview-winning sentence is not "the app runs on Kubernetes". It is "no image reaches the cluster without a passing scan and an immutable SHA tag, because the only identity allowed to push to the registry is the CI role, and Argo CD only syncs from a protected branch." That is one sentence describing four Days of work as a single closed loop.

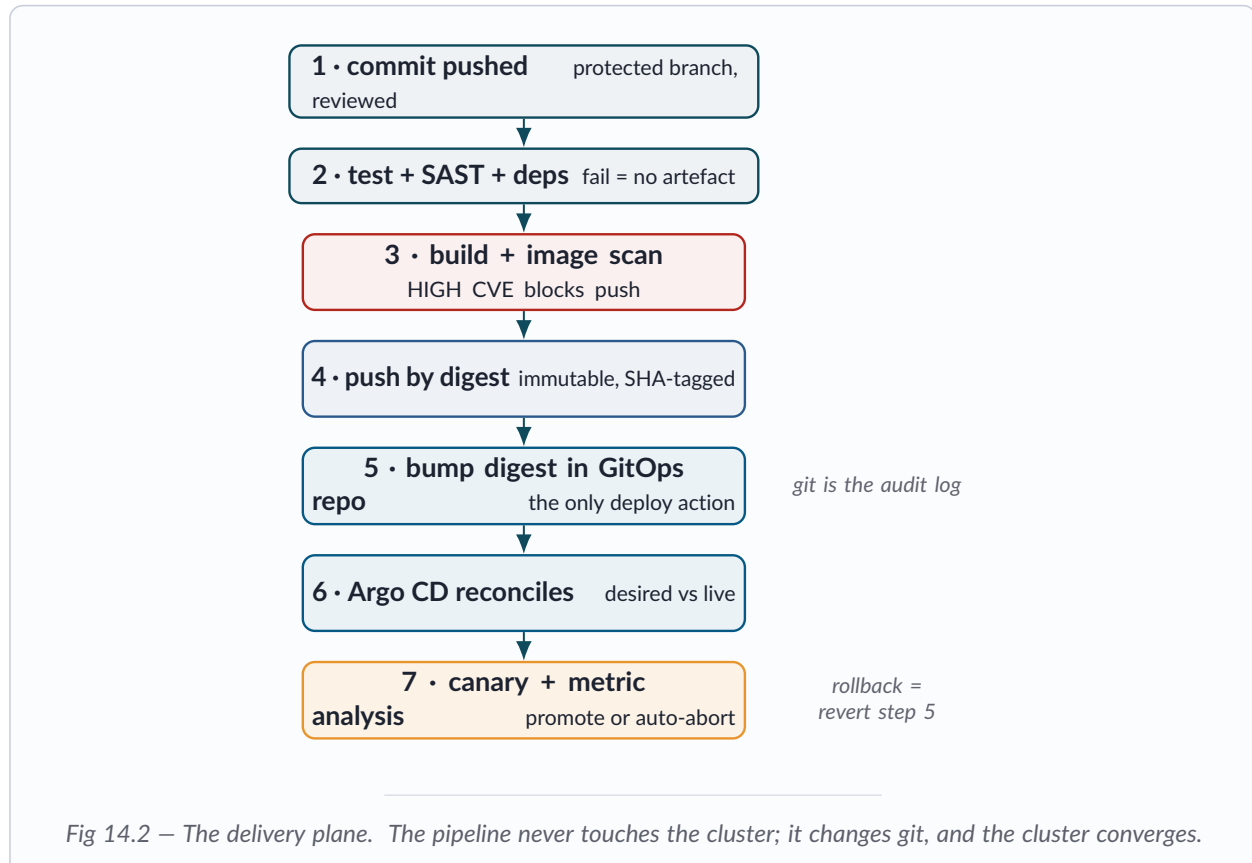
3.2 Non-Functional Requirements: the Ones That Decide Whether You Sleep

What is it? A non-functional requirement (NFR) constrains *how* the system does what it does — its reproducibility, security posture, delivery discipline, observability and documentation. NFRs are what separate a working system from an operable one.

Concern	Requirement
Infrastructure as Code	The entire environment is provisioned by Terraform with remote state and locking. No console clicks that are not later codified.
Configuration management	Any non-containerised host configuration is applied idempotently with Ansible — re-runnable to the same end state.
Security	Least-privilege IAM and Kubernetes RBAC throughout; NetworkPolicies restrict lateral movement; image scanning is a CI gate, not a report.
Networking	Database and application tiers sit in private subnets; only the load balancer and NAT gateways are public.
Delivery	Immutable SHA-tagged images promoted <i>unchanged</i> across environments, plus a rollback path that has actually been rehearsed.
Observability	A dashboard someone would genuinely open during an incident, and an alert tied to a user-visible SLO rather than raw metric noise.
Documentation	A README explaining architectural <i>decisions and trade-offs</i> , not merely how to run it.

Why does it exist? Every NFR above is the scar tissue of a real class of outage. Remote state with locking exists because two engineers ran `apply` at the same time and corrupted a state file. Immutable promotion exists because "rebuild for staging, rebuild for prod" quietly picked up a different transitive dependency between the two builds. Alerting on SLOs exists because alerting on CPU trained an entire generation of on-call engineers to ignore their pagers.

How does it work internally? "Promoted unchanged" has a precise mechanical meaning: the artefact's identity is a content digest. When CI pushes `registry/cloudbase-api@sha256:...`, staging and production reference *that digest*, so the bytes running in production are provably the bytes the tests ran against. A mutable tag like `latest` breaks the proof, because the name can be repointed after the test passed.



⚖️ Design trade-off

GitOps buys you an auditable, revertible record of every production change and removes cluster credentials from CI entirely. It costs you a second repository to keep in sync, a reconciliation delay between merge and effect, and a new failure mode — drift between what a human `kubectl apply`-ed and what git says. Accept the trade-off knowingly and write it in the README; "we chose Argo CD" is not an architecture decision, "we chose Argo CD *because* we wanted CI to hold no cluster credentials, accepting a 3-minute reconciliation lag" is.

3.3 High Availability: Multi-AZ and the Failure Modes It Does Not Cover

What is it? High availability is redundancy plus automatic failure detection plus automatic traffic redirection. Remove any of the three and you have expensive idle capacity rather than availability.

Why does it exist? An Availability Zone is a distinct set of data centres with independent power, cooling and networking inside one region. Spreading across two AZs exists because correlated failure

is the enemy: two servers in the same rack share a power feed, and a system whose "redundant" replicas all die together is not redundant.

How does it work internally? Multi-AZ works only if every layer is spread and every layer's failure detector is faster than your users' patience. A pod spread constraint places replicas across nodes in different AZs. The ALB health-checks targets and stops routing to failed ones within seconds. RDS Multi-AZ maintains a synchronous standby in the second AZ and, on failover, repoints a DNS CNAME — which is why your application must handle a connection drop and reconnect, and why an over-long JVM DNS cache turns a 60-second failover into a 20-minute outage.

Failure	What multi-AZ does	What still bites you
One node dies	Scheduler reschedules pods onto surviving nodes.	Capacity: if each AZ runs at 90% you cannot absorb the other half.
One AZ lost	ALB drains targets; RDS fails over to the standby.	Stale DNS caches; single-AZ resources you forgot (a lone NAT gw).
Whole region lost	Nothing. Multi-AZ is not multi-region.	Only a DR strategy helps — see 3.4.
Bad deploy	Nothing. The bug is in every AZ simultaneously.	Only canary analysis and rollback help.
Database corrupted	Nothing — corruption replicates synchronously.	Only point-in-time restore from backup helps.

Common misconception

"We are Multi-AZ, so we are covered." Multi-AZ addresses *infrastructure* failure in one facility. It offers zero protection against the three failure classes that actually cause most outages: a bad deploy, a bad configuration change, and data corruption — all of which replicate faithfully to your standby. Redundancy protects against things breaking; it never protects against things being wrong.

Reliability principle

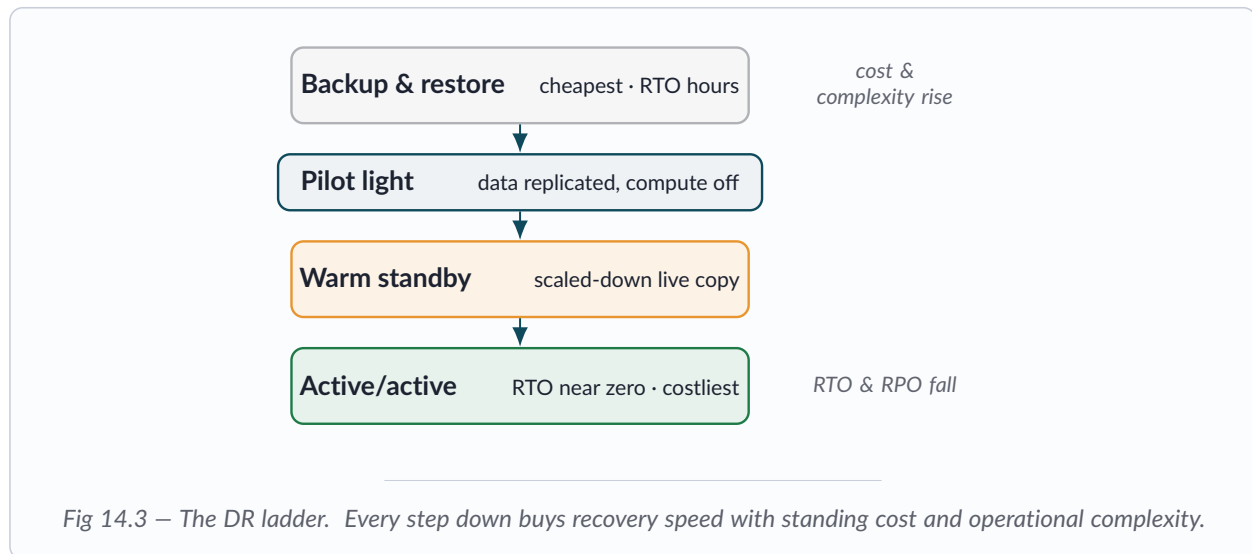
Design the capacity, not just the topology. If your HPA minimum is 2 replicas across 2 AZs, losing one AZ leaves a single replica serving 100% of traffic — which will then fall over from load, and the incident you get is not "AZ failure" but "cascading overload". The rule of thumb is $N + 1$ per AZ: each zone must be able to carry the full load alone, or you must accept degradation and say so explicitly in the README.

3.4 Disaster Recovery: RTO, RPO and Choosing a Strategy Honestly

What is it? AWS's Well-Architected Reliability Pillar defines the two numbers precisely. **RTO** (Recovery Time Objective) is the maximum acceptable delay between the interruption of service and its restoration — how long you may be down. **RPO** (Recovery Point Objective) is the maximum acceptable amount of time since the last recovery point — how much data you may lose. They are *targets set by the business*, not properties of your tooling.

Why does it exist? Without stated numbers, "we have backups" is unfalsifiable. With them, every DR conversation becomes an engineering problem: a 15-minute RPO means transaction-log shipping at least every 15 minutes; a 1-hour RTO means an automated restore, because a human paging in at 3am and reading a wiki will not make it.

How does it work internally? The four canonical strategies trade cost against recovery. Each one is a different amount of infrastructure kept warm in the recovery region.



When should I use it? For CloudBase, backup and restore with a written, *rehearsed* procedure is the correct choice, and saying so is a stronger answer than building an active/active setup you cannot justify. Declare it: "RPO 24 hours from automated RDS snapshots, plus 5-minute point-in-time recovery within the retention window; RTO 2 hours, measured in a restore drill on 2026-07-19."

When should I not? Do not build multi-region for a portfolio project or an internal tool. Multi-region doubles your surface area, forces you to solve data residency and split-brain, and roughly doubles standing spend — for a failure mode that is far rarer than the bad deploys you have not yet defended against.

🔧 Disaster recovery tip

A backup is a hypothesis; a restore is the experiment. Until you have restored into a fresh environment and connected the application to it, you do not have backups — you have snapshots of unknown validity. Put the restore drill on a schedule and record the measured RTO in the runbook each time. When the number drifts upward, that is a finding.

💰 Cloud cost optimization

DR cost is dominated by what you keep running, not by what you store. Snapshots and object storage are cheap; an idle warm-standby cluster is not. The cost-aware pattern is to keep *data* continuously replicated (cheap, and it is what sets your RPO) while keeping *compute* defined in Terraform but not applied — your recovery step is `terraform apply` in the second region, which is exactly why the whole environment had to be code in the first place.

3.5 Cost as a First-Class Engineering Constraint

What is it? Cost optimisation is not a finance activity that happens to you quarterly. It is a design constraint with the same standing as latency: every architectural choice has a monthly price, and an engineer who cannot state it is only half-designing.

Why does it exist? On-premises, capacity was a capital purchase reviewed by committee. In the cloud, any engineer can create a recurring monthly liability with one `terraform apply`, and nothing forces them to look at it again. The discipline exists to replace the committee with attribution and feedback.

How does it work internally? Three mechanisms carry most of the value:

- **Attribution.** Cost allocation tags on every resource (owner, environment, service), applied by Terraform `default_tags` so they cannot be forgotten. Untagged spend is unownable spend.
- **Right-sizing.** Kubernetes requests are a *reservation*: the scheduler subtracts them from node capacity whether or not the pod uses them. A deployment requesting 2 CPU and using 100m is billing you for idle cores across every replica. Set requests from observed p95 usage, and limits from the point where you would rather throttle than let one pod starve its neighbours.
- **Elasticity and lifecycle.** Non-production environments do not need to exist at 2am. Log retention that nobody has ever queried past 14 days is a subscription to your own past. Both are one-line policy changes with permanent effect.

When should I not? Do not optimise cost on the critical path of an incident, and do not trade away redundancy for savings without writing down the availability you just sold. The classic self-inflicted outage is deleting the second NAT gateway to save money and discovering months later that one AZ lost all egress.

Scalability note

Cost per request is the metric that scales; total bill is the metric that panics people. A system whose bill grows linearly with traffic is healthy. A system whose bill grows while traffic is flat has a leak — orphaned volumes, forgotten load balancers, a log pipeline indexing debug output, a cross-AZ chatty service paying data-transfer charges on every hop. Track the ratio, and the leaks announce themselves.

3.6 Operability: Runbooks, SLOs and the Evidence a Reviewer Reads

What is it? Operability is the property that someone other than the author can run the system. Its artefacts are a runbook (what to do when the alert fires), an SLO (what "working" means, numerically), and an architecture README (why it is shaped this way).

Why does it exist? Before runbooks, operational knowledge lived in the one engineer who had seen the failure before — so incidents were resolved at the speed of that person's availability. The runbook is a mechanism for moving knowledge out of a head and into the repository, next to the code that will trigger the next incident.

How does it work internally? A good runbook entry is not prose; it is a decision procedure. Alert name → what the user is experiencing → the first three commands or dashboard panels to check → the two most likely causes with their distinguishing evidence → the mitigation, including the rollback command, verbatim and copy-pasteable. Anything requiring judgement is marked as requiring judgement, so nobody pastes it half-asleep.

docs/runbook.md — one alert entry, abbreviated

```
## ALERT: CloudBaseAPIErrorBudgetBurn (severity: page)

User impact: checkout requests failing; SLO 99.5% success over 30d.

Triage (in order, 5 min budget):
1. Grafana "CloudBase / API overview" -> error rate by endpoint + version
2. argocd app history cloudbase-api      # was there a sync in the window?
3. kubectl -n cloudbase get rollout cloudbase-api

Most likely causes:
A. Bad release      evidence: errors confined to the canary version label
   -> mitigate: kubectl argo rollouts abort cloudbase-api
B. DB saturation    evidence: errors across ALL versions, latency up,
                    pool wait time rising in the app dashboard
```

```
-> mitigate: scale down non-critical consumers; DO NOT restart pods
      (restart storms make pool exhaustion worse)
```

Escalate after 15 min with no improvement. Post-mortem required: yes.

☰ Operational excellence

Write the runbook *before* the incident, and edit it *during* the next one. The half-hour after an incident, while the details are still loaded in your head, is worth more than any planned documentation sprint. If an incident produced no runbook diff, either nothing was learned or nothing was recorded — both are failures.

👤 Interview insight

Expect: "What is your SLO and why that number?" The weak answer names 99.9% because it sounds serious. The strong answer is: "99.5% success over a rolling 30 days, which is a budget of about 3.6 hours of failed requests a month. We chose it because our deploy cadence spends roughly an hour of it, leaving headroom for one genuine incident. When the budget is half-spent we stop shipping features and fix reliability." That answer proves you understand an error budget as a *decision rule*, not a dashboard number.

✓ Best practices — what experienced engineers do

- Every requirement in the spec is paired with the automated mechanism that enforces it.
- Artefacts are immutable and promoted by digest; environments differ only by configuration.
- RTO and RPO are written numbers, and the restore has actually been performed and timed.
- Alerts fire on user-visible symptoms tied to an SLO, and every alert has a runbook entry.
- Every resource carries owner and environment tags applied automatically by Terraform.
- The README explains decisions and trade-offs, and honestly names the system's weak points.

✗ Common mistakes beginners make

- Adding more services instead of operating one service properly.
- Treating multi-AZ as disaster recovery, and snapshots as a tested restore path.
- A rollback procedure that has never been executed by anyone.
- Alerting on CPU and memory rather than on error rate and latency.
- Requests and limits copied from a tutorial, so the cluster bills for idle reservations.
- A README that explains *how to run it* but never says *why* anything is the way it is.

4

Visualizations to Internalize

You will be asked to draw these on a whiteboard, from memory, with someone watching. Practise now, on paper, with nothing open.

 Redraw: the full CloudBase network — VPC, both AZs, public vs private subnets, ALB, NAT, nodes, RDS primary/standby — and mark which resources are reachable from the internet

 Redraw: the delivery plane from commit to production, naming every gate and the exact rollback action at each stage

5**Guided Hands-On Lab — Build CloudBase in the Right Order**

Goal. Assemble the whole system in a sequence where each step is verifiable before the next depends on it. The order matters: it is chosen so that every layer can be tested in isolation, and so that a failure tells you unambiguously which layer broke.

1**Network layer in Terraform**

VPC, public and private subnets in two AZs, internet gateway, NAT gateways, route tables. Remote state with locking configured *first*, before any resource exists.

 **Why:** The network is the only layer nothing else can be tested without, and the hardest to change later because every subsequent resource references its IDs. Prove correctness by reading the private route table and confirming there is no 0.0.0.0/0 route to the IGW — do not infer it from the resource names.

2**Containerise the application**

Multi-stage Dockerfile, non-root user, pinned base image digest, no build tooling in the final layer. Run it locally against a local database before any cluster is involved.

 **Why:** A container that fails in a cluster is ambiguous — image, manifest, network or RBAC. A container proven

locally reduces the next failure to the orchestration layer, which is worth the ten minutes.

3 Stand up the cluster and deploy with Helm

Chart with configurable image digest, resource requests and limits, distinct liveness and readiness probes, a pod anti-affinity or topology spread across AZs, and an HPA.

💡 *Why: Probes are the single most consequential five lines in the chart. Liveness failing restarts the container; readiness failing only removes it from load balancing. Pointing both at a handler that touches the database converts a slow database into a cluster-wide restart storm.*

4 Expose it: Ingress and TLS

Ingress terminating TLS, certificate issued and renewed automatically, DNS pointing at the load balancer.

💡 *Why: This is the first moment the system has real users' shape. It is also where you discover whether your readiness probe is honest — if it reports ready before the app can serve, the load balancer will send traffic into a 502.*

5 Wire the pipeline

test → SAST and dependency scan → build → image scan → push by SHA. Each stage must be able to fail the build; prove it by breaking one deliberately.

💡 *Why: An unproven gate is decoration. Push a deliberately vulnerable dependency once and watch the pipeline stop — that single experiment is what turns "we scan images" into a claim you can defend.*

6 Move deployment to GitOps

Install Argo CD (or Flux), point it at a manifests repository, and remove every cluster credential from CI. CI's last act becomes a commit that bumps the image digest.

💡 *Why: This inverts the trust direction: the cluster pulls, rather than CI pushing. A compromised CI runner can now propose a change but cannot reach the cluster's API server — and every production change is a reviewable commit.*

7 Layer in observability

Application metrics, a dashboard organised by user-facing symptom, structured JSON logs carrying a correlation ID, and exactly one alert bound to your SLO.

💡 *Why: One good alert beats forty. Every alert you add that does not require human action trains the team to ignore the channel, and the cost of that training is paid during the incident that mattered.*

8 Add canary rollout and rehearse the rollback

Configure a canary with automated analysis against your Prometheus success-rate query, then deliberately ship a broken version and watch it abort. Time the rollback.

💡 *Why: You are not testing the canary; you are testing yourself. The number you want is "how many seconds from noticing to healthy", and the only way to know it is to have done it once when nothing was at stake.*

9 Security pass

Default-deny NetworkPolicies with explicit allows, secrets migrated out of manifests into the secrets manager with workload identity, and an audit of every IAM role and Kubernetes Role for unused permissions.

💡 *Why: Doing security last is a deliberate choice here, not a mistake — you cannot write a least-privilege policy for traffic you have not yet observed. Follow Monzo's pattern: observe real traffic, derive the rules from it, run them in dry-run, then enforce.*

 Production tip

Destroy and rebuild the entire environment from scratch at least once before you call it done. `terraform destroy` followed by `terraform apply` into an empty account is the only honest test of "provisioned from code". Every project has one resource somebody created by hand and forgot; this is how you find it, and it is exactly what an interviewer means when they ask whether they could reproduce your environment.

6 Thinking Exercises **Think it through**

Your CloudBase is Multi-AZ, GitOps-managed and monitored — and the worst outage you experience is caused by a configuration change that was reviewed, merged and reconciled exactly as designed. Every safety mechanism worked, and the outage still happened. What does that tell you about where to invest the next unit of reliability effort, and why is "add more redundancy" the wrong answer?

 Think it through

You are asked to cut CloudBase's monthly cost by 40% without an availability regression. List the changes you would make in order, and for each one name the failure mode you are now closer to. Which change would you refuse to make, and what sentence would you write in the README to justify the refusal?

 Think it through

An interviewer says: "Show me the weakest part of this architecture." Answer it truthfully for your own build — name the component, the failure it is vulnerable to, why you accepted that risk given your constraints, and what you would build first if the system had ten times the traffic.

7

Debugging Practice

 Debugging Lab • Intermittent 5xx — roughly one request in twenty

Symptom. Error rate rose from 0.02% to 4% forty minutes after a routine deploy. Not every request fails; retries usually succeed. Nobody rolled back because "it is only sometimes".

error rate by pod, and one failing request's log line

```
$ kubectl -n cloudbase logs -l app=cloudbase-api --since=10m | grep '"level":"ERROR"' | head -3
{"ts":"...", "level":"ERROR", "corr":"9f3a", "pod":"api-7c9d-x2k", "msg":"upstream timeout"}
{"ts":"...", "level":"ERROR", "corr":"b81c", "pod":"api-7c9d-x2k", "msg":"upstream timeout"}
{"ts":"...", "level":"ERROR", "corr":"44de", "pod":"api-7c9d-x2k", "msg":"upstream timeout"}

$ kubectl -n cloudbase get pods -o wide
api-7c9d-x2k    1/1    Running    0    41m    10.0.12.9    node-az-b
api-7c9d-p4m    1/1    Running    0    41m    10.0.11.4    node-az-a
api-7c9d-t8v    1/1    Running    0    41m    10.0.11.7    node-az-a
```

- ✓ **Observe.** Every error carries the same `pod` field. One replica of three is failing — which is why the rate is a fraction, not 100%, and why retries "fix" it.
- ✓ **Hypothesise.** This is not a code defect (the same image runs on all three). It is environmental and specific to that pod or its node — a node-local network path, a stale DNS resolution, or a per-pod resource condition.
- ✓ **Investigate.** Compare the failing pod against a healthy one: is it in a different AZ? `kubectl describe` for throttling and restart counts, `kubectl exec` to resolve the upstream name from inside both pods, and check whether the upstream's endpoint list changed at the time errors began.
- ✓ **Root cause.** The pod on `node-az-b` was resolving the database writer endpoint to an address that stopped serving after an RDS maintenance failover; the client had cached the DNS answer for the process lifetime.
- ✓ **Fix.** Delete the affected pod so a fresh process re-resolves, restoring service immediately.
- ✓ **Prevent.** Bound the client-side DNS cache TTL explicitly, add a readiness probe that actually exercises a datastore round trip so a pod with a dead connection removes *itself* from load balancing, and alert on per-pod error-rate divergence rather than only on the aggregate — an aggregate hides one-in-three.

 Debugging Lab • Connection pool exhaustion that looks like a slow database

Symptom. p99 latency climbed from 120 ms to 9 s across *all* pods and all versions. The database's own CPU is at 25% and its slow-query log is empty. Somebody suggests restarting the pods.

application metrics – the two series that matter

```

cloudbase_db_pool_active{pod="api-7c9d-p4m"}    10
cloudbase_db_pool_idle{pod="api-7c9d-p4m"}      0
cloudbase_db_pool_pending{pod="api-7c9d-p4m"}   87
cloudbase_db_pool_wait_seconds{quantile="0.99"} 8.4

# replicas after the morning traffic ramp
NAME          READY    UP-TO-DATE    AVAILABLE
cloudbase-api 24/24    24            24

```

- ✓ **Observe.** Time is spent *waiting for a connection*, not executing queries. The database is idle because requests never reach it. Latency is uniform across pods, so this is systemic rather than pod-local.
- ✓ **Hypothesise.** The HPA scaled to 24 replicas; each holds a pool of 10, so the deployment demands 240 connections against a database limit well below that — or, equally likely, a query path is holding connections while doing slow non-database work.
- ✓ **Investigate.** Multiply replicas by pool size and compare with the server's `max_connections`. Check whether pending grew at exactly the moment the HPA scaled. Look for a request path that opens a transaction and then calls an external API inside it.
- ✓ **Root cause.** Horizontal scaling multiplied a fixed per-pod pool past the database's connection ceiling. Adding replicas made it worse, which is why the auto-scaler and the incident amplified each other.
- ✓ **Fix.** Cap total connections: reduce per-pod pool size, or put a connection pooler in front of the database so pods multiplex over a bounded server-side set. Do not restart pods — a restart storm makes every pod re-establish connections simultaneously.
- ✓ **Prevent.** Treat connections as a cluster-wide finite resource with an explicit budget ($\text{replicas} \times \text{pool} \leq \text{ceiling}$, with headroom for migrations and admin sessions). Alert on pool saturation and wait time, not just on query latency. Add a maximum-replicas bound that respects the budget.

🔧 Debugging Lab • The rollback that made the incident longer

Symptom. A release is clearly bad. Under pressure an engineer runs the CI pipeline against the previous commit to "rebuild the good version". Eleven minutes later the build fails on an unrelated dependency resolution error, and the site is still down.

what should have happened instead

```

# WRONG under pressure: rebuild from source
$ git checkout v1.4.2 && ./ci/build.sh      # 11 min, and it can fail

# RIGHT: redeploy an artefact that already exists and already passed
$ argocd app history cloudbase-api
ID  DATE          REVISION  IMAGE
14  2026-07-19 09:41 UTC  a91f0c3   cloudbase-api@sha256:7d2f...
13  2026-07-19 08:02 UTC  4b7e118   cloudbase-api@sha256:1c8a...  <- last known good

$ argocd app rollback cloudbase-api 13      # or: git revert the digest bump

```

- ✓ **Observe.** Recovery time is being spent in a build system, which is neither deterministic nor fast, while users are down.

- ✓ **Hypothesise.** The team's rollback path was never rehearsed, so under stress people fell back to the workflow they use daily — building.
- ✓ **Investigate.** Ask the only question that matters mid-incident: does a known-good, already-tested artefact exist? With SHA-tagged immutable images the answer is always yes, and its identity is in the deploy history.
- ✓ **Root cause.** A procedural gap, not a technical one. The mechanism (immutable digests plus GitOps history) was in place; the rehearsed procedure was not.
- ✓ **Fix.** Redeploy the last known-good digest — a git revert of the digest bump, reconciled by Argo CD, typically under two minutes.
- ✓ **Prevent.** Put the exact rollback command in the runbook, verbatim. Rehearse it monthly in a game day. Measure and publish the rollback time, and treat any regression in that number as a reliability defect in its own right.

8 Mini-Labs

Mini-Lab • Beginner

60 min

Goal. Run a cost audit of CloudBase and produce a costed architecture diagram.

Business context. Finance asks what the environment costs and which parts are optional; nobody currently knows.

Architecture. Your existing Terraform state plus the provider's cost and usage data.

Requirements. List every billable resource with an estimated monthly cost; identify the top three line items; find at least one resource nobody owns; propose two reductions with the availability impact of each stated.

Constraints. No change may reduce redundancy without an explicit written trade-off.

Hints. Start from `terraform state list` rather than the billing console — it tells you what *should* exist, and the difference between the two lists is the interesting part. Check NAT gateways, idle load balancers, unattached volumes and log retention.

Expected outcome. A one-page costed diagram and a ranked reduction list committed to docs/.

Production considerations. Add `default_tags` in the provider block so future resources are attributable automatically.

Common pitfalls. Optimising the cheapest resources because they are easiest; deleting the second NAT gateway without noting the AZ-isolation risk you just accepted.

Mini-Lab • Intermediate

90 min

Goal. Perform a disaster-recovery restore drill and measure your real RTO and RPO.

Business context. The README claims a 2-hour RTO. Nobody has ever tested the claim.

Architecture. A fresh namespace or a second Terraform workspace, restored from the latest automated snapshot.

Requirements. Restore the database into a new instance from a snapshot; point a copy of the application at it; verify data integrity with a known query; record the wall-clock time of each phase and the age of the recovered data.

Constraints. Use only what is written in the runbook — if a step is missing, stop, add it, and restart the timer. That *gap* is the finding.

Hints. Time four phases separately: decide, provision, restore, verify. Most teams discover their RTO is dominated by "decide", not by the restore.

Expected outcome. A dated drill record with measured RTO and RPO, plus a runbook diff.

Production considerations. Schedule the drill quarterly and treat an unrehearsed quarter as an open risk.

Common pitfalls. Restoring the data but never connecting the application, so schema or credential mismatches stay hidden until the real disaster.

Mini-Lab • Production-style

2–3 h

Goal. Run a game day: inject a failure on purpose, with the team watching, and let your alerts and runbook do the work.

Business context. You claim the system survives losing an Availability Zone. Prove it before reality asks.

Architecture. Your live non-production CloudBase, plus a hypothesis written down *before* you touch anything.

Requirements. State a hypothesis ("if AZ-b becomes unavailable, error rate stays under 1% and recovery is automatic within 3 minutes"); define an abort condition; inject the failure — cordon and drain all AZ-b nodes, or fail the database over deliberately; observe whether the alert fires, whether the dashboard shows it, and whether the runbook is sufficient; then restore and write it up.

Constraints. Blast radius must be bounded and reversible; one person runs the experiment, another observes only the dashboards, as an on-call engineer would.

Hints. A managed fault-injection service can script this reproducibly, but a cordon-and-drain plus a forced database failover teaches the same lesson today.

Expected outcome. A short post-mortem: hypothesis, what actually happened, every gap found in alerting or documentation, and dated follow-up actions.

Production considerations. Escalate the blast radius only as confidence grows — one pod, then one node, then one AZ. Netflix took years to get from Chaos Monkey to Chaos Kong.

Common pitfalls. Running the experiment with no stated hypothesis (then any outcome looks acceptable); silently fixing what breaks instead of recording it; skipping the write-up, which is the only durable output.

9

Engineering Notes

Senior engineer insight

The strongest signal a portfolio project can send is not sophistication — it is *completeness under constraint*. A small API with a rehearsed rollback, a tested restore, one meaningful alert and a README that names its own weaknesses outranks a sprawling microservice estate that only its author can deploy. Senior engineers recognise the difference instantly, because they have maintained both.

 Security warning • the audit nobody performs

Permissions accumulate and never shrink, because every grant is added during an urgency and removed during nobody's urgency. Schedule the audit as work: for every IAM role and Kubernetes Role, ask which specific action requires each permission, and delete the ones nobody can justify. Do the same for NetworkPolicies. A permission you cannot explain is a permission an attacker will explain to you later.

 Automation tip

Anything you did twice by hand during this capstone is a candidate for automation, but the criterion is not repetition — it is *consequence of getting it wrong at 3am*. Automate the rollback, the restore and the environment rebuild first, because those are executed under stress by a tired person. Leave the pleasant, low-stakes manual steps alone; a script nobody trusts is worse than a documented command.

 Infrastructure insight

The deepest lesson of the fourteen days is that infrastructure is a *program* whose runtime is a cloud account. It has source (Terraform, Helm, manifests), a build (CI), a deploy (GitOps reconciliation), tests (plan, policy checks, game days), observability (metrics and alerts) and a rollback. Every engineering practice you already trust for application code applies without modification. Teams that internalise this stop treating infrastructure work as a lesser discipline — and their environments stop being unique snowflakes.

 Reliability principle

Availability is not bought once; it decays. Certificates expire, quotas tighten, base images accrue CVEs, a dependency deprecates an API, an alert threshold drifts out of relevance as traffic grows. A system that is production-ready today and untouched for six months is not production-ready. The maintenance loop — patch, re-scan, re-drill, re-audit — is part of the architecture, not an afterthought to it.

10 Build-Along Project — CloudBase

Thirteen instalments built the parts: a hardened host and strict-mode scripts (Day 1), a protected repository (Day 2), a network design (Day 3), an image (Day 4), a cluster and workloads (Days 5–6), Terraform with remote state (Day 7), idempotent host configuration (Day 8), a pipeline (Day 9), a real cloud account (Day 10), metrics and an SLO (Day 11), least privilege and secrets (Day 12), GitOps and progressive delivery (Day 13). Today you assemble them into one environment and — just as importantly — produce the evidence that it is operable by somebody who is not you.

 Build-Along • Day 14 — final assembly, the runbook and the architecture README

- Assemble the full environment from code: `terraform destroy` then `terraform apply` into a clean state, then deploy the application through the pipeline and GitOps, touching nothing by hand.
- Confirm every functional requirement from 3.1 and every NFR from 3.2, naming for each one the mechanism that enforces it — not the intention.
- Write `docs/runbook.md`: one entry per alert with user impact, a time-boxed triage sequence, the two most likely causes with distinguishing evidence, verbatim mitigation commands, escalation criteria, plus standalone procedures for rollback, database restore and full environment rebuild.

- Write `docs/README.md` as an architecture document: the diagram, the request path, and a numbered list of decisions, each in the form *decision – alternatives considered – why – what it costs us*. Include a "known weaknesses and what I would do at 10x" section.
- Record the measured numbers, dated: rollback time, restore time (RTO), data-loss window (RPO), monthly cost, and the SLO with its error budget.
- Run one game day against a non-production copy; commit the post-mortem and the runbook diff it produced.
- Perform the security pass: default-deny NetworkPolicies, secrets sourced at runtime from the secrets manager, and an IAM/RBAC audit removing every permission you cannot justify out loud.
- Tag the repository `v1.0`, and record in the README exactly what a reviewer must run to reproduce the environment from zero.

Definition of done: a stranger with cloud credentials, your repository and no access to you can bring the entire environment up from an empty account with the documented commands, watch a change reach production through the pipeline, break it, and restore it using only `docs/runbook.md` — and the measured rollback time, RTO, RPO and monthly cost are written down and dated.

Production tip

Record a five-minute screen capture of the whole loop: commit → pipeline → canary → dashboard → deliberate break → rollback. It costs one afternoon and it converts every claim in your README into something a reviewer can verify in the time it takes to drink a coffee. It is also the artefact you will be most grateful for six months from now, when you have forgotten which command does what.

11 Chapter Summary

Key takeaways

- ✓ Production-ready is a set of answered questions — deploy, failure, data, paging, rollback — not a size or a technology list.
- ✓ A requirement is real only when an automated mechanism enforces it; otherwise it is an intention.
- ✓ Immutable, digest-identified artefacts make promotion provable and rollback a redeploy rather than a rebuild.
- ✓ Multi-AZ addresses infrastructure failure only; bad deploys, bad config and data corruption replicate faithfully.
- ✓ RTO and RPO are business-set targets; an untested backup is a hypothesis, not a recovery plan.
- ✓ Cost is a design constraint with the same standing as latency, and attribution comes before optimisation.
- ✓ The runbook, the README's decision log and the measured numbers are what make the system operable by someone else.

Things to remember

- Replicas × per-pod pool size must stay under the database connection ceiling — horizontal scaling can amplify an incident.
- Errors confined to one pod point at the environment; errors spread across all versions point at a shared dependency.
- Never rebuild from source under incident pressure. Redeploy a known-good digest.
- "I would improve X next" is a stronger interview answer than "it is complete".

Common pitfalls (last look)

- Adding services instead of depth • untested restores • unrehearsed rollbacks
- Alerting on CPU rather than symptoms • a README that only says how to run it

📋 One-Page Cheat Sheet

Production-ready — deploy path, failure behaviour, data recovery, paging, rollback — all answered in writing.

RTO / RPO — max acceptable downtime / max acceptable data loss. Targets set by the business, proven by a drill.

DR ladder — backup-restore → pilot light → warm standby → active/active; cost up, recovery time down.

Multi-AZ — covers facility failure; covers neither bad deploys nor corruption nor region loss.

Immutable promotion — same digest across environments; environments differ only by configuration.

Error budget — allowed SLO violation per window; when it is spent, reliability outranks features.

Game day — stated hypothesis, bounded blast radius, abort condition, written post-mortem.

Runbook entry — impact → triage steps → likely causes with evidence → verbatim mitigation → escalation.

CLI quick reference

Day 14 — assembly, verification and incident commands

```
# rebuild the world from code (the honest reproducibility test)
terraform destroy -auto-approve && terraform apply -auto-approve
terraform state list                                # what SHOULD exist; diff vs the bill

# verify the runtime
kubectl -n cloudbase get deploy,hpa,ingress,networkpolicy
kubectl -n cloudbase describe pod <pod> | sed -n '/Events/, $p'
kubectl -n cloudbase top pods                       # requests vs actual usage (rightsizing)

# delivery + rollback
argocd app history cloudbase-api                    # find the last known-good digest
argocd app rollback cloudbase-api <id>
kubectl argo rollouts status cloudbase-api
kubectl argo rollouts abort cloudbase-api

# game day: bounded, reversible AZ failure
kubectl cordon -l topology.kubernetes.io/zone=eu-west-1b
kubectl drain -l topology.kubernetes.io/zone=eu-west-1b --ignore-daemonsets
kubectl uncordon -l topology.kubernetes.io/zone=eu-west-1b
```

```
# DR drill
aws rds describe-db-snapshots --db-instance-identifier cloudbase
aws rds restore-db-instance-to-point-in-time --help
```

Interview questions to master

- › Walk me through your architecture, then tell me its weakest point and why you accepted that risk.
- › What are your RTO and RPO, how did you measure them, and when did you last test the restore?
- › The app is intermittently returning 5xx. Walk me through your process, in order.
- › How do you safely roll back a bad deploy under pressure — and why not rebuild from source?
- › Why put the database in a private subnet if security groups already restrict access?
- › Multi-AZ versus a read replica — which problem does each one solve?
- › Why tag images by commit SHA instead of `latest`, and why promote rather than rebuild?
- › What would you change about this system at ten times the traffic, and what would you leave alone?

Further reading & official docs

- AWS Well-Architected Framework, Reliability Pillar — "Disaster Recovery (DR) objectives" and "Plan for Disaster Recovery" (docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/). The authoritative definitions of RTO and RPO used in this chapter.
- AWS Architecture Blog — "Disaster Recovery (DR) Architecture on AWS, Part I: Strategies for Recovery in the Cloud". The four-strategy ladder in Fig 14.3.
- AWS Fault Injection Service User Guide (docs.aws.amazon.com/fis/latest/userguide/) — experiment templates, targets, actions and stop conditions for scripted game days.
- Monzo engineering blog — "We built network isolation for 1,500 services". How default-deny NetworkPolicies were derived from code and rolled out in dry-run before enforcement.
- Argo Rollouts documentation — "Analysis & Progressive Delivery" and the Prometheus provider page (argo-rollouts.readthedocs.io) — AnalysisTemplate, background analysis and automatic abort.
- Kubernetes documentation — "Network Policies" and "Configure Liveness, Readiness and Startup Probes" (kubernetes.io/docs/).
- Google SRE Book, chapters on Service Level Objectives and Postmortem Culture — error budgets as a decision rule, and blameless write-ups.

Production checklist

- ☐ All thirteen prior days' concepts are represented somewhere in the infrastructure.
- ☐ `terraform apply` brings up the full environment from a clean state, verified by an actual destroy-and-rebuild.
- ☐ The CI/CD pipeline is green and genuinely gates on tests and security scans — each gate proven by breaking it once.
- ☐ A dashboard and at least one alert exist and reflect a real SLO, with a runbook entry per alert.

- ☐ Database and application tiers are in private subnets; only the load balancer and NAT gateways are public.
- ☐ Secrets are sourced at runtime from a secrets manager; nothing sensitive is in git or in an image layer.
- ☐ Rollback, restore and full rebuild have each been executed at least once and timed.
- ☐ The README explains architecture decisions and trade-offs, and names the system's known weaknesses.

Key idea

What comes next (after Day 14). You now hold the thing this book existed to produce: a system you can defend line by line. The next step is not another tool — it is repetition under different constraints. Run the game day again with a failure you did not choose. Re-cost the environment after a traffic change. Hand the runbook to somebody else and watch where they hesitate; that hesitation is your next ticket. The senior engineer is not the one who knows the most services — it is the one who has broken their own system on purpose, most often, and written down what happened each time. Now go build something and break it on purpose.

APPENDIX A

DevOps Interview Cheat Sheet

A

The mechanism behind the tool — review this before every interview

Interviewers at the junior/mid level test *judgment* as much as tool syntax: the ability to explain why a decision was made, not recite a flag. Every answer below states the mechanism. Read a question, answer it out loud from memory, then check yourself.

Linux

› What is PID 1 responsible for?

PID 1 (`systemd` on modern distros) is the first userspace process the kernel starts. It reaps orphaned processes and starts every other service in dependency order.

› Hard link vs symlink?

A hard link is a second directory entry pointing at the same inode — the same file, and it survives the original being deleted. A symlink is a separate file containing a path, and it breaks if the target moves.

› Why does SIGKILL risk data corruption that SIGTERM does not?

SIGKILL is handled by the kernel and cannot be caught, so the process gets no chance to flush buffers, close files or release locks before dying.

› What does load average actually mean?

The average number of processes running or waiting on CPU/IO over 1/5/15 minutes. A load average above the core count means work is queued.

Git

› Merge vs rebase — the mechanical difference?

Merge creates a new commit joining two histories, rewriting nothing. Rebase replays commits onto a new base, generating new commit hashes and a linear history.

› Why is `--force-with-lease` safer than `--force`?

It refuses to push if the remote branch has commits you have not fetched, preventing you from silently overwriting a teammate's work.

› How do you find which commit introduced a bug?

`git bisect` — a binary search between a known-good and known-bad commit, testing each candidate until it isolates the exact one.

Docker

› Image vs container?

An image is a read-only, layered template; a container is a running (or stopped) instance of that image plus one writable layer on top.

› Why do multi-stage builds reduce attack surface, not just size?

Build tools, compilers and dev dependencies never ship in the final image — fewer packages for a scanner to flag, and fewer tools available to an attacker who gets a shell.

› CMD vs ENTRYPOINT?

`ENTRYPOINT` defines the fixed executable that always runs; `CMD` supplies default arguments that can be overridden at `docker run` time.

Kubernetes

› What actually happens when you run `kubectl apply`?

`kubectl` sends the desired state to the API server, which persists it to etcd. Controllers then reconcile actual cluster state toward that desired state, asynchronously.

› Why does a Service keep working when Pods are replaced?

A Service selects Pods dynamically by label, not by a fixed IP list — as old Pods die and new ones start, the endpoint list updates automatically.

› Liveness vs readiness, concretely?

Liveness failing restarts the container. Readiness failing only removes it from load balancing. Conflating the two turns a slow dependency into a restart storm.

› What is a DaemonSet for?

Ensuring exactly one copy of a Pod runs on every (or every matching) node — used for node-level agents such as log shippers and monitoring agents.

Terraform

› Why can Terraform not work correctly without state?

State is the only record mapping configuration blocks to the real IDs of what was created. Without it, Terraform cannot tell what exists from what needs creating.

› What does `terraform plan` actually do?

Refreshes real resource state, computes the diff against the desired configuration, and prints exactly what would change — without touching anything.

› Why remote state with locking instead of local?

A team needs one source of truth. Without a lock, two simultaneous applies can corrupt the state file or make conflicting changes.

CI/CD

› Continuous Delivery vs Continuous Deployment?

Delivery means every passing build is one click from production. Deployment removes the click and ships automatically.

› Why tag images by commit SHA instead of `latest`?

Every deployed artifact is traceable to the exact source commit, and rollback means redeploying a known-good tag rather than guessing what `latest` currently points at.

› Why promote the same image across environments instead of rebuilding?

Rebuilding risks a different dependency resolution or base-image patch sneaking in between builds — exactly the drift a tested pipeline exists to eliminate.

AWS & Cloud

› Why put a database in a private subnet?

Private subnets have no route to an internet gateway, so nothing on the internet can reach the database directly regardless of security-group rules. It is a second layer of defence, not the only one.

› Why prefer IAM roles over access keys for compute identity?

Roles issue short-lived, automatically rotated credentials scoped to the resource. A static access key is a permanent secret that works from anywhere the moment it leaks.

› Multi-AZ vs read replica?

Multi-AZ is synchronous replication for automatic failover (availability). A read replica is asynchronous replication for scaling reads. Different problems — often combined.

Monitoring & Observability

› Why does p99 latency matter more than average latency?

Averages smooth outliers into invisibility. p99 exposes the tail of genuinely slow requests, which is where real user pain and SLO breaches live.

› What is an error budget, practically?

The allowed amount of SLO violation over a period. Once it is spent, team priority should shift from shipping features to reliability work.

› Counter vs gauge?

A counter only increases (resetting on restart); a gauge moves freely up and down. Using a gauge where a counter belongs breaks `rate()` calculations.

Security

› First move when a secret is committed and pushed?

Treat it as compromised immediately — rotate or revoke it — *then* separately clean git history. Deleting the file in a later commit does not remove it from history.

› Why run SAST, dependency scanning and image scanning as three separate gates?

Each catches a different class of risk: insecure code you wrote, known CVEs in libraries you depend on, and known CVEs in the OS packages baked into the image. No single scan covers all three.

› What does a Kubernetes NetworkPolicy actually restrict?

Pod-to-pod traffic by label selector. By default every Pod can reach every Pod; a NetworkPolicy is what limits lateral movement if one is compromised.

System Design & Troubleshooting

› How would you scale a read-heavy API?

Add a cache in front of the database, add read replicas, and consider CDN caching for public/static data before scaling the database vertically.

› The app is intermittently returning 5xx — what is your process?

Check dashboards for error-rate/latency spikes correlated with a deploy or traffic change; check logs by correlation ID; check downstream dependency health; check resource saturation (CPU, memory, connection pool, disk).

› How do you safely roll back a bad deploy under pressure?

Redeploy the last known-good, SHA-tagged image. Never rebuild from source under incident pressure — which is exactly why CI/CD promotes immutable, tagged artifacts.

› Vertical vs horizontal scaling?

Vertical is a bigger machine: simple, but with a ceiling and a single point of failure. Horizontal is more machines behind a load balancer, which requires statelessness to work.

💡 Senior engineer insight

The candidates who get offers are not the ones who recite the most flags. They are the ones who can say “here is the trade-off I made, and here is what I would do differently at ten times the scale.” That sentence is worth more than a perfect, over-engineered capstone. Practise answering every question above with a *because*.

APPENDIX B

The CloudBase Build-Along Master Tracker B

One platform, fourteen increments — tick each as you finish it

Your single source of truth for the capstone. By Day 14, `terraform apply` should bring up the whole environment from a clean state and the pipeline should deploy into it unattended.

Phase 1 — Foundations (Days 1–3)

- ☐ **Day 1:** Base host — strict-mode `bootstrap.sh`, dedicated non-root service user, `/opt/cloudbase` with correct modes, systemd unit with a restart policy, verified via `journalctl`.
- ☐ **Day 2:** Repo conventions — branch protection on `main` (passing CI + one review), `CODEOWNERS`, commit convention, pre-commit hook blocking secret patterns.
- ☐ **Day 3:** Network design on paper — one /16 VPC, public and private subnets across two AZs, a bastion as the single audited SSH entry point, least-privilege security groups.

Phase 2 — Containers & Orchestration (Days 4–6)

- ☐ **Day 4:** Hardened multi-stage Dockerfile — non-root `USER`, pinned base tag, `.dockerignore`, layers ordered for cache reuse.
- ☐ **Day 5:** Kubernetes deployment — Deployment with resource requests/limits, Service, ConfigMap and Secret, separated liveness and readiness probes.
- ☐ **Day 6:** Production Kubernetes — Helm chart with per-environment values, Ingress with TLS termination, HPA, least-privilege RBAC.

Phase 3 — Infrastructure as Code & Delivery (Days 7–10)

- ☐ **Day 7:** Terraform — VPC, subnets, IGW, NAT gateway and route tables in code, with a remote backend and state locking.
- ☐ **Day 8:** Ansible — an idempotent role replacing `bootstrap.sh`, per-environment inventory, secrets in Vault.
- ☐ **Day 9:** CI/CD — test → scan → build → push, SHA-tagged images, secrets from the CI store, manual gate before production.
- ☐ **Day 10:** AWS — ALB as the only public entry, cluster compute, RDS in private subnets, least-privilege IAM roles rather than static keys.

Phase 4 — Operate It Like Production (Days 11–14)

- ☐ **Day 11:** Observability — Prometheus and Grafana, structured logs with correlation IDs, one real SLO with an alert tied to it.
- ☐ **Day 12:** Security — secrets in a manager or Vault, image scanning as a blocking gate, default-deny NetworkPolicies, IAM/RBAC audit.
- ☐ **Day 13:** GitOps — Argo CD (or Flux) reconciling from git, a canary rollout gated on the Day 11 metrics, and a *rehearsed* rollback.
- ☐ **Day 14:** Capstone — full assembly, a disaster-recovery restore drill, an operational runbook, and an architecture review.

 Build-Along • Final completion checklist

- ☐ All 13 prior days' concepts are represented somewhere in this infrastructure.
- ☐ `terraform apply` brings up the full environment from a clean state.
- ☐ The CI/CD pipeline is green and gates on tests plus security scans.
- ☐ A dashboard and at least one real alert exist and reflect actual SLOs.
- ☐ Secrets exist in a secrets manager — never in git, never in the image.
- ☐ A rollback has been rehearsed, not just designed.
- ☐ The README explains architecture *decisions* and trade-offs, not just how to run it.

APPENDIX C

Command & Manifest Quick Reference



The commands you will actually reach for, grouped by tool

Linux triage

the first five minutes of an incident

```
uptime                                # load average: 1/5/15 min
ps aux --sort=-%cpu | head            # top CPU consumers
df -h && sudo lsof +L1                # disk full? deleted-but-open files?
journalctl -u <unit> -p err --since "1 hour ago"
ss -tulpn                             # what is listening on which port
```

Git recovery

when something went wrong

```
git reflog                            # every HEAD position - your undo history
git revert <sha>                       # safe on shared branches
git bisect start / bad / good <sha>    # binary-search a regression
git push --force-with-lease            # never bare --force
```

Docker

build, inspect, debug

```
docker build -t app:${(git rev-parse --short HEAD)} .
docker history <image>                 # see layer sizes - find the bloat
docker run --rm -it --entrypoint sh <image>
docker logs -f --tail 100 <container>
docker system df && docker system prune # reclaim disk
```

Kubernetes

the debugging loop

```
kubectl get pods -o wide --sort-by=.status.startTime
kubectl describe pod <pod>             # EVENTS at the bottom - read these first
kubectl logs <pod> -c <container> --previous # logs from the crashed instance
kubectl exec -it <pod> -- sh
kubectl get events --sort-by=.lastTimestamp
kubectl rollout status/undo deployment/<name>
```

Terraform

the safe loop

```
terraform init -backend-config=env/prod.hcl
terraform plan -out=tfplan             # ALWAYS plan to a file
terraform apply tfplan                 # apply exactly what you reviewed
```

```
terraform state list / show <addr>
terraform force-unlock <lock-id>      # only when you are certain
```

Ansible

idempotency in practice

```
ansible-inventory --list -i inventory/prod
ansible-playbook site.yml --check --diff # dry run, show changes
ansible-playbook site.yml --limit web --tags nginx
ansible-vault edit group_vars/prod/vault.yml
```

APPENDIX D

Glossary of Key Terms

The vocabulary of a DevOps engineer, in plain language

D

Namespace (Linux) — Kernel feature giving a process an isolated view of a resource (PIDs, network, mounts). One half of what a container is.

cgroup — Kernel feature limiting and accounting a process group's CPU, memory and IO. The other half of a container.

PID 1 — First userspace process; reaps orphans and supervises services. In a container, your app usually is PID 1.

SIGTERM / SIGKILL — Catchable, graceful stop versus uncatchable kernel kill. $\text{Exit } 137 = 128 + 9 = \text{SIGKILL}$.

Idempotency — Running an operation again produces the same state. The property that makes automation safe to re-run.

Image layer — An immutable, content-addressed filesystem diff. Cached and shared between images.

Multi-stage build — A Dockerfile that builds in one stage and copies only artifacts into a lean runtime stage.

Control plane — Kubernetes API server, scheduler, controller manager and etcd — the brain that records and reconciles desired state.

Reconciliation loop — A controller continuously driving actual state toward desired state. The core idea of Kubernetes and GitOps.

Liveness vs readiness — Liveness failure restarts the container; readiness failure only removes it from load balancing.

Service / Ingress — A Service gives stable in-cluster networking by label selector; an Ingress is an HTTP reverse proxy for external traffic, usually terminating TLS.

HPA — Horizontal Pod Autoscaler — scales replica count from a metric ratio.

Helm chart — A templated, versioned package of Kubernetes manifests with per-environment values.

Terraform state — The map from configuration to real resource IDs. Contains secrets — never commit it.

Plan / apply — Plan computes and prints the diff without changing anything; apply executes exactly that

diff.

Drift — Reality diverging from what the code declares, usually because someone changed it by hand.

Remote backend — Shared, locked state storage so a team cannot corrupt state with concurrent applies.

CIDR — $10.0.0.0/16$ — the prefix length fixes the network portion; the rest addresses hosts. $/24 = 254$ usable.

Security group vs NACL — Stateful, instance-level, allow-only versus stateless, subnet-level, ordered with explicit deny.

NAT gateway — Lets private-subnet hosts reach the internet outbound without being reachable inbound. Bills hourly plus per GB.

Bastion / jump host — A single hardened, audited SSH entry point to hosts with no public route.

IAM role vs access key — A role issues short-lived rotated credentials to a workload; a static key is a permanent secret that works from anywhere.

Least privilege — Grant exactly the permissions needed, no more. Explicit deny always wins in IAM evaluation.

TLS termination — Decrypting HTTPS at the load balancer or ingress so backends handle plain HTTP inside the trust boundary.

DNS TTL — How long resolvers cache an answer. Lower it days before a planned cutover, never during one.

Artifact promotion — Building once and moving the same immutable, SHA-tagged image through environments.

GitOps — Git as the single source of truth, with an agent continuously reconciling the cluster to match it.

Blue-green / canary — Two full environments with an instant switch, versus shifting a small traffic percentage while watching metrics.

SLI / SLO / error budget — The measured indicator, the target for it, and the amount of failure you may spend before reliability work takes priority.

p99 latency — The 99th percentile — the slow tail where real user pain lives, invisible in an average.

Counter vs gauge — Monotonically increasing versus free-moving. Using the wrong one breaks `rate()`.

Cardinality — Number of distinct label combinations in a metric. High cardinality is the usual cause of a melted Prometheus.

NetworkPolicy — Label-selector firewall for pod-to-pod traffic; without one, everything can reach everything.

RTO / RPO — How fast you must recover, and how much data you may lose. Both are business decisions, not technical ones.

Good luck — now go build something, and break it on purpose before production does it for you.